Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 1
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

Titulari de Curs

- Prof. Emil-Ioan Sluşanschi – Seriile CB & CD
  emil.slusanschi@cs.pub.ro

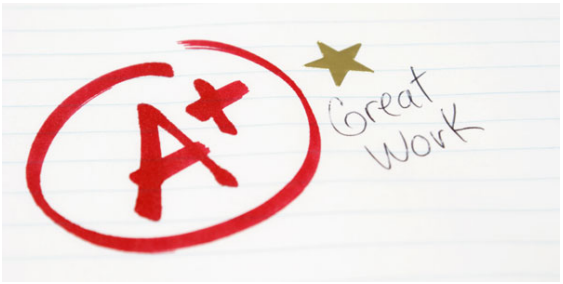
- Șl. Voichița Iancu – Seria CA & CC
  voichita.iancu@cs.pub.ro

Cuprins

- Notare
- Sumarul Cursului
- Bibliografie
- Educatia Inginereasca
- De ce Arhitectura Sistemelor de Calcul?
- Idei Importante in Arhitectura Sistemelor de Calcul

Notarea la ASC

- Laborator: 5/5.5
  - Trei teme: 4.5:
    - Python: 1.5
    - GP-GPU: 1.5
    - Optimizare C: 1.5
  - Activitate la laborator: 1
  - Bonusuri:
    - Max 20% @teme
    - Participări concursuri
- Examen: 5 (teorie)
  - Min > 2.5 lab
  - Oral || Scris



Colaborarea in **Negru** si Alb

- **Colaborarea Buna**
  - Discutii, brainstorming, dezbateri on/off-line
  - Participarea activa la prezentarile temelor in laboratoarele dedicate
- **Colaborarea Rea**
  - Lucrul in comun/revizie pe cod
  - Reluare/oferire /partajare/copiere de (segmente de) cod
- Ar trebui sa fie de la sine inteles, dar…
  - Nu cumparati teme!
  - Nu rugati pe cineva din afara cursului sa va ajute
  - Nu cautati pe internet / GitHub / ChatGPT solutii pentru teme
  - Nu puneti teme rezolvate pe site-uri nesecurizate sau pe GitHub / GitLab
  - Doar din propriile teme veti invata cu adevarat

Sumarul Cursului

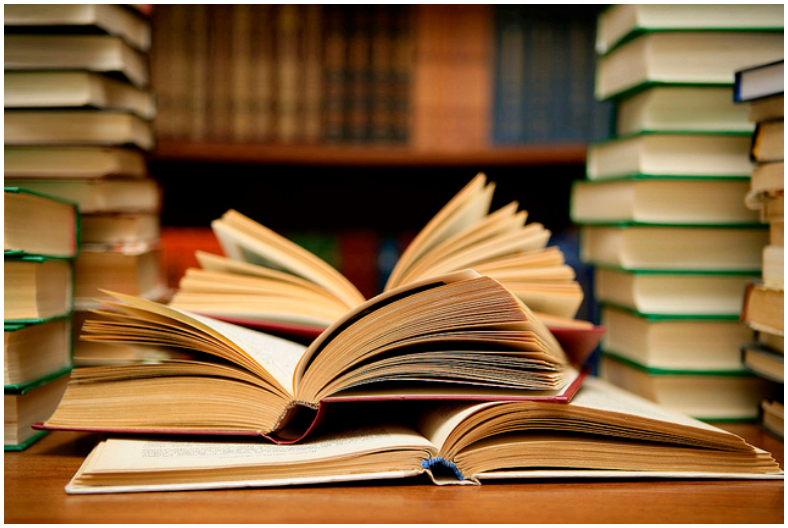
Un Scurt Sumar al Cursului

- Intro & Recapitulare
- Nivelul structural de descriere a sitemelor numerice - PMS
- Ierarhia de memorii, localitatea datelor, cache
- Arhitecturi omogene/eterogene: CPU / GP-GPU
- Comutatoare ierarhice & neierarhice
- Structuri de calcul cu prelucrare paralela – taxonomii
- Structuri SIMD
- Intercalarea perfecta si conectarea inversa

Un Scurt Sumar al Cursului (2)

- Arhitecturi MIMD – Caracteristici
- Arhitectura Cm* - comunicare intra- si inter-cluster
- Interconectarea in sisteme cu resurse multiple
- Retele de comutare (Ierarhice & Delta)
- Debugging, profiling, analiza de performanta
- Top 500, Benchmarking & the current Top 10

  Sumarul cursului se poate modifica pe parcursul semestrului



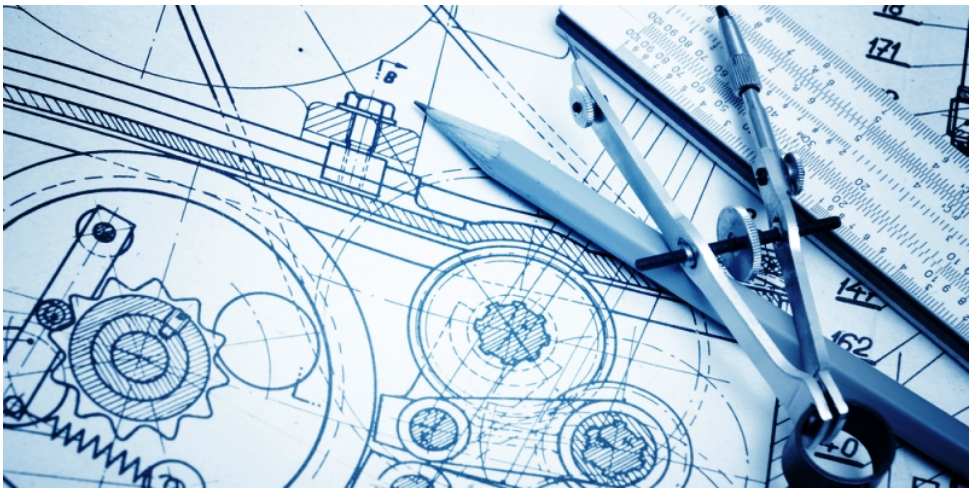
Bibliografie

Bibliografie

- Structura si Arhitectura Sistemelor Numerice; T. Moisa, N. Tapus, C. Morarescu 1999
- Introduction to Parallel Computing: Design & Analysis of Algorithms; V. Kumar, A. Grama, A. Gupta, G. Karypis; Addison Wesley; 2nd Edition 2003
- The Sourcebook of Parallel Computing; J. Dongarra, I. Foster, W. Grapp, K. Kennedy; Morgan Kaufmann 2002
- Computer Organization & Design – The Hardware Software Interface; D.A. Patterson, J. Hennesy; Morgan Kaufmann; 5th Edition 2014
- Computer Architecture: A Quantitative Approach; J. Hennesy, D.A. Patterson; Morgan Kaufmann; 6th Edition 2017
- Introduction to Parallel Processing; Algorithms and Architectures; Behrooz Parhami; Kluwer 2002
- Techniques for Optimizing Applications: High Performance Computing; Rajat P. Garg, Ilya Sharapov; Sun 2001.
- Inside the Machine; Jon Stokes; No Starch Press 2006.
- www.top500.org

Bibliografie vs. Cursuri

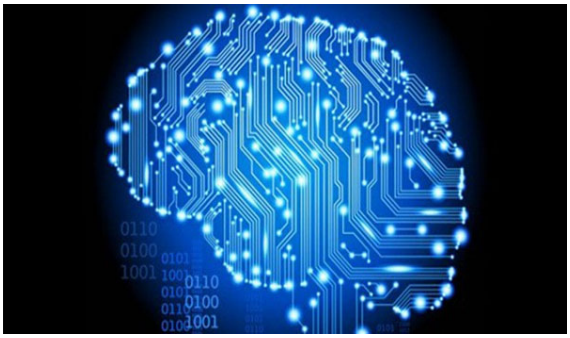
- Structura si Arhitectura Sistemelor Numerice
  - Cursurile 2, 3, 6, 7, 8, 9, 10, 11, 12
- Introduction to Parallel Computing: Design & Analysis of Algorithms
  - Cursurile 6, 8, 11
- The Sourcebook of Parallel Computing, www.top500.org, netlib.org, icl.cs.utk.edu/hpcc/
  - Cursurile 13, 14
- Computer Organization & Design – The Hardware Software Interface
  - Cursurile 2-14
- Computer Architecture: A Quantitative Approach
  - Cursurile 3, 4, 7, 9, 10, 11
- Introduction to Parallel Processing; Algorithms and Architectures
  - Cursurile 6-11
- Techniques for Optimizing Applications
  - Cursul 4
- Inside the Machine; Jon Stokes
  - Cursurile 1, 4, 5, 7, 8



Educatie Inginereasca

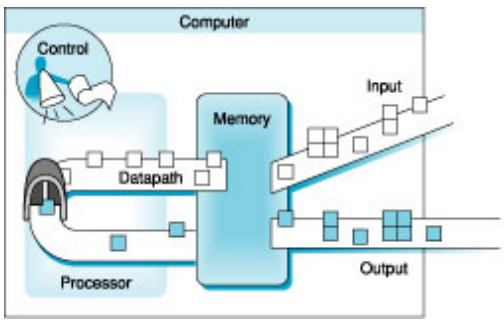
Ce este un Computer?

- Un instrument pentru a rezolva probleme
  - Prelucrarea datelor/numerelor
  - Prelucrarea algoritmilor
- Un procesor ce controleaza un sistem (avion, fabrica, monitor de inima, trafic, robot, etc.)
  - Senzori / Intrari
  - Actuator / Iesiri
  - Functii
  - Stari



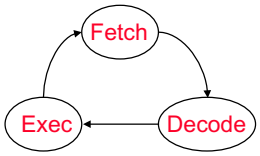
@Yale Patt - University of Texas at Austin

Ce este un Computer? (2)



- Components:
  - Processor (Datapath & Control)
  - Input (mouse, keyboard)
  - Output (display, printer)
  - Memory (cache, main memory, disks, CD/DVD)
  - Network
- Our primary focus: the Processor (**Datapath** & **Control**)
  - Implemented using billions of transistors
  - Impossible to understand by looking at each transistor
  - We need abstraction!

Processor Organisation



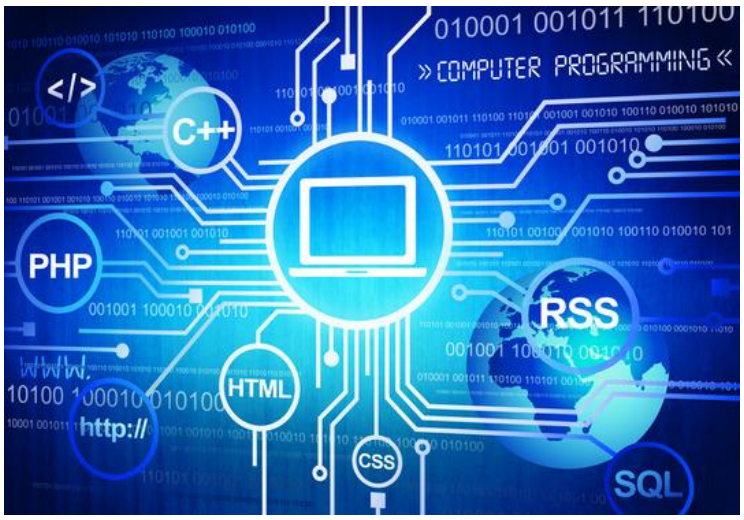
- **Control** needs to have circuitry to
  - Decide which is the next instruction and input it from memory
  - Decode the instruction
  - Issue signals that control the way information flows between datapath components
  - Control what operations the datapath's functional units perform
- **Datapath** needs to have circuitry to
  - Execute instructions - functional units (e.g. adder, multiplier, etc.) and storage locations (e.g. register file)
  - Interconnect the functional units so that the instructions can be executed as required
  - Load data from and store data to memory

Typical Design Goals for Processors

- Maximize performance (runtime, throughput, etc.)
- Minimize memory space (embedded systems)
- Minimize power consumption (mobile systems)
- Reduce design time (time-to-market)
- Minimize cost ($)

The Four Design Principles (CN)

- Simplicity favours regularity
- Smaller is faster
- Make the common case fast
- Good design demands good compromises



De ce ASC?

Why Computer Organization?

- Embarrassing if you are a BS in CSE and can't make sense of the following terms: DRAM, pipelining, cache hierarchies, I/O, virtual memory, …
- Embarrassing if you are a BS in CSE and can't decide which processor to buy wrt different metrics: 2.2 GHz Xeon or 3.6 GHz A8
  - E.g. Performance vs. Power
- Obvious first step for chip designers, compiler & OS writers
- **Will knowledge of the hardware help you write better programs?**

Must a Programmer **care** about Hardware?

- Must know how to reason about program performance and energy
- Memory management: if we understand how/where data is placed, we can help ensure that **relevant data** is **nearby**
- Thread management: if we understand how threads **interact**, we can write **smarter** multi-threaded programs
- Why do we care about multi-threaded programs?
  - **Because most HW is now multi/many-core**

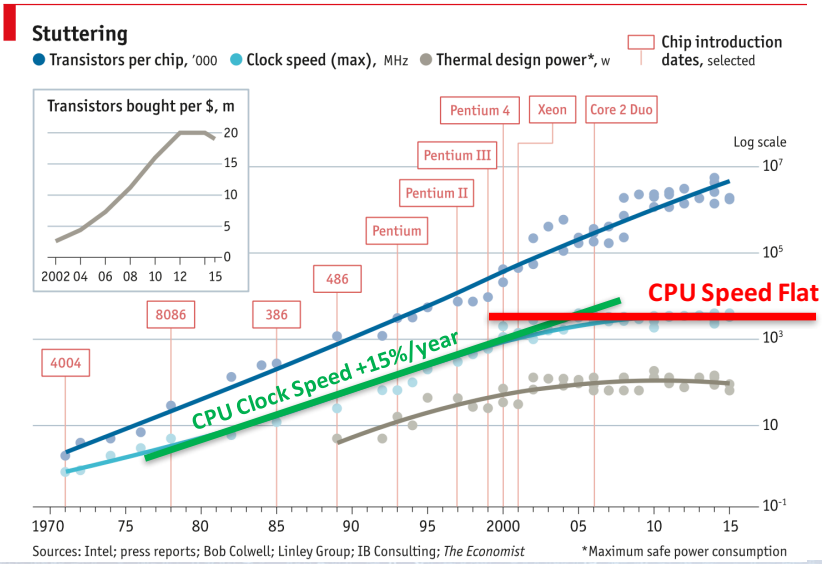
Ce vom discuta la ASC?

- Discussions about the hardware-software interface
  - What do you need to know to achieve best possible performance
- Introduction to the design of major components of a computer system, how they function together in an executing program
- But what do we mean by a computer?
  - Different types: mobile, embedded, laptop, desktop, server,…
  - Different uses: AI, auto, graphics, finance, genomics,…
- Best way to learn:
  - Focus on a specific instance and learn how it works
  - While learning general principles and historical perspectives

De ce sa discutam de HW/SW?

- You want to call yourself a “computer scientist/engineer”
- You want to build HW/SW people use
  - So you need to deliver **performance** at **low cost**
- You need to make a purchasing decision or offer “expert” advice
- Both hardware and software affect performance
  - The **algorithm** determines number of **source-level statements**
  - The **language/compiler/architecture** determines the number of machine **instructions**
  - The **processor/memory** determines **how fast** machine-level instructions are **executed**

Why is Architecture Exciting Today?



ACM Communications, February 2019

Important Trends

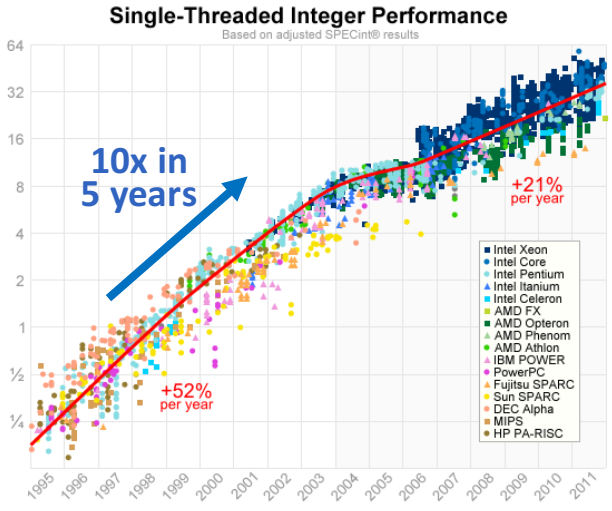
- Running out of ideas to improve single thread performance
- Power wall makes it harder to add complex features
- Power wall makes it harder to increase frequency
- Historical contributions to performance:
  - Better processes (faster devices) ~**20%** (eventually disappearing)
  - Better circuits/pipelines ~**15%** (trending lower)
  - Better organization/architecture ~**15%**

Practical Example

- +**300x** speedup for matrix vector multiplication
  - Data level parallelism: **3.8x**
  - Loop unrolling and out-of-order execution: **2.3x**
  - Cache blocking: **2.5x**
  - Thread level parallelism: **14x**
- Further, one can use accelerators to get an additional **100x**

Predicting the Future

- Predictable performance improvements
- Predictable market
  - Everybody buys a new CPU every X years
- Predictable expenses
  - Cost of new integrated circuit fab
- CEOs and Wall Street love this



Opportunities

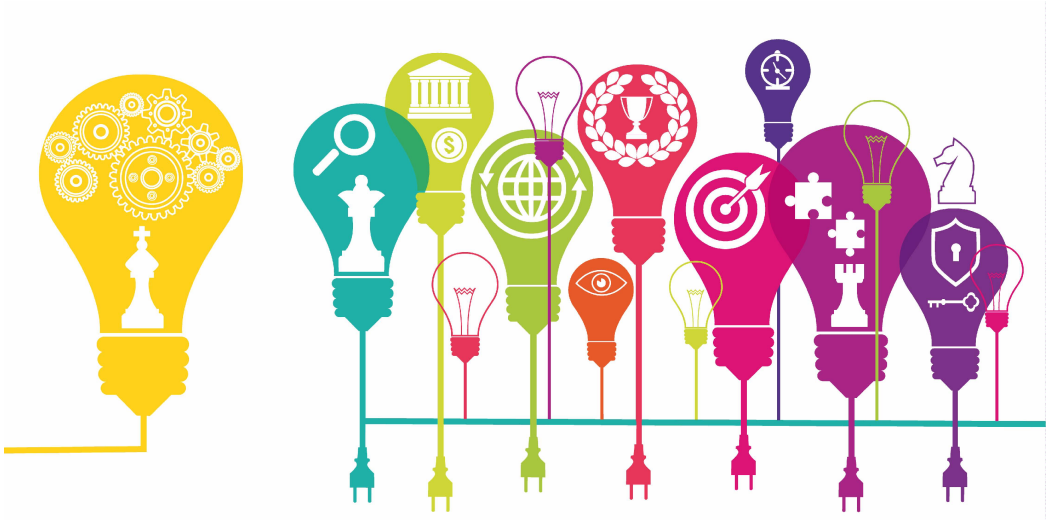
- Computer architecture today
  - General purpose: optimized for speed
  - Few players: AMD, ARM, Intel, NVidia…
- Future directions?
  - Applications have different requirements
    - Battery life / energy
    - Security & privacy
- Best solutions depend on application
  - Many players: HW / SW
  - **Innovation matters more than Moore's Law**

What Does This Mean to a Programmer?

- Today, one can expect **at most a 20% annual improvement** / even lower if the program is not multi-threaded
  - A program needs **many threads**
  - The threads need **efficient synchronization** and **communication** (Tema 1)
  - **Accelerators** should be used whenever possible (Tema 2)
  - **Data placement** in the **memory hierarchy** is important (Tema 3)

Challenges for Hardware Designers

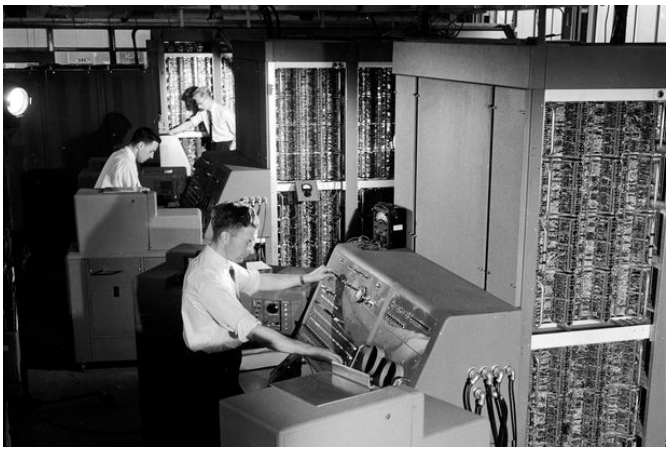
- Find efficient ways to
  - **Improve data sharing & synch techniques** (Tema 1)
  - **Boost programmer productivity** (Temele 1-3)
  - **Manage the memory system** (Temele 2-3)
  - **Build accelerators for important kernels** (Tema 2)
  - **Boost single-thread performance** (Tema 3)
  - Reduce system energy per instruction

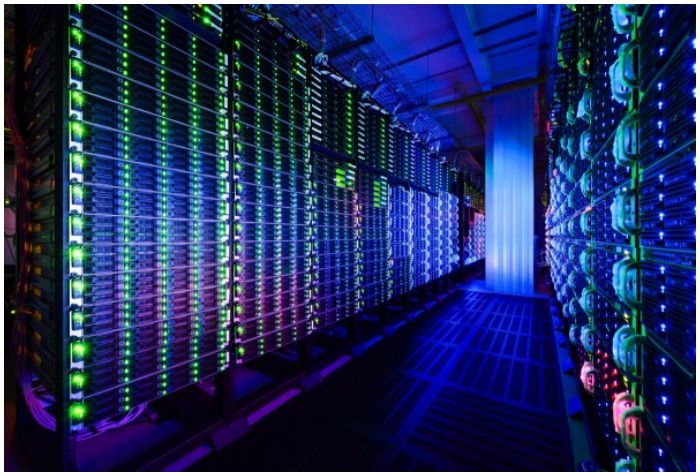


Idei Importante in Arhitectura Sistemelor de Calcul

Sisteme de Calcul

Atunci…





Sisteme de Calcul

Acum.

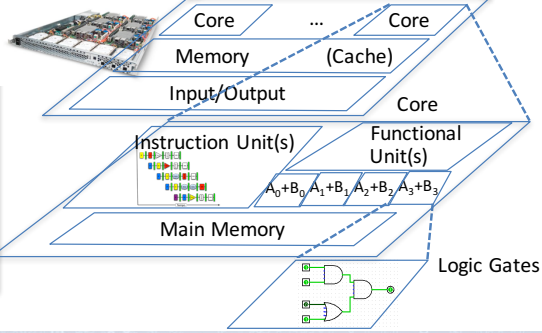
Arhitectura Sistemelor de Calcul

Software

- Parallel Requests
  Assigned to computer
  e.g., Search "cats"
- Parallel Threads
  Assigned to core
  e.g., Lookup, Ads
- Parallel Instructions
  >1 instruction @ one time
  e.g., 5 pipelined instructions
- Parallel Data
  >1 data item @ one time
  e.g., Add of 4 pairs of words
- Hardware descriptions
  All gates functioning in parallel at same time

Harness Parallelism & Achieve High Performance

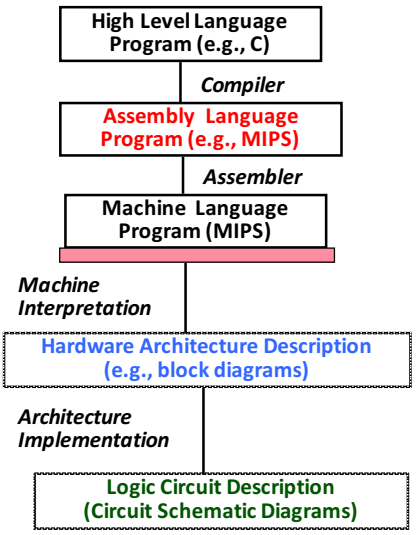
Hardware



Cinci Idei Importante in Arhitectura Sistemelor de Calcul

1. Abstraction - Layers of Representation / Interpretation
2. Moore's Law - Designing through trends
3. Principle of Locality - Memory Hierarchy
4. Parallelism
5. Dependability via Redundancy

Ideea 1: Abstractizarea

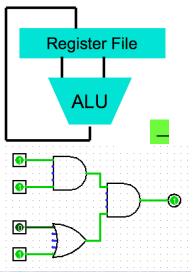



```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

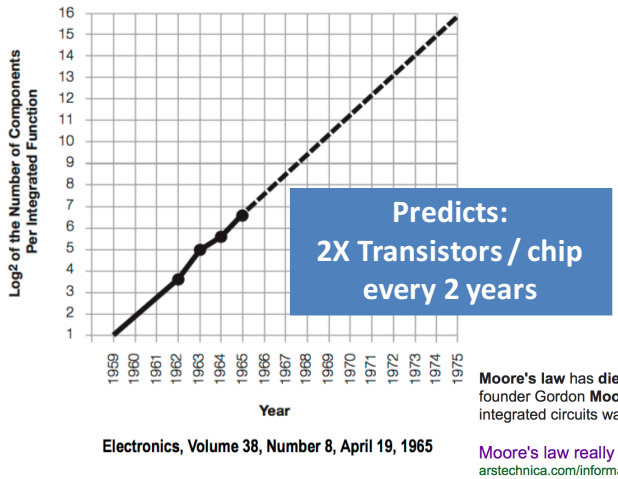
```
lw   $t0, 0($2)
lw   $t1, 4($2)
sw   $t1, 0($2)
sw   $t0, 4($2)
```

Anything can be represented as a *number*, i.e., data or instructions

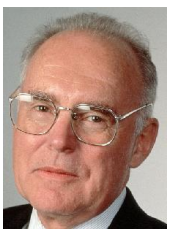
```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 [illegible]  1
0101 1000 0000 1001 1100 [illegible]  1
```



Ideea 2: Legea lui Moore



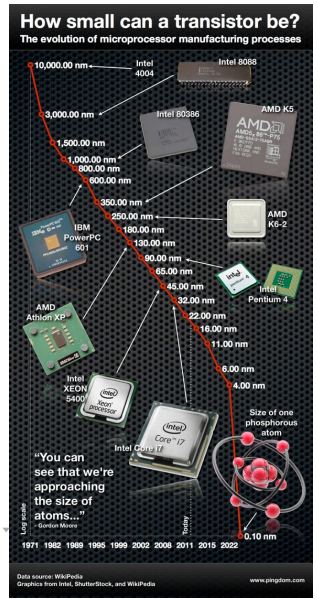
Electronics, Volume 38, Number 8, April 19, 1965



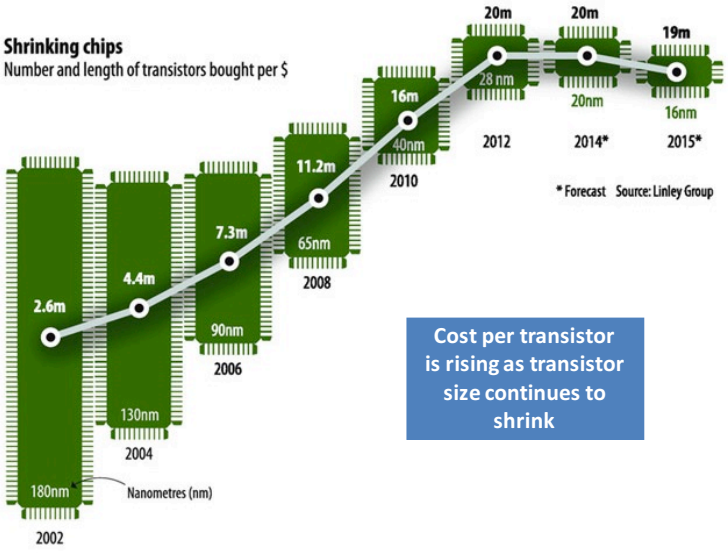
Gordon Moore
Intel Cofounder
B.S. Cal 1950!

Moore's law has **died** at the age of 51 after an extended illness. In 1965, Intel co-founder Gordon **Moore** made an observation that the number of components in integrated circuits was doubling every 12 months or so. Feb 11, 2016

Moore's law really is dead this time | Ars Technica
arstechnica.com/information.../2016/.../**moores-law**-really-is-dead-this-time... Ars Technica



Numarul de tranzistori si latenta

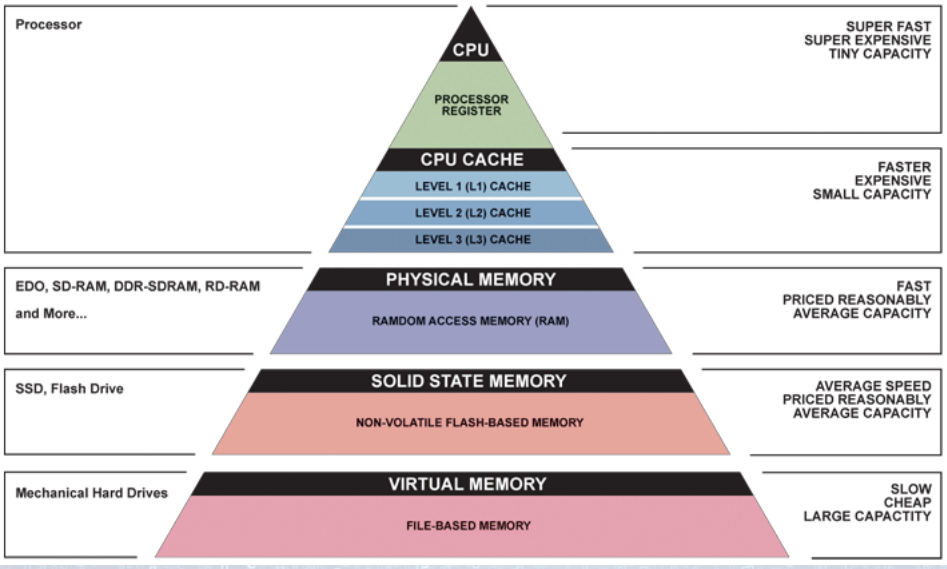


Cost per transistor is rising as transistor size continues to shrink

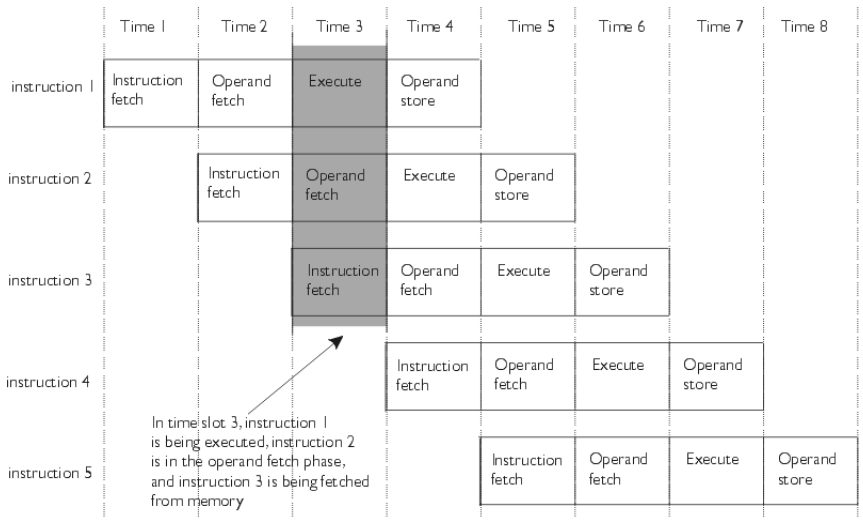
Jim Gray
Turing Award
B.S. Cal 1966
Ph.D. Cal 1969!

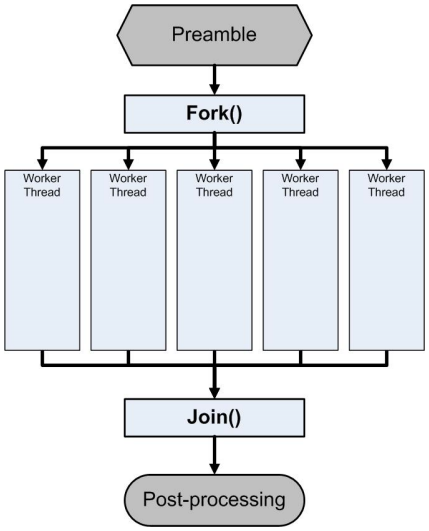
10^9	Banda Robot Optic	Andromeda 2,000 Ani
10^6	Disk	Pluto 2 Ani
100	Memorie Principala	Brasov 1.5 h
10	On Board Cache	Politehnica 10 min
2	On Chip Cache	Sala de curs
1	Registri	Capul meu 1 min
(ns)		

Ideea 3: Localitate - Ierarhia de Memorii



Ideea 4: Paralelism





Ideea 5: Fiabilitate prin Redundanta

- Applies to everything from datacenters to storage to memory to instructors
  - Redundant **datacenters**: can lose 1 datacenter but Internet service stays online
  - Redundant **disks**: can lose 1 disk but not lose data (Redundant Arrays of Independent Disks - RAID)
  - Redundant **memory** bits: can loose 1-2 bits but no data is lost (Error Correcting Code - ECC Memory)

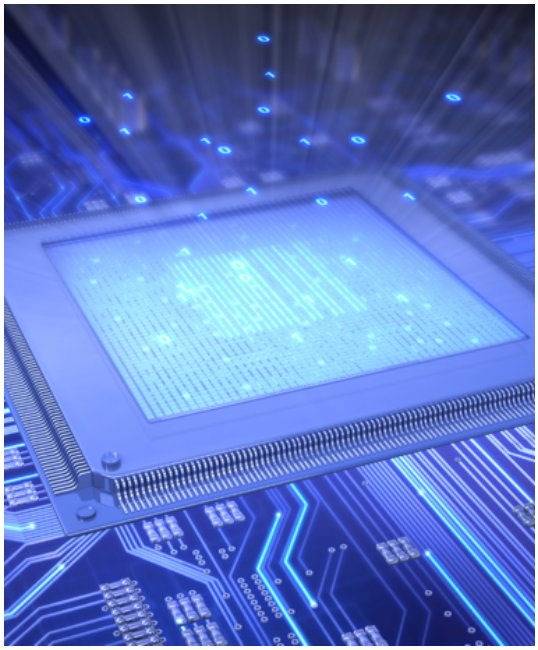
Ce am aflat in aces curs?

- Despre notarea de la ASC
- Sumarul cursului de ASC
- Bibliografia cursului de ASC
- Despre educatia Inginereasca
- De ce Arhitectura Sistemelor de Calcul?
- Idei Importante in Arhitectura Sistemelor de Calcul

In cursul urmator

- Componentele Sistemelor de Calcul
- Nivelul Structural de Descriere al Sistemelor Numerice – PMS (Processor Memory Switches)
- Exemple de descrieri PMS
- Structuri multiprocesor organizate pe o magistrala comuna (SBC)

Slides & Support Material Acknowledgements



Yale Patt - University of Texas at Austin 2007
CSE 331 - Penn State University 2007
CS61C - University of Berkeley 2016
CS/ECE 3810 - University of Colorado 2017
Ryan J. Leng

Thank you for your attention

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 2
cs.pub.ro
cs.curs.pub.ro

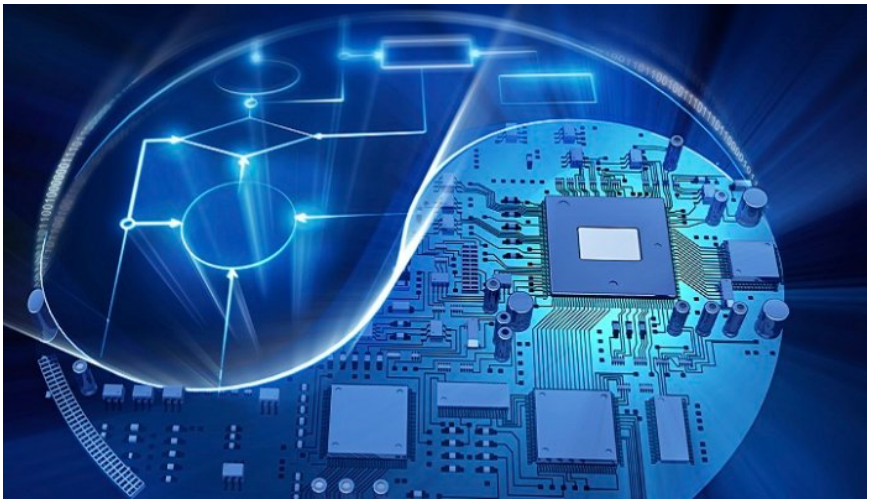
University Politehnica of Bucharest

In cursul anterior

- Notare
- Sumarul Cursului
- Bibliografie
- Educatia Inginereasca
- De ce Arhitectura Sistemelor de Calcul?
- Idei Importante in Arhitectura Sistemelor de Calcul

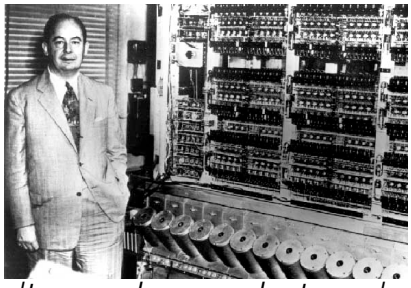
Cuprins

- Componentele Sistemelor de Calcul
- Nivelul Structural de Descriere al Sistemelor Numerice – PMS (Processor Memory Switches)
- Exemple de descrieri PMS
- Structuri multiprocesor organizate pe o magistrala comuna (SBC)



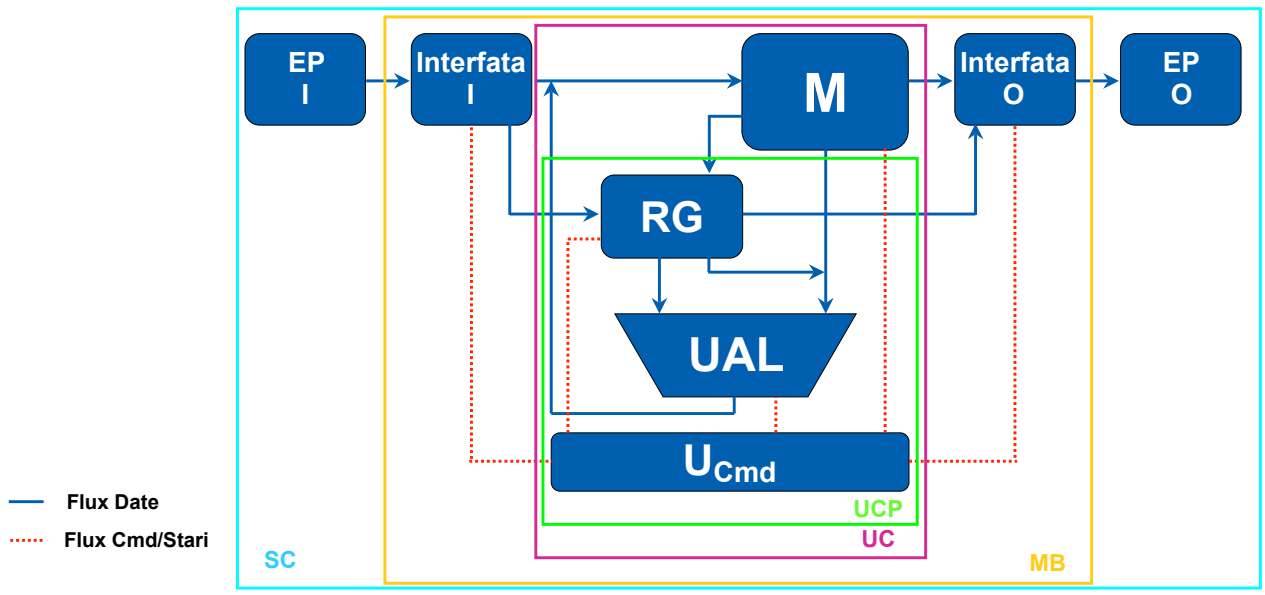
Componentele Sistemelor de Calcul

Masina von Neumann



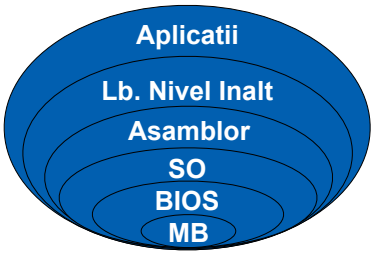
- John von Neumann (1903 – 1957)
- Structura cu acces secvential
  - CPU
  - Memorie (Instructiuni & Date)
  - Ex: EDVAC (1945) – binar
- *“There's no sense in being precise when you don't even know what you're talking about.”*
- *“In mathematics you don't understand things. You just get used to them.”*
- *“It would appear that we have reached the limits of what it is possible to achieve with computer technology, although one should be careful with such statements, as they tend to sound pretty silly in 5 years.” (Said in 1949)*

Arhitectura masinii von Neumann

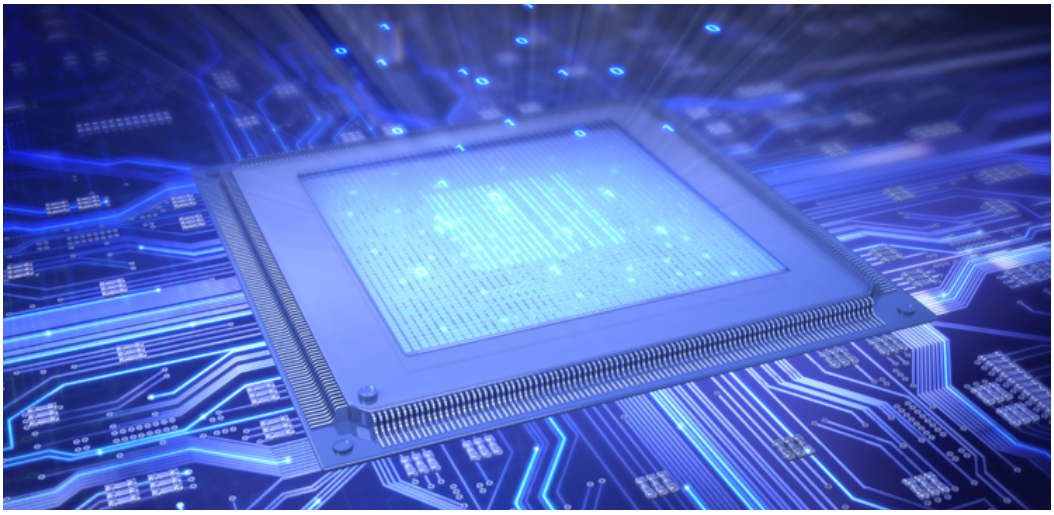


Componentele Arhitecturii

- UCP = {RG, UAL, U_{Cmd}} = Unitatea Centrala de Prelucrare
- UC = {UCP, M} = Unitatea Centrala
- MB = {UC, I/E} = Masina de Baza
- SC = {MB, EP, Software de Baza} = Sistem de Calcul



Software de Baza



Directive PMS

Nivelul Structural de Descriere al Sistemelor Numerice

- PMS (Processor Memory Switches)
- La nivel structural sistemele numerice (SN) sunt specificate si analizate considerand urmatoarele elementele primare:
  - PC = procesorul central
  - M = memoria
  - S = comutatorul
  - L = linia de legatura
  - K = unitatea de comanda
  - PI/E = procesorul de intrare/iesire sau interfata
  - D = operatorul de date
  - T = terminalul

Primitivele PMS

- Fiecare primitiva
  - Este caracterizata printr-un atribut:
    - a_i = atributul asociat primitivei
- Fiecare atribut
  - Are o anumita valoare:
    - v_i = valoarea corespunzatoare atributului
  - Astfel un sistem de calcul devine:
    - SN (a_1:v_1; a_2:v_2; … a_n:v_n)

Memoria – M

- Rolul memoriei:
  - Pastreaza informatia
  - Actualizeaza informatia prin operatii de citire/scriere
  - Necesita un sistem de adresare cu o corespondenta liniara intre adresa si continut
- Atribute:
  - Functie: primara | auxiliara
  - Tehnologie: bipolara | MOS | statica | dinamica
  - Operatii: citire | citire/scriere
  - Mod acces: aleator | secvential | FIFO | LIFO | asociativ
  - Lungime cuvant: 8+1 | 16+2 | 32+4 | 64+8
  - Capacitatea: 4Mb | 8Mb | 16Mb | …| 1Gb | 2Gb | 4Gb | 8Gb
  - Ciclu de lucru: 2 | 4ms | … | 400ns | 200ns | 100ns

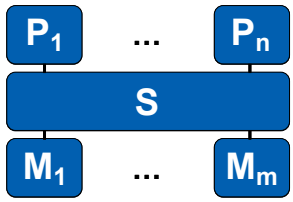
Procesorul Central – P_C

- Rolul PC:
  - Citeste, interpreteaza si executa instructiuni masina
  - In acest proces are loc generarea comenzilor spre toate resursele sistemului + citirea starilor acestora
- Atribute:
  - Functie: universal | specializat (de semnale, etc)
  - Implementarea: conventionala | μprogramata | μprogramata specializata
  - Formatul instructiunilor: fix | variabil
  - Lungimea instructiunilor: 8 | 16 | 32 | 64 | 128 biti
  - Ciclu instructiune: ciclu fix | ciclu varibil (S cicli masina)
  - Tehnologia: LSI | VLSI | …

Switch/Comutatorul – S

- Rolul S:
  - Asigura conexiuni intre componentele sistemului
  - Poate evolua
    - De la structuri foarte simple = buffer 3 state de acces pe magistrala
    - La structuri complexe cu unitati de comanda proprii = comutatoare cu discipline de servire a cererilor de stabilire a legaturilor
- Atribute:
  - Structura: ierarhica | neierarhica
  - Tip: simplex | semiduplex | duplex (e vorba de sensuri)
  - Numar de legaturi realizate:
    - m intrari | n iesiri din S
    - n intrari | m iesiri din S
  - Concurenta: c

Maxim de paralelism: min (n, m)



Legatura – L

- Rolul L:
  - Asigura legatura fizica intre diverse componente ale sistemului
  - Nu prelucreaza informatia ci doar asigura traseul transferului spatial al datelor
  - E fie o magistrala, fie o interfara seriala/paralela
- Atribute:
  - Functia: legatura seriala | legatura paralela | legatura de tip magistrala
  - Lungimea cuvantului: 5 | 6 | 7 | 8 | 16 | 32 | 64 + comanda
  - Mod de dialog: sincron | asincron | cu Q&A: cu/fara interblocare sau cu interblocare completa | fara A
  - Mod de control al accesului: inlantuire seriala (token) | interogare | cereri independente
  - Standard: paralel (SCSI) | serial (RS232; 485; 482) | magistrala (MultiBus; MCI(IBM); HPIBus; etc.)

Unitatea de Comanda – K

- Rolul K:
  - Componenta care exprima controlul in diverse subansamble ale sistemului
  - Are o functie de comanda a resurselor asociate unitatilor functionale
    - Nu citeste & interpreteaza instructiuni aritmetice logice, ci doar **comanda** & **control**
- Atribute:
  - Functia: comanda resursa I
  - Implementare: conventionala | μprogramata
  - Numar de stari: n
    - Daca are 2 stari este un bistabil
    - Daca are 64 de stari este un automat complex

Unitatea de Comanda – K

- Citireste, interpreteaza si executa instructiuni masina de **comanda si control**
- Legata de:
  - Setul de instructiuni
    - Intructiuni Universale; Ciclu variabil
    - RISC cu ciclu fix
    - Complex CISC – emularea unei masini intermediare
- Codificarea instructiunilor
  - 0 adrese

  - 1 adresa

  - 2 adrese

  - 3 adrese

- Moduri de adresare multiple la ora actuala

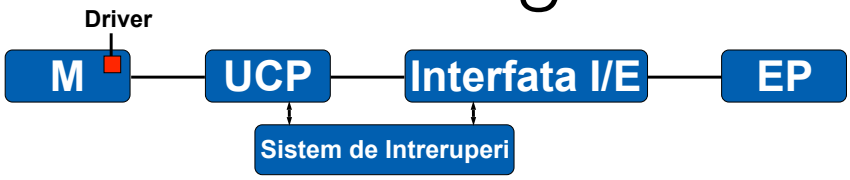
Procesorul de I/E – $P_{I/E}$

- Rolul $P_{I/E}$:
  - Poate fi incorporat in PC-ul in care are functie de I/E sau poate fi privit independent
  - Daca este o primitiva independenta are urmatoarele atribute
- Atribute:
  - Functie: DMA | canal I/E | procesor specializat I/E
  - Tip implementare: conventional | μprogramat | (co)procesor specializat I/E
  - Rata de transfer: Mb/s | Gb/s | Tb/s
  - Mod de transfer: cum rezolva conflictul de acces la UC
    - Prin furt de ciclu
    - Prin rafala
  - Lungimea cuvantului: 8 | 16 biti (in general caractere)

Subsistemul de I/E

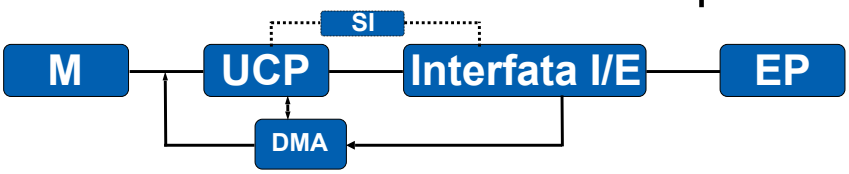
- Asigura transferul datelor intre UC si mediul extern
- Interfetele I/E asigura
  - O adaptare electrica intre EP si UCP
  - Sincronizarea temporala intre EP si UCP printr-un cuvant de stare
    - Diferenta este de cateva ordine de marime
  - Transfer de date intre EP si UC:
    - Transfer programat
    - Acces direct la memorie
    - Canal de intrare/iesire
    - Calculator de intrare/iesire (Front End Computer/Processor - FEP)

Transfer Programat



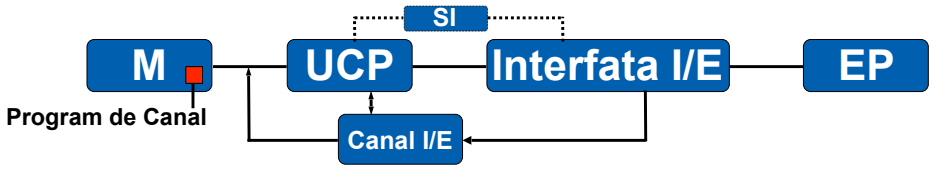
- UCP-ul se ocupa de fiecare caracter in parte
- Driver-ul se ocupa de transferul datelor I/O
- Sincronizarea se poate face
  - Prin citirea ciclica a starii (busy waiting)
    - UCP asteapta dupa EP si ajunge sa lucreze la viteza EP-ului
    - Driver-ul citeste si intrerupe cuvintele de date/stare/comanda
  - Prin intreruperi
    - Se dau ordine EP si se continua prelucrarile
    - Cand se termina se genereaza o intrerupere
    - UCP-ul nu se adapteaza la viteza EP-ului

Acces Direct la Memorie prin DMA



- Asigura transferul de date intre EP si M fara interventia UCP
- Dialogul intre UCP si DMA are loc doar la initiere & incheiere
- Dialogul se face prin transfer programat (Intreruperi)
- Initiere: UCP → DMA: adrese date, sensul (R/W), adresa EP
- La orice eroare DMA cheama UCP-ul
- UCP-ul nu poate initia un alt transfer pana nu s-a incheiat cel curent
- Arbitrarea conflictului de acces la M al UCP/DMA se face prin
  - Furt de ciclu
  - Rafala

Canal de I/E



- Canalul I/E e un procesor specializat ce asigura transferul intre EP si M fara interventia UCP
- Dialogul UCP – EP se face printr-un program de canal (din M – se da doar adresa de inceput a programului)
- UCP poate inlantui mai multe programe de canal = mai multe transferuri de blocuri de date (mai bine ca la DMA)
- Canalul aduce informatia necesara transferului in registrele proprii prin DMA (acces direct)
- Canalul reincearca transferul in caz de eroare

Calculator de I/E

- In cazul calculatorului de I/E, SC poate avea toate facilitatile de mai sus:
  - DMA
  - Canal I/E
  - Transfer programat
- Mecanism util pentru a nu irosi timpul unui procesor / calculator puternic
- Ofera datele rapid si eficient pentru prelucrare
- Este folosit des in sistemele multiprocesor

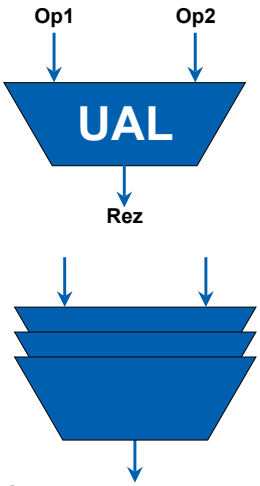
Operatorul de Date – D

- Rolul D:
  - “Produce” unitati de informatie cu semnificatie noua
  - Efectueaza operatii aritmetice & logice + prelucrari primare de compactare, expandare si asociere asupra datelor
  - Este unitate de prelucrare: in virgula mobila si zecimala
  - Sunt module optionale, de sine statatoare, independente de PC
- Atribute:
  - Functie: prelucrare in virgula mobila | zecimala(BCD) | vectoriala | matriceala | cu liste
  - Tip implementare: conventionala | procesor specializat (coprocesor matematic)
  - Structuri de date asupra carora opereaza: scalari | vectori | matrice | liste
  - Operatii: + | - | * | / | cautare atomica | memorare/extragere atomica

Evolutia Operatorului de Date D/UAL

- UAL: in virgula fixa si virgula mobila
- Formatul general al instructiunilor

- Evolutia spre SIMD (masini vectoriale)
  - UAL multiple
  - Fiecare element are o prelucrare dedicata
  - Rezultatele sunt predate etajului adiacent
- Op UAL: +, -, *, /, ^, %, Shiftari, etc
- In timp UAL-ul s-a extins catre coprocesorul matematic



Terminalul – T

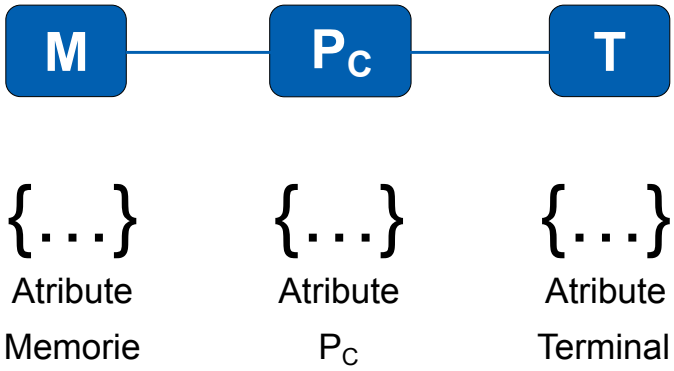
- Rolul T:
  - Asigura conversia din punct de vedere fizic a datelor
    - Adaptare electrica
    - Asigura sincronizarea intre terminal si UC de prelucrare sau PI/E care se ocupa de acel terminal
  - Terminalul este format fie
    - Dintr-o singura componenta
    - Din 3 componente
      - Terminal
      - Unitate de comanda a T: interpreteaza comenzi/stari de la UC
      - Switch T

- Atribute:
  - Functie: cupleaza terminalul i
  - Tip cuplare: seriala | paralela
  - Caracteristici: viteza de transfer | capacitate | pagini/min

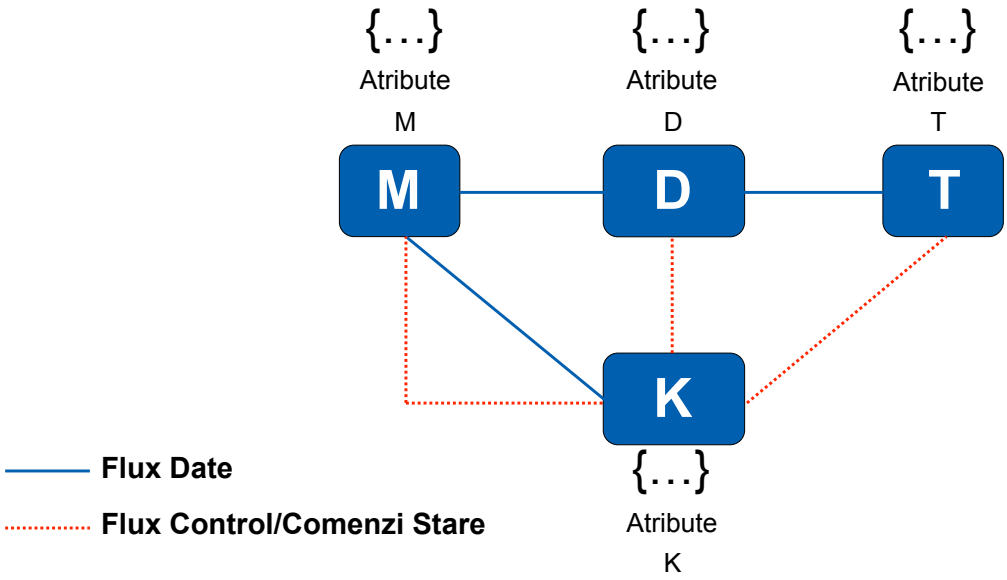
Exemple de Descrieri PMS

Exemple de Folosire ale Primitivelor PMS

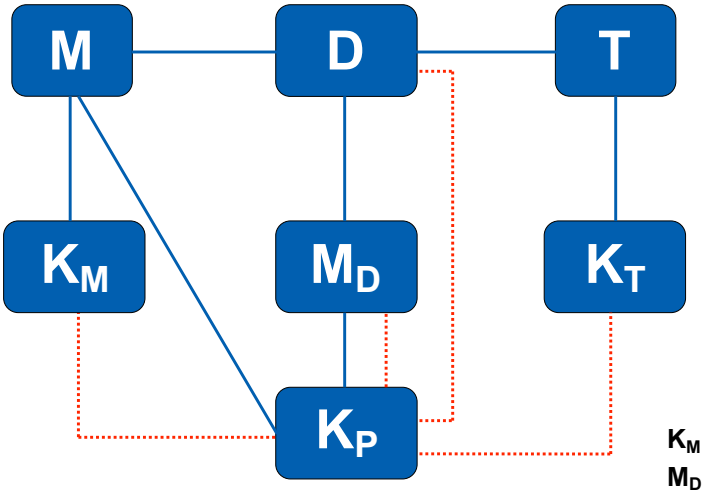
- Sa incercam o structura von Neumann



Detaliem P_C



Detaliem M & T



Flux Date

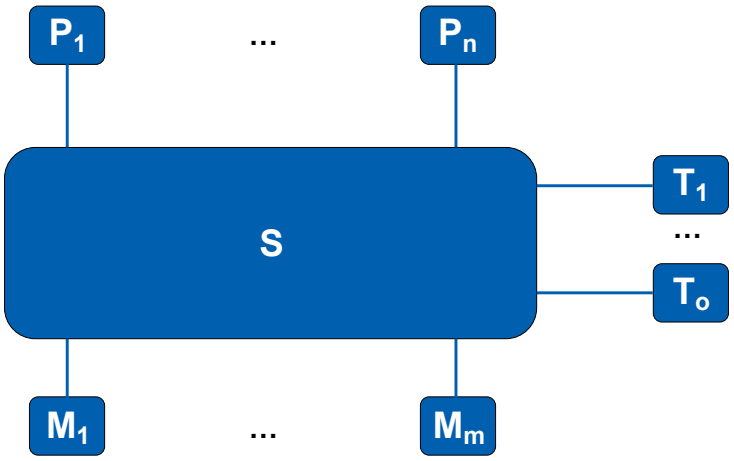
Flux Control/Comenzi Stare

K_M – Unitate de comanda a memoriei

M_D – set de registrii generali

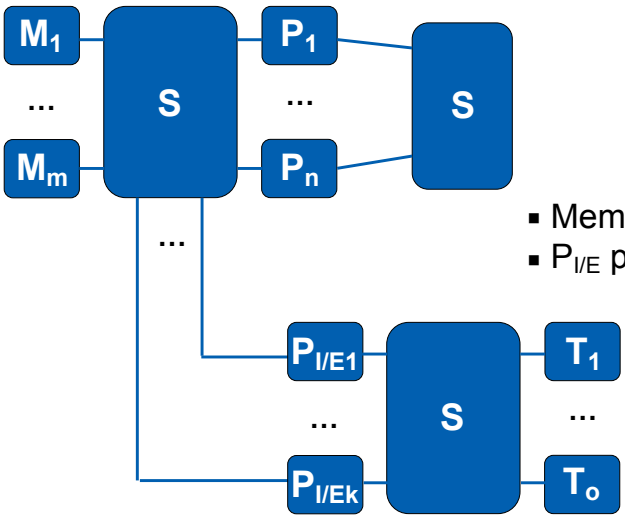
K_P – procesorul ce interpreteaza si executa

Sistem cu mai multe Procesoare si Memorii

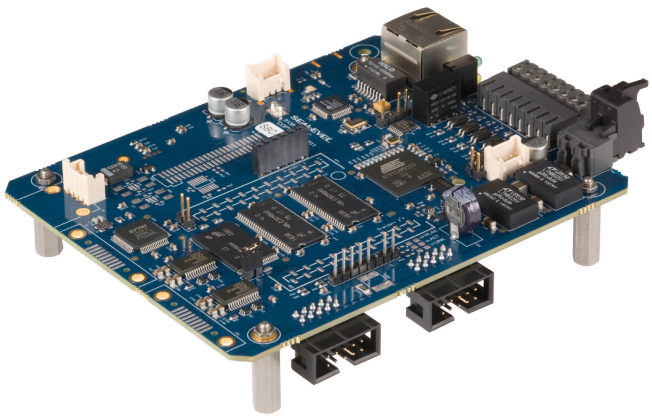


S este cel mai important element pentru ca asigura conexiunile intre $P_i \leftrightarrow M_j$ sau $P_k \leftrightarrow T_l$

Sisteme Multiprocesor



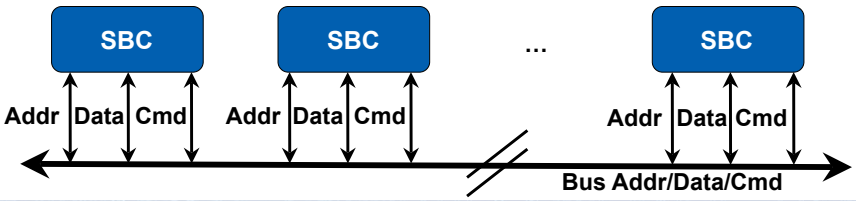
- Memoriile sunt partajate de procesoare
- $P_{I/E}$ pot conecta orice T la orice P



Single Board Computers (SBC)

Single Board Computer

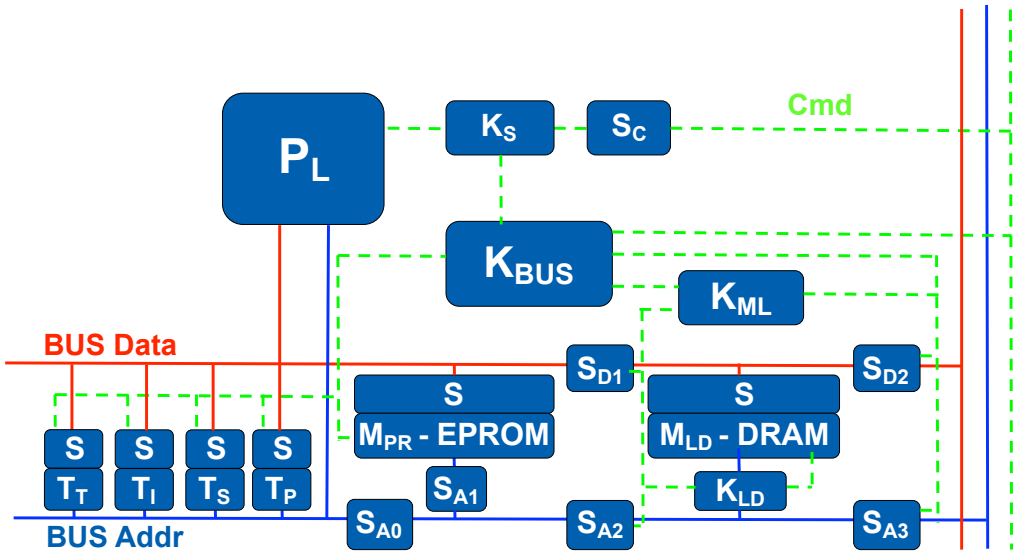
- Are procesor, memorie + interfete I/O
- Conecteaza pe o magistrala comenzi, date si adrese bidirectional
- Resurse SBC:
  - Private: procesor, EPROM, Interfete (S/P), RT Clock, Sistem de Intreruperi
  - Locale: Memoria locala
  - Globale: RAM (in spatiul de adresare)



SBC – Mecanisme Acces Magistrala

- Pentru a gestiona accesul la magistrala (exclusiv) sunt necesare:
  - K_{BUS}: unitate de comanda pt accesul la MAG
  - K_{ML}: unitate de control al accesului la memoria locala
  - K_{LD}: unitate de comanda a memoriei locale (refresh/etc)
  - K_s: unitate de comanda a intregului sistem – asigura controlul procesoarelor la resursele sistemului
- Fiecare procesor al SBC-ului vede memoria locala incapand de la 0 si memoriile celorlalte SBC-uri in spatii adiacente superioare

Structura PMS a SBC



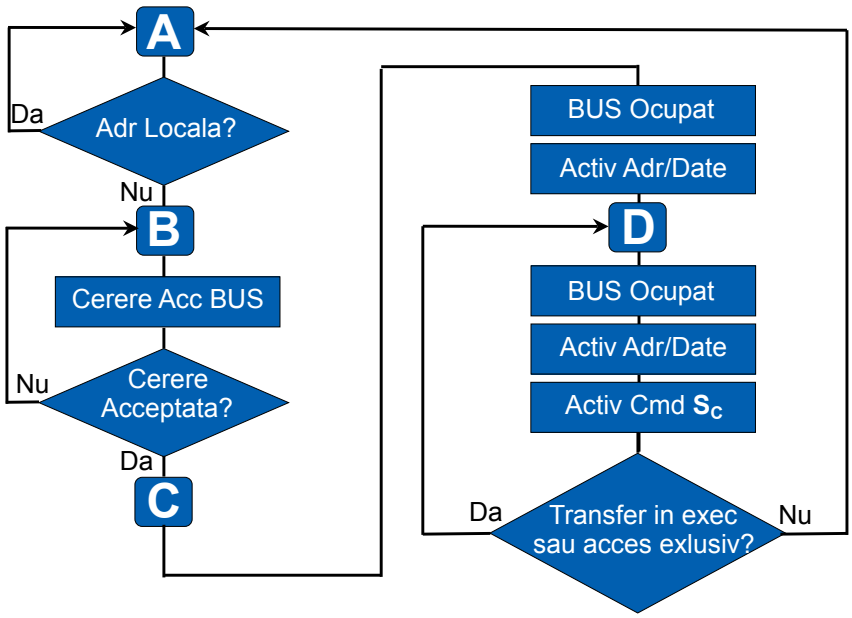
Elementele Structurii PMS a SBC

- P_L: procesor local – furnizeaza date si adrese
- Interfete:
  - T_T – de ceas de timp real
  - T_I – sistemului de intreruperi
  - T_S – seriala & T_P – paralela
  - Fiecare interfata are adresele portului sau de I/O
  - Daca isi recunoaste adresa isi pune datele pe magistrala
- M_{LD} – DRAM: memoria poate fi accesata de P_L si de procesoare din exterior
  - Exista niste S-uri care acorda accesul fie procesorului local fie celor externe
- S_{D1} si S_{D2} S-uri locale de date – separa procesoarele intre ele
- S_{A0} si S_{A1} – S-uri interne de adrese
- S_{A2} si S_{A3} – S-uri externe de adrese
- S_C – S prin care se transmit comenzi catre exterior
- Obs: structura PMS poate fi detaliata pe componente cu directive PMS

SBC – Mecanisme Acces Magistrala

- K_s e specifica fiecarui procesor
  - In procesoarele moderne e inclusa in chip
  - K_s genereaza semnalele de control ale resurselor locale
  - Generarea se face prin interpretarea semnalelor de stare date de microprocesor
- Trebuie sa existe o unitate de comanda K_{BUS}:
  - Fiecare procesor trebuie sa intre intr-o stare de asteptare pana primeste dreptul de acces la magistrala
  - Procesorul face doar un ciclu care apoi se poate prelungi prin stari de T_{WAIT}
- K_{ML} controleaza S-urile de adrese si de date
  - Accesul local sau extern la memorie
  - Daca deschide toate S-urile se ajunge la accesul direct al procesorului local in exterior

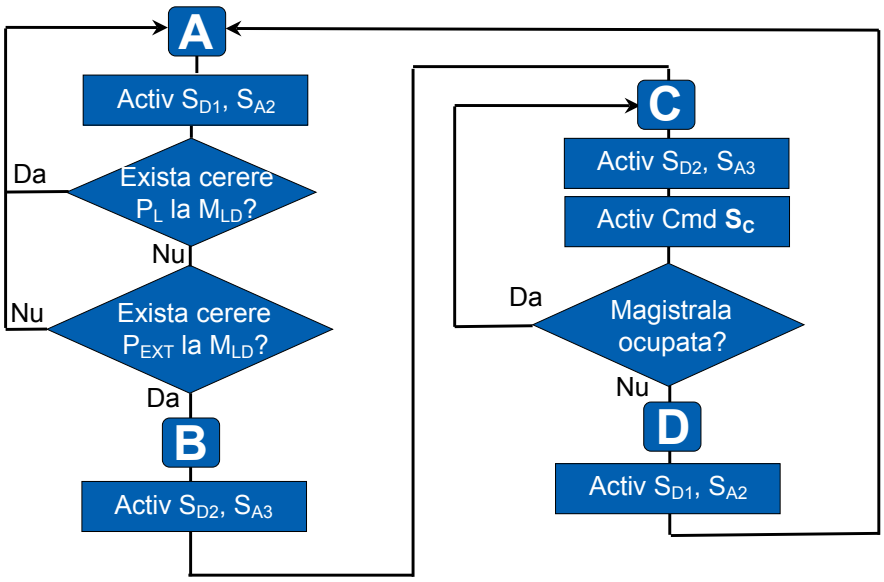
K_{BUS}



Descriere K_{BUS}

- K_{BUS} – reprezinta procesorul local in relatie cu magistrala externa
- Starea A: adresa e locala?
  - Da = se adreseaza resurse locale, nu trebuie iesit pe magistrala
  - Nu = adr nu e locala → se trece in starea B
- Starea B: cerere de acces la magistrala; daca cererea e acceptata se trece in starea C
- Starea C: se emite Bus Busy (mag ocupata) – doar in C & D; accesul pe BUS e exclusiv & cu rafala (overwrite) raman in D pt un nou ciclu daca mai e cazul; automatul preia functia procesorului; Dupa preluarea comenzii, procesorul face functia de secventiere a Adr, Date, Cmd
- Intrebare: de ce si starea C si starea D? Din cauza ciclului instructiune curent. Trebuie sincronizate comenzile cu decodificarea adreselor & stabilirea datelor pe magistrala (e un Δ_t acolo = T_{Wait} datorat interpretarii adreselor) → se introduce starea D pentru a compensa acest timp de latenta: starea de overwrite
- Obs: K_{BUS} trebuie sa dispuna de unitati logice combinationale ce identifica:
  - Daca adresa mem/port I/O implicate in transfer sunt locale
  - Semnalul de cerere acceptata e combinat din mai multe conditii: mag libera & modul SBC activ

K_{ML}



Descriere K_{ML}

- K_{ML} – asigura accesul la Mem locala a PL prin intermediul S_{D1} si S_{A2} & a proc externe prin S_{D2} si S_{A3}
- PL are prioritate fata de proc externe (P_{EXT}) == daca vrea P_L la M_{LD} NU ma intereseaza ce vor celelalte P_{EXT}
- Starea A: P_L poate avea comenzi locale
- Starea B: activez accesul ext la M_{LD} – semnalele asociate starii
- Obs: Mem locala e vazuta de fiecare procesor acelasi spatiu de memorie, iar de procesoarele ext in spatiul complementar → trebuie sa existe o unitate de translatare a adreselor
- Mag ocupata: nu stii ciclii masina ai unui proc extern. Numai cand magistrala e libera un P_{EXT} a terminat ciclul masina curent → abordare top-down

Ce am invatat in aces curs?

- Care sunt componentele sistemelor de calcul
- Processor Memory Switches - PMS
- Exemple de descrieri PMS
- Single Board Computers - Unitati de command

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 6
cs.pub.ro
cs.curs.pub.ro

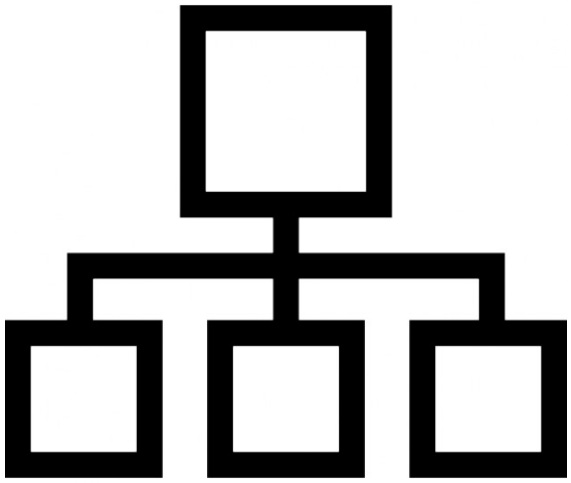
University Politehnica of Bucharest

In cursul anterior

- Structura si Setul de Instructiuni SIMD
- Implementarea Salturilor Conditionate in Structuri SIMD
- Organizarea Datelor in Structuri SIMD
- Probleme de Comunicatii intr-un Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

Cuprins

- Comutatoare Ierarhice
- Comutatoare Neierarhice
- Exemple de Comutatoare Procesor Memorie
- Interconectarea in Sistemele cu Resurse Multiple
- Retele de Comutare Ierarhice
- Retele de Comutare de tip Delta
- Retele Bazate pe Rutare
- Performantele Retelelor de Comutare



Comutatoare Ierarhice

Comutatoare

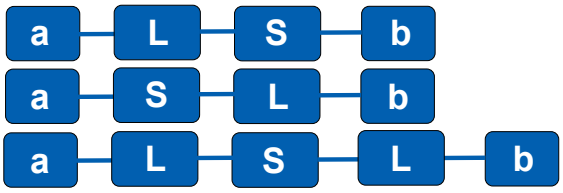
- Se impart in doua mari categorii
  - Ierarhice = conecteaza o componenta de tip a cu una de tip b
    - $a_i \leftrightarrow b_k$ (P ↔ M)

  - Neierarhice = conecteaza doua componenta de acelasi tip a
    - $a_i \leftrightarrow a_k$ (P ↔ P)

Comutatoare Ierarhice

- Comutatorul Simplu - Poarta
  - Asigura comunicarea intre componente de tip a si b
  - Este descris ca : S {poarta, 1a, 1b}

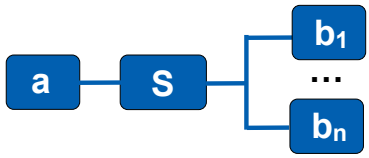
  - Variante posibile:
    - S {poarta, 1a, 1b, comutare la b}
    - S {poarta, 1a, 1b, comutare la a}
    - S {poarta, 1a, 1b, cu legatura la a si b}



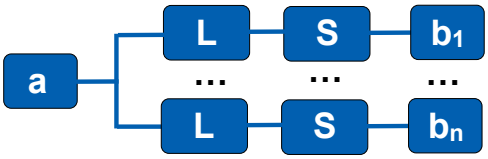
  - Poate conecta o resursa la magistrala sau o magistrala la alta magistrala

Comutatoare Ierarhice Duplex

- Asigura comunicatia intre o componenta de tip a si mai multe componente de tip b
  - Este descris ca: S {poarta, 1a, nb, concurenta 1, 1Sp}

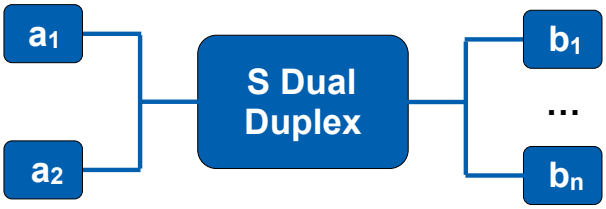


  - Se pot considera toate variantele de la comutatoarele simple



Comutatoare Ierarhice Dual Duplex

- Asigura comunicatia intre doua componente de tip a si mai multe componente de tip b
  - Este descris ca: S {poarta, 2a, nb, concurenta 2, 2nSp}



  - Sunt posibile aceleasi variante ca la comutatoarele simple

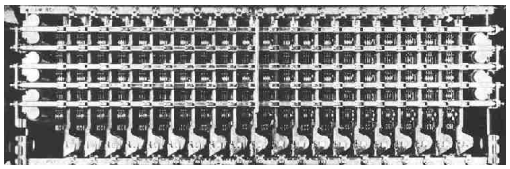
Comutatoare Ierarhice Legatura Multipla

- Asigura comunicatia intre m componente de tip a si n componente de tip b
- Este cunoscut si ca Cross-Bar-Switch
  - S {CB, ma, nb, c = min(m, n), m*nSp}

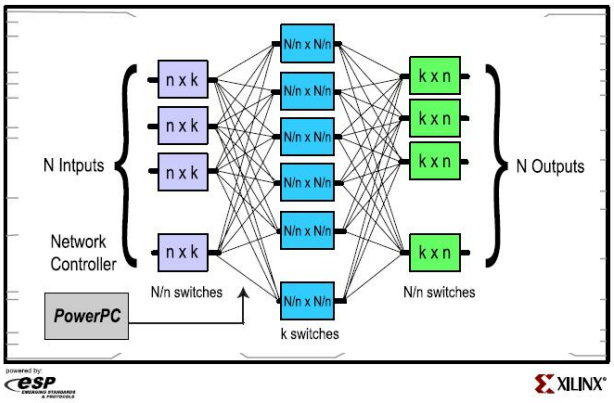
- Orice a poate comunica cu orice b
- Pot avea loc transferuri multiple si simultane!
- Cel mai dificil este de implementat U_{Cmd} pentru S_{CB}
  - Trebuie sa rezolve toate posibilele conflicte

Comutatoare Ierarhice CrossBar Switch

- Cross bar de la Bell System din 1960 cu 20 de verticale si 10 nivele

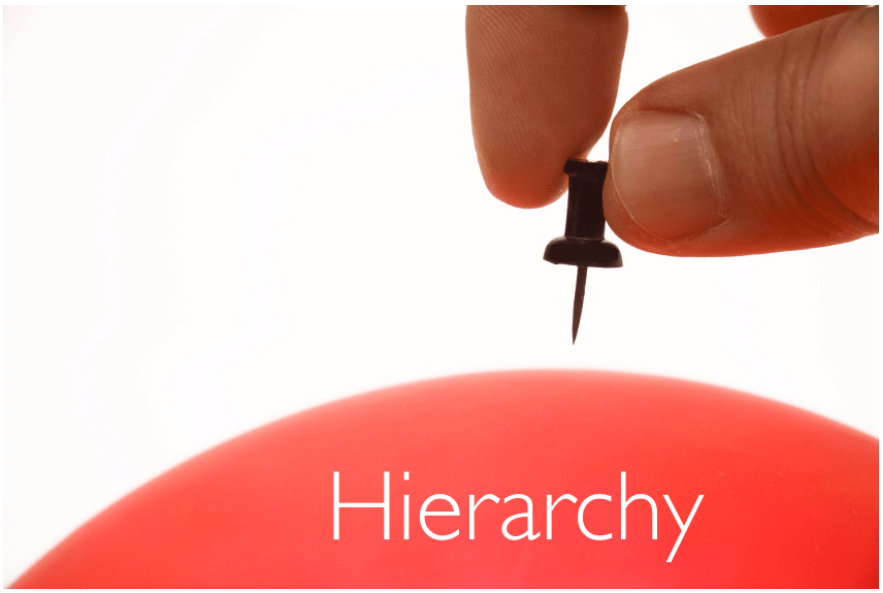


- Un crossbar de la Xilinx in FPGA-urile Virtex-II / Virtex-II Pro/ProX



Comutatoare Ierarhice Trunchi K

- Imbina conectivitatea oferita de S legatura multipla cu costul redus al celorlalte tipuri de comutatoare
  - S {TK, ma, nb, c = k, k*(m+n)Sp}
- Orice a poate comunica cu orice b
- Nu totdeauna vei avea min(m, n) comunicatii simultane
  - Asa a aparut idea de a folosi k linii pentru comunicatie
- In functie de m si n si de trafic, se va determina un k optim
  - 8 procs si 4 memorii:
    - Pot fi maxim 4 accese la memorii → k=4
    - Nu are sens mai mult → k < n si k < m



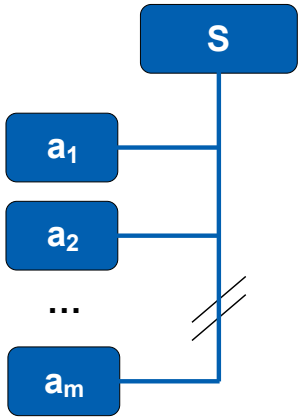
Comutatoare Neierarhice

Comutatoare Neierarhice

- Asigura conexiunea intre elemente de acelasi tip (e.g. P-P; M-M)
- Tipuri:
  - Comutatoare Duplex
  - Legatura multipla – tip central
  - Trunchi K neierarhic
- Structurile cu comutatoare asigura infrastructura de comunicatie

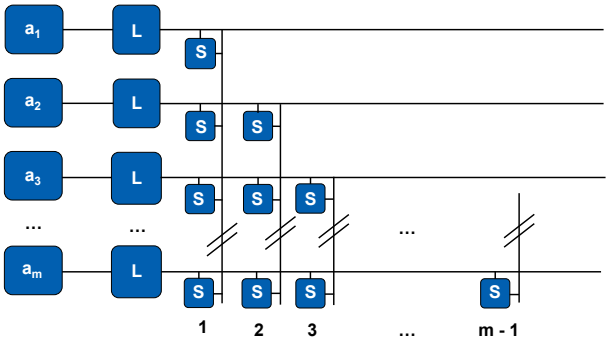
Comutatoare Neierarhice Duplex

- Comutatoare Duplex Neierarhice
  - S{Dn; ma, c:1; mSp}
  - Bus Inlantuit



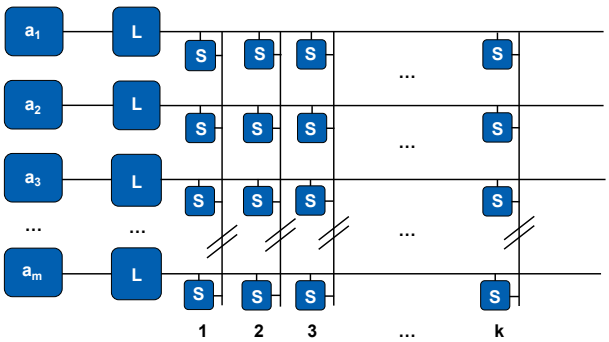
Comutatoare Neierarhice Legatura Multipla

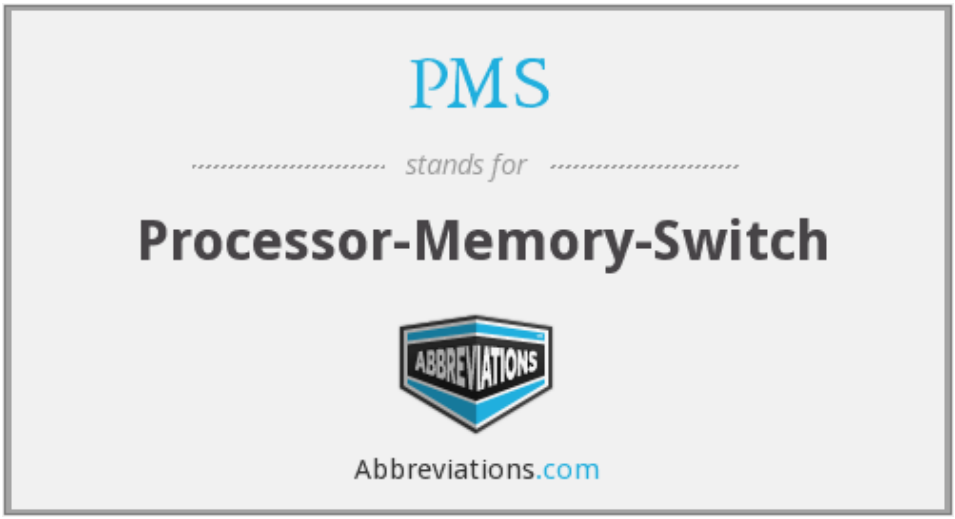
- Legatura multipla – tip central
  - S{lm-neierarhica; ma, c:m/2; m(m-1)/2 Sp}



Comutatoare Neierarhice Trunchi-K

- Trunchi-K neierarhic
  - S{tk; ma, c:min(k,m/2); m*k Sp}

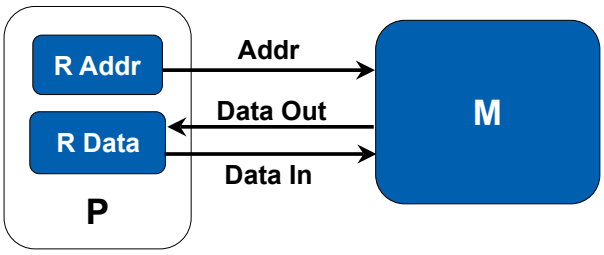




Exemple de Comutatoare Procesor Memorie

Comutatoare Procesor Memorie

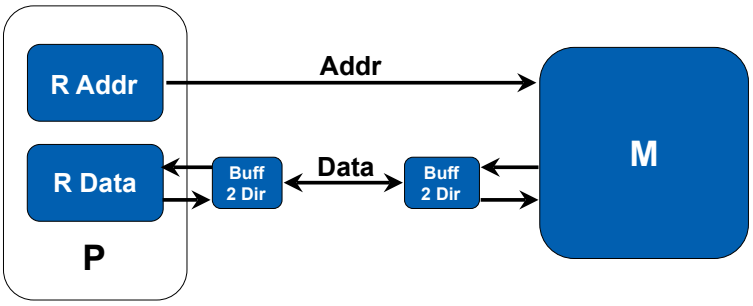
- Switch Processor Memory → S_{PM}
- Liniile de adrese si date sunt complet separate



R/W – Disjuncte → No Switch!
Nu se justifica ambele linii

Comutatoare Procesor Memorie

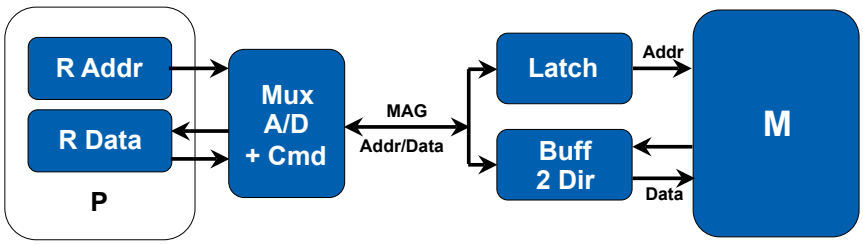
- Switch Processor Memory → S_{PM}
- D_{In} si D_{Out} sunt multiplexate prin magistrala de date



Bufferele sunt fie MUX fie TS

Comutatoare Procesor Memorie

- Switch Processor Memory → S_{PM}
- Addr, D_{In} si D_{Out} sunt multiplexate prin aceeasi magistrala

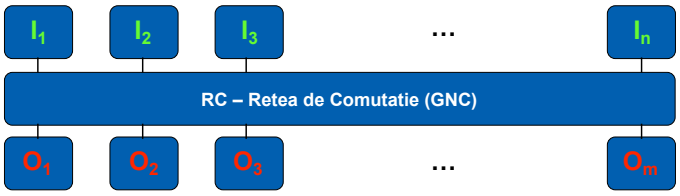


Bufferele sunt fie MUX fie TS

Interconectarea in Sistemele cu Resurse Multiple

Interconectarea in Sistemele cu Resurse Multiple

- In structurile SIMD si MIMD, retelele de comutare (de interconectare a resurselor), au rolul cel mai important in asigurarea performantelor sistemului!
- Realizarea interconectarii resurselor (P, M & I/O) se face printr-o retea de comutatie (RC)



- I_i = Intrari & O_j = Iesiri; RC conecteaza n·m resurse
- Net (n, m) = Generalised Connection Network – graf in care arcele conecteaza perechi de resurse

Retele de Comutatie

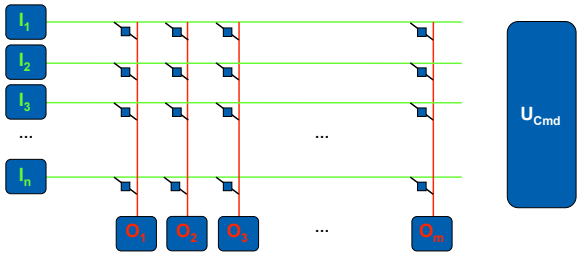
- O astfel de RC trebuie sa respecte niste reguli:
  - O intrare poate fi conectata la orice iesire
  - O intrare poate fi conectata simultan la mai multe iesiri
    - Astfel vom avea Broadcast partial sau total
  - O iesire poate fi conectata numai la o intrare
  - Concurenta maxima e data de min (m, n)
- Din aceste reguli rezulta ca RC ar trebui sa realizeze n·m corespondente == GCN!

GCN – Generalised Connection Network

- Un GCN poate fi implementat
  - Printr-o structura organizata pe un singur nivel in care se stabilesc legaturi directe intre intrari si iesiri (crossbar)
  - Prin utilizarea unor subretele de dimensiuni (mai) reduse conectate ierarhic pe mai multe nivele
- Caracteristici pentru GCN sunt doi parametrii
  - Intarzierea de propagare (Δt) – timpul de raspuns ce depinde de numarul maxim de comutatoare elementare (de tip poarta) ce asigura legatura intre intrare si iesire
  - Complexitatea (nC) – numarul total de comutatoare elementare utilizate in implementarea GCN-ului

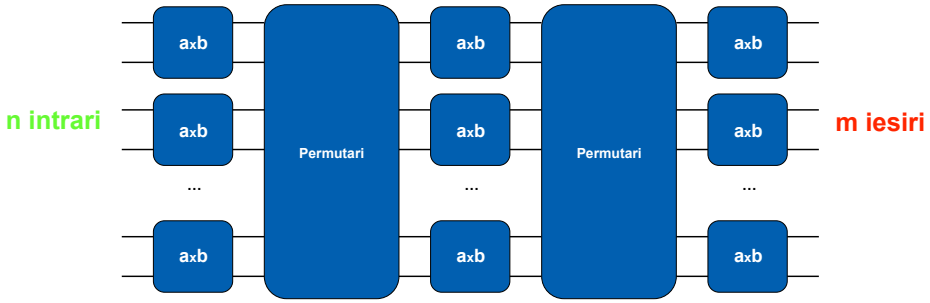
Implementari ale RC (GCN)

- Comutare directa – Cross Bar cu n·m comutatoare (Δt = 1)
- Unitatea de comanda trebuie sa rejecteze 2 I (intrari) / 1 O (iesire)
- O implementare cu comutare directa
  - Este greu de proiectat fizic (nC = n·m)
  - Nu se preteaza la VLSI & este inflexibila la dezvoltare ulterioara
- U_{Cmd} controleaza toate switch-urile elementare



Implementari ale RC (GCN)

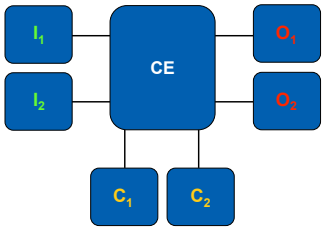
- Prin realizarea unor subretele de comutare organizate ierarhic, pe mai multe niveluri
- Aceasta implementare necesita k ≈ $\log_2 n$ comutatoare elementare → reduce complexitatea (nC) & creste timpul de comutare (Δt)
- Comutatoarele elementare sunt de tip Cross Bar (axb), unde a « n si b « m
- Intre nivele se aseaza permutarile de interconectare



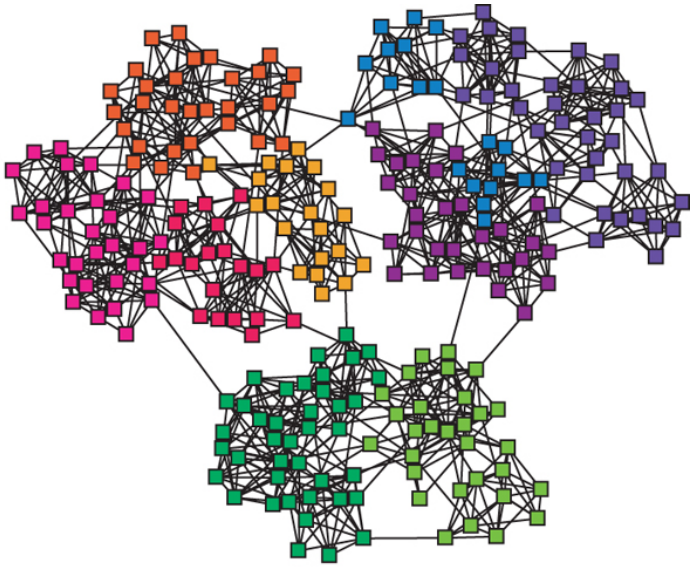
Comutatoare Elementare

- Un comutator elementar are 2 intrari, 2 iesiri si 2 biti de comanda:

C_1	C_2	Connection	
0	0	$O_1 = I_1$; $O_2 = I_2$	Direct
0	1	$O_1 = I_2$; $O_2 = I_1$	Inversat
1	0	$O_1 = I_1$; $O_2 = I_1$	Broadcast Superior
1	1	$O_1 = I_2$; $O_2 = I_2$	Broadcast Inferior



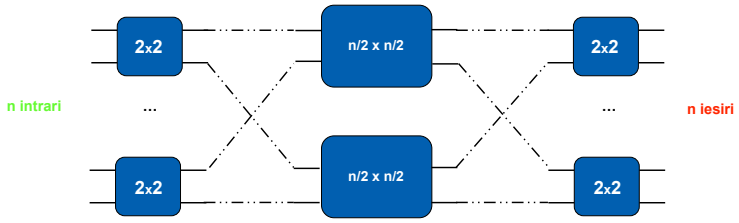
- Aceste module se pot integra pe baza permutarilor elementare
- In acelasi mod se pot construi si comutatoare elementare 4x4, 8x8, etc.



Retele de Comutare Ierarhice

Retele de Comutare Ierarhice

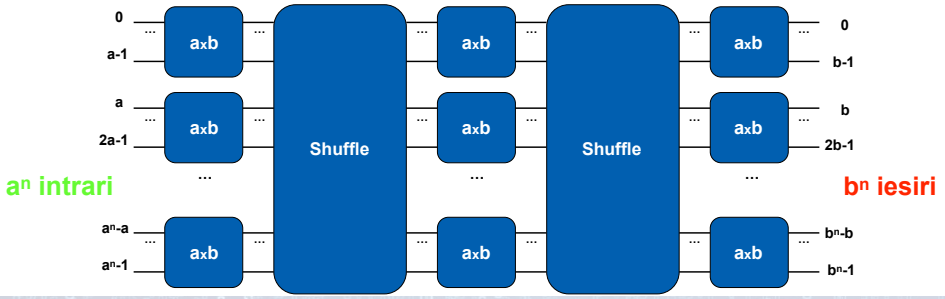
- Realizate din CrossBar-uri de dimensiuni mici asezate pe mai multe nivele de interconectare:
  - Creste timpul de intarziere datorita utilizarii mai multor module – dar se mentine ∀ intrare ↔ ∀ iesire
- Exemplu – structura Beizer (Benes):
  - Retea nxn realizata cu doua module n/2 x n/2 (se pot imparti la randul lor in subretele mai mici) si 4n module de tip 2x2
  - Complexitatea este de (4n log n – 2n)



Retele de Comutare de tip Delta

Retele de Comutare de tip Delta

- Sunt comutatoare ierarhice cu a^n intrari si b^n iesiri ce utilizeaza CrossBar-uri (CB) axb si retele de permutare de tip intercalare perfecta (Shuffle)
- Reteaua Delta are n nivele ierarhice:
  - Nivelul 0 contine a^{n-1} CB-uri de tip axb, pentru a conecta an intrari cu a^{n-1}xb iesiri
  - Nivelul 1 contine asadar a^{n-1}xb intrari conducand la a^{n-2}xb module CB

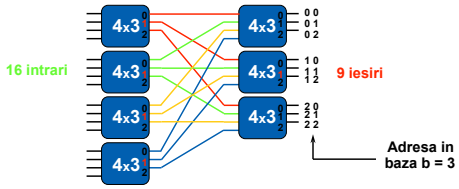


Retele de Comutare de tip Delta

- Astfel, in general:
  - Nivelul i contine $a^{n-i}b^{i-1}$ module CB de tip axb
- Intreaga retea contine:
  - $(a^n-b^n)/(a-b)$ CB-uri de tip axb cand $a \neq b$
  - nb^{n-1} CB-uri de tip axb cand $a = b$
- Destinatia este $D = (d_{n-1}\ d_{n-2} \ldots d_1\ d_0)$ cu $0 \leq d_i < b$
- Cifrele de reprezentare in baza b, a adresei destinatiei vor controla modulul CB de pe nivelul respectiv (d_i controleaza CB-ul i)
- Functia Shuffle-ului este de a interconecta nivelele retelei Delta

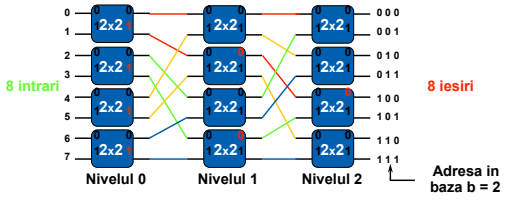
Exemplu – Delta 16x9

- Presupunem ca avem 16 procesoare si 9 memorii
- $16 = 4^2$ si $9 = 3^2$ → folosim 2 nivele de CB 4x3
- Daca vrem acces la memoria 4 (11 in baza 3) atunci se merge pe 1 (niv 0) si 1 (niv 1)!



Exemplu – Delta 8x8

- Presupunem ca avem 8 procesoare si 8 memorii
- $8 = 2^3$ → folosim 3 nivele de CB 2x2
- Daca vrem acces la memoria 4 (100 in baza 2) atunci se merge pe 1 (niv 0), 0 (niv 1) si 0 (niv 2)!



Retele Bazate pe Rutare

Retele Bazate pe Rutare

- Au ca nucleu legaturi directe intre o resursa si cele considerate vecine
- Resursele pot fi distincte dar toate au in comun un router ce se ocupa de comunicarea prin mesaje
- Traditional aceste retele sunt modelate prin grafuri cu urmatoarele proprietati de baza:
  - Gradul nodului (al resursei) – defineste numarul de canale de conexiune ale resursei la vecinii sai
  - Diametrul – distanta maxima intre doua resurse ale retelei
  - Regularitatea – nodurile/resursele care au/nu au acest grad
  - Simetria – o retea este simetrica daca “arata la fel” pentru orice nod
- Obs: acest tip de retea nu foloseste comutarea de circuite (CB), ci rutarea – legatura intre noduri/resurse e permisa doar cand este ceruta & permisa

Retele Bazate pe Rutare – Caracteristici

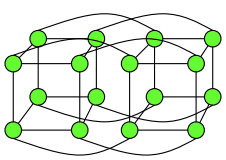
- Topologia – defineste modul in care nodurile sunt interconectate
- Rutarea – stabileste calea de date pe care un mesaj o urmeaza de la sursa la destinatie
- Comutarea – mecanismul ce determina cum si cand un canal de intrare este conectat la unul de iesire
- Obs: trebuie avute in vedere:
  - Alocarea de buffere
  - Controlul fluxului de date

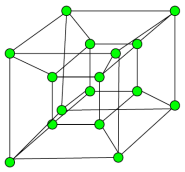
Retele Bazate pe Rutare – Topologii

- Topologii ortogonale:
  - Nodurile sunt aranjate intr-un spatiu n-dimensional ortogonal
  - Fiecare legatura produce o deplasare intr-o singura dimensiune
- Topologii strict ortogonale (mai interesante):
  - Fiecare nod are cel putin o legatura in fiecare dimensiune
  - Trecerea intr-o noua dimensiune se poate face din orice nod
  - Rutarea e mai simpla si implementarea HW e mai eficienta
  - Nodurile pot fi numerotate folosind coordonatele lor in spatiul n-dimensional
  - Procesul de rutare se face pe baza diferentei intre coordonate
  - Obs: Distanta dintre noduri poate fi calculata direct folosind adresele nodurilor respective

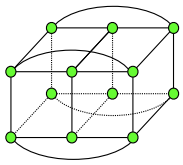
Retele Bazate pe Rutare – Topologii

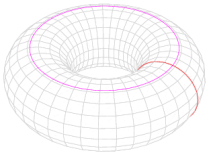
- Hipercub:
  - 2 cuburi unul intr-altul
  - Din orice nod se poate ajunge pe orice nivel
  - Se poate optimiza drumul

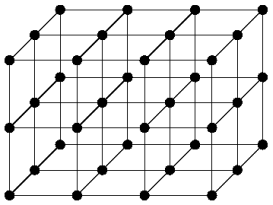




- Tor:



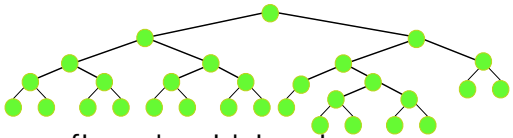




- Plasa n-dimensionala:

Retele de Tip Arbore

- Sunt un alt tip de retele de comutare

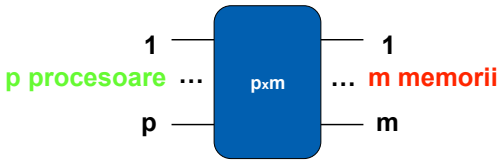


- Nu e neaparat necesar sa fie arbori binari
- Fiecare nod al arborelui poate avea la randul sau un subarbore – subarbori multiplii
- Similar – la structurile cu hipercub, fiecare nod poate fi inlocuit cu un nou cub!
- Exista mai multe cai posibile pentru mesaje – totdeauna se cauta drumul de lungime minima

Performantele Retelelor de Comutare

Performantele Retelelor de Comutare

- Analizam performantele retelelor ce asigura comutarea intre p procesoare si m memorii



- Analiza va fi realizata pentru retele de tip CB si Delta
- Rata de servire – media cererilor de acces la memorii acceptate intr-un ciclu (toate memoriile)
- Ciclu de servire – timpul necesar pentru ca o cerere de acces la memorie sa se propage prin logica de comutare + timpul efectiv de acces la memorie (propagarea inversa)

Presupuneri / Restrictii

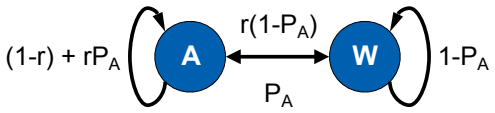
- Nu se fac deosebiri intre ciclurile de citire si de scriere
- Fiecare procesor va genera cereri independente pentru acces la un modul de memorie
- Cererile sunt uniform distribuite in cadrul tuturor modulelor de memorie – fiecare procesor cu un modul de memorie
- La inceputul fiecarui ciclu
  - Fiecare procesor genereaza o noua cerere de acces cu probabilitatea r (media cererilor generate de fiecare procesor intr-un ciclu)
  - Nu exista coada de asteptare → dialog direct
- Cererile neacceptate intr-un ciclu sunt ignorate
  - Cererile din ciclul urmator sunt independente de cele din ciclul anterior
  - Cererile neacceptate trebuiesc relansate intr-un ciclu urmator

Comportarea unui Procesor intr-un CB

- Se va utiliza un graf Markov
- Un procesor are 2 stari posibile:
  - A = starea activa – cererea este acceptata
  - W = starea de asteptare (wait) – cererea este respinsa
- q_A si q_W sunt probabilitatile ca procesorul sa se afle in starile A si W respectiv
- P_A = probabilitatea ca cererea sa fie acceptata – adica de a trece din starea W in starea A
- Astfel, putem defini:

$$q_A = \frac{P_A}{P_A + r(1 - P_A)} \qquad q_W = 1 - q_A$$

Comportarea unui Procesor intr-un CB



- Se considera rata de cereri r ca fiind statica
  - In realitate r e diferita luandu-se in considerare conflictele de acces
- Modulele de memorie primesc r' > r cereri de acces:

$$r' = rq_A + q_W = \frac{r}{r + (1-r)P_A} \qquad P_A = \frac{m}{r'p}\left(1 - \left(1 - \frac{r'}{m}\right)^p\right)$$

Analiza Retelei de Tip CrossBar

- Relatiile dinainte, definesc r' ca o functie de PA si PA ca o functie de r' → definesc un proces iterativ:
  - Initiem rata de transfer r' = r
  - P_A poate fi considerat ca o masura a cererilor de acces nerezolvate
  - Daca P_A este mare atunci sunt putine stari de asteptare
- Fie $\overline{w}$ media numarului de cicli de asteptare pentru o cerere
  - O cerere rejectata de i ori va astepta i ciclii $\overline{w} = \sum_{i=1}^{p} i(1-P_A)^i = \frac{1-P_A}{P_A}$
- In practica, cererile ce produc conflicte de acces la memorii, nu sunt rejectate, ci introduse intr-o coada de asteptare si servite FIFO

Analiza Retelei de Tip Delta

- Consideram o retea Delta ($a^n x b^n$) formata din CB-uri elementare (axb)
  - a^n procesoare si b^n memorii
- Fiecare nivel al retelei Delta e controlat de unul din bitii de reprezentare a adresei destinatie, in baza b!
- Orice modul CB axb e independent de celelalte module → putem considera:
  - Cererile independente si uniform distribuite
- Daca r este rata de cereri la intrarea in axb atunci: $B(a,b) = b - b\left(1 - \frac{r}{b}\right)^a$
- Impartim la b → rata de cereri la fiecare iesire din CB-ul curent: $\frac{B(a,b)}{b} = 1 - \left(1 - \frac{r}{b}\right)^a$

Analiza Retelei de Tip Delta

- Pentru fiecare nivel intermediar al retelei Delta:
  - r_{out} (rata de cereri pentru fiecare iesire) este o functie de r_{in} (rata de cereri de la intrare)

$$r_{out} = 1 - \left(1 - \frac{r_{in}}{b}\right)^a$$

- Rata de cereri de la iesirea unui nivel r_n este rata de cereri de intrare pentru nivelul adiacent:
  - Rata de cereri de la ultimul nivel determina rata de transfer a retelei de tip Delta
- Fie r_i rata de cereri de la iesirea unui nivel in reteaua Delta → rata de cereri a unei RC Delta, generata de fiecare procesor este:

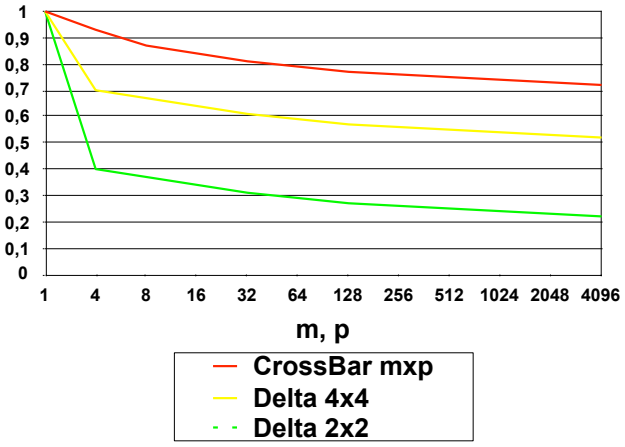
$$B(a^n, b^n) = b^n \cdot r_n \quad \text{unde} \quad r_i = 1 - \left(1 - \frac{r_{i-1}}{b}\right)^a \quad \text{si} \quad r_0 = r$$

Analiza Comparativa Delta / CB

- Probabilitatea ca o cerere sa fie acceptata este:

$$P_A = \frac{b^n \cdot r_n}{a^n \cdot r}$$

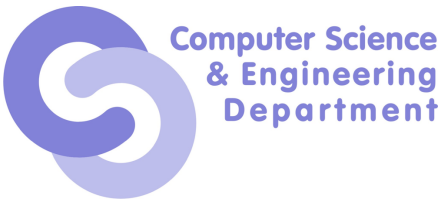
- Ce influenta au m si p-ul unei retele?
  - CrossBar **m**x**p** cu r=1 rata de cereri a fiecarui procesor
  - Delta 4^nx4^n cu 4x4
  - Delta 2^nx2^n cu 2x2
- Pentru m mare, P_A-ul scade puternic
  - Multe rejectii si conflicte
  - Limita la 16^nx16^n este 0.6



Ce am invatat in aces curs?

- Comutatoare Ierarhice
- Comutatoare Neierarhice
- Exemple de Comutatoare Procesor Memorie
- Interconectarea in Sistemele cu Resurse Multiple
- Retele de Comutare Ierarhice
- Retele de Comutare de tip Delta
- Retele Bazate pe Rutare
- Performantele Retelelor de Comutare

Thank you for your attention.



emil.slusanschi@cs.pub.ro
cs.pub.ro

Arhitectura Sistemelor de Calcul

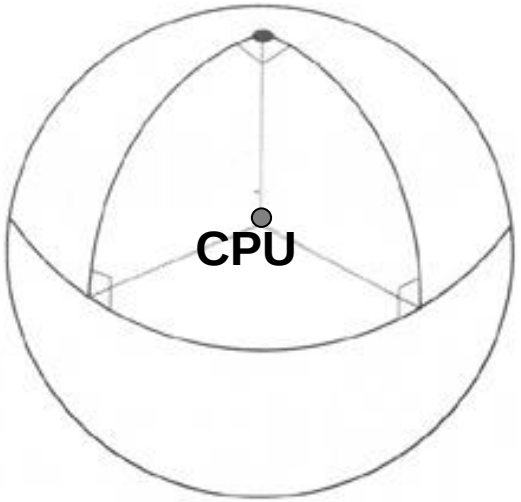
Universitatea Politehnica Bucuresti
Facultatea de Automatica si Calculatoare
curs.upb.ro

Cuprins

- De ce avem nevoie de paralelism?
- Structuri de calcul cu prelucrare paralela
- Clasificarea sistemelor de calcul / Flynn: SISD, SIMD, MISD, MIMD
- Exemple de utilizare a structurilor:
  - SISD
  - SIMD
  - MIMD
- Exemplu cu/fara dependenta de date pe sisteme de calcul MIMD

Un procesor de 10TFlops?

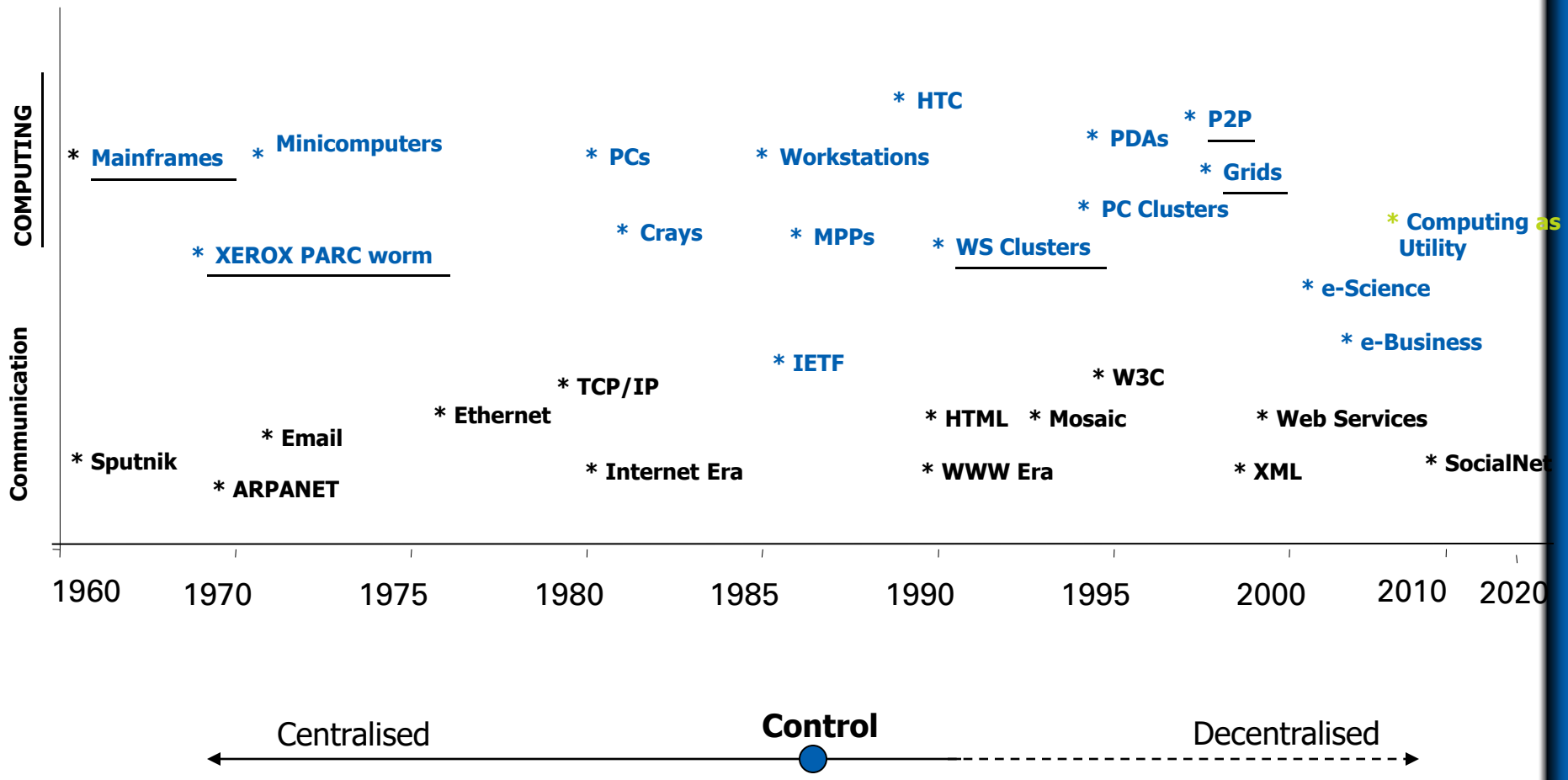
- Putem construi un CPU neconcurent care
  - Ofera (10 TFlops) -10,000 de miliarde de operatii in virgula mobila pe secunda?
  - Opereaza pe 10,000 miliarde de bytes (10 TByte)?
- Este reprezentativ pentru necesitatile cercetatorilor din ziua de azi
- Ceasul procesorului trebuie sa fie de 10,000 GHz
- Presupunem ca datele circula cu viteza luminii
- Presupunem ca procesorul este o sfera “ideala”



Un procesor de 10TFlops?

- Masina genereaza o instructiune pe ciclu, si de aceeas ceasul trebuie sa fie de aproximativ 10,000GHz ~ 10^{13} Hz
- Datele au de parcurs o distanta intre memorie si CPU
- Fiecare instructiune necesita cel putin 8 bytes de memorie
- Presupunem ca datele circula cu viteza luminii c = $3x10^{8}$ m/s
- Astfel, distanta intre memorie si CPU trebuie sa fie r < c / 10^{13} ~ $3e^{-6}$ m
- Apoi, trebuie sa avem 10^{13} bytes de memorie in $4/3\pi r^3 = 3.7x10^{-17}$ m^3
- Si de aceea, fiecare cuvant de memorie trebuie sa ocupe maxim $3.7e^{-30}$ m^3
  - Ceea ce inseamna **3.7 Angstrom^3**
- Aceasta dimensiune corespunde unei molecule foarte mici, formata din cativa atomi…
- Densitatea curenta a memoriei este de 10 GB/cm^3, sau cu un factor de ordinul 10^{20} mai mica decat ceea ce ar fi necesar!
- Concluzie: Acest procesor nu va fi disponibil pana cand quantum computing nu va deveni mainstream ☺

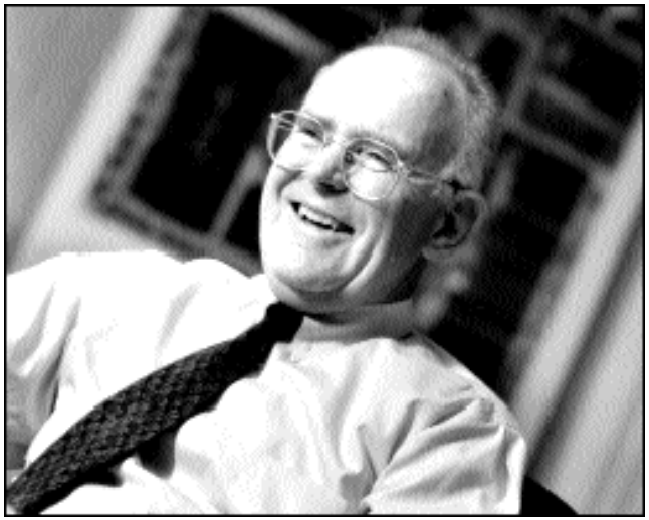
Computing and Communication Technologies Evolution: 1960-2020!

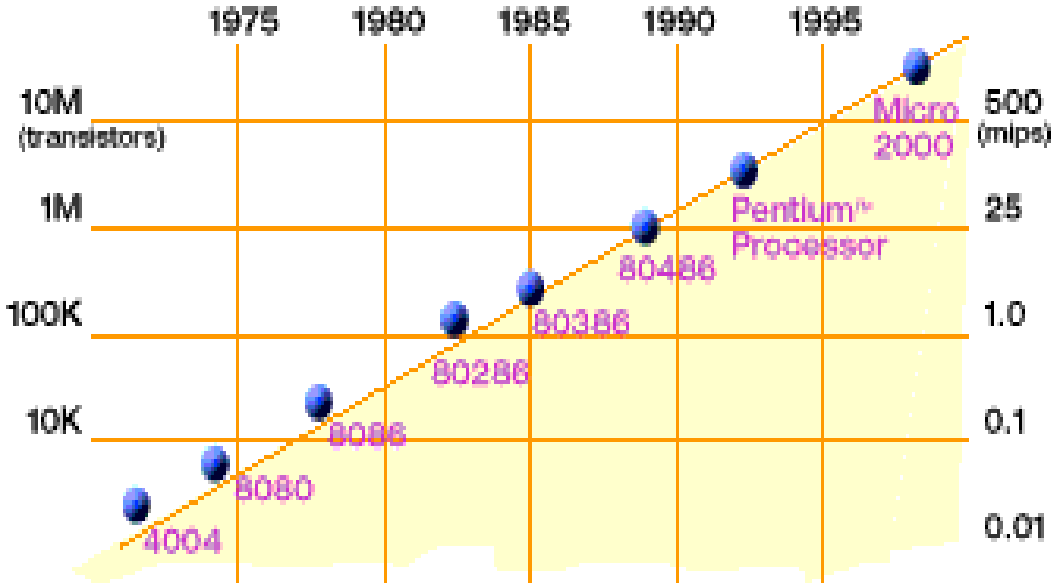


Words of Wisdom !

- “I think there is a world market for maybe five computers.”
  - Thomas Watson, chairman of IBM, 1943.
- “There is no reason for any individual to have a computer in their home”
  - Ken Olson, President and founder of Digital Equipment Corporation, 1977.
- “640KB [of main memory] ought to be enough for anybody.”
  - Bill Gates, Chairman of Microsoft,1981.

Evolutia Microprocesoarelor





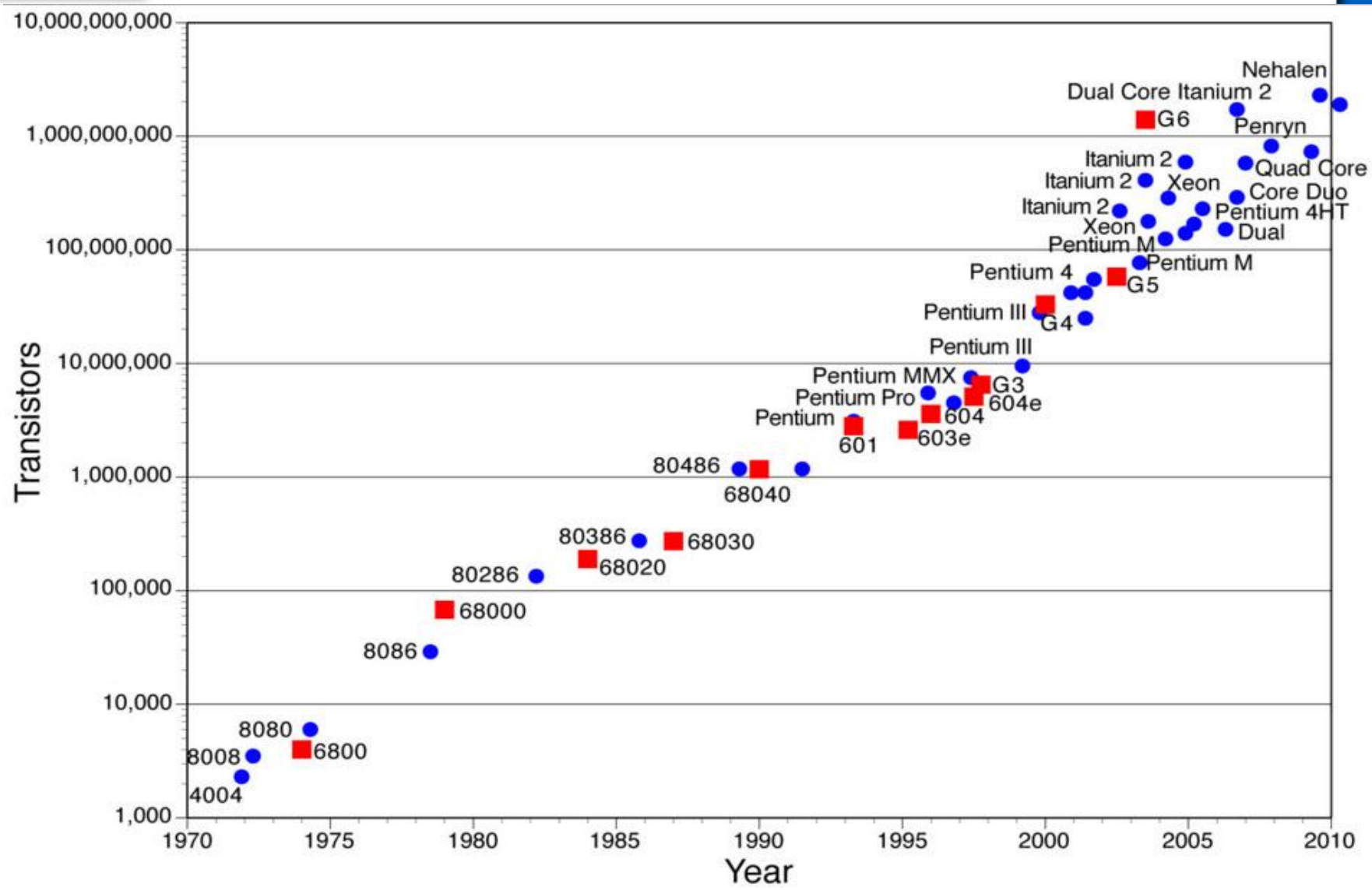
Gordon Moore (cofondator Intel) a prezis in 1965 ca densitatea tranzistoarelor in cipurile cu semiconductori se va dubla intr-un interval aproximativ de 18 luni

Legea lui Moore se confirma!

Intel 4004 2300 transistors;

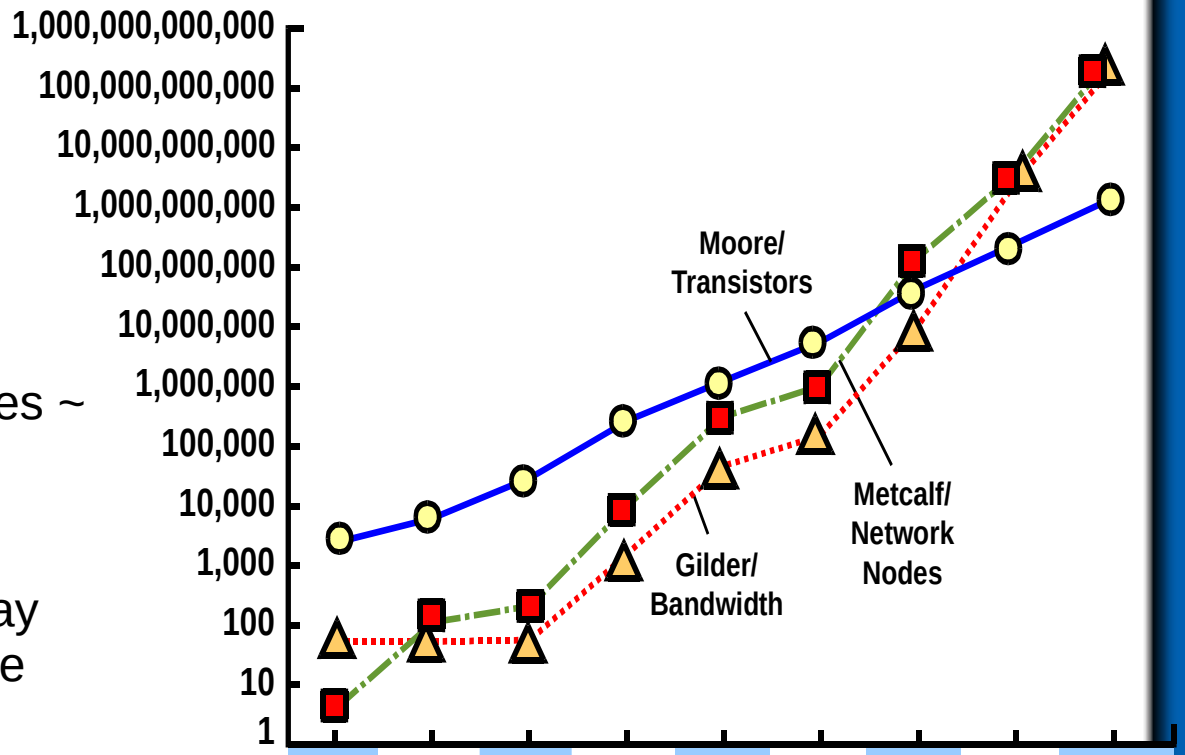
AMD Epyc Rome (64-bit, SIMD, caches) CPU with 39.5 billion transistors

Tranzistori / microprocesor



Performance, Capability, Value of ICT as defined by the three Laws of Computing

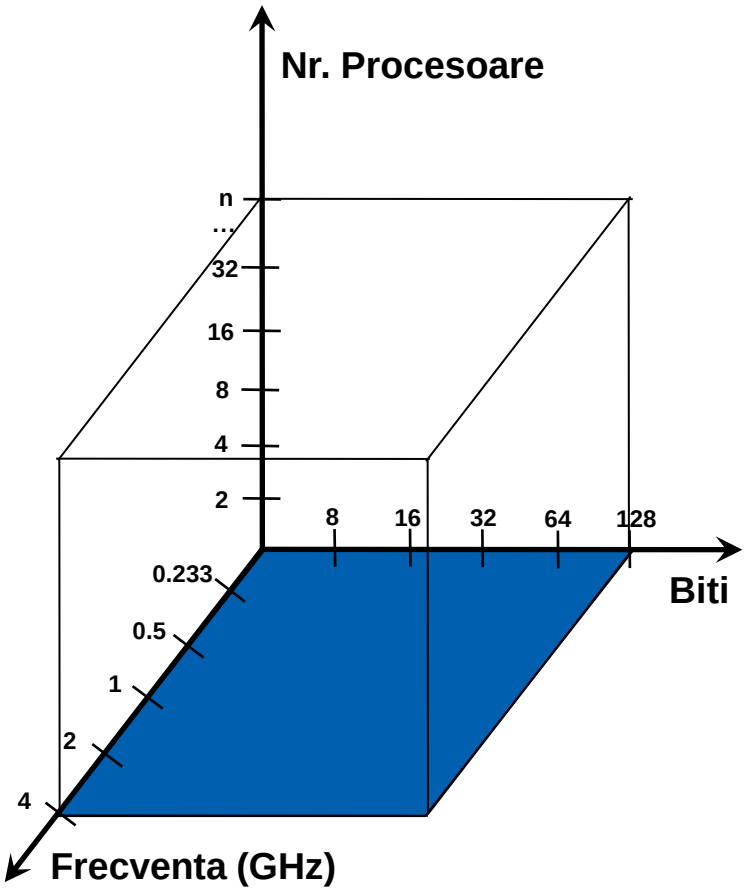
- Moore's Law.
  - Transistors on a single chip doubles ~ every 18–24 months.
- Gilder's Law.
  - Aggregate bandwidth triples ~ every year.
- Metcalfe's Law.
  - The value of a network may grow exponentially with the number of participants.



	1970	1975	1980	1985	1990	1995	2000	2005	2010
○	2,300	6,000	29,000	275,000	1.2 mil	5.5 mil	42 mil	252 mil	1.344 bil
△	50	50	56	1,544	45,000	145,000	10 mil	2.43 bil	200.49 bil
□	4	111	200	10,000	300,000	1 mil	140 mil	3.5 bil	300 bil

Source: Cambridge Energy Resource Associates

Structuri de Calcul cu Prelucrare Paralela



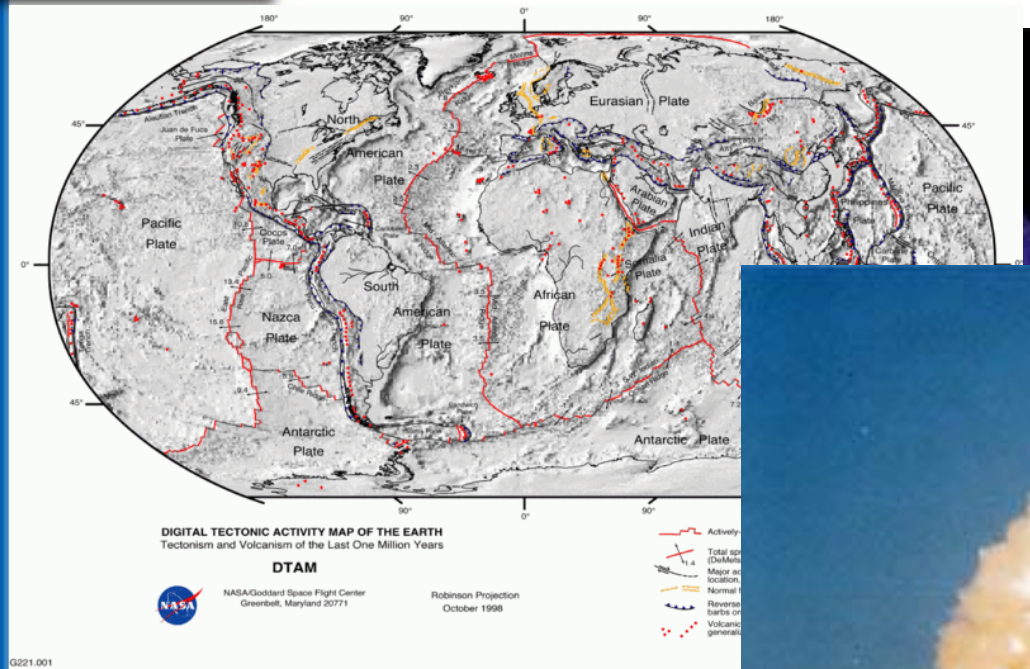
In general gradul de paralelism este de 10%

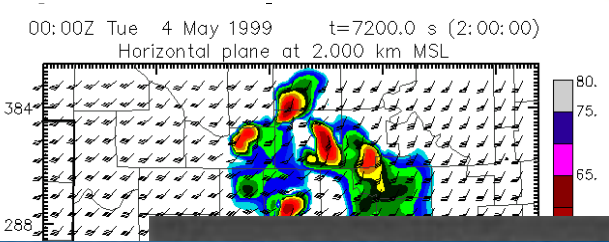
- Aria → puterea de calcul
- Marimea puterii de calcul: volumul paralelipiped-ului –
- “n” e limitat doar de cost
- arhitecturi paralele de calcul
- Orice problema are un grad de paralelism intrinsec ce depinde de natura ei
- Trebuie tinut seama de:
  - Algoritmi paraleli specifici
  - Limbaje adecvate de programare
  - Limitarile SO
  - Arhitectura intrinseca a SC utilizat

Paralelismul este necesar!

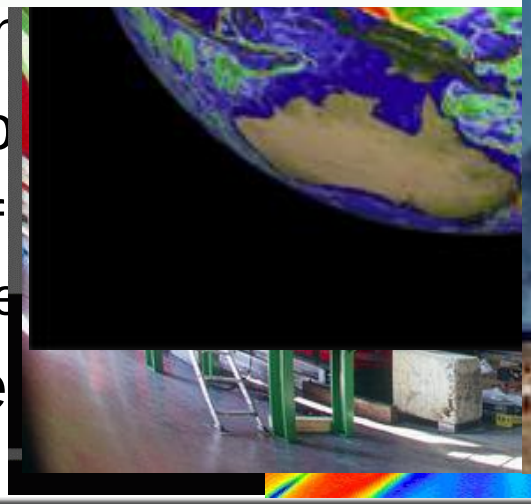
- Paralelism la nivel de Bit (Bit-level parallelism)
  - Operatii in virgula mobila
- Paralelism la nivel Instructiune (ILP)
  - Mai multe instructiuni pe ciclu
- Paralelism la nivelul memoriei
  - Suprapunerea intre operatii cu memoria si de calcul
- Paralelism al sistemului de operare
  - Mai multe thread-uri, procese pe CPU-uri multiple, in cadrul aceleasi masini
- Paralelism distribuit
  - Mai multe masini conectate impreuna

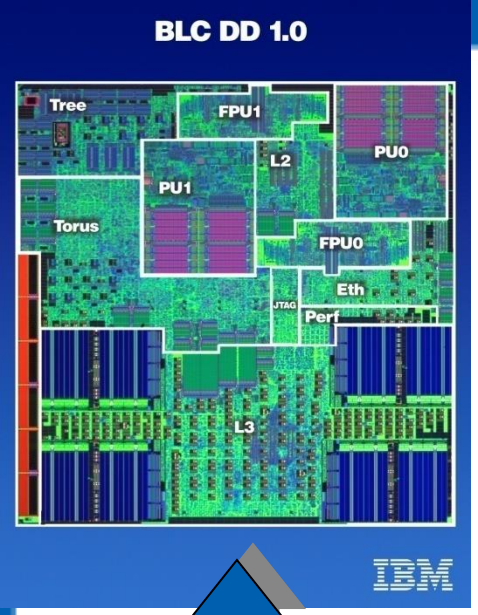
Aplicatii Paralele





- Bioch
- Astro
- Geof
  - Ge
- Mete





BlueGene/L
Compute ASIC

System
(64 racks, 64x32x32)
131,072 procs

Rack
(32 Node boards, 8x8x16)
2048 processors

Node Card
(32 chips, 4x4x2)
16 Compute Cards
64 processors

Compute Card
(2 chips, 2x1x1)
4 processors

Chip
(2 processors)

180/360 TF/s
32 TB DDR

2.9/5.7 TF/s
0.5 TB DDR

90/180 GF/s
16 GB DDR

5.6/11.2 GF/s
1 GB DDR

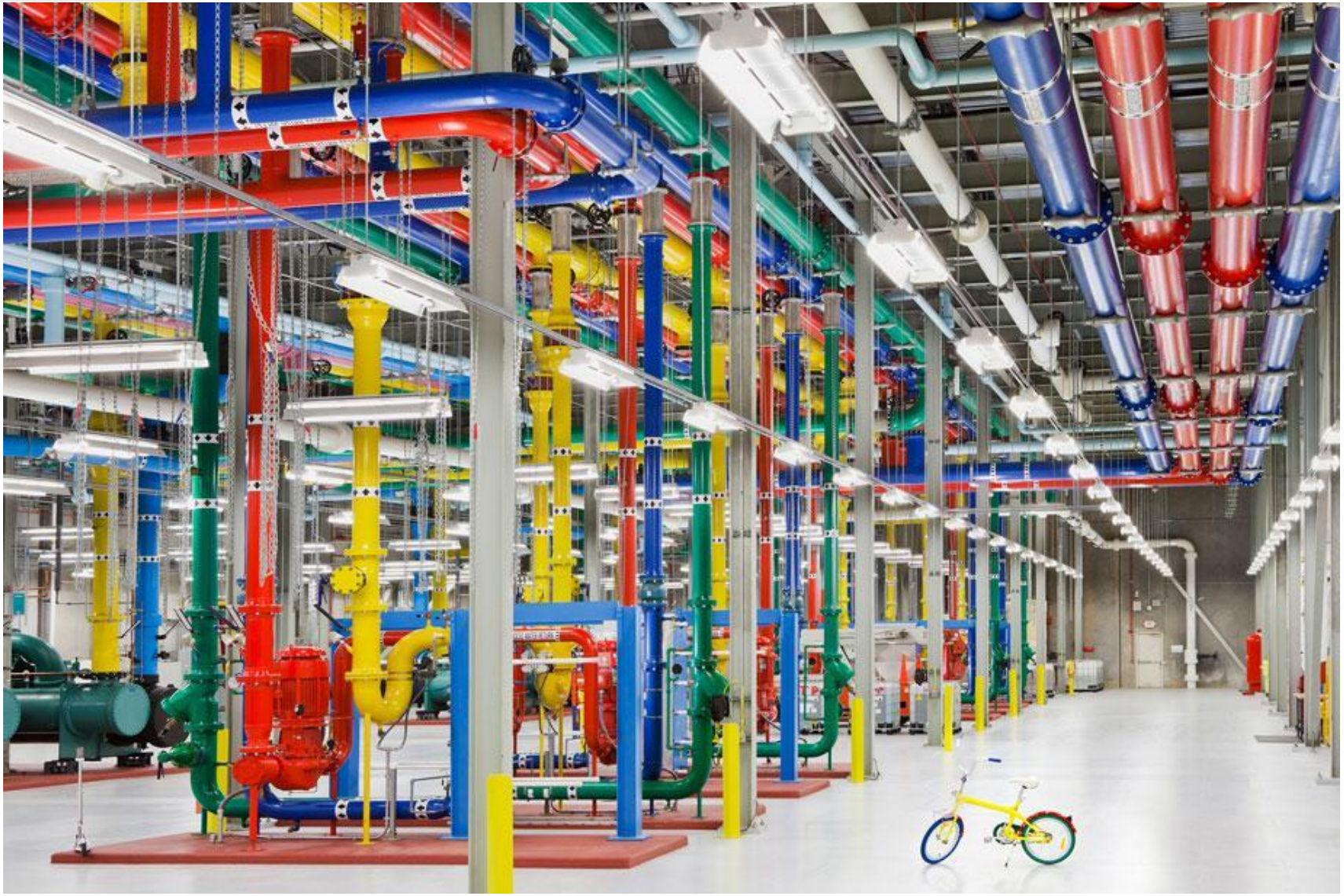
2.8/5.6 GF/s
4 MB (cache)

IBM BlueGene/L
131,072 Processors

Data Center

Google Data Center

Infrastructura Google



Infrastructura

Data center locations

We own and operate data centers around the world to keep our products running 24 hours a day, 7 days a week. Find out more about our data center locations, community involvement, and job opportunities in our locations around the world.

Americas

Berkeley County, South Carolina
Council Bluffs, Iowa
Douglas County, Georgia
Quilicura, Chile
Mayes County, Oklahoma
Lenoir, North Carolina
The Dalles, Oregon

Asia

Singapore
Changhua County, Taiwan

Europe

Hamina, Finland
St Ghislain, Belgium
Dublin, Ireland

TOP 10 Sites for November 2019

https://www.top500.org/lists/2019/11/

Rank	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	**Summit** - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband , IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148,600.0	200,794.9	10,096
2	**Sierra** - IBM Power System AC922, IBM POWER9 22C 3.1GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband , IBM / NVIDIA / Mellanox DOE/NNSA/LLNL United States	1,572,480	94,640.0	125,712.0	7,438
3	**Sunway TaihuLight** - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway , NRCPC National Supercomputing Center in Wuxi China	10,649,600	93,014.6	125,435.9	15,371
4	**Tianhe-2A** - TH-IVB-FEP Cluster, Intel Xeon E5-2692v2 12C 2.2GHz, TH Express-2, Matrix-2000 , NUDT National Super Computer Center in Guangzhou China	4,981,760	61,444.5	100,678.7	18,482
5	**Frontera** - Dell C6420, Xeon Platinum 8280 28C 2.7GHz, Mellanox InfiniBand HDR , Dell EMC Texas Advanced Computing Center/Univ. of Texas United States	448,448	23,516.4	38,745.9	

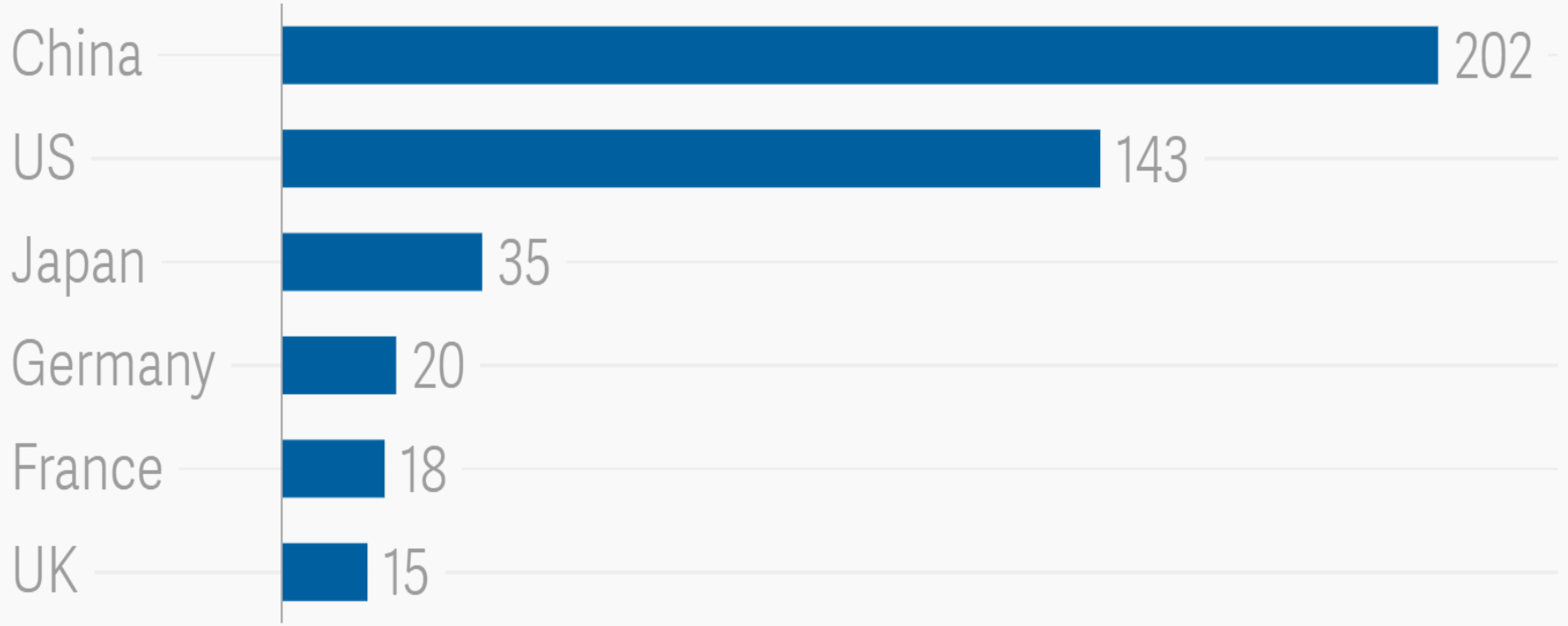
summit
IBM

TOP 10 Sites for November 2019 cont

6	**Piz Daint** - Cray XC50, Xeon E5-2690v3 12C 2.6GHz, Aries interconnect , NVIDIA Tesla P100 , Cray/HPE Swiss National Supercomputing Centre (CSCS) Switzerland	387,872	21,230.0	27,154.3	2,384
7	**Trinity** - Cray XC40, Xeon E5-2698v3 16C 2.3GHz, Intel Xeon Phi 7250 68C 1.4GHz, Aries interconnect , Cray/HPE DOE/NNSA/LANL/SNL United States	979,072	20,158.7	41,461.2	7,578
8	**AI Bridging Cloud Infrastructure (ABCI)** - PRIMERGY CX2570 M4, Xeon Gold 6148 20C 2.4GHz, NVIDIA Tesla V100 SXM2, Infiniband EDR , Fujitsu National Institute of Advanced Industrial Science and Technology (AIST) Japan	391,680	19,880.0	32,576.6	1,649
9	**SuperMUC-NG** - ThinkSystem SD650, Xeon Platinum 8174 24C 3.1GHz, Intel Omni-Path , Lenovo Leibniz Rechenzentrum Germany	305,856	19,476.6	26,873.9	
10	**Lassen** - IBM Power System AC922, IBM POWER9 22C 3.1GHz, Dual-rail Mellanox EDR Infiniband, NVIDIA Tesla V100 , IBM / NVIDIA / Mellanox DOE/NNSA/LLNL United States	288,288	18,200.0	23,047.2	

Sunway TaihuLight: National Supercomputing Center in Wuxi

Nations with the world's most powerful computers



Summit - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband

Site:	DOE/SC/Oak Ridge National Laboratory
System URL:	http://www.olcf.ornl.gov/olcf-resources/compute-systems/summit/
Manufacturer:	IBM
Cores:	2,414,592
Memory:	2,801,664 GB
Processor:	IBM POWER9 22C 3.07GHz
Interconnect:	Dual-rail Mellanox EDR Infiniband
Performance	
Linpack Performance (Rmax)	148,600 TFlop/s
Theoretical Peak (Rpeak)	200,795 TFlop/s
Nmax	16,473,600
HPCG [TFlop/s]	2,925.75
Power Consumption	
Power:	10,096.00 kW (Submitted)

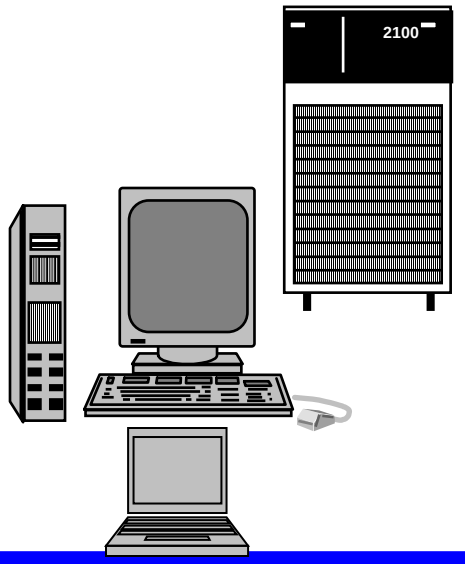
Sunway TaihuLight - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway

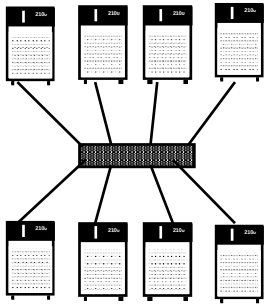
Site:	National Supercomputing Center in Wuxi
Manufacturer:	NRCPC
Cores:	10,649,600
Memory:	1,310,720 GB
Processor:	Sunway SW26010 260C 1.45GHz
Interconnect:	Sunway
Performance	
Linpack Performance (Rmax)	93,014.6 TFlop/s
Theoretical Peak (Rpeak)	125,436 TFlop/s
Nmax	12,288,000
HPCG [TFlop/s]	480.848
Power Consumption	
Power:	15,371.00 kW (Submitted)
Power Measurement Level:	2
Software	
Operating System:	Sunway RaiseOS 2.0.5

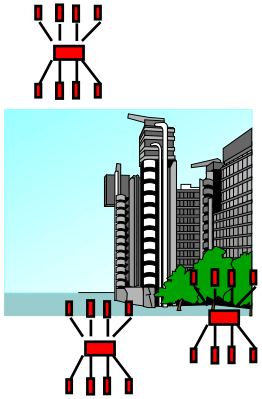
Computing is Scaling: Towards Inter-Planetary Level

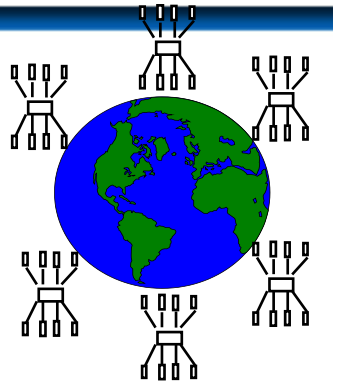
InterPlaNet

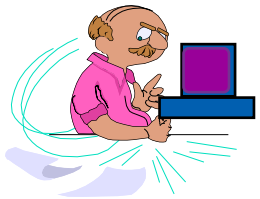
SERVICES + PERFORMANCE











Administrative Barriers

- Individual
- Group
- Department
- Campus
- State
- National
- Globe
- Inter Planet
- Universe

Personal Device	SMPs or SuperComputers	Local Cluster	Enterprise Cluster/Grid	Global Grid	Inter Planet Grid

Nivelele Prelucrarii Paralele – 1

- 1: Paralelismul la nivel de Job:
  - Intre Job-uri diverse
    - Sunt necesare: un mecanism de salvare a contextului; un ceas pentru divizarea timpului; canale I/O pt transfer
  - Intre fazele unui Job
    - Citirea sursei programului
    - Compilare
    - Link-are
    - Executarea codului obiect
    - Salvarea rezultatelor
  - Anumite faze ale unui job pot fi suprapuse
- 2: Paralelismul la nivel de subansamble Hardware:
  - Intre elemente de prelucrare ale vectorilor
  - Intre componentele logice ale dispozitivelor aritmetico/logice (carry look ahead)

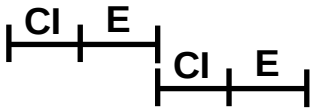
Nivelele Prelucrarii Paralele – 2

- 3: Paralelismul la nivel de Program:
  - Intre diverse sectiuni (independente) ale unui program
  - Intre buclele (loop-urile) unui program
  - Ambele presupun analiza dependentelor de date!
- 4: Paralelismul la nivel de Instructiune:
  - Intre diverse faze ale ciclului instructiune:

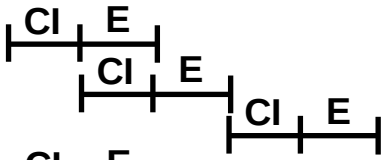
CI = Citire & Interpretare

E = Executie

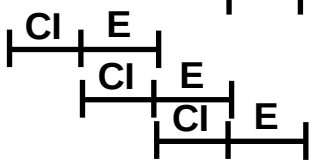
    - Implementare seriala:



    - Implementare serie-paralela:



    - Implementare paralela:



  - Este necesar un mecanism de predictie al salturilor

Cuprins

- Structuri de calcul cu prelucrare paralela
- Clasificarea sistemelor de calcul / Flynn: SISD, SIMD, MISD, MIMD
- Exemple de utilizare a structurilor:
  - SISD
  - SIMD
  - MIMD
- Exemplu cu/fara dependenta de date pe sisteme de calcul MIMD

Clasificarea Sistemelor de Calcul

- Sisteme Matriceale (Processor Array):
  - Unitatea de baza a informatiei este vectorul
  - Dispun de instructiuni similare SC Von Neumann – operatiile asupra vectorilor sunt efectuate in aceeasi instructiune
- Sisteme Multiprocesor (Multiprocessor Systems):
  - Formate din N unitati de prelucrare interconectate printr-o retea de comutare (Strans/Slab cuplata)
  - Sistemele lucreaza independent la realizarea aceluiasi Job
- Sisteme Pipeline:
  - Dispun de mai multe unitati de prelucrare asezate in banda de asamblare (RISC):
    - Fiecare UC executa o prelucrare specifica si transfera rezultatul subansamblului adiacent

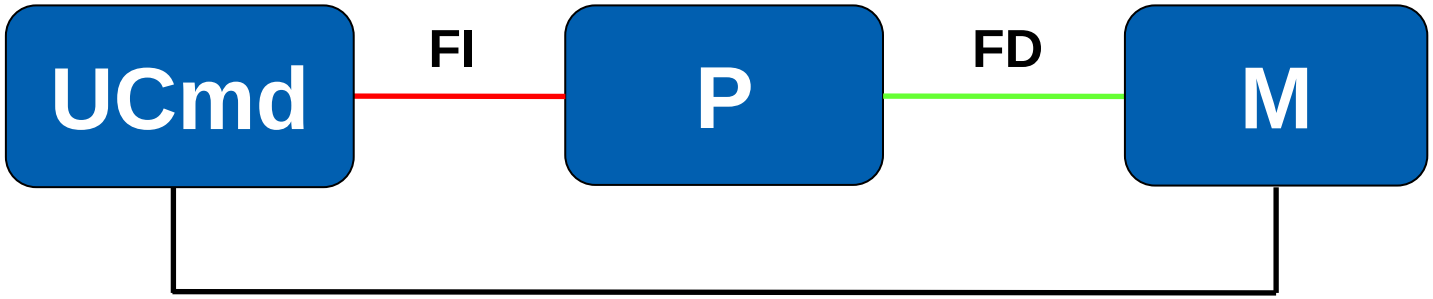
Taxonomia lui Flynn

- Impartirea sistemelor de calcul in functie de:
  - Fluxul de Instructiuni – secventa de instructiuni executate de procesor
  - Fluxul de Date – secventa de operanzi manipulata de procesor
- Bazate pe acest criteriu se desprind:
- I – SISD = Single Instruction Single Data Stream (structura Von Neumann)
- II – SIMD = Single Instruction Multiple Data Stream
- III – MISD = Multiple Instruction Single Data Stream
- IV – MIMD = Multiple Instruction Multiple Data Stream

I – SISD

- **UCmd** = Unitate de comanda
- **P** = Unitate de prelucrare aritmetica
- **M** = Memorie
- FI = Flux Instructiuni
- FD = Flux Date

SISD = 1 FI & 1 FD

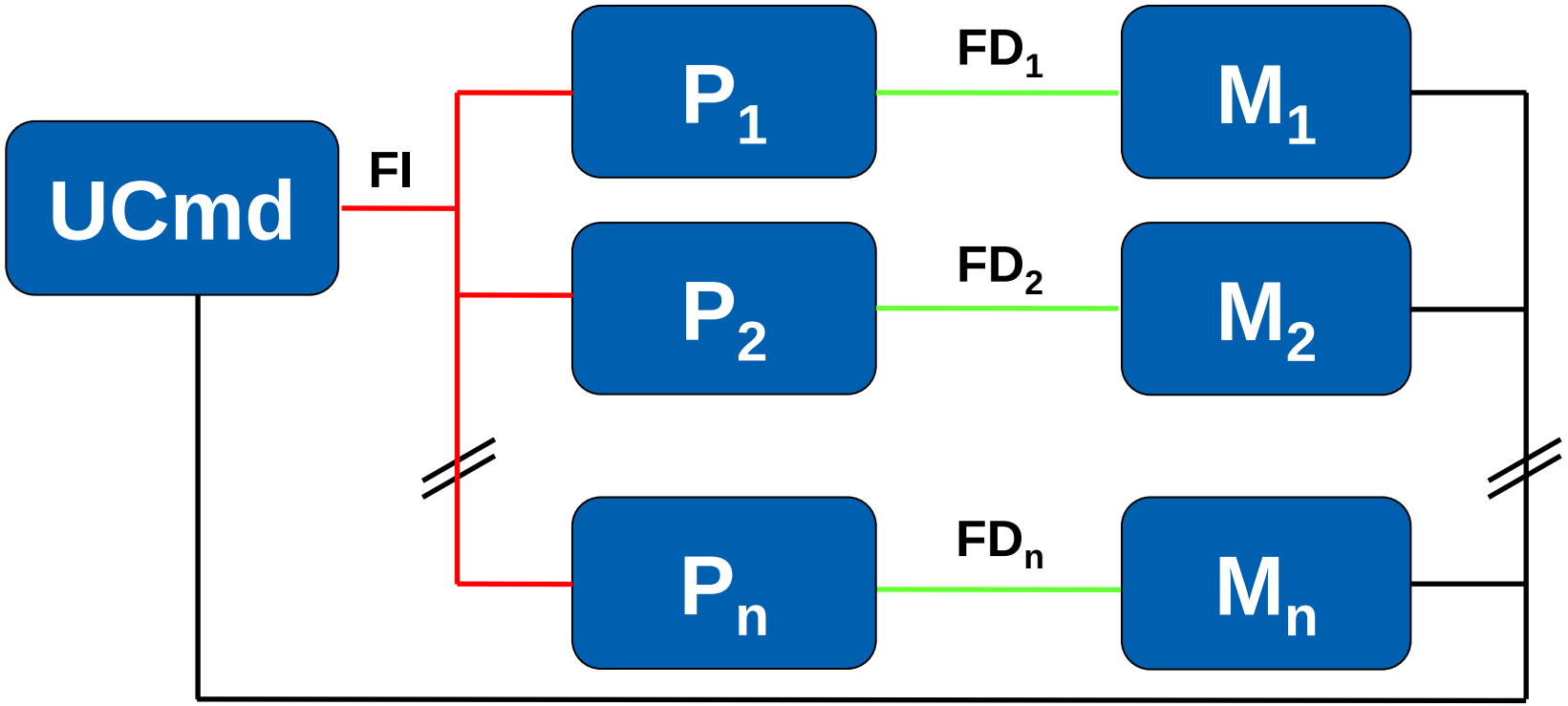


II – SIMD

- SIMD sunt masini vectoriale:
  - O singura UCmd
  - Mai multe procesoare si module de memorie (orice procesor vede orice memorie)

SIMD = 1 FI & n FD

- Toate procesoarele fac aceeasi op impusa de UCmd prin FI

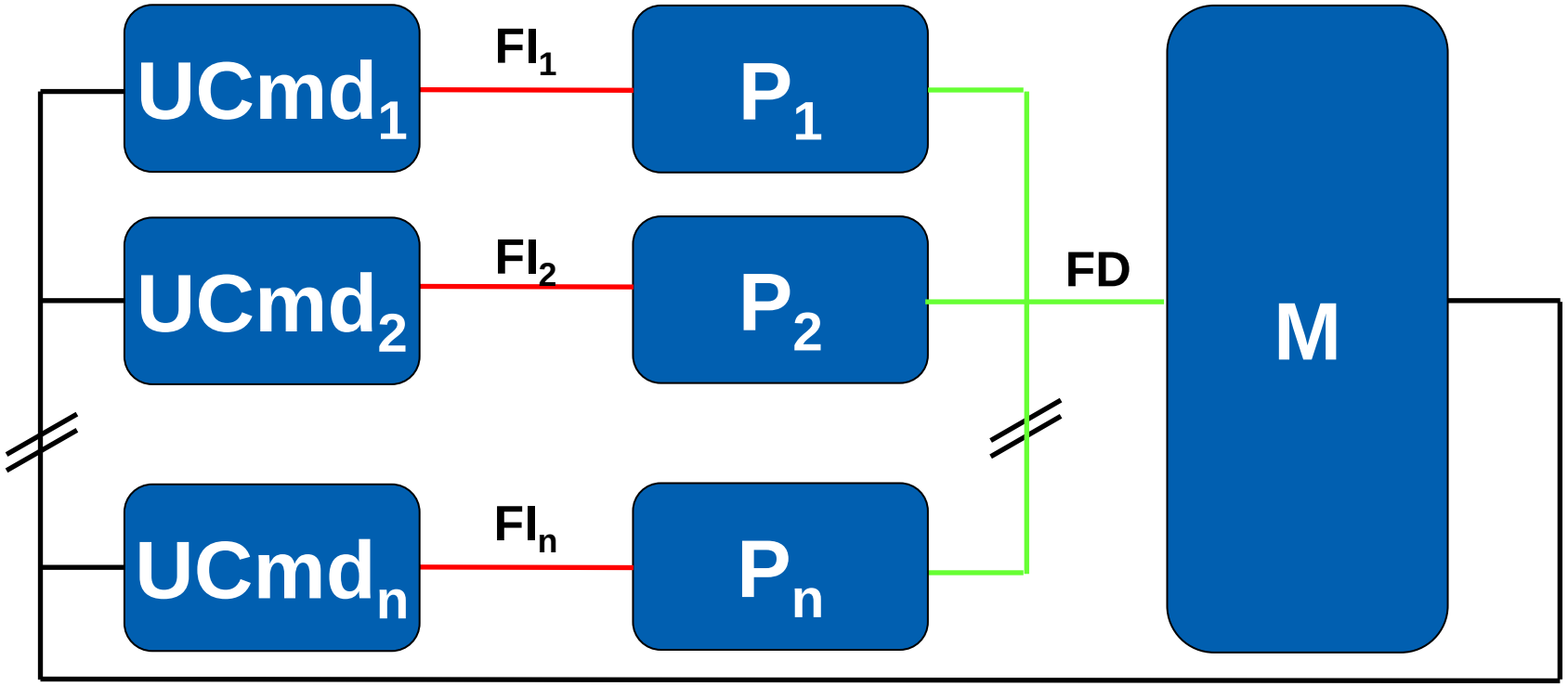


III – MISD

- MISD au:
  - Mai multe UCmd si procesoare
  - Un singur modul de memorie

MISD = n FI & 1 FD

- Domeniu de aplicatie restrans si special: aplicarea altor algoritmi pe aceleasi date (Apps: meteo/evidenta populatiei)

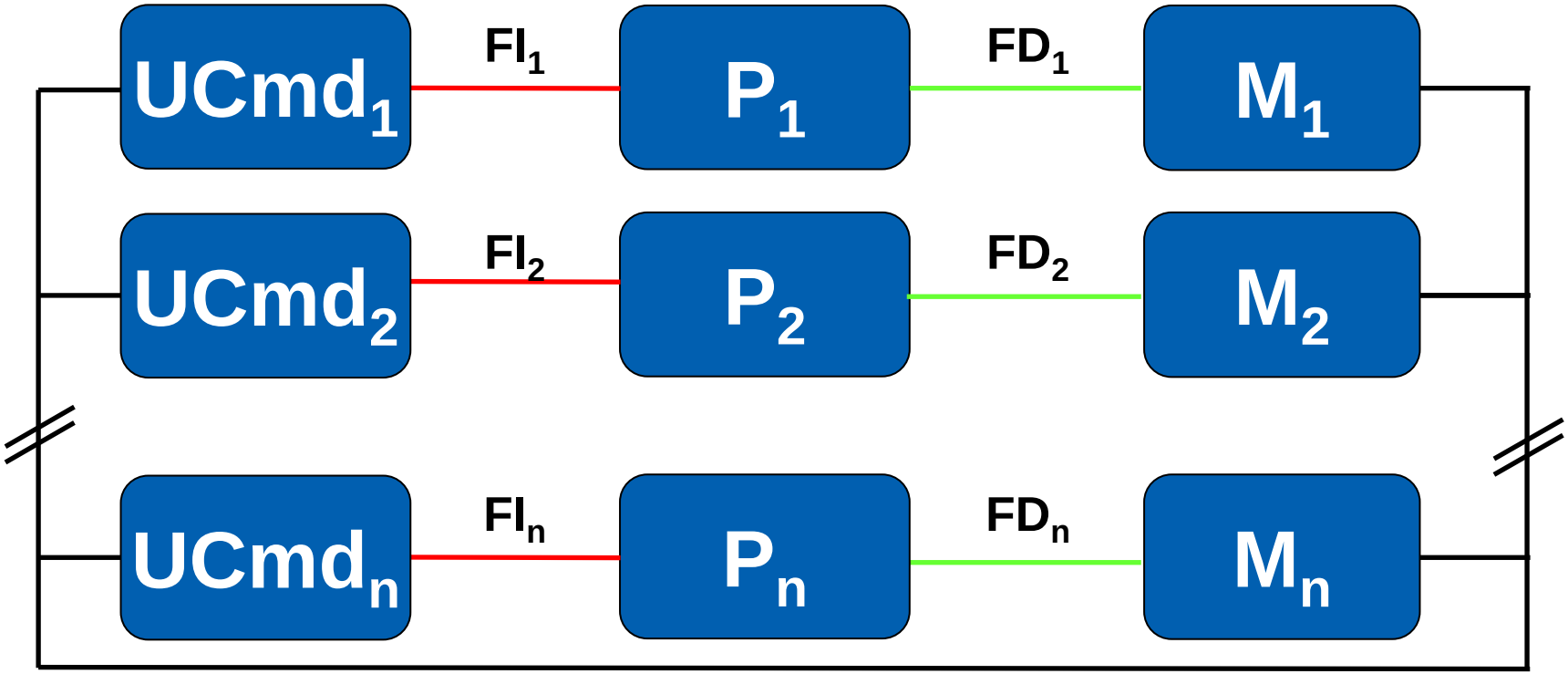


IV – MIMD

- MIMD pot comunica: (P-P sau P-M)

MIMD = n FI & n FD

- Toate procesoarele participa la acelasi program
- Mod programare:
  - Shared memory (strans cuplate) – memorie partajata (e.g. OpenMP)
  - Distributed memory (slab cuplate) – transfer de mesaje (e.g. MPI)



Cuprins

- Structuri de calcul cu prelucrare paralela
- Clasificarea sistemelor de calcul / Flynn: SISD, SIMD, MISD, MIMD
- Exemple de utilizare a structurilor:
  - SISD
  - SIMD
  - MIMD
- Exemplu cu/fara dependenta de date pe sisteme de calcul MIMD

Exemplu de Utilizare – SISD

- Problema: A[n, n], B[n, n], C = A x B
- Pe structuri de calcul SISD – 3 for-uri:

```
for i = 1 to n
    for k = 1 to n
        c_ik = 0
        for j = 1 to n
            c_ik = c_ik + a_ij * b_jk
        end j loop
    end k loop
end i loop
```

- Complexitatea acestui algoritm este… $O(n^3)$! → nesatisfacatoare! (mai ales daca n e mare…)

Exemplu de Utilizare – SIMD

- Aceeasi problema: A[n, n], B[n, n], C = A x B
- Avem n procesoare & toate executa aceeasi instructiune odata → in fiecare calcul se calculeaza cate o linie si nu doar un element
- Considerand ($1 \leq k \leq n$) → se opereaza pentru toti indicii k simultan, adica se calculeaza pe linii

```
for i = 1 to n
    c_ik = 0 (0 ≤ k ≤ n )
    for j = 1 to n
        c_ik = c_ik + a_ij * b_jk (1 ≤ k ≤ n)
    end j loop
end i loop
```

- Complexitatea acestui algoritm este… $O(n^2)$! → considerabil mai bine ca in cazul SISD

Comentarii & Observatii – SIMD

- Fiecare element al matricei produs C, este o suma ce se efectueaza secvential
- Cele n sume se calculeaza apoi in paralel !?
- a_{ij} NU depinde de k → accesul la aceasta memorie se face aproximativ secvential → NU e chiar “sumele se calculeaza in paralel”!
- Solutia: structurile SIMD trebuie sa dispuna de o instructiune de Broadcast & o retea de comutare (RC) ce sa asigure acest Broadcast
- P_j citeste a_{ij} prin RC (constanta a_{ij} e difuzata catre toate procesoarele)
- Concluzie SIMD: probleme mari la comunicatiile inter-procesoare & accesul si organizarea datelor!

Exemplu de Utilizare – MIMD

- Aceeasi problema: A[n, n], B[n, n], C = A x B
- O UCmd/P trebuie sa preia functia de master: organizare si control + partajarea calculelor pe procesoare individuale
- Conway a propus o metoda cu 2 primitive: FORK & JOIN
  - FORK: desface un FI in n FI executabile simultan pe proc indep
  - JOIN: reuneste n FI intr-unul singur cand cele n FI s-au terminat

```
for k = 1 to n – 1(nu si pt el insusi)
   FORK Adr
end k loop
Adr:
   for i = 1 to n
       c_ik = 0 (1 ≤ k ≤ n ) – pe coloane k fix
       for j = 1 to n
             c_ik = c_ik + a_ij * b_jk (1 ≤ k ≤ n)
       end j loop
   end i loop
   JOIN
```

Complexitatea este… $O(n^2)$!

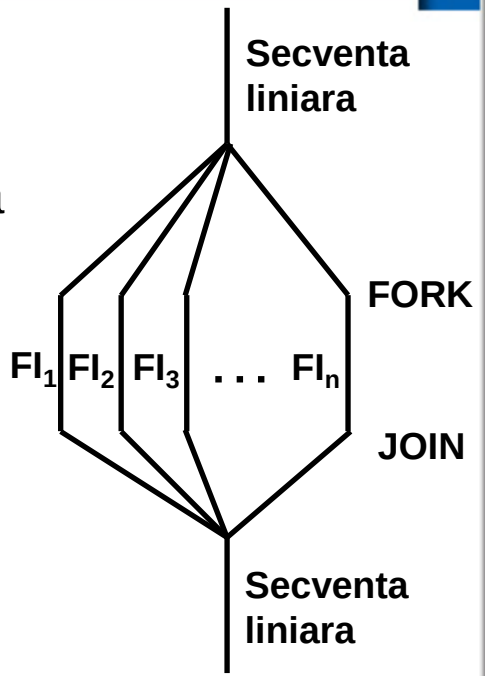
→ la fel ca SIMD

Comentarii & Observatii – MIMD

- In mod deliberat programul pt SIMD a fost scris a.i. actiunile P-urilor sa simuleze actiunile din structura MIMD
- Fiecare P_j calculeaza un C_{ik} in paralel
- Diferente SIMD/MIMD:
  - In SIMD procesoarele se sincronizau instructiune (It) cu It
  - In MIMD nu exista (neaparat) sincronizari intre FI ale P-urilor din structura
  - La SIMD se calculeaza elementele lui C pe linie si coloana (FI unic)
  - La MIMD orice procesoare pot calcula orice elemente din C (fiecare P are un FI propriu!)
- Castigul e acelasi; se pot folosi mai multe perechi FORK/JOIN
- Concluzie MIMD: mai usor de programat & utilizat decat SIMD – dar, mai scump! (Totul se plateste…)

Implementarea FORK & JOIN

- P_1 (master) pregateste taskurile intr-o coada de asteptare impreuna cu contextele asociate lor
- Celelalte procesoare P_2 … P_n inspecteaza coada pana gasesc un task ce asteapta & il executa
- Daca numarul de procesoare = numarul de procese → se executa simultan toate
- Daca numarul de procesoare << numarul de procese → dupa executia unui proces, un procesor preia din coada un nou proces pt exec
- Atentie: aici procesoarele sunt considerate omogene si procesele independente intre ele (caz f rar in realitate!)



- Instructiunea JOIN n:
  - Pentru fiecare FI exista un contor unic ce e initializat cu 0 la inceput
  - Pentru fiecare FI contorul este incrementat si comparat cu n
  - Trebuie ca toate procesele sa se fi terminat (pana la n) si abia apoi se continua cu urmatoarea secventa liniara pe P_1 (master)

Cuprins

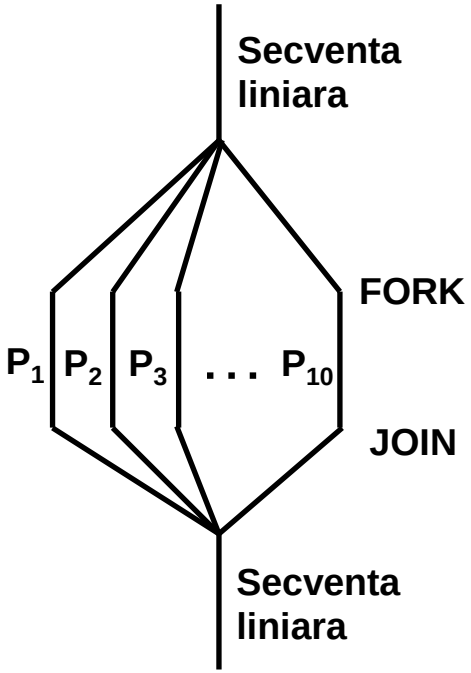
- Structuri de calcul cu prelucrare paralela
- Clasificarea sistemelor de calcul / Flynn: SISD, SIMD, MISD, MIMD
- Exemple de utilizare a structurilor:
  - SISD
  - SIMD
  - MIMD
- Exemplu cu/fara dependenta de date pe sisteme de calcul MIMD

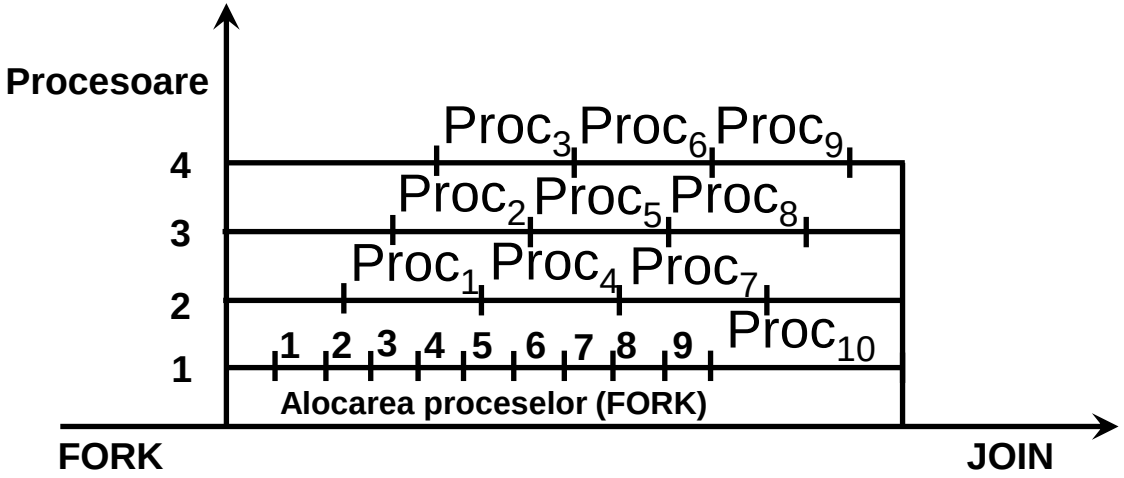
Exemplu fara dependenta de date

- Patru procesoare P_1, P_2, P_3, P_4 si 10 procese $Proc_i$ (i = 1…10)
- Modelul de executie este:

```
for i = 1 to 10
   FORK Adr
end i loop
Adr:
   Proc_i
   JOIN
```

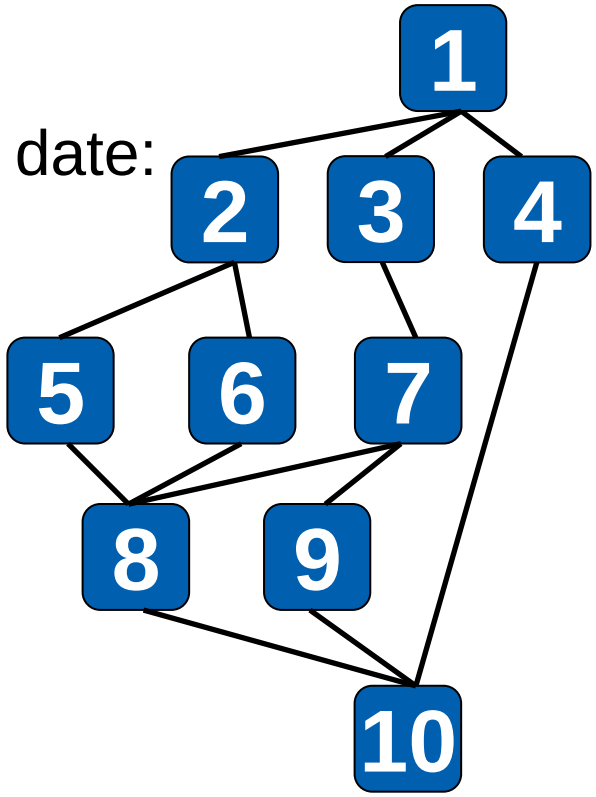
Coada de asteptare a proceselor:

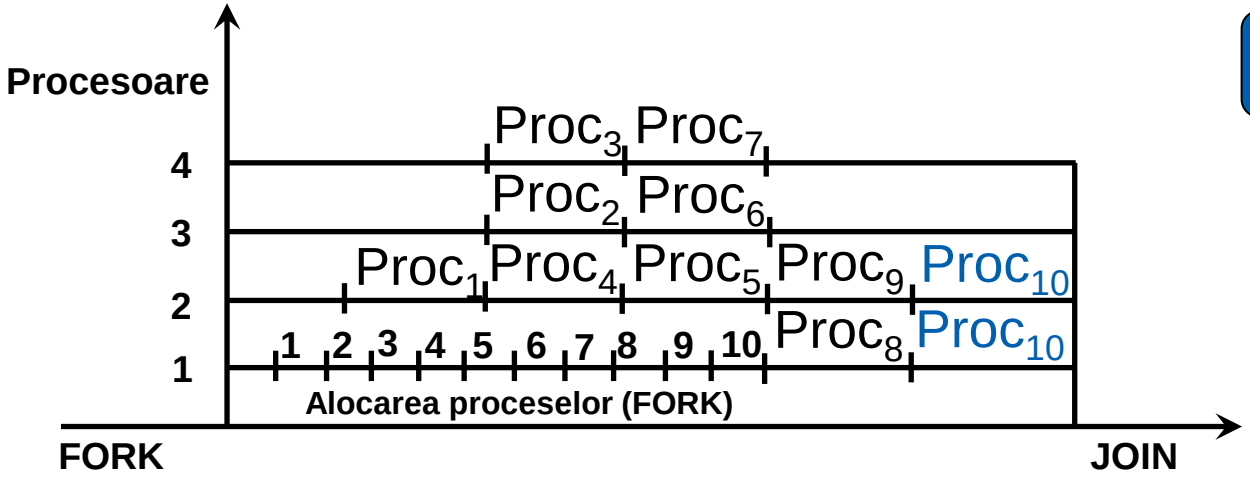




Exemplu cu dependenta de date

- Patru procesoare P_1, P_2, P_3, P_4 si 10 procese $Proc_i$ (i = 1…10)
- Aceeasi coada de asteptare
- Presupunem urmatoarea dependenta de date:
- Lucrurile stau altfel: exista procesoare ce asteapta datorita dependentei de date





Concluzii Dependenta de Date

- In exemplele prezentate nu apar restrictii de precedenta
- Planificarea FIFO nu e necesar cea mai potrivita → alocare statica
- Alocarea dinamica e mai buna → mai ales cand ∃ procesoare specializate
- Programe de uz general → 10% paralelism
- Programe dedicate → 90% paralelism
- Dependenta de date → sincronizari in timp
- Eficienta MIMD depinde de:
  - Eficienta algoritmilor de planificare
  - Consumul de timp impus de precedenta de date
- Problema repartizarii pe procesoare → NP completa!

What Next?

- Q & A?

- Next time:
  - The Cell/B.E.:
    - Motivation
    - Architecture
    - Programming
    - Applications
    - Performance
    - Summary & Conclusions
    - Cell Resources & Documentation

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 7
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

In cursul anterior

- Comutatoare Ierarhice
- Comutatoare Neierarhice
- Exemple de Comutatoare Procesor Memorie
- Interconectarea in Sistemele cu Resurse Multiple
- Retele de Comutare Ierarhice
- Retele de Comutare de tip Delta
- Retele Bazate pe Rutare
- Performantele Retelelor de Comutare

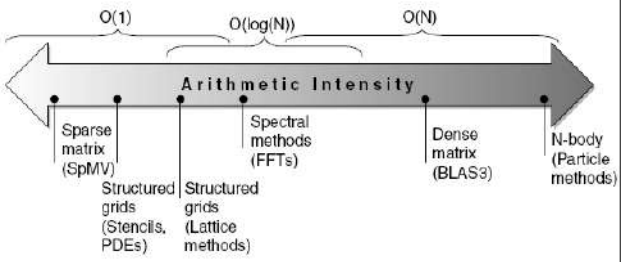
Cuprins

- Roofline Model Reloaded
- Architecture
- Programming
- Performance
- Going Forward

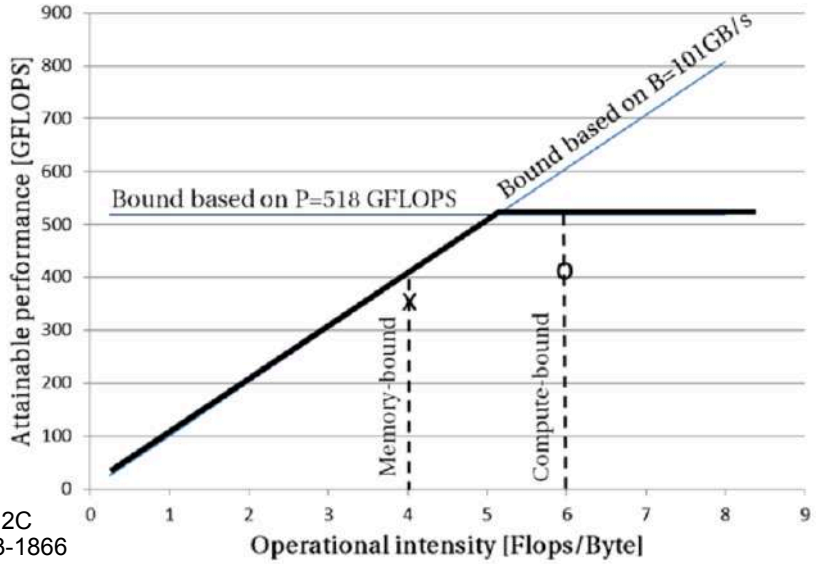
Roofline Model Reloaded

The Roofline Performance Model

- Basic Idea
  - Plot peak floating-point throughput as a function of arithmetic intensity
  - Ties together floating-point performance and memory performance for a target machine
- Arithmetic Intensity
  - Floating-point operations per byte read



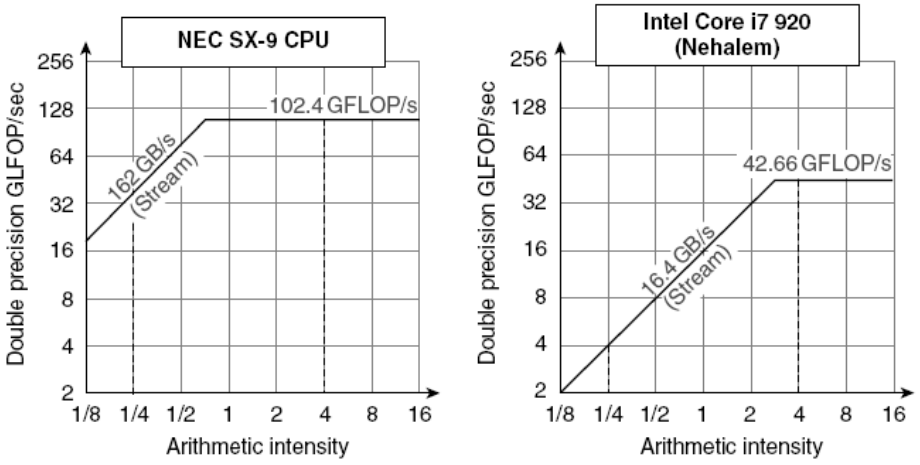
Modelul Roofline



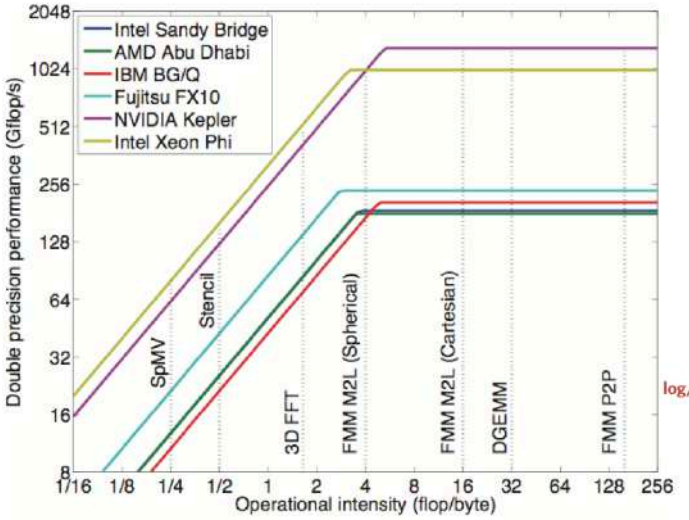
Intel XeonE5-2697v2 12C
2.7GHz cu memorie DDR3-1866

Roofline Performance Examples

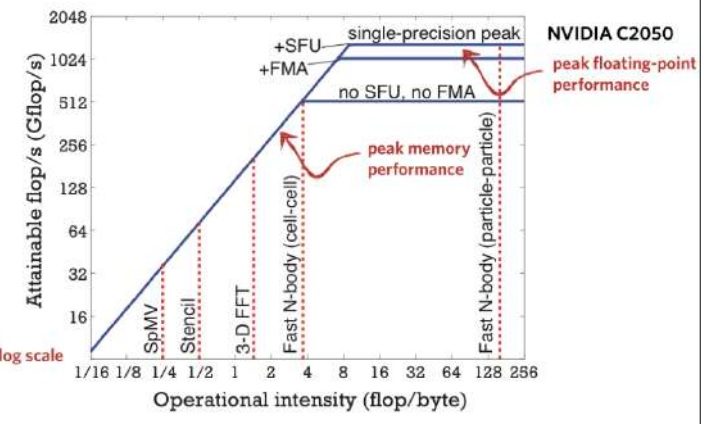
- Attainable GFLOPs/sec Min = (Peak Memory BW × Arithmetic Intensity, Peak Floating Point Performance)



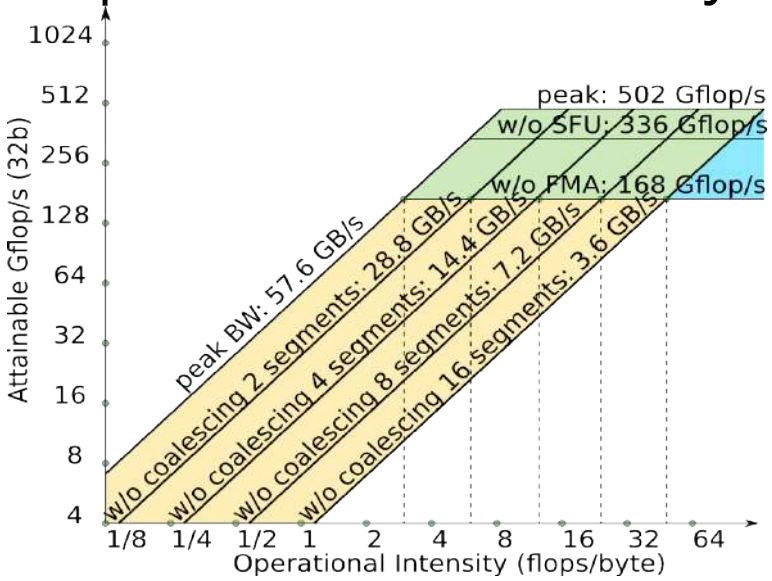
Roofline Performance Examples



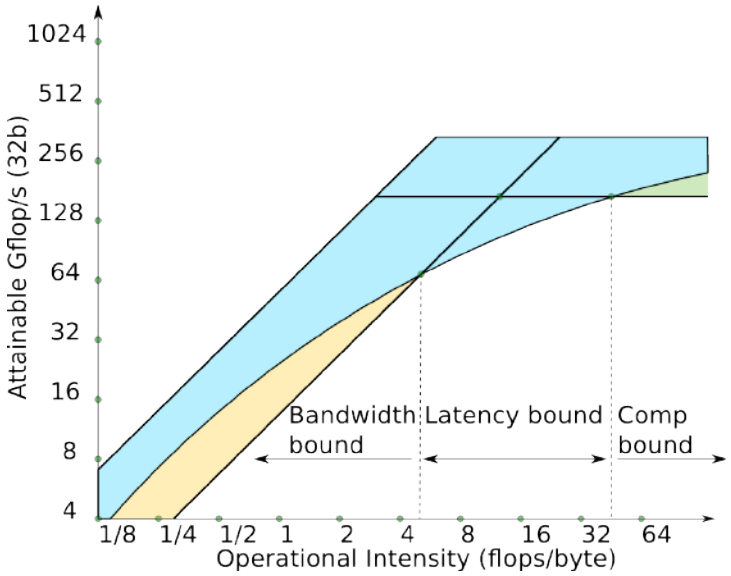
log/log scale



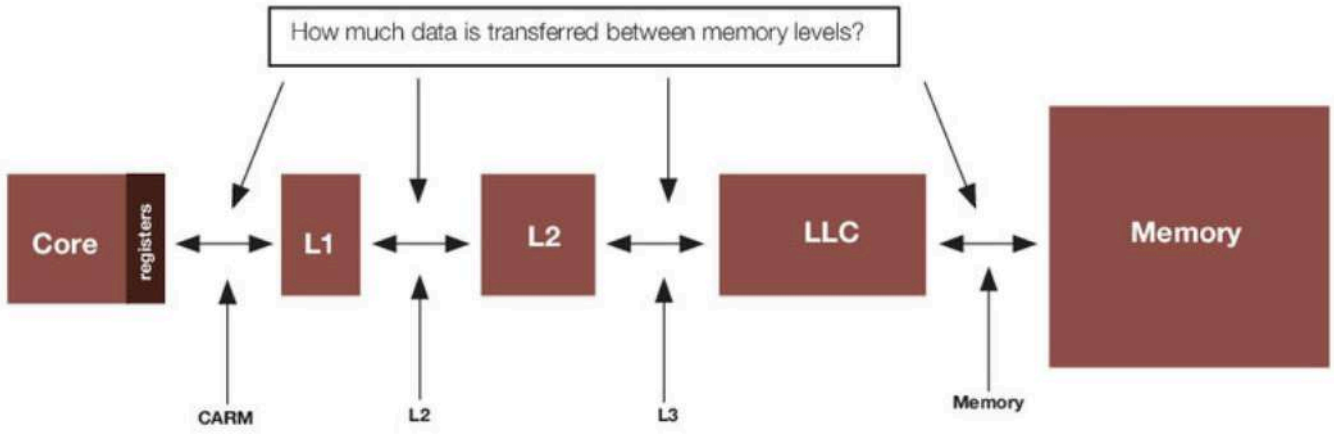
Optimization is Key



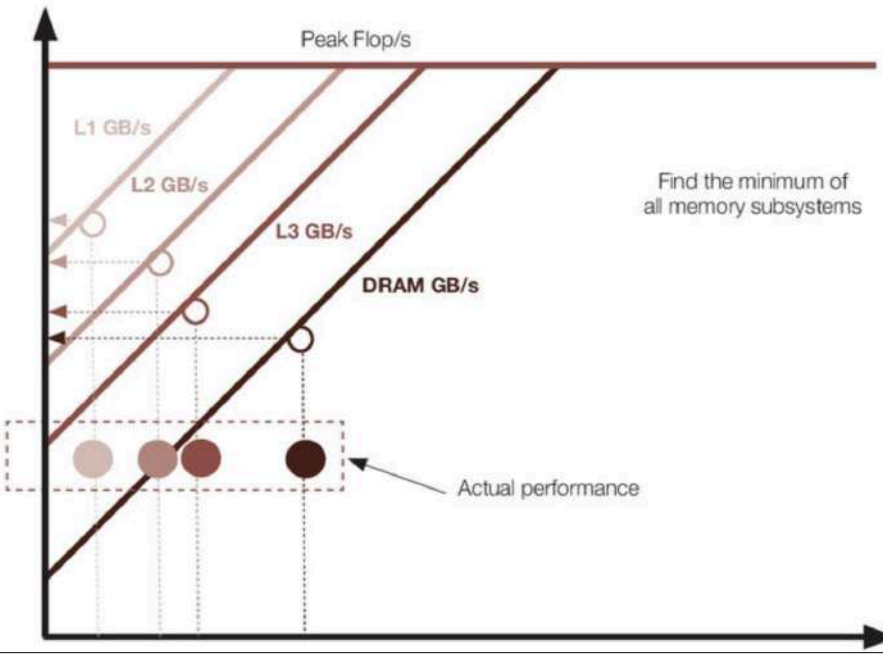
Bottlenecks



Cache-Aware Roofline Model



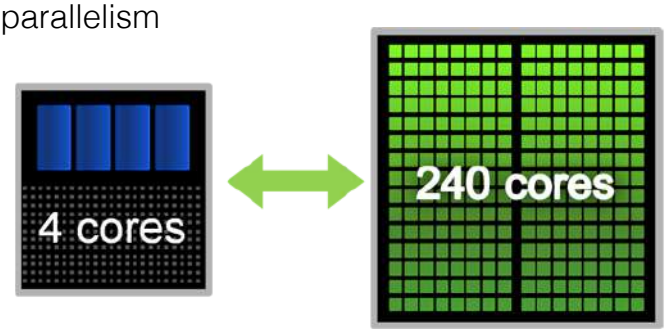
Performance Limitations



Architecture

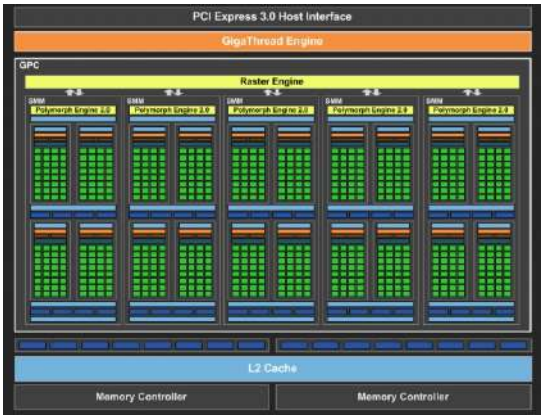
CPUs & GPUs

- Typically GPUs and CPUs coexist in a heterogeneous setting
- “Less” computationally intensive part runs on CPU
  - Coarse-grained parallelism
- More intensive parts run on GPU
  - Fine-grained parallelism

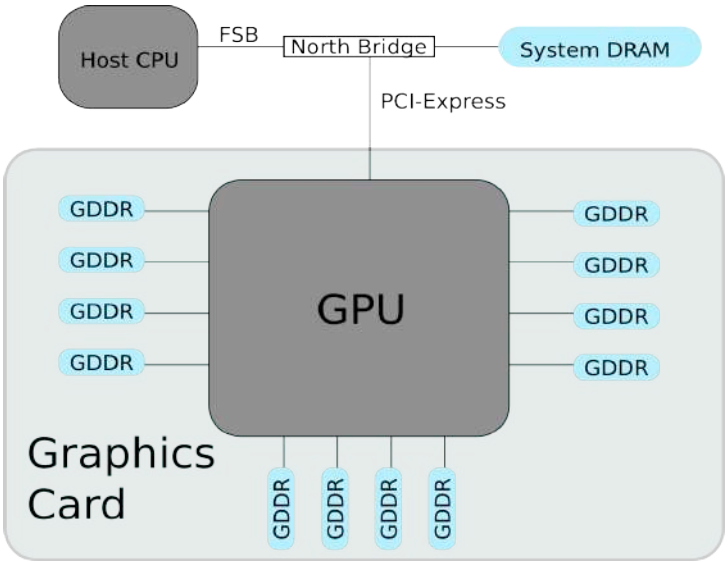


NVIDIA Kepler K40

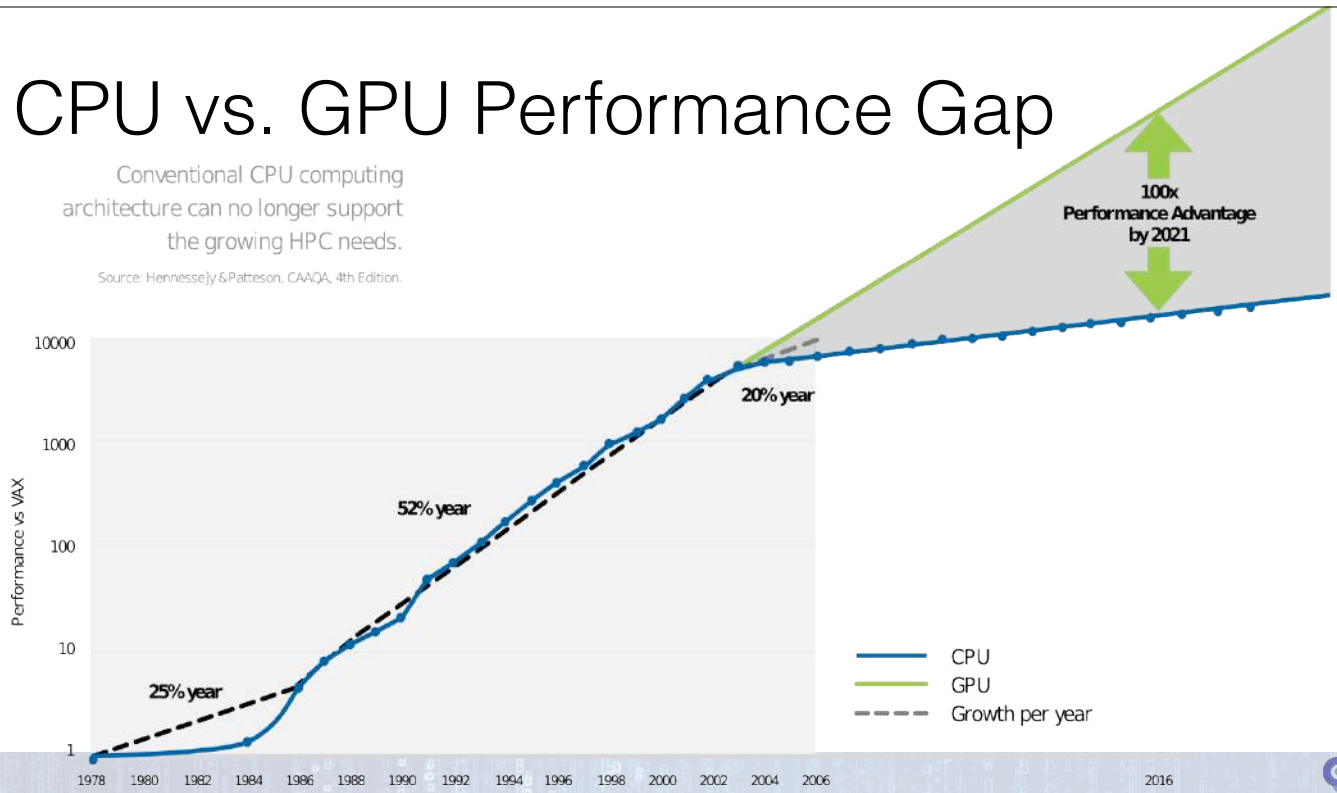
- 2880 Streaming Processors/cores (SPs) organized as 15 Streaming Multiprocessors (SMs)
- Each SM contains 192 cores
- Memory size of the GPU system: 12 GB
- Clock speed of a core: 745 MHz



System Architecture

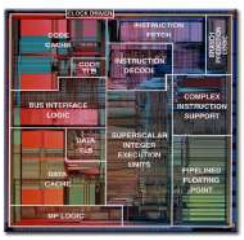


CPU vs. GPU Performance Gap

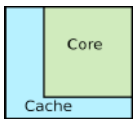


Pentium Processors Evolution

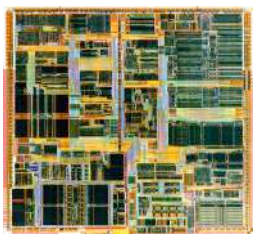
Pentium I

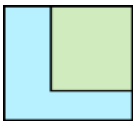


Chip area breakdown

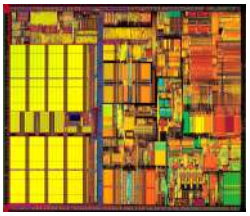


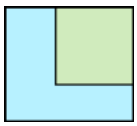
Pentium II



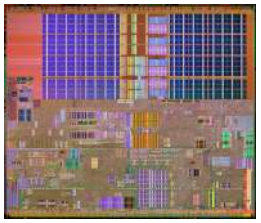


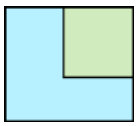
Pentium III





Pentium IV

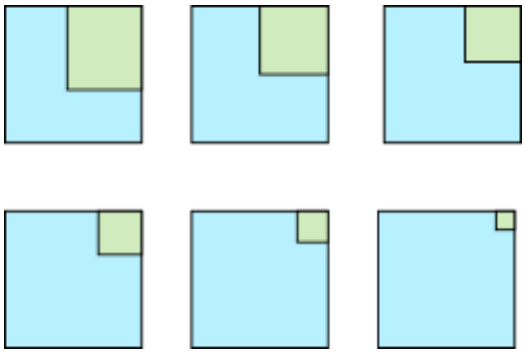




Q: What can you observe? Why?

Extrapolation of Single Core CPUs

If we extrapolate the trend, in a few generations, Pentium will look like this:

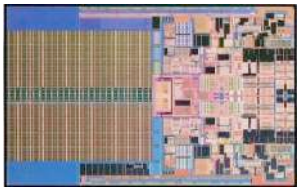


Of course, we know it did not happen.

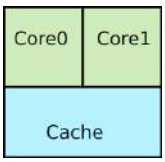
Q: What happened instead? Why?

Multi-Core CPUs Evolution

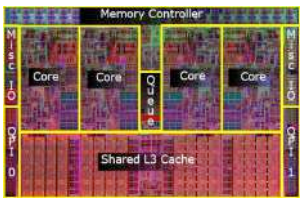
Penny

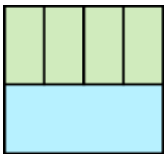


Chip area breakdown

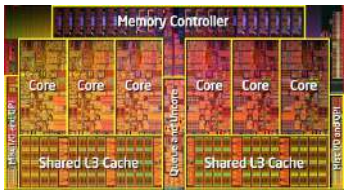


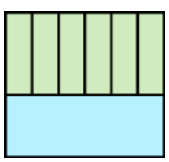
Bloomfield



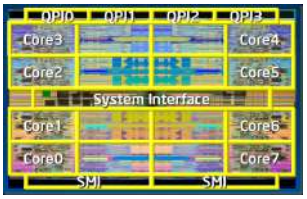


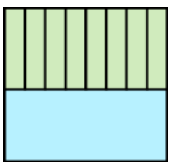
Gulftown





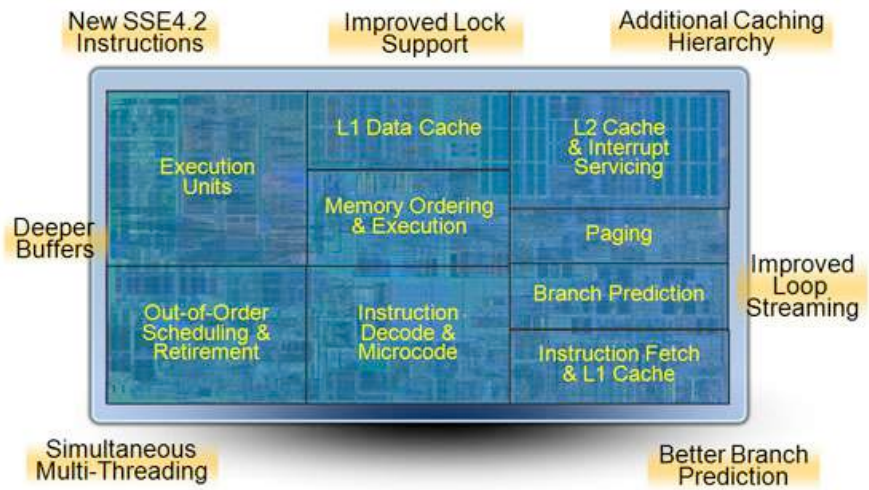
Beckton

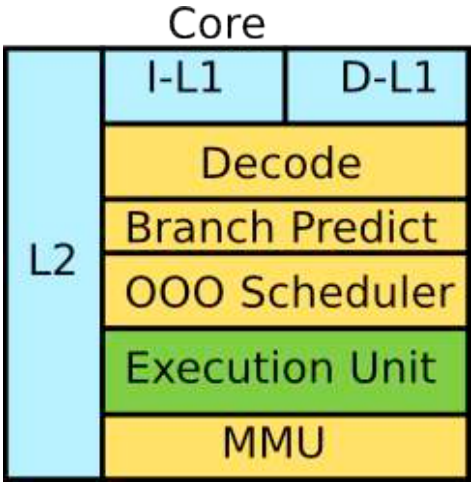




Q: What can you observe? Why?

Let's Zoom(in)to a Core

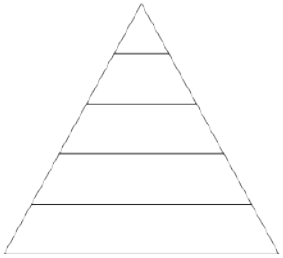




Less than **10%** of total chip area is used for the real execution.

Q: Why?

Revisiting the Memory Hierarchy



	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	1K	10	1	1000
L1	32K	20	5	100
L2	256K	100	10	100
L3	8M	200	50	100
Off-chip	4G	2000	100	10

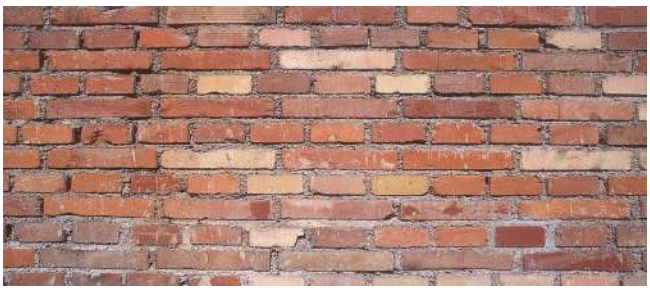
Notes on Energy at 45nm:
64-bit Int ADD takes about 1 pJ.
64-bit FP FMA takes about 200 pJ.

It seems we can not further increase the computational density!

The Brick Wall

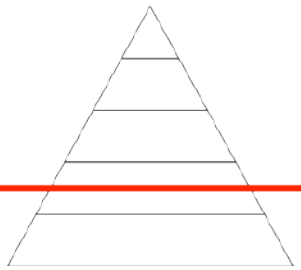
Power Wall: power expensive, transistors “free”
Memory Wall: Memory slow, multiplies fast
ILP Wall: diminishing returns on more ILP HW

Power Wall + Memory Wall + ILP Wall = Brick Wall



Q: How to Break the Wall?
Exploit the parallelism inside the application…

Step 1: Trade Latency with Throughput

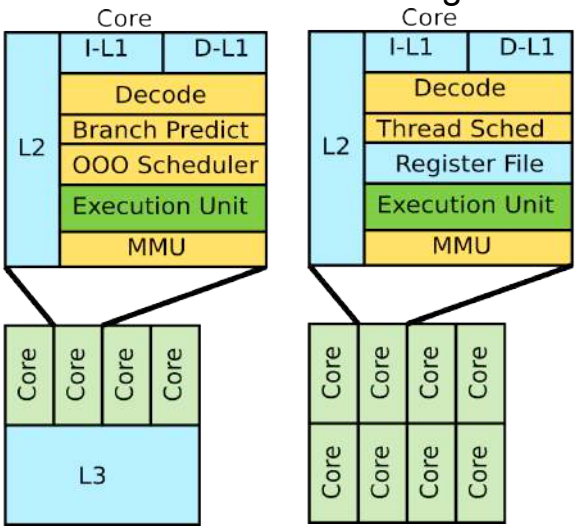


	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	~~1K~~ 16K	~~10~~ 20	~~1~~ 2~3	1000
L1	32K	20	5	100
L2	256K	100	10	100
~~L3~~	~~8M~~	~~200~~	~~50~~	~~100~~
Off-chip	4G	2000	100	10

Hide the memory latency through fine-grained interleaved threading

Fine-Grained Interleaved Threading

Without and with fine-grained interleaved threading



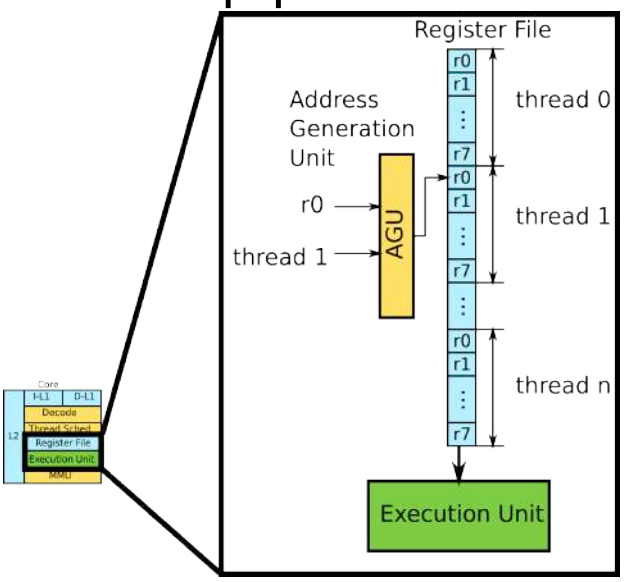
Pros:
Reduce cache size
No branch predictor
No OOO scheduler

Cons:
Register pressure
Thread scheduler
Requires huge parallelism

OOO = Out-of-order Execution

Hardware Support

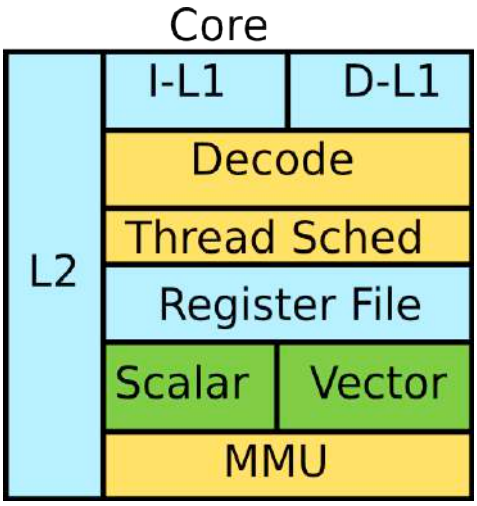
Register file supports zero overhead context switch between interleaved threads



Can we make Further Improvements?

- Reducing large cache gives 2x computational density.
- Q: Can we make further improvements?

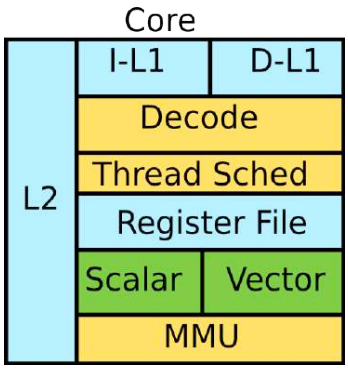
Hint:
We have only utilized thread level parallelism (TLP) so far.



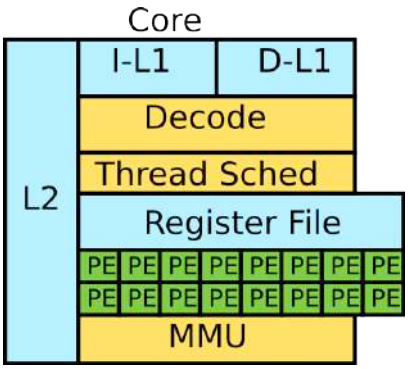
Step 2: Single Instruction Multiple Data

GPU uses wide SIMD: 8/16/24/... processing elements (PEs)
CPU uses short SIMD: usually has vector width of just 4

SSE has 4 data lanes

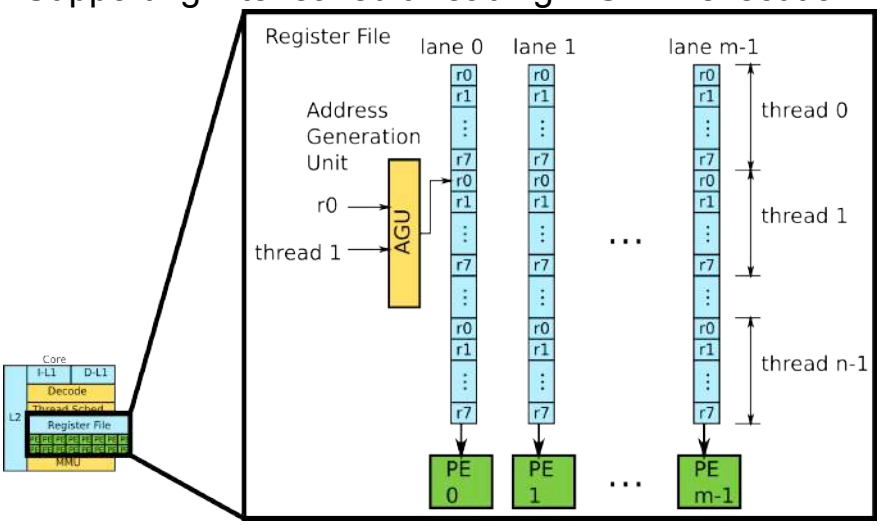


GPU has 8/16/24/... data lanes



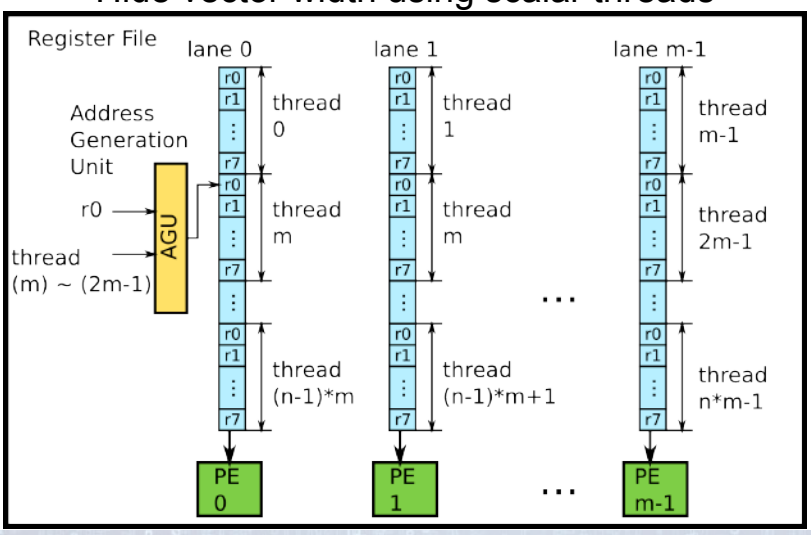
Hardware Support

Supporting interleaved threading + SIMD execution



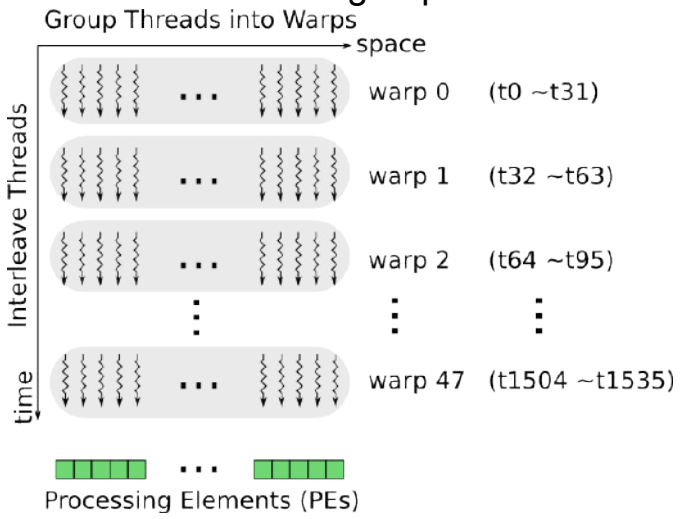
Single Instruction Multiple Thread (SIMT)

Hide vector width using scalar threads



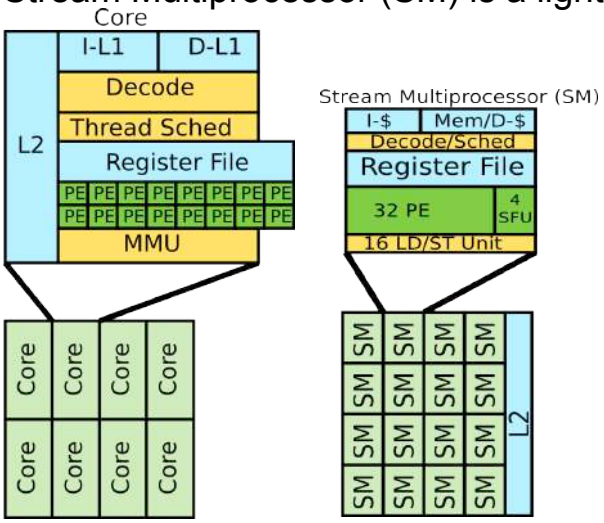
Example of SIMT Execution

Assume 32 threads are grouped into one warp.



Step 3: Simple Core

The Stream Multiprocessor (SM) is a light weight core compared to IA core



Light weight PE:

Fused Multiply Add (FMA)

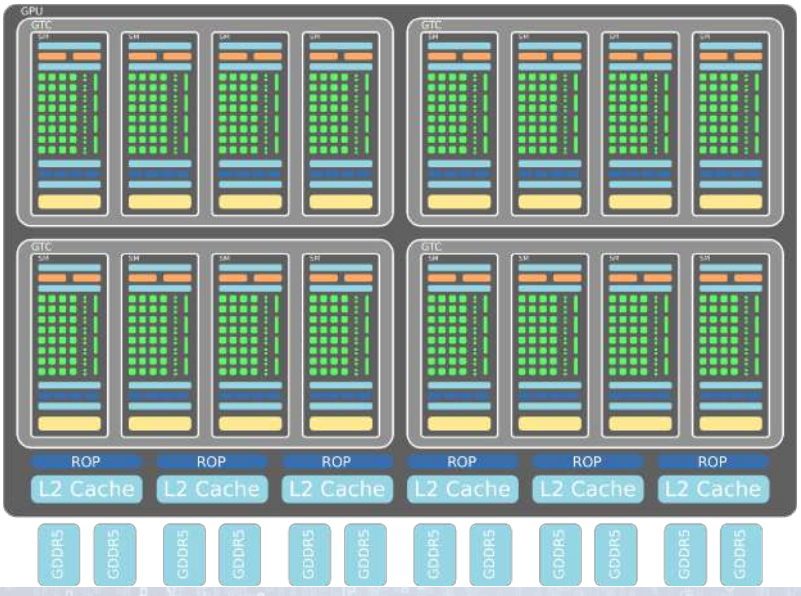
SFU:

Special Function Unit

NVIDIA's Motivation of a Simple Core

"This [multiple IA-core] approach is analogous to trying to build an airplane by putting wings on a train."
Bill Dally, NVIDIA

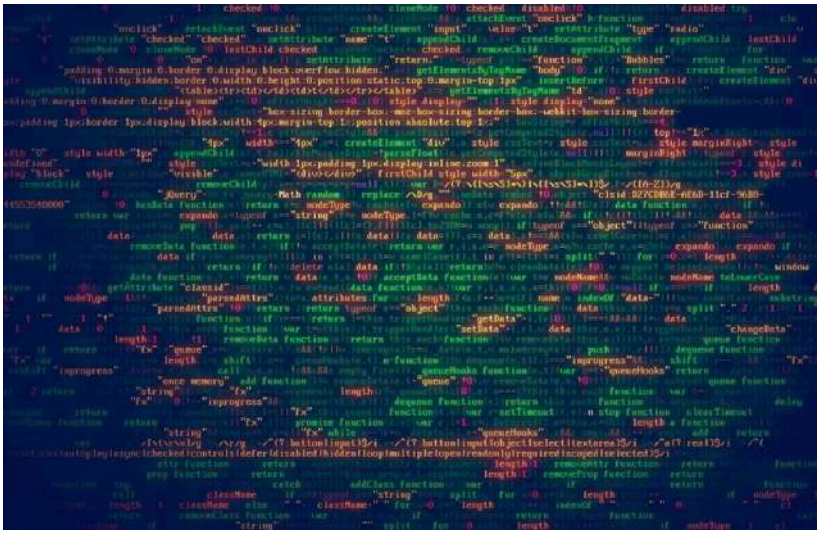
How Do We Reach Here?



NVIDIA Fermi, 512 Processing Elements (PEs)

Throughput Oriented Architectures

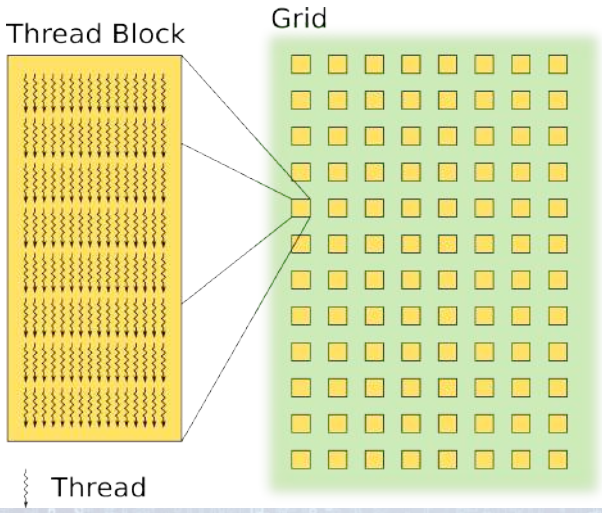
- Fine-grained interleaved threading (~2x comp density)
- SIMD/SIMT (>10x comp density)
- Simple core (~2x comp density)



Programming

Multi-Threading at another Level

- Massive number (>10000) of light-weight threads

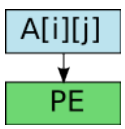


Vector Programming

- Vector width is exposed to programmers

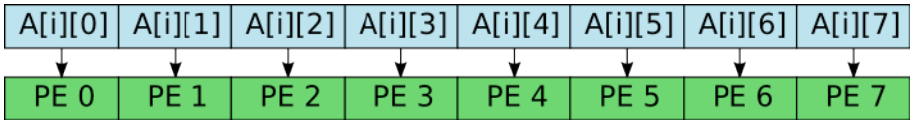
Scalar program

```
float A[4][8];
do-all(i=0;i<4;i++){
    do-all(j=0;j<8;j++){
        A[i][j]++;
    }
}
```



Vector program (vector width of 8)

```
float A[4][8];

do-all(i=0;i<4;i++){
    movups xmm0, [ &A[i][0] ]
    incps xmm0
    movups [ &A[i][0] ], xmm0
}
```



GPU Program

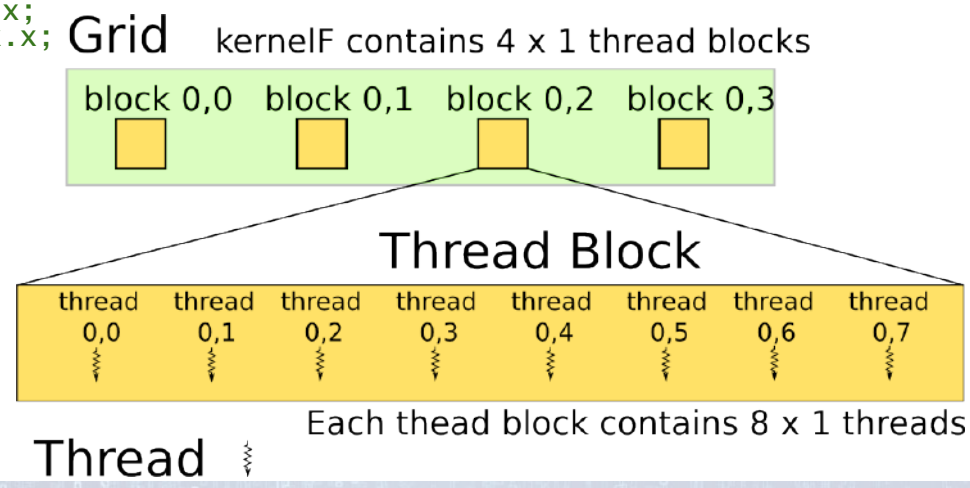
- Program expresses data level parallelism (DLP) in terms of thread level parallelism (TLP)
- Hardware converts TLP into DLP at run time

```
Scalar program

float A[4][8];
do-all(i=0;i<4;i++){
    do-all(j=0;j<8;j++){
        A[i][j]++;
    }
}
```

```
GPU program

float A[4][8];

kernelF<<<(4,1),(8,1)>>>(A);

__device__    kernelF(A){
    i = blockIdx.x;
    j = threadIdx.x;
    A[i][j]++;
}
```

Two Levels of Thread Hierarchy

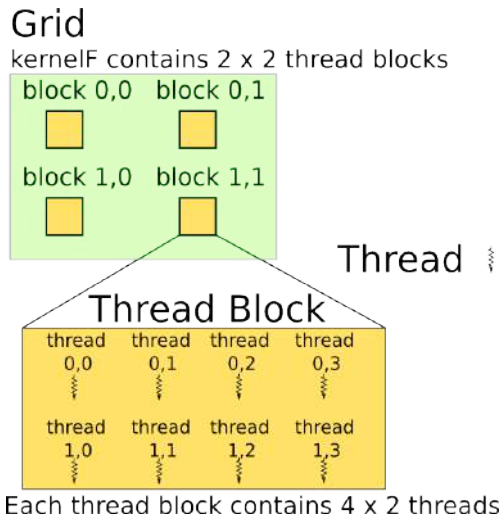
```
kernelF<<<(4,1),(8,1)>>>(A);

__device__    kernelF(A){
    i = blockIdx.x;
    j = threadIdx.x;
    A[i][j]++;
}
```



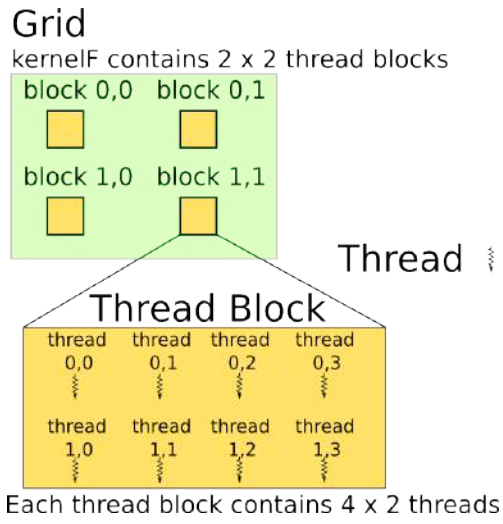
Multi-dimension Thread and Block ID

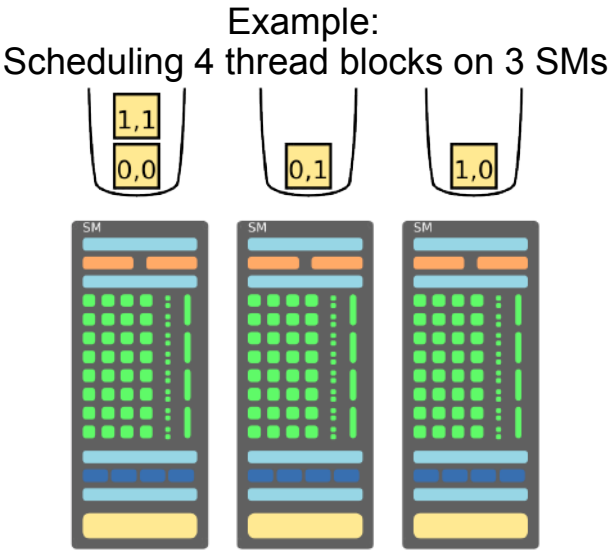
- Both grid and thread block can have two dimensional index

```
kernelF<<<(2,2),(4,2)>>>(A);

__device__    kernelF(A){
    i = blockDim.x * blockIdx.y
        + blockIdx.x;
    j = threadDim.x * threadIdx.y
        + threadIdx.x;
    A[i][j]++;
}
```



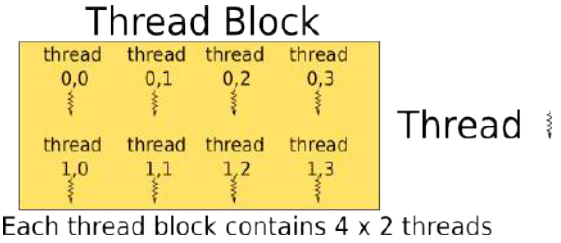
Scheduling Thread Blocks on SM



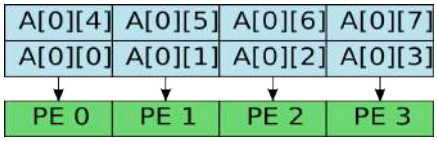


Executing Thread Block on SM

```
kernelF<<<(2,2),(4,2)>>>(A);

__device__    kernelF(A){
    i = blockDim.x * blockIdx.y
        + blockIdx.x;
    j = threadDim.x * threadIdx.y
        + threadIdx.x;
    A[i][j]++;
}
```

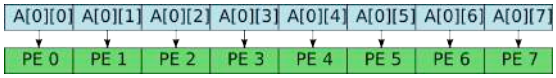


Executed on machine with width of 4:

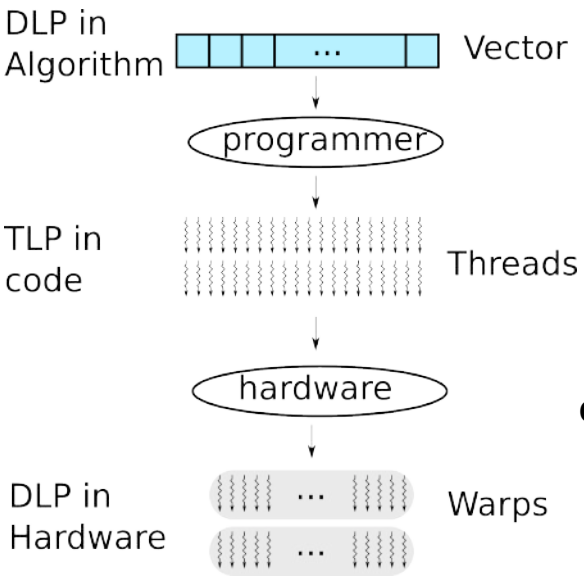


Note: The number of Processing Elements (PEs) is transparent to programmer

Executed on machine with width of 8:



Programmers Think in Threads



Q: Why make this hassle?

Why Use Thread instead of Vector?

- Thread **Pros**:
  - Portability: Machine width is transparent in ISA (Instruction Set Architecture)
  - Productivity: Programmers don't take care of vector width on processor
- Thread **Cons**:
  - Manual sync: Give up lock-step within vector
  - Scheduling of threads could be inefficient
  - Debug: "Threads considered harmful". Thread programming is notoriously hard to debug
- Programmers explicitly express DLP in terms of TLP
- Programmers explicitly manage memory hierarchy

Hierarchical Parallelism

- Parallel computations arranged as grids
- One grid executes after another
- Grid consists of blocks
- Blocks assigned to SM
  - A single block assigned to a single SM
  - Multiple blocks can be assigned to a SM
- Block consists of elements
- Elements computed by threads

Thread Blocks

- Thread block
  - Array of concurrent threads that execute the same program and can cooperate to compute the result
  - Consists of up to 1024 threads (K40)
  - Has shape and dimensions (1d, 2d or 3d) for threads
- A thread ID has corresponding 1d, 2d or 3d indices
- Each SM executes up to 16 thread blocks concurrently (K40)
- Threads of a thread block share memory
- Maximum threads per SM: 2048 (K40)

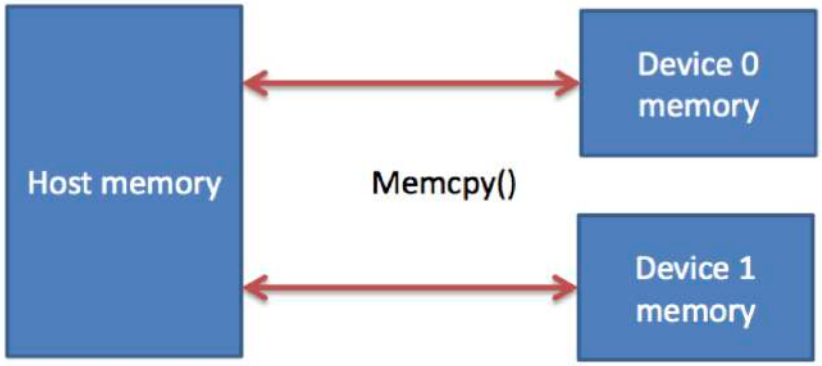
General GPU Programming Steps

1. Initialize the data on the CPU / space on the GPU
2. Copy data from CPU to GPU
3. Compute on GPU
4. Copy data back from GPU to CPU

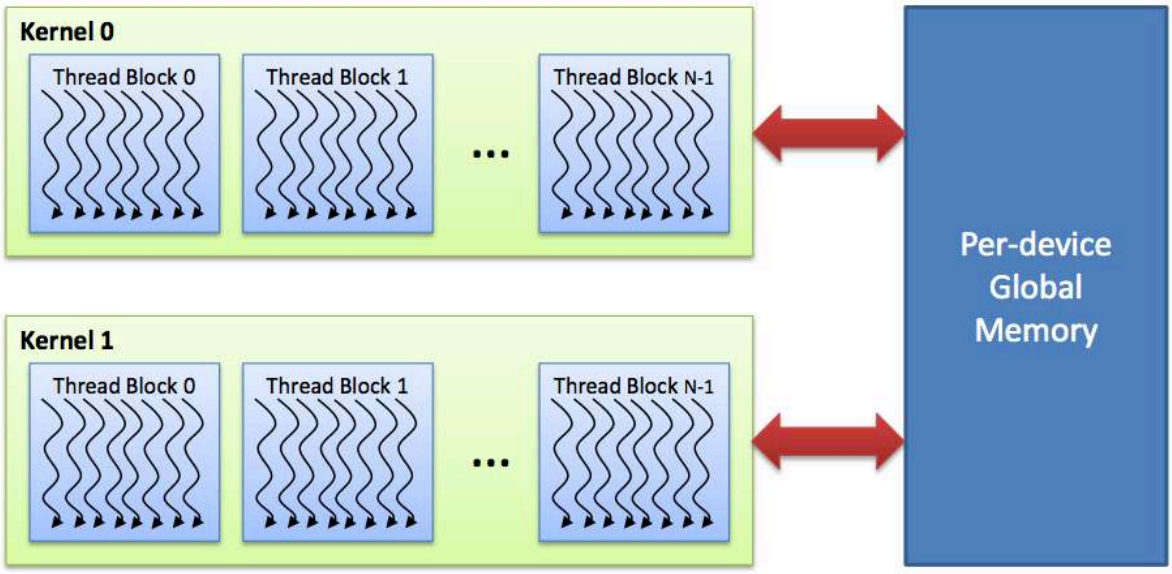
- By default, execution on host doesn't wait for kernel to finish
- General rules:
  - **Minimize** data transfer between CPU & GPU
  - **Maximize** number of threads on GPU

GPU Memory Hierarchy

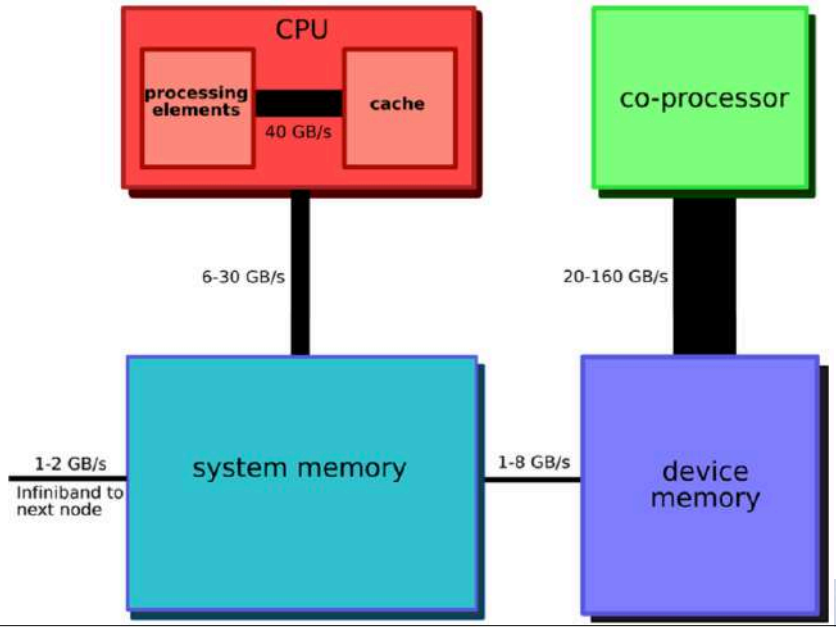
- Global (device) memory
  - Accessible by all threads as well as host (CPU) – Data lifetime is from allocation to deallocation



GPU Memory Hierarchy

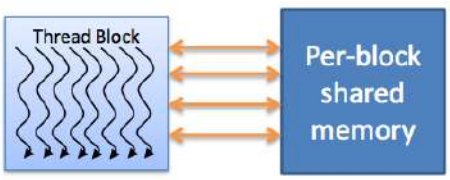


Bandwidth in a CPU-GPU System



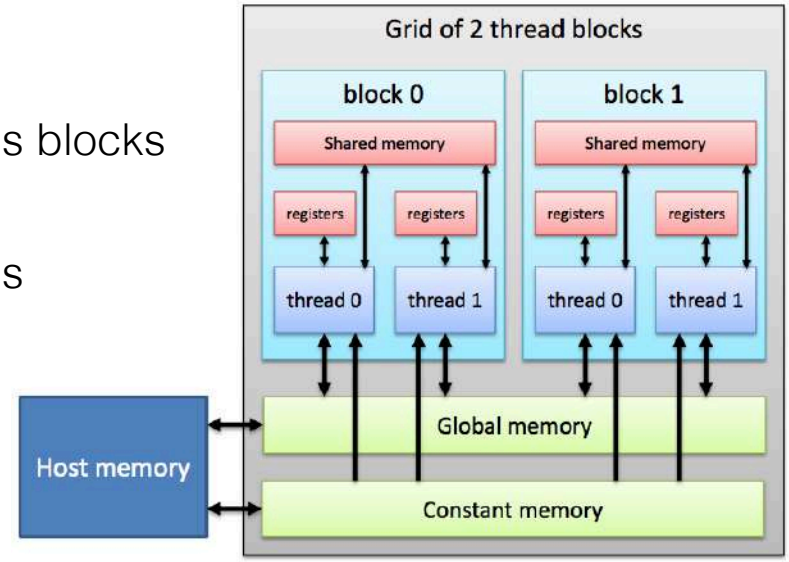
GPU Memory Hierarchy

- Local storage
  - Each thread has own its local storage
  - Mostly registers - managed by the compiler
  - Data lifetime = thread lifetime
- Shared memory
  - Each thread block has its own shared memory
    - Accessible only by threads within that block
  - Data lifetime = block lifetime

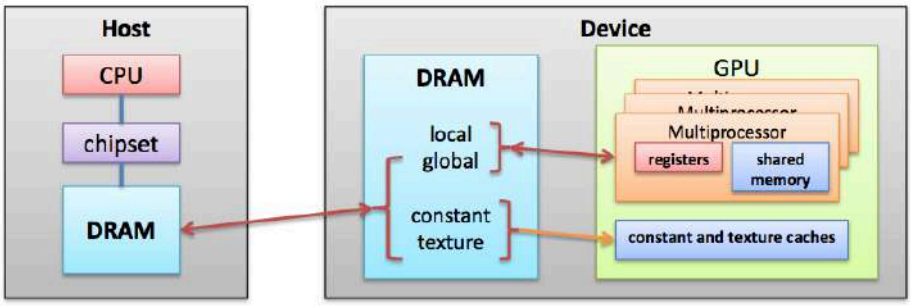


GPU Memory Hierarchy

- 1D grid
  - 2 threads blocks
- 1D block
  - 2 threads

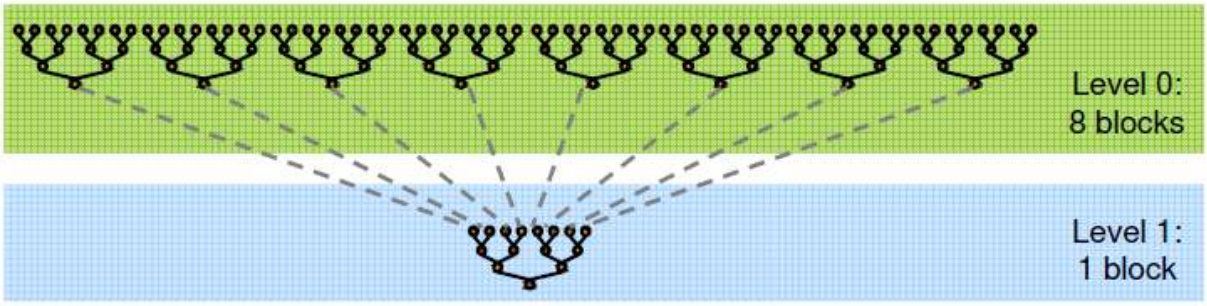


GPU Memory Hierarchy



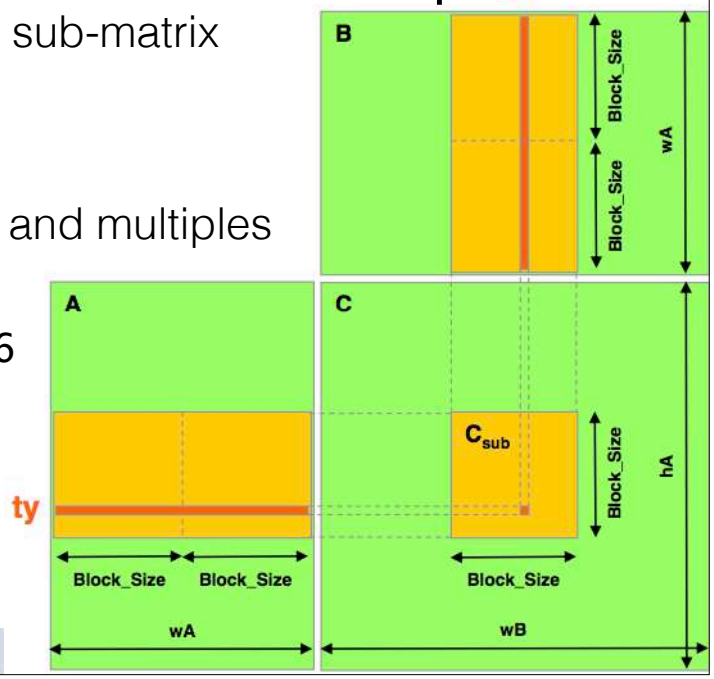
Memory	Location	Cached	Access	Scope	Lifetime
Register	On-chip	N/A	R/W	One thread	Thread
Local	Off-chip	No	R/W	One thread	Thread
Shared	On-chip	N/A	R/W	All threads in a block	Block
Global	Off-chip	No	R/W	All threads + host	Application
Constant	Off-chip	Yes	R	All threads + host	Application
Texture	Off-chip	Yes	R	All threads + host	Application

Reduction Example

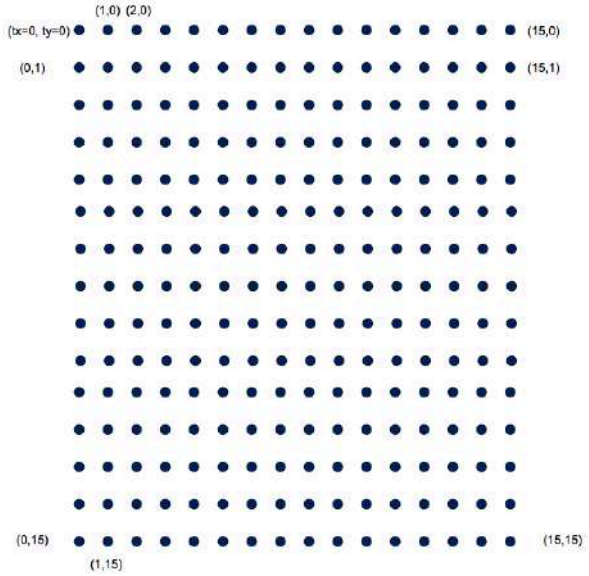


Matrix Multiplication Example

- One block computes one square sub-matrix C_{sub} of size `Block_Size`
- One thread one entry of C_{sub}
- Assume that of A & B are square and multiples of `Block_Size`
  - We will have `Block_Size=16x16`

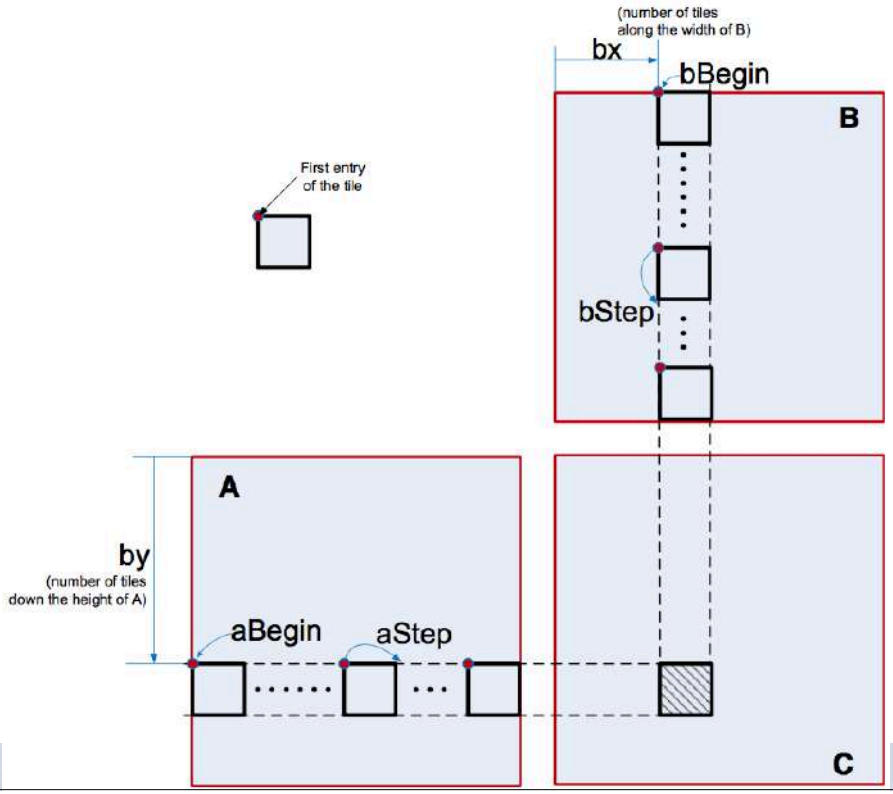


A Block of 16 X 16 Threads



Matrix Multiplication - Host Code

```
// Thread block size
#define BLOCK_SIZE 16

// Forward declaration of the device multiplication func.
__global__ void Muld(float*, float*, int, int,
float*);

// Host multiplication function
// Compute C = A * B
// hA is the height of A
// wA is the width of A
// wB is the width of B
void Mul(const float* A, const float* B, int hA, int
wA, int wB, float* C) {
 int size;

 // Load A and B to the device
 float* Ad;
 size = hA * wA * sizeof(float);
 cudaMalloc((void**)&Ad, size);
 cudaMemcpy(Ad, A, size, cudaMemcpyHostToDevice);

 float* Bd;
 size = wA * wB * sizeof(float);
 cudaMalloc((void**)&Bd, size);
 cudaMemcpy(Bd, B, size, cudaMemcpyHostToDevice);

 // Allocate C on the device
 float* Cd;
 size = hA * wB * sizeof(float);
 cudaMalloc((void**)&Cd, size);

// Compute the execution configuration assuming
// the matrix dimensions are multiples of BLOCK_SIZE
 dim3 dimBlock (BLOCK_SIZE, BLOCK_SIZE);
 dim3 dimGrid (wB/dimBlock.x, hA/dimBlock.y);

 // Launch the device computation
 Muld<<<dimGrid, dimBlock>>>(Ad, Bd, wA, wB, Cd);

 // Read C from the device
 cudaMemcpy(C, Cd, size, cudaMemcpyDeviceToHost);

 // Free device memory
 cudaFree(Ad);
 cudaFree(Bd);
 cudaFree(Cd);
}
```



Matrix Multiplication - Device Code

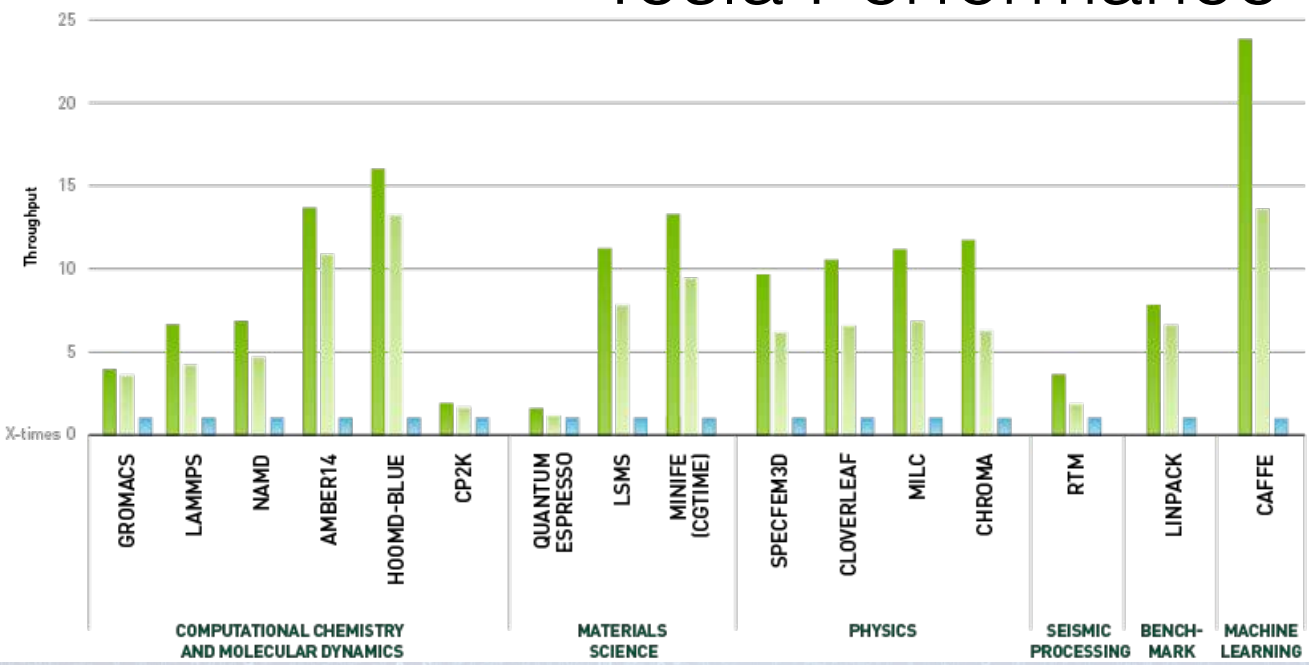
```
// Device multiplication function called by Mul()
// Compute C = A * B
// wA is the width of A
// wB is the width of B
__global__ void Muld(float* A, float* B, int wA,
int wB, float* C){

  // Block index
  int bx = blockIdx.x;  // the B & C matrix sub-block column index
  int by = blockIdx.y;  // the A & C matrix sub-block row index

  // Thread index
  int tx = threadIdx.x;  // the column index in the sub-block
  int ty = threadIdy.y;  // the row index in the sub-block

  // Index of the first sub-matrix of A processed by the block
  int aBegin = wA * BLOCK_SIZE * by;
  // Index of the last sub-matrix of A processed by the block
  int aEnd = aBegin + wA - 1;
  // Step size used to iterate through the sub-matrices of A
  int aStep = BLOCK_SIZE;
  // Index of the first sub-matrix of B processed by the block
  int bBegin = BLOCK_SIZE * bx;
  // Step size used to iterate through the sub-matrices of B
  int bStep = BLOCK_SIZE * wB;
  // The element of the block sub-matrix that computed by the thread
  float Csub = 0;
  // Shared memory for the sub-matrix of A
  __shared__ float As[BLOCK_SIZE][BLOCK_SIZE];
  // Shared memory for the sub-matrix of B
  __shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE];
  // Loop over all the sub-matrices of A and B required to compute
  // the block sub-matrix
  for (int a = aBegin, b = bBegin;
   a <= aEnd;
   a += aStep, b += bStep){
    // Load the matrices from global memory to shared memory;
    // each thread loads one element of each matrix
    As[ty][tx] = A[a + wA * ty + tx];
    Bs[ty][tx] = B[b + wB * ty + tx];
    // Synchronize to make sure the matrices are loaded
    __syncthreads();
    // Multiply the two matrices together; each thread computes
    // one element of the block sub-matrix
    for (int k = 0; k < BLOCK_SIZE; ++k)
      Csub += As[ty][k] * Bs[k][tx];
    // Synch to make sure the preceding computation is done before
    // loading two new sub-matrices of A and B in the next iteration
    __syncthreads();
    }
    // Write the block sub-matrix to global memory;
    // each thread writes one element
    int c = wB * BLOCK_SIZE * by + BLOCK_SIZE * bx;
    C[c + wB * ty + tx] = Csub;
}
```

Further Matrix-Multiply Performance

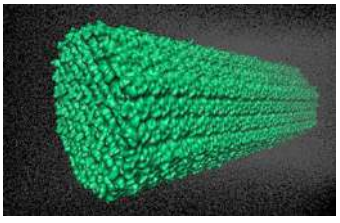
- Different implementations
  - Reference jki-ordered version on a 3GHz Xeon SC is 1.7 GFLOPS
  - V1 SIMD 32 i loop: 27 GFLOPS
  - V2 SIMD 32 k loop + Local Memory: 44 GFLOPS
  - V3 unroll 2 values from i loop: 43 GFLOPS
  - V4 unroll 2 values from j loop: 169 GFLOPS
  - V3+V4: 173 GFLOPS
- The bottom line
  - Relatively easy to get something to run on GPUs
  - Difficult to get it to run fast… (e.g. algorithms, memory, I/O, etc.)

Performance

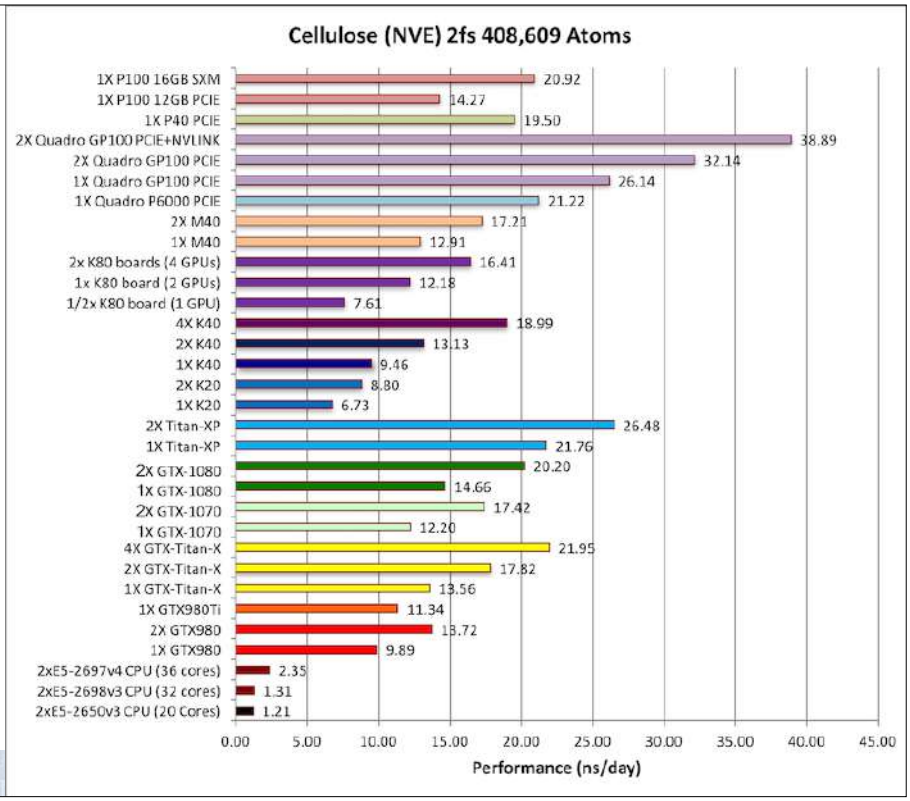
Tesla Performance



Amber Molecular Simulations



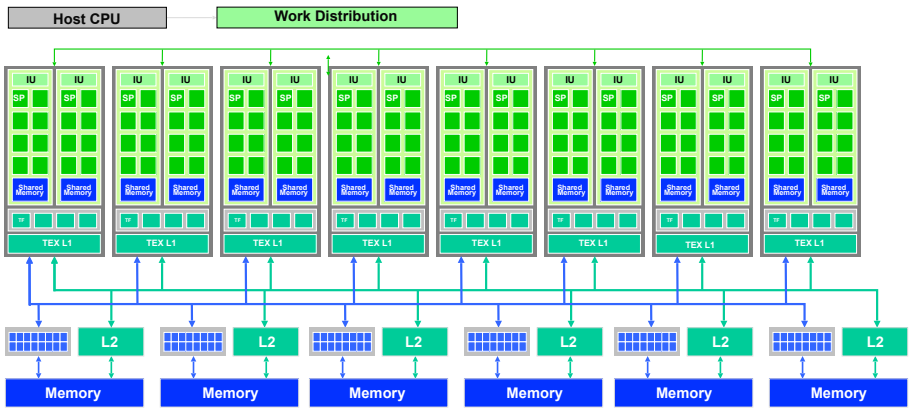
Cellulose



Going Forward

The Future of Computing is Parallel

- CPU clock rate growth is ~slowing, future computing growth will be from parallelism
- GPU Accelerators are a massively parallel computing platforms
  - 12,288 concurrent threads, hardware managed
  - 128 SP Thread Processor cores at 1.35 GHz == 518 GFLOPS peak



Implications and Opportunities

- Massively parallel computing allows
  - Drastic reduction in “time to discovery”
  - The new 3rd paradigm for research: computational experimentation
  - The “democratization of supercomputing”
    - $1,000/Teraflop in personal computers today
    - $1,000,000/Petaflops in clusters today
    - HW cost will no longer be the main barrier for big science
    - Global competition will be won with abilities to create and use parallel systems for discovery
- Future winner academic institutions will be leaders in
  - Research in Parallel Programming and Parallel Architecture
  - Teaching massively parallel programming to CS/EE students & scientists

Ce am invatat in aces curs?

- The Roofline Model
- GPU Architecture
- GPU Programming
- GPU Performance
- Going Forward

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

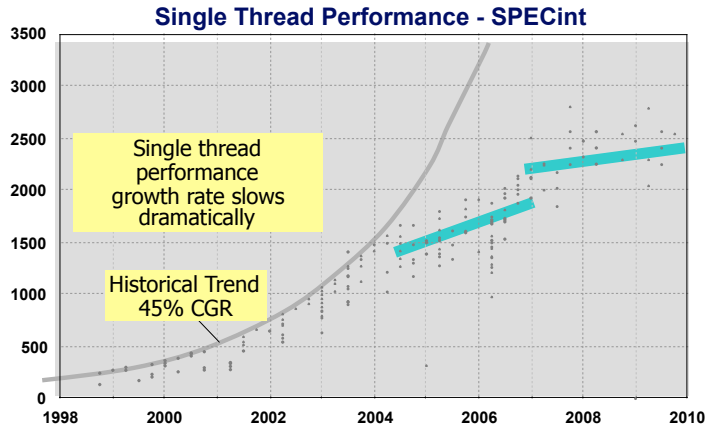
Curs 7
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

Cell BE Architecture

Computing Paradigm Shift

- Today:
  - Single thread performance hitting limits
- Happening now:
  - Paradigm shift to application specific system customization
- Future:
  - Greater performance demands
    - Immersive Interaction
    - Sensory Computing

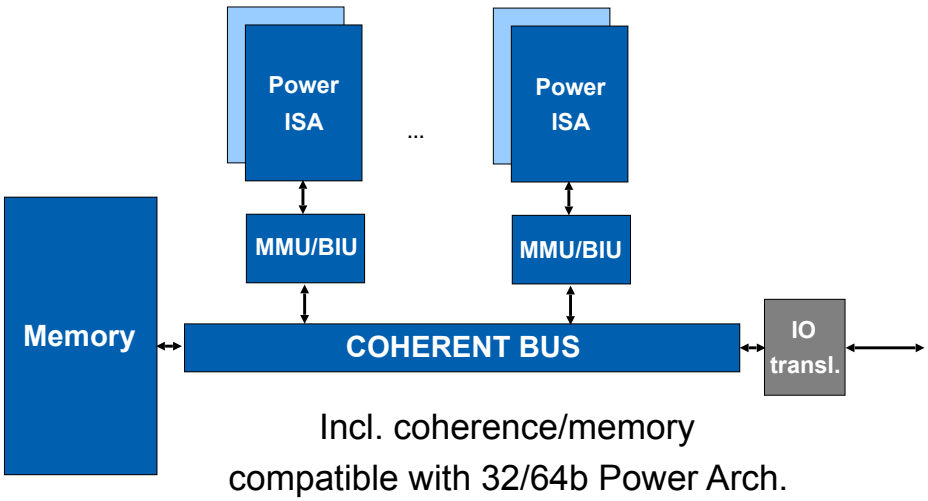


Cell Concept

- Compatibility with 64b Power Architecture™
- Increased efficiency and performance
  - Non Homogenous Coherent Chip Multiprocessor attacks “Frequency Wall”
  - Streaming DMA architecture attacks “Memory Wall”
  - High design frequency, low operating voltage attacks “Power Wall”
- Interface between user and networked world
  - Flexibility and security
  - Multi-OS support, including RTOS/non-RTOS
  - Architectural extensions for real-time management

Cell Architecture is…

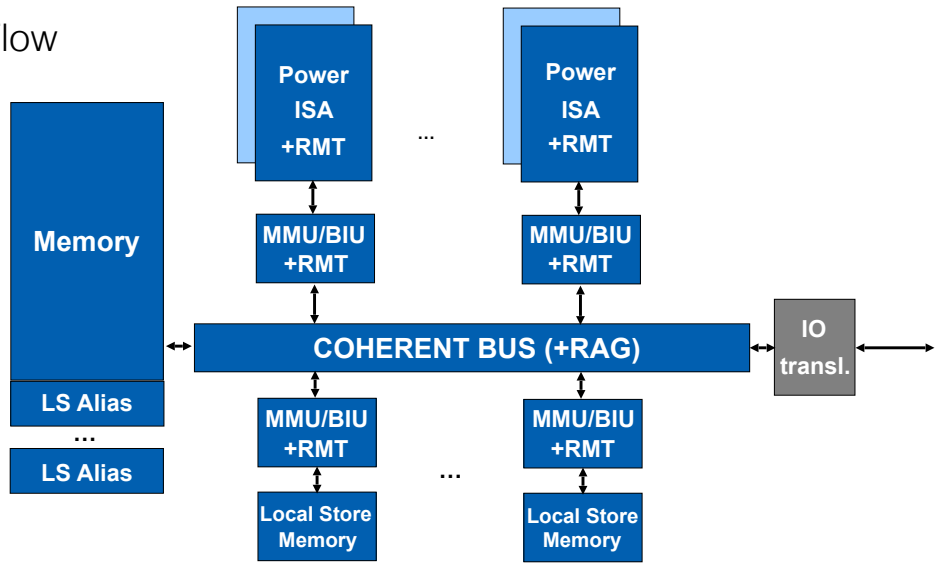
- 64b Power Architecture™



Incl. coherence/memory
compatible with 32/64b Power Arch.

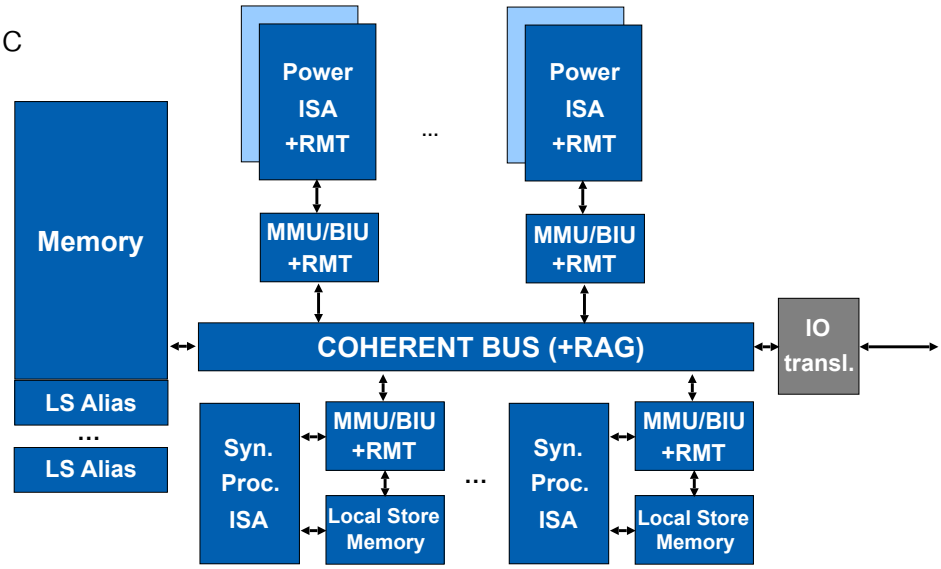
Cell Architecture is…64b Power

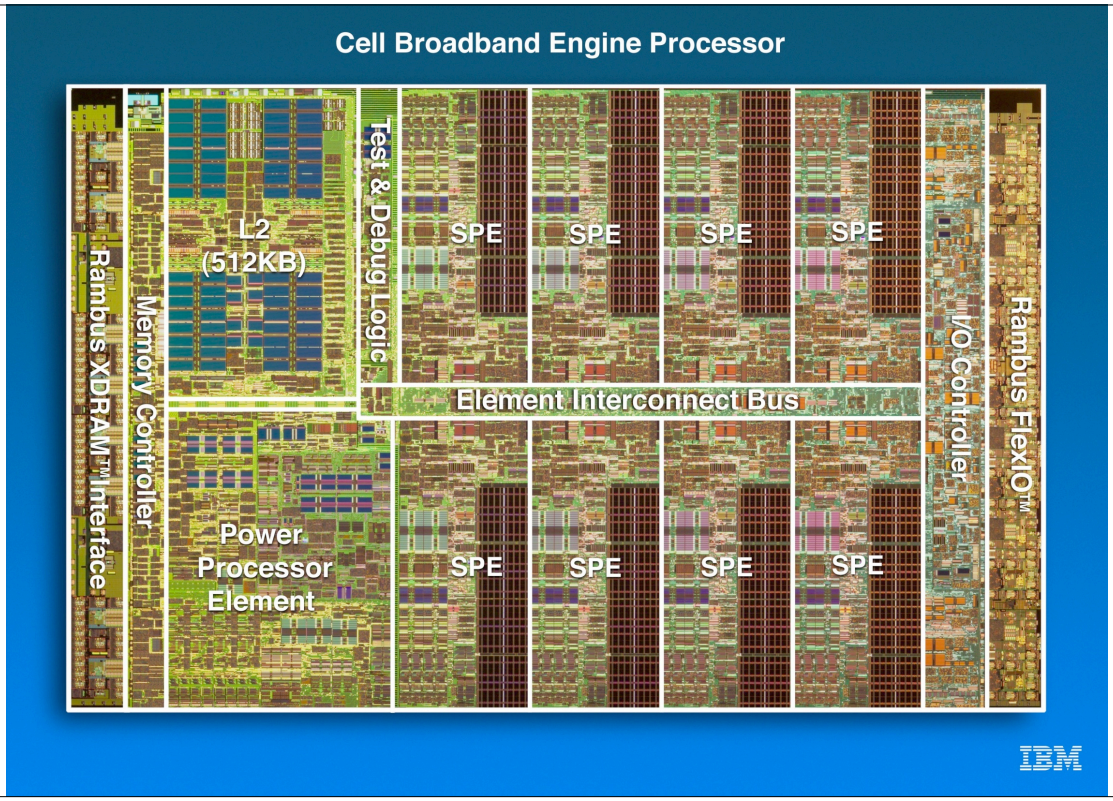
- Plus Memory Flow Control (MFC)



Cell Architecture is…64b Power + MFC

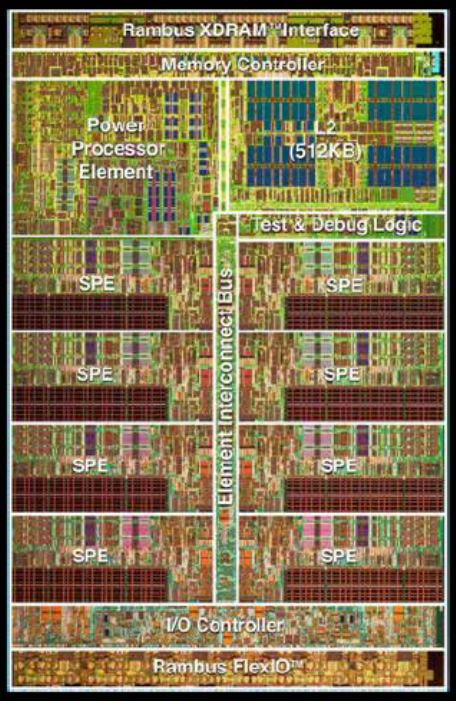
- Plus Synergistic Processors





Highlights (3.2 GHz)

- 241M transistors
- 235mm2
- 9 cores, 10 threads
- >200 GFlops (SP)
- >20 GFlops (DP)
- Up to 25 GB/s memory B/W
- Up to 75 GB/s I/O B/W
- >300 GB/s EIB
- Top frequency >4GHz
  (observed in lab)



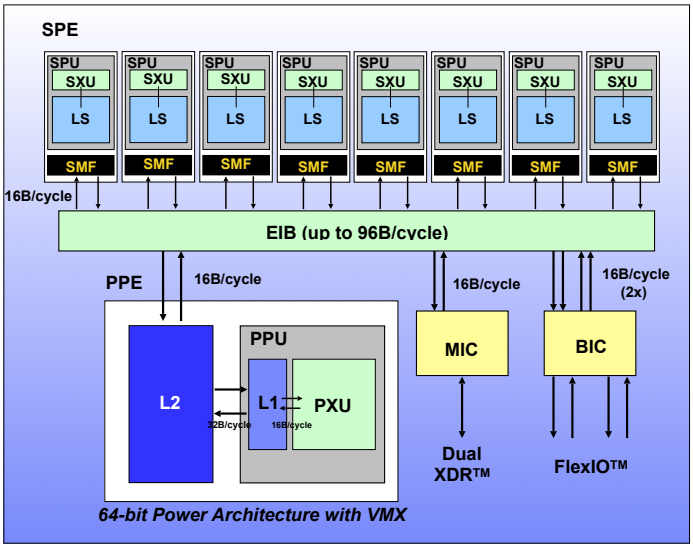
Cell BE vs. Pentium D

- Combines multiple high performance processors in one chip
  - 9 cores, 10 threads
  - A 64-bit Power Architecture™ core (PPE)
  - 8 Synergistic Processor Elements (SPEs) for data-intensive processing
- Current implementation — roughly 10 times the performance of Pentium for computational intensive tasks
  - Clock: 3.2 GHz (measured at >4GHz in lab)

	Cell	Pentium D
Peak I/O BW	75 GB/s	6.4 GB/s
Peak SP Performance	> 200 GFLOPS	> 30 GFLOPS
Area	235 mm^2	206 mm^2
Transistors	241 M	230 M

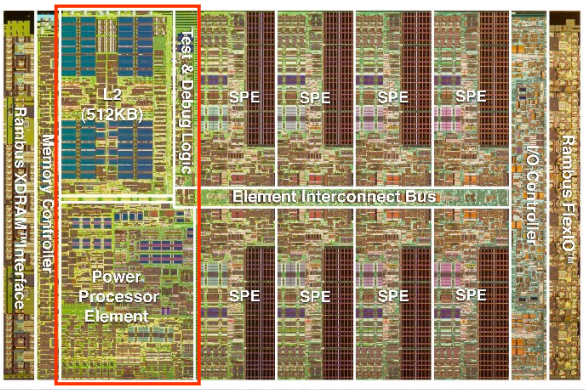
Cell BE Processor Features

- Heterogeneous multi-core system architecture
  - Power Processor Element for control tasks
  - Synergistic Processor Elements for data-intensive processing
- Synergistic Processor Element (SPE) consists of
  - Synergistic Processor Unit (SPU)
  - Synergistic Memory Flow Control
    - Data movement and synchronization
    - Interface to high-performance Element Interconnect Bus

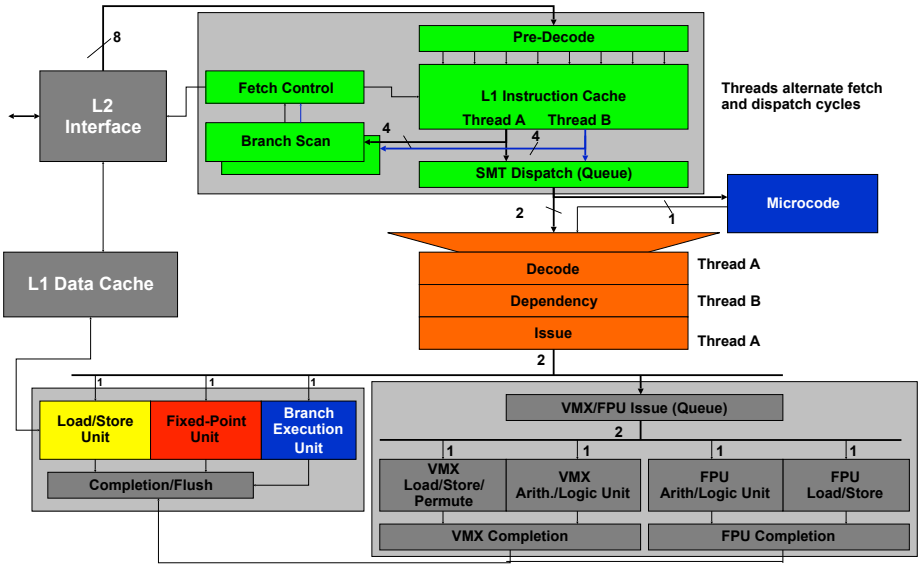


Power Processor Element

- PPE handles operating system and control tasks
  - 64-bit Power ArchitectureTM with VMX
  - In-order, 2-way hardware simultaneous multi-threading (SMT)
  - Coherent Load/Store with 32KB I & D L1 and 512KB L2

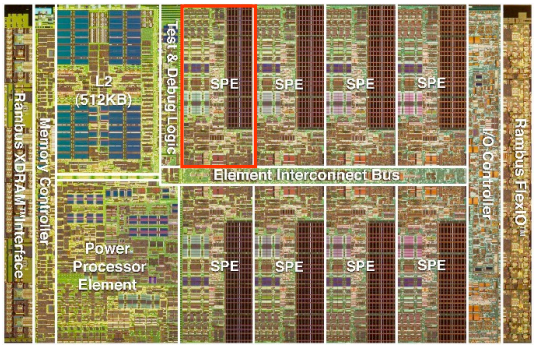


PPE Block Diagram

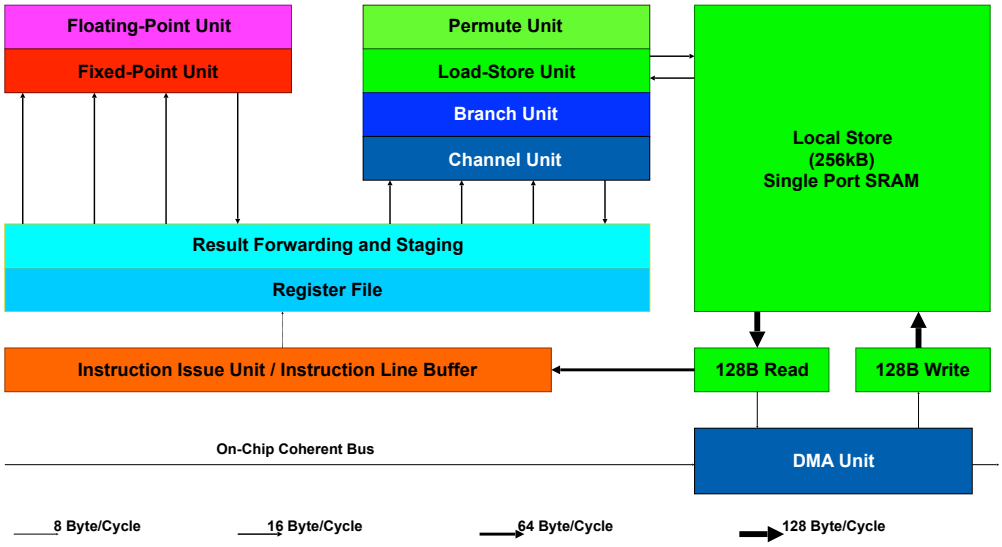


Synergistic Processor Element

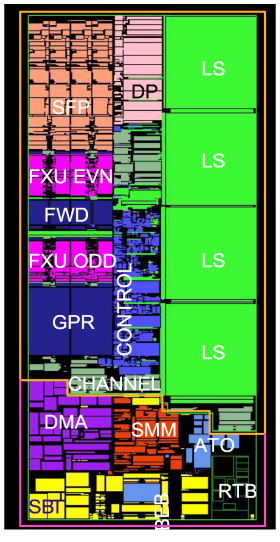
- SPE provides computational performance
  - Dual issue, up to 16-way 128-bit SIMD
  - Dedicated resources: 128 128-bit RF, 256KB Local Store
  - Each can be dynamically configured to protect resources
  - Dedicated DMA engine: Up to 16 outstanding requests



SPE Block Diagram



SPE Highlights

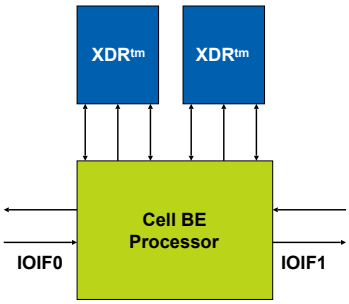


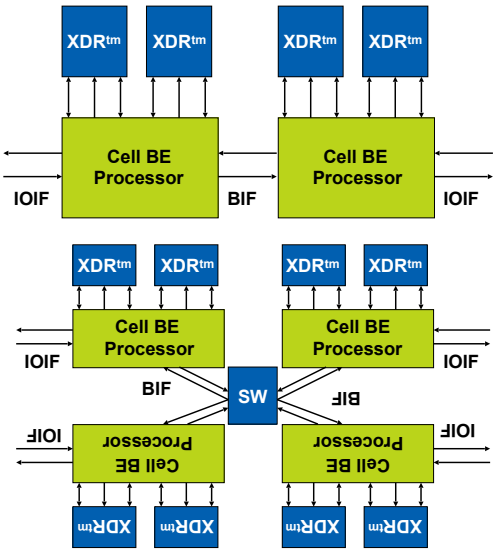
14.5mm² (90nm SOI)

- RISC like organization
  - 32 bit fixed instructions
  - Clean design – unified Register file
- User-mode architecture
  - No translation/protection within SPU
  - DMA is full Power Arch protect/x-late
- VMX-like SIMD dataflow
  - Broad set of operations (8 / 16 / 32 Byte)
  - Graphics SP-Float
  - IEEE DP-Float
- Unified register file
  - 128 entry x 128 bit
- 256KB Local Store
  - Combined Instruction & Data

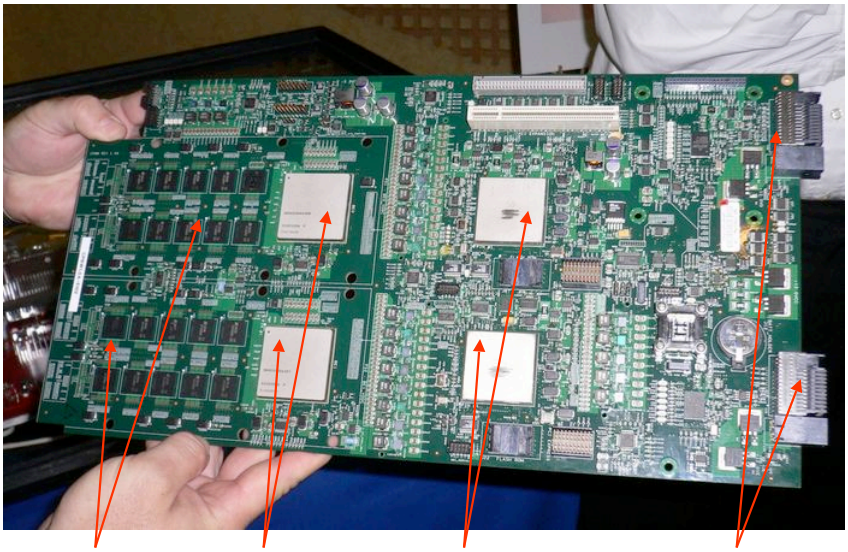
Cell BE Systems

- Game console systems
- Blades
- HDTV
- Home media servers
- Supercomputers

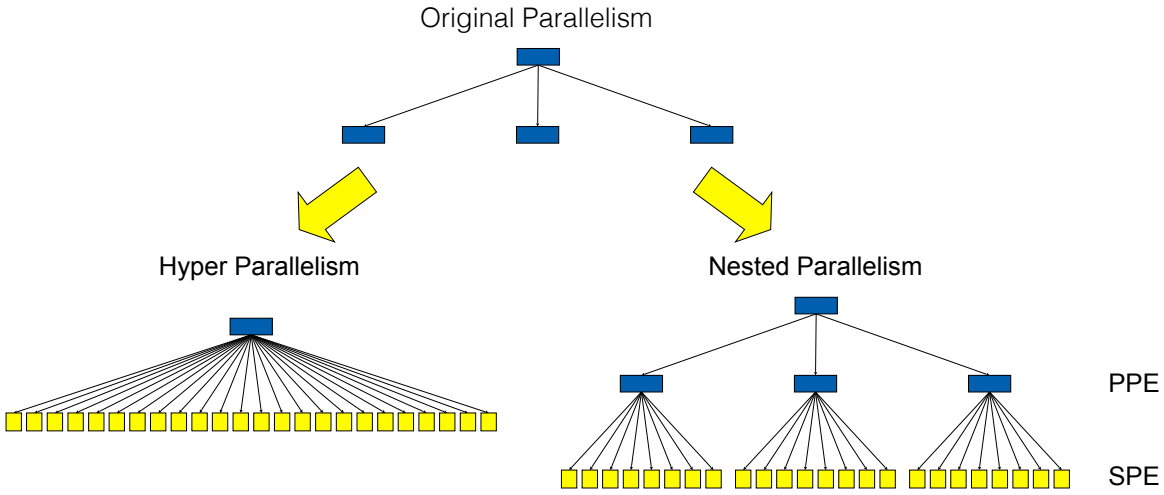




First Generation Cell Blade



Node-Level Parallelism



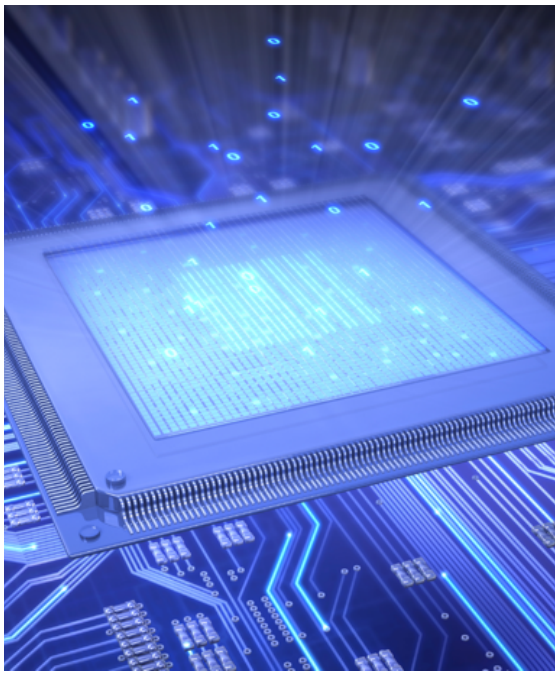
Usages of the Cell

- Sony
  - Playstation 3
  - HDTVs
- Toshiba
  - Cell on ATX form Cards
  - HDTVs
- IBM
  - Engineering & Technology Services - R&D & Military
  - Cell Based Systems - HPC, Scientific, Medical

Type	Algorithm	3.2 GHz GPP	3.2 GHz Cell BE Processor	Comparison Factor
HPC	Matrix Multiplication (S.P.)	24 Gflops (w/SIMD)	200 GFlops* (8SPEs)	8x
	Linpack (S.P.)	16 GFlops (w/SIMD)	156 GFlops* (8SPEs)	9x
	Linpack (D.P.): 1kx1k matrix	7.2 GFlops (IA32/SSE3)	9.67 GFLops* (8SPEs)	1.3x
	FFTW (S.P.)	1.7 GFlops (3GHz Woodcrest)	25.2 GFLops+ (8SPEs)	13.5x
	FFTW (D.P.)	1.3 GFlops (3 GHz Woodcrest)	6.38 GFLops+ (8SPEs)	4.9x
Graphics	Transform-light	170 MVPS (G5/VMX)	256 MVPS* (per SPE)	12x
	TRE	1 fps (G5/VMX)	30 fps* (Cell)	30x
Security	AES encryp. 128-bit key	1.03 Gbps	2.06Gbps** (per SPE)	16x
	AES decryp. 128-bit key	1.04 Gbps	1.5Gbps** (per SPE)	11x
	TDES	0.12 Gbps	0.16 Gbps** (per SPE)	10x
	DES	0.43 Gbps	0.49 Gbps** (per SPE)	9x
	SHA-1	0.85 Gbps	1.98 Gbps** (per SPE)	18x
Medical Imaging	3D Image Reconstruction	6.88 sec (3GHz Woodcrest)	0.35 sec+ (8 SPEs)	19x
Video Processing	mpeg2 decoder (CIF) mpeg2 decoder (SDTV) mpeg2 decoder (HDTV)	---- 354 fps (IA32) ----	1267 fps* (per SPE) 365 fps** (per SPE) 73 fps* (per SPE)	-- 8x --

Slides & Support Material Acknowledgements

IBM Control Center 2011-2016



Arhitectura Sistemelor de Calcul

Universitatea Politehnica Bucuresti

Facultatea de Automatica si Calculatoare

cs.ncit.pub.ro

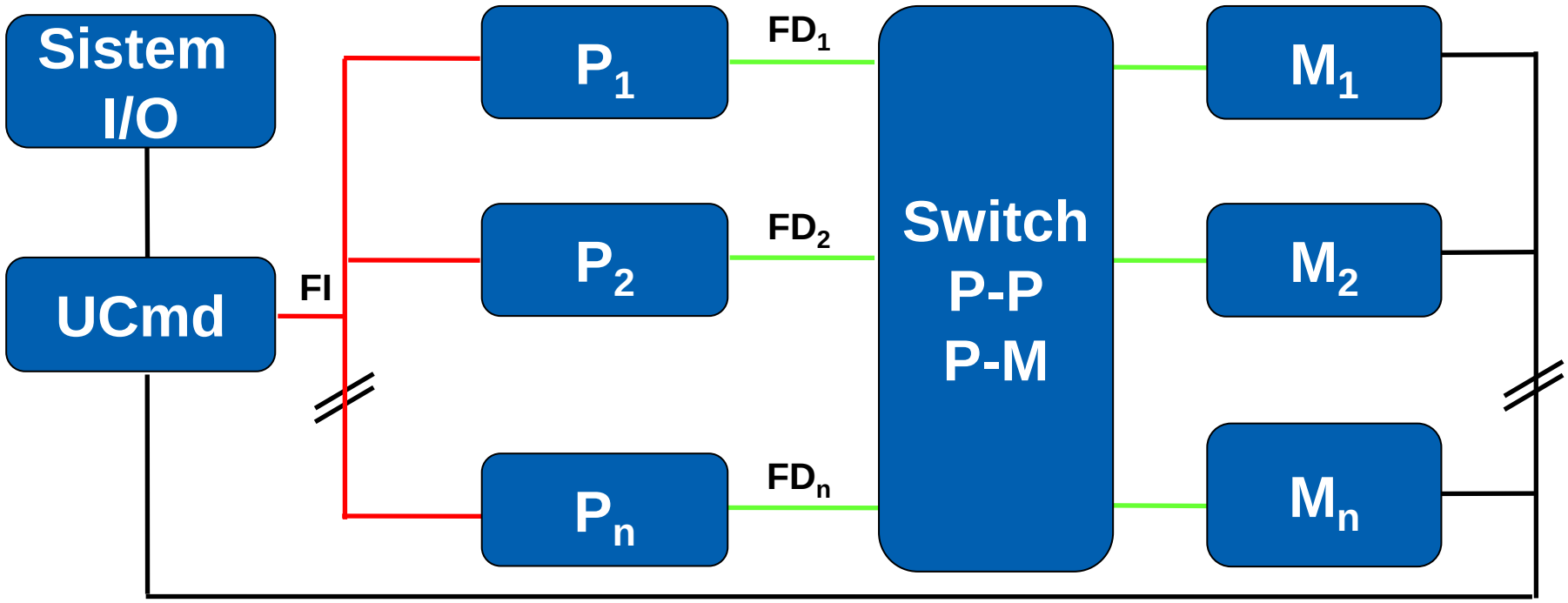
curs.cs.pub.ro

Cuprins

- Structura SIMD
- Setul de Instructiuni Masina SIMD
- Implementarea Salturilor Conditionate in Structura SIMD
- Organizarea Datelor in Structura SIMD

Structura SIMD

- Arhitecturile SIMD sunt adecvate prelucrarilor vectoriale:
  - O singura UCmd
  - Mai multe procesoare aritmetice si module de memorie
  - Retea de comutatie P-P & P-M
- Structuri adecvate prelucrarii vectoriale → esentiala organizarea algoritmului de calcul
- Pentru aplicatii neorientate pe aplicatii vectoriale → ineficiente



Structura SIMD - Componente

- UCmd: Citeste, interpreteaza si executa instructiuni de control
  - Trebuie sa aiba registre generale rapide, unitate de prelucrare si memorie locala
  - Executa si instructiunile de ramificatie (nu ca P_i)
- P_i: Sunt sincronizate la nivel de instructiune → fiecare procesor lucreaza pe un alt set de date
  - Trebuie sa aiba registre locale
- M: Formata din n module
  - Contine instructiuni (de comanda & prelucrare) si date

Structura SIMD - Componente

- Instructiuni:
  - De control → executate pe UCmd
  - De prelucrare vectoriala → executate de P_i
  - Fluxul de instructiuni e similar cu cel din sisteme secventiale
- Registre index:
  - Globale → in UCmd; afecteaza cu aceeasi cantitate adresa operanzilor
    - Accesul pe linii
  - Locale → in fiecare P_i; pot fi independente si se refera la zona de memorie M_i asociata fiecarui procesor
    - Acces pe coloane
    - Altfel se poate ajunge la secventialitatea accesului la date!

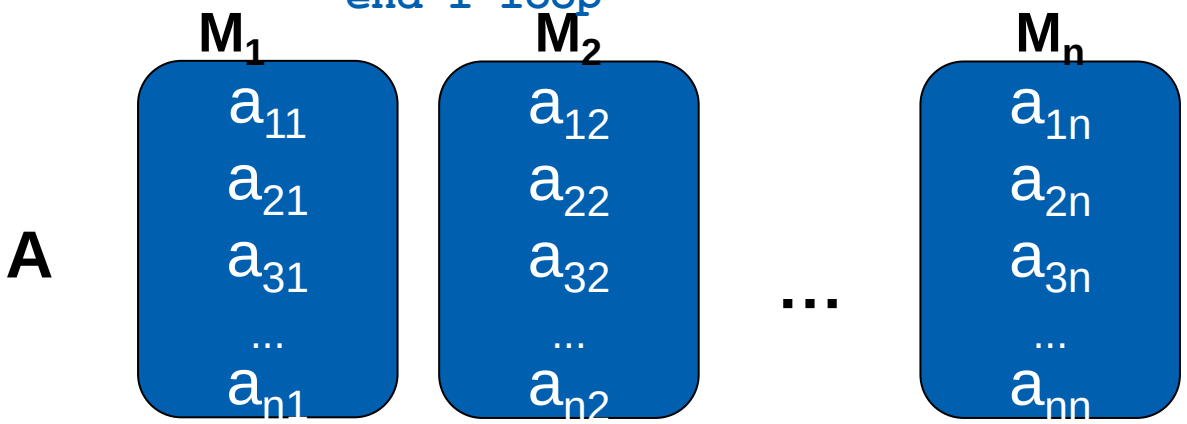
Cuprins

- Structura SIMD
- Setul de Instructiuni Masina SIMD
- Implementarea Salturilor Conditionate in Structura SIMD
- Organizarea Datelor in Structura SIMD

Exemplu – Inmultirea de Matrice

- Problema: A[n, n], B[n, n], C = A x B
- Considerand ($1 \le k \le n$) → se opereaza pentru toti indicii k simultan, adica se calculeaza pe linii

```
for i = 1 to n
    c_ik = 0 (1 ≤ k ≤ n)
    for j = 1 to n
        c_ik = c_ik + a_ij * b_jk (1 ≤ k ≤ n)
    end j loop
end i loop
```



- Elementele unei linii sunt in module diferite de memorie!
- Aceeasi discutie cu a_{ij} care nu depinde de k (Curs 5)
- Matricele B & C sunt aranjate similar
- Linia rezultat: c_{i1}, c_{i2} … c_{in} este calculata in paralel pe P_1, P_2 … P_n

Setul de Instructiuni Masina SIMD

- P_i are un registru acumulator ACC_i → info locala; UCmd are un registru index INDEX[i]
- Instructiuni masina cu vectori: $(1 \leq k \leq n)$
  - Incarcare:
    - vector: LOAD A ;ACC[k] ← A[1,k] (pentru vectori e suficient A[k])
    - vector indexat: LOAD A[i] ;ACC[k] ← A[INDEX[i],k]
  - Memorare:
    - vector: STO A ;A[1,k] ← ACC[k]
    - vector indexat STO A[i] ;A[INDEX[i],k] ← ACC[k]
  - Adunare vector: ADD A[i] ;ACC[k] ← ACC[k] + A[INDEX[i],k]
  - Inmultire vector: MUL A[i] ;ACC[k] ← ACC[k] * A[INDEX[i],k]
  - Difuzie scalara: BCAST i ;ACC[k] ← ACC[INDEX[i]] (1 perioada de ceas)
- [INDEX[i], k] e organizarea pe linii; [k, INDEX[i]] e organizarea pe coloane
- Instructiunile sunt similare SISD (von Neumman) cu diferenta k-ului ce indica paralelismul

Instructiuni de Ciclare & Comparare

- Instructiuni referitoare le registre INDEX:
  - Incarcare:
    - ENXC i, ct ;INDEX[i] ← ct
    - LDNX i, Adr ;INDEX[i] ← MEM[Adr]
  - Incrementare:
    - INCX i, ct ;INDEX[i] ← INDEX[i] + ct
  - Compara index & salt la adresa:
    - CPNX i, j, Adr ;if INDEX[i] < INDEX [j]
; then Jump Adr
;else continue
- Adresele de genul A[i], B[j] sunt adrese indexate cu continutul unui registru INDEX de pe UCmd

Exemplu - Inmultire de Matrice


```
for i = 1 to n
    c_ik = 0  (1 ≤ k ≤ n)
    for j = 1 to n
        c_ik = c_ik + a_ij * b_jk  (1 ≤ k ≤ n)
    end j loop
end i loop
```

- i, j si n sunt incarcate in registre INDEX

```
            ENXC    i, 1        ;INDEX[i] = 1
            SUB     ACC         ; resetez toate ACC-urile  ACC[k] = ACC[k] – ACC[k] ~ ACC[k] =0
            STO     C[i]        ; C [INDEX[i],k] ← ACC[k] ;  ~ c_ik  =0
nexti:      ENXC     j, 1       ;INDEX[j] = 1
            LDNX    n, adrN     ;la adrN se afla valoarea n+1
nextj:      LOAD    A[i]        ;ACC[k] ← A [INDEX[i],k]
            BCAST   j           ;ACC[k] ← ACC[INDEX[j]] (f important!)
            MUL     B[j]        ;ACC[k] ← ACC[k] * B [INDEX[j],k]
            ADD     C[i]        ;ACC[k] ← ACC[k] + C [INDEX[i],k]
            STO     C[i]        ;C [INDEX[i],k] ← ACC[k] (depunere rezultat)
            INCX    j, 1        ;INDEX[j]++
            CPNX    j, n, nextj ;salt la “nextj” cand  j < n+1
            INCX    i, 1        ;INDEX[i]++
            CPNX    i, n, nexti ;salt la “nexti” cand  i < n +1
```

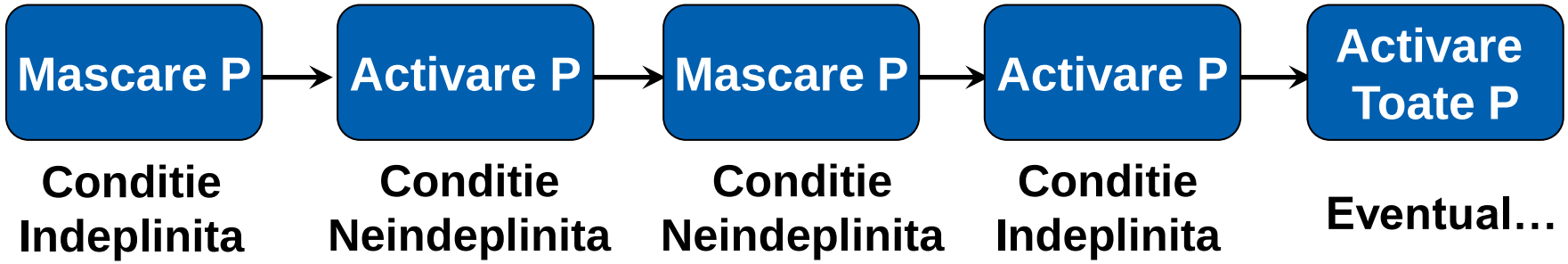
Cum credeti ca este alocat B-ul in memorie?

Cuprins

- Structura SIMD
- Setul de Instructiuni Masina SIMD
- Implementarea Salturilor Conditionate in Structura SIMD
- Organizarea Datelor in Structura SIMD

Implementarea Salturilor Conditionate

- In sistemele SIMD salturile conditionate se implementeaza prin divizarea fluxului de instructiuni (doar 1FI posibil!)
  - Un flux pentru conditia indeplinita
  - Alt flux pentru conditia neindeplinita
  - Cele doua FI se executa la momente diferite de timp si secvential! → paralelism n/2
  - Pentru implementarea acestor salturi conditionate se vor utiliza operatii de mascare a activitatii procesoarelor P_i:



Mascare/Activare Procesoare

- Se considera cate un bit de mascare asociat fiecarui procesor P_i, intr-un registru de masca (n biti): $P_i \leftrightarrow$ MASK[i]:
  - MASK[i] = 1 → procesorul P_i functioneaza
  - MASK[i] = 0 → procesorul P_i in asteptare & nu executa operatia curenta
- Operatiile de testare a mastilor se fac in fiecare P si rezultatele → Reg Index din UCmd → necesare instructiuni logice & de acces la registre Index:
  - Pt stabilirea de conditii
  - Pt implementarea salturilor
  - Lucru cu registre de masti (mascarea efectiva)

Instructiuni cu Registre de Masti

- Compara vector:

```
CLSS A, i        ;if ACC[k] < A[1,k] then
                 ;      INDEX[i].bit_k ← 1
                 ;else INDEX[i].bit_k ← 0
CEQL A, i        ;similar doar cu conditia '='
```

- Ramificatie fortata:

```
BRALL i, Adr     ;Salt la Adr numai daca INDEX[i]=1..1
BRNON i, Adr     ;Salt la Adr numai daca INDEX[i] = 0
```

- Operatii logice:

```
AND/OR/XOR i, j  ;INDEX[i] ← INDEX[i] op INDEX[j]
```

- Complementare index:

```
CMP i            ;INDEX[i] ← NOT INDEX[i]
```

Instructiuni cu Registre de Masti (cont)

- Incarca/Memoreaza masca de la index:
  - LDMSK i ;MASK ← INDEX[i]
  - STMSK i ;INDEX[i] ← MASK
- Mentionarea/extragerea primului bit (prioritatea):
  - FRST i ;if INDEX[i] ≠ 0 then
; ramane nemodificat
;else primul bit 1 ramane asa & restul
; de biti 1 se trec pe 0
  - FRST se foloseste cand o parte din Procs indeplinesc o conditie si trebuiesc activate
  - NXFIR i ;INDEX[i] = indicele primului bit 1
;if INDEX[i] = 0 then INDEX[i] =1..1 (FFF…)
  - NXFIR se utilizeaza in colaborare cu BCAST

Exemplu - Normalizarea unei Matrice

- Normalizarea prin inlocuirea A[i,j] ← A[i,j]/A[1,k] cu A[1,k] ≠ 0:

```
for i = 2 to n
   if (A[1,k]!=0) then A_ik = A_ik / A_1k (1 ≤ k ≤ n )
end i loop
```

- Fie M[k] = (A[1,k]!=0) in corespondenta cu mastile P_i:

```
for i= 2 to n
   A_ij /= A_1k, M[k]
end i loop
```

```
        ENXC i, 2              ;INDEX[i] = 2 (contor)
        SUB ACC                ;resetezi toate ACC-urile (XOR AX, AX)
        CEQL A, M              ;if ACC[k] = A[1,k] then bit k din INDEX[M] = 1
                               ;   else bit k din INDEX[M] = 0
        CMP M                  ;INDEX[M] ← NOT INDEX[M] (masca ok?)
        LDMSK M                ;incarcam masca & activam procs
        LDNX n, adrN           ;la adrN e valoarea n+1
nexti:  LOAD A[i]              ;ACC[k] ← A[INDEX[i],k]
        DIV A                  ;doar pe procs unde MASK = 1
        STO A[i]               ;A[i] ← ACC[k] (depunere rezultat)
        INC i, 1               ;INDEX[i]++
        CPNX i, n, nexti       ;salt la “nexti” reia ciclul  daca i < n+1
```

Cuprins

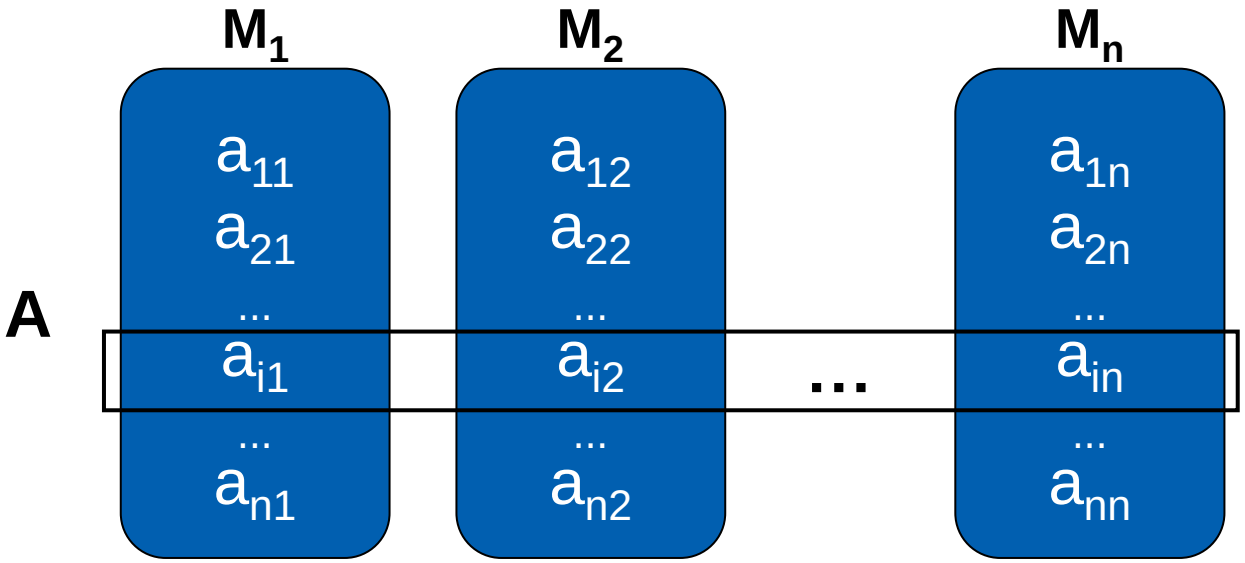
- Structura SIMD
- Setul de Instructiuni Masina SIMD
- Implementarea Salturilor Conditionate in Structura SIMD
- Organizarea Datelor in Structura SIMD

Organizarea Datelor in Structura SIMD

- Probleme:
  - Repartizarea in memoriile M_i & accesul paralel
  - Citirea vectorilor intr-o singura instructiune
  - La matrice: acces la linii si la coloane → necesara transpunerea/deplasarea datelor?
- Memoria:
  - Poate face cel mult un acces la un ciclu de lucru
  - n operanzi in n module diferite pot fi cititi simultan
  - Defavorabila plasarea in acelasi modul al operanzilor → citirea va fi secventiala in n cicli de acces!

Reprezentarea Directa pe Linii

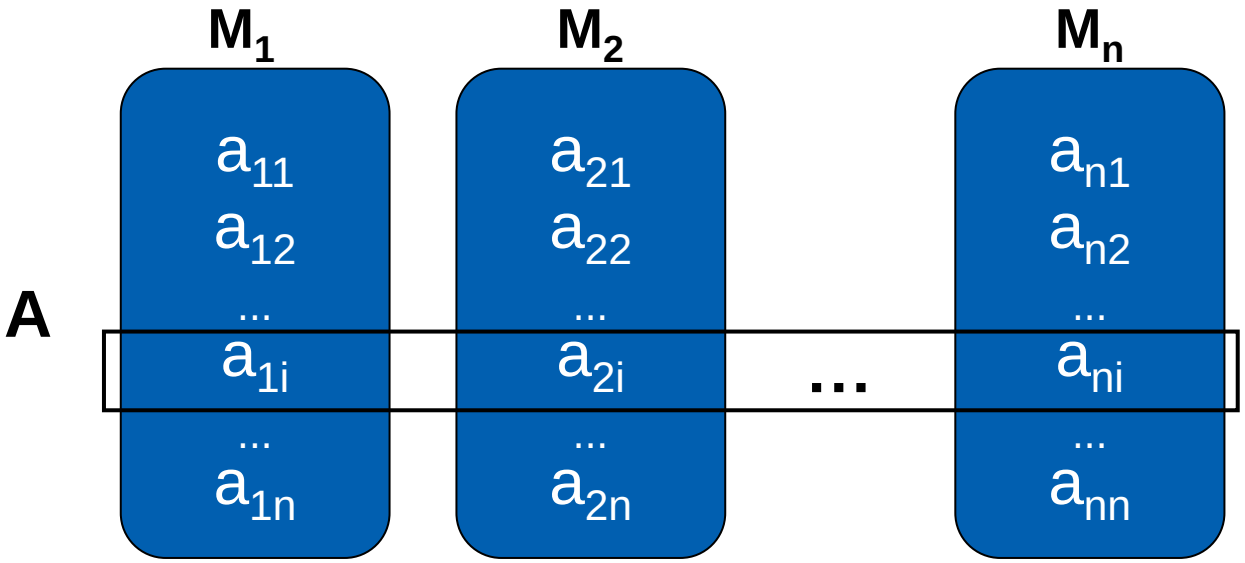
- Matrice A[n,n] & structura SIMD (n procs/memorii)
- Cu acces simultan la liniile matricei:
  - Acces paralel la linii
  - Acces secvential la coloane



LOAD A[i] – citirea liniei i printr-un singur acces

Reprezentarea Directa pe Coloane

- Matrice A[n,n] & structura SIMD (n procs/memorii)
- Cu acces simultan la coloanele matricei:
  - Acces secvential la linii
  - Acces paralel la coloane

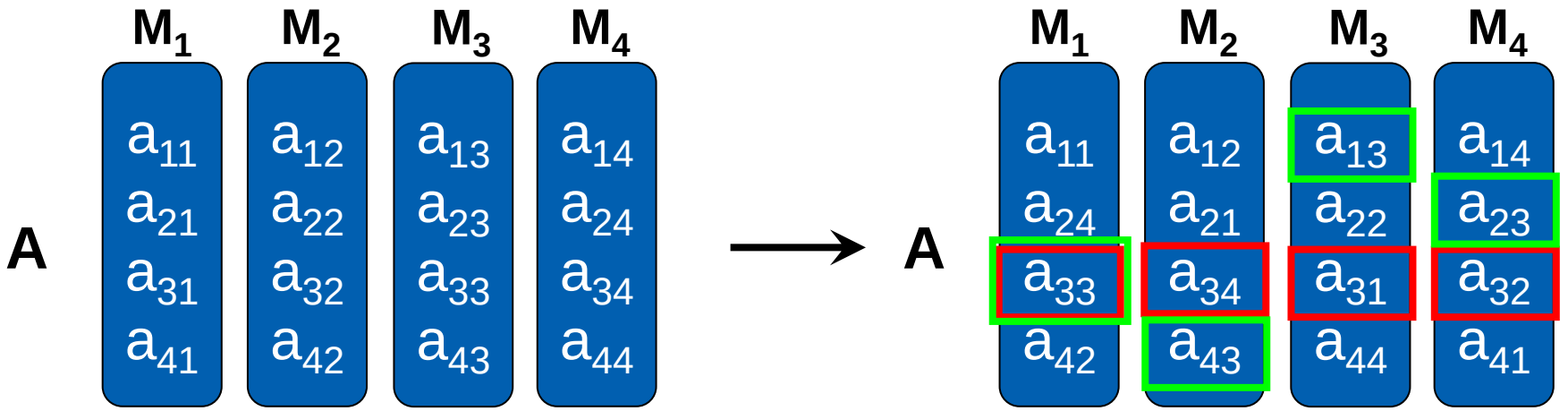


LOAD A[i] – citirea coloanei i printr-un singur acces

Reprezentarile directe pe linii/coloane sunt incompatibile la acelasi moment de timp

Reprezentarea Deplasata Ciclica

- Matrice A[4,4] & structura SIMD (4 procs/memorii)
- Cu acces paralel la liniile & coloanele matricei:
  - Exemplu valid pt inmultirea matricelor (linia & coloana 3)



Organizarea originala (pe linii)

Organizarea deplasata

Necesar un shuffling cu reteaua de comutatie (a_{i1} nu e pe P_1)

Concluzii Organizarea Datelor

- Trade-off simplu:
  - Programe simple → acces secvential la linii & coloane
  - Programe complicate → acces paralel la linii & coloane
- Cu organizarea deplasata accesul la linii & coloane se face intr-un singur ciclu:
  - La linii – adresa fizica/actuala a liniei
  - La coloane – sunt necesari registre index private in P_i:
    - Pt acces la Col i, vectorul index e deplasat ciclic pana 0 e la M_i
    - Se difuzeaza adresa liniei 0 cu indicatia de indexare locala
    - Aducerea in pozitia corecta se face prin reteaua de comutatie P-P
- Costul platit paralelismului este consistenta datelor!
- Cu RC cross-bar → nu conteaza asezarea datelor (setare cross-bar sau index local inainte de operatii)

Exemplu – Suma pe Coloane

- Problema: A[n, n] → se doreste suma pe coloane a matricei A intr-un vector SUM[n]
- Cu organizarea pe linii – suma se face secvential, linie cu linie, fiecare P_i calculeaza suma pe cate o coloana

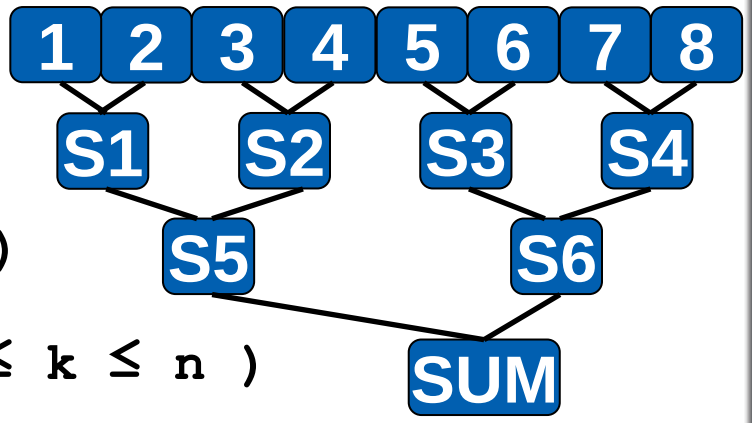
```
SUM[k] = 0 (1 ≤ k ≤ n )
for i = 1 to n
   LOCINDEX[k] = i (1 ≤ k ≤ n )
   SUM[k] += A[LOCINDEX[k],k] (1 ≤ k ≤ n )
end i loop
```

- LOCINDEX e un index local din P
- Fiecare citire din memorie aduce n elemente ce sunt adunate la cele n sume partiale pe cele n P
- Complexitatea este… $O(n)$

Exemplu – Suma pe Linii

- Problema: A[n, n] → se doreste suma pe linii intr-un vector SUM[n]
  1. Organizarea pe linii duce la o complexitate $O(n^2)$
  2. Organizarea pe coloane duce la un algoritm de transpunere
  3. Folosind registre temporare asociate P_i-urilor (TEMP[k]) $O(n \log_2 n)$
  4. Se doreste o complexitate $O(n)$ astfel incat trebuie cautata alta solutie
- Solutia 3 este:

```
for i = 1 to n
   TEMP[k] = A[i,k] (1 ≤ k ≤ n )
   for j = 2 to n - 1
       TEMP[k] += TEMP[k-j] (j ≤ k ≤ n )
   end j loop
   SUM[i] = TEMP[n-1]
end i loop
```



$O(n \log_2 n)$

Exemplu – Suma pe Linii (cont)

- Solutia 4: pentru o complexitate $O(n)$ → se va folosi cate un index local pentru fiecare P_i (e absolut necesar)
- Algoritmul este:

```
SUM[k] = 0 (1 ≤ k ≤ n )
LOCINDEX[k] = k (1 ≤ k ≤ n )
for i = 1 to n
   SUM[k] += A[LOCINDEX[k],(k – i) mod n] (1 ≤ k ≤ n)
end i loop
```

- Se aduna cate o diagonala la suma curenta

Diagonala e deplasata ciclic spre elementul adiacent

A [LOCINDEX[k],k] nu este altceva decat accesul pe coloane!

TA: Rezolvarea sumelor pe linii & coloane cu utilizarea organizarii deplasate a datelor

A

M_1	M_2	M_3	M_4
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}

Concluzii

- Elaborarea unor algoritmi eficienti necesita
  - Organizarea corespunzatoare a structurilor de date
  - Registre locale ale sistemului de calcul
  - Existenta unei retele de comutatie P-P
- Etapele elaborarii unui program sunt:
  - Descrierea algoritmului
  - Paralelizarea algoritmului
  - Optimizarea paralelizarii curente

What Next?

- Q & A?

- Next time:
  - Structuri SIMD:
    - Probleme de Comunicatii intre Procesoarele unui Sistem SIMD
    - Deplasarea Ciclica a Datelor intre Procesoare
    - Intercalarea Perfecta
    - Conectarea Inversa
    - Permutari Elementare

Arhitectura Sistemelor de Calcul

Universitatea Politehnica Bucuresti

Facultatea de Automatica si Calculatoare

cs.ncit.pub.ro

curs.cs.pub.ro

Cuprins

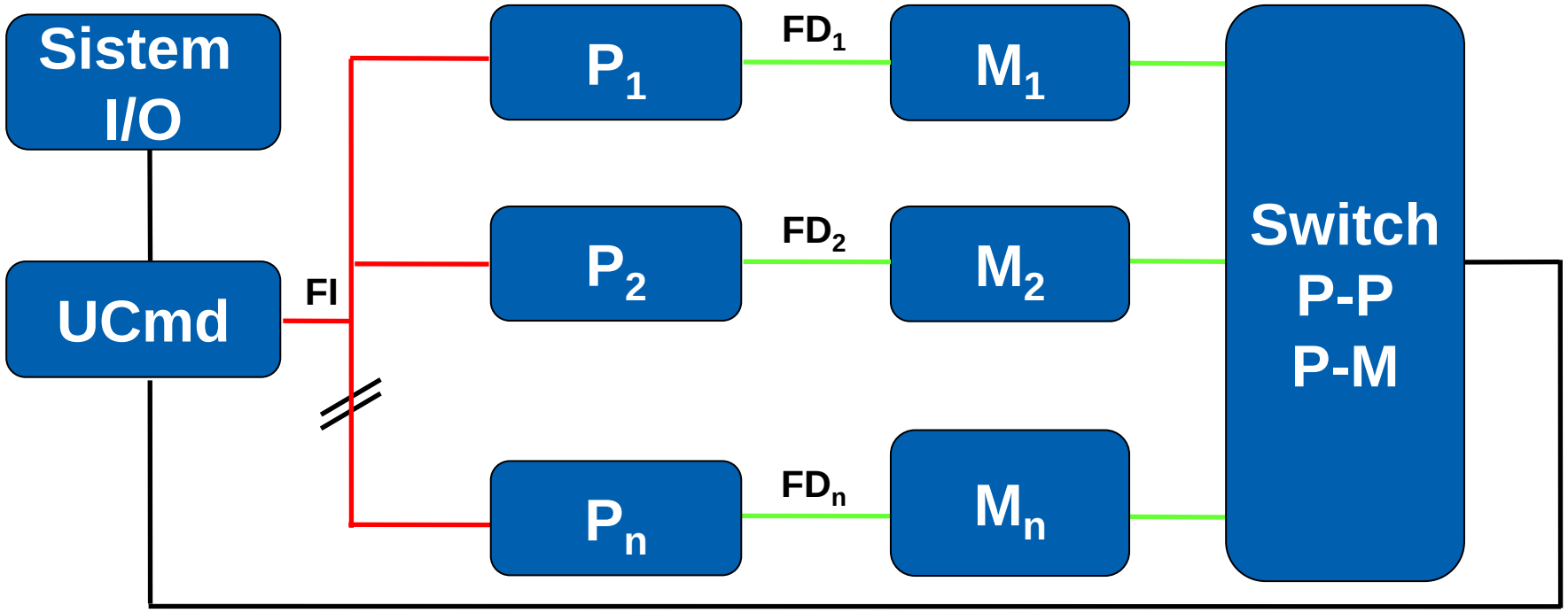
- Structura SIMD
- Probleme de Comunicatii intre Procesoarele unui Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

Structura SIMD – Probleme

- Arhitecturile SIMD – adecvate prelucrarilor vectoriale

1. Cresterea vitezei masinii SIMD:
   - Datele trebuiesc plasate in module distincte de memorie
2. Utilizarea eficienta a procesoarelor:
   - Datele trebuiesc permutate pt a furniza operanzi pentru calcule

- Pt rezolvarea celor doua probleme: necesare RC P-P si P-M



Retele de Comutatie

- Retelele de comutatie (RC) P-P sunt implementate la nivelul procesoarelor
- Retelele de comutatie P-M pot fi implementate facil cu switch-uri Cross-Bar
  - Pt. n procesoare sunt necesare n(n-1)/2 SP → costul devine prohibitiv (65k Procs → 4Mrd+ Switches)
- Sunt necesare RC adecvate fiecarei aplicatii (masini dedicate); aceste RC ofera:
  - Fiabilitate
  - Costuri reduse
  - Viteza mare de comutatie/transfer
  - Facilitati de intretinere

The Trade-off…

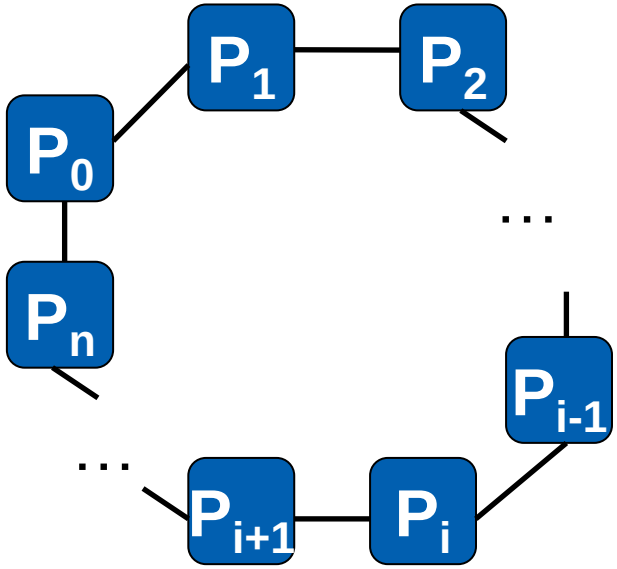
Reduced Parallelism!

Cuprins

- Structura SIMD
- Probleme de Comunicatii intre Procesoarele unui Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

Interconexiunea Circulara

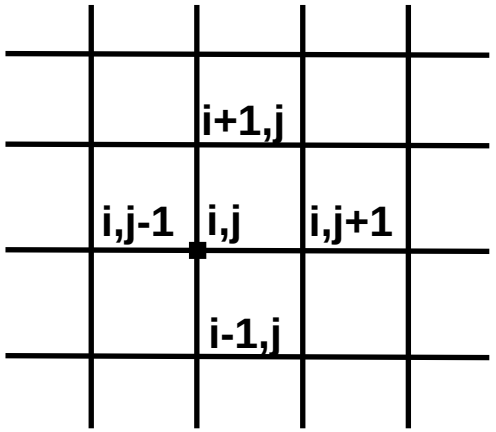
- Deplasarea ciclica a datelor intre procesoare
- Conexiunile sunt permise numai cu vecinii
- Nu exista conexiuni directe intre i si j
- Sunt maxim [**`(i-j-1)% n`**] noduri intermediare

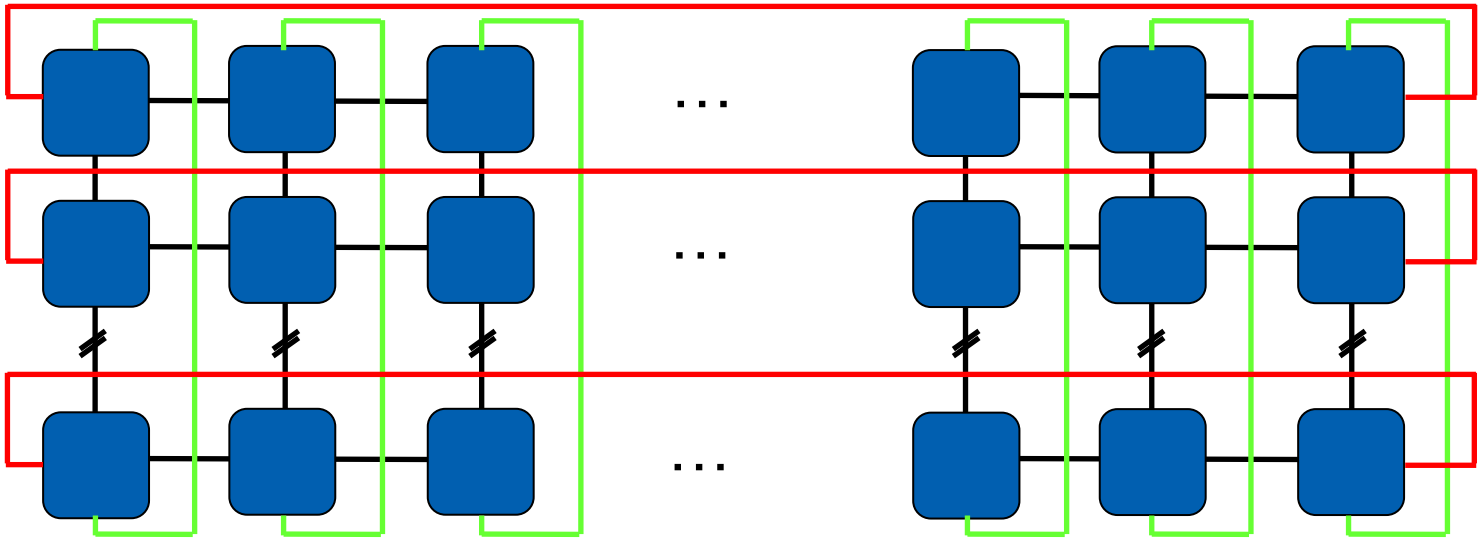


- Exemple de domenii de aplicatie:
  - Adecvate rezolvarii ecuatiilor diferentiale de ordinul 1: `x[i] = x[i-1] + x[i+1] – 2x[i]`
  - Propagarea caldurii printr-o bara
  - Aplicatii meteo – prognoza vremii in diferite puncte ale globului se face in paralel cu aceleasi date

Interconexiunea Matriceala

- Asigura comunicarea cu patru procs vecine
- Daca dependenta este de felul: `x[i,j] = x[i+1,j] + x[i-1,j] + x[i,j+1] + x[i,j-1] - 4x[i,j]` → de ex: evoluatia temperaturii in volumul unui material
- ILLIAC 4 – primul SIMD (64 procs) – 90% la fel si acum



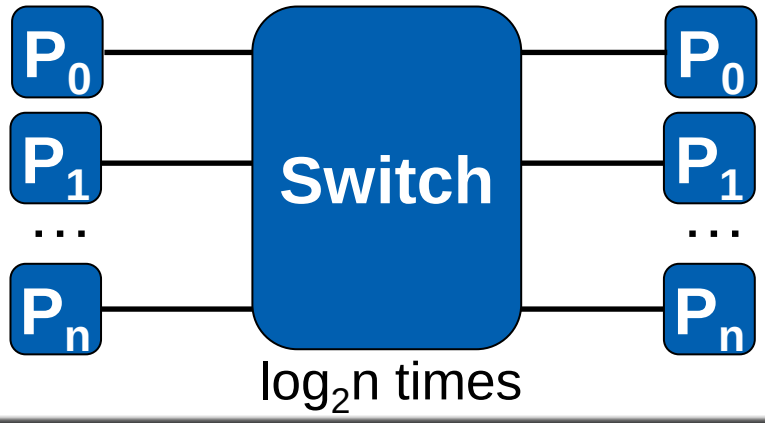


Cuprins

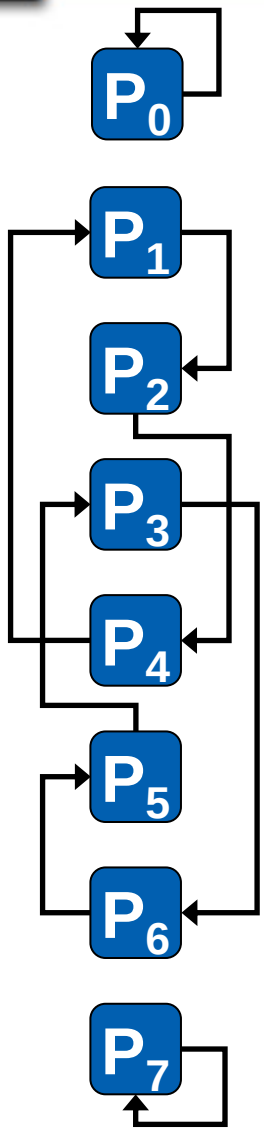
- Structura SIMD
- Probleme de Comunicatii intre Procesoarele unui Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

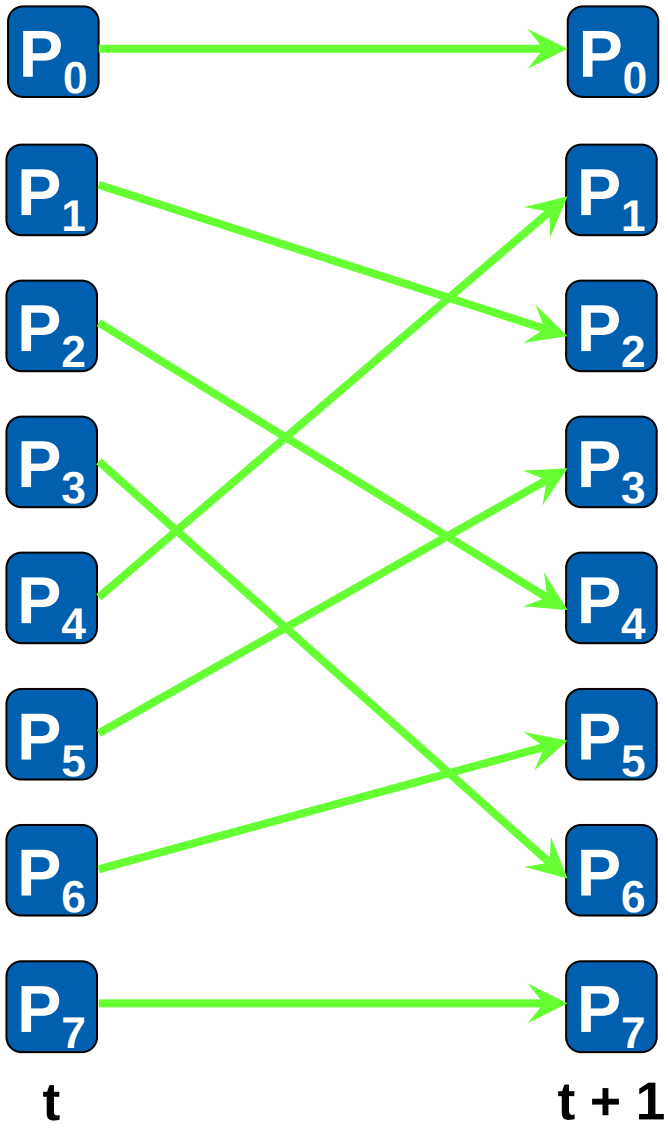
Intercalarea Perfecta – Shuffle

- Este adecvata pentru:
  - Transformata Fourier (FFT – Fast Fourier Transformation)
  - Algoritmi de sortare
  - Transpuneri de matrice
- Asigura conectarea intre procesoarele unei structuri SIMD de n procesoare cu:
  - n-2 comunicatii simultane
  - dupa $\log_2 n$ iteratii datele ajung pe procesorul sursa

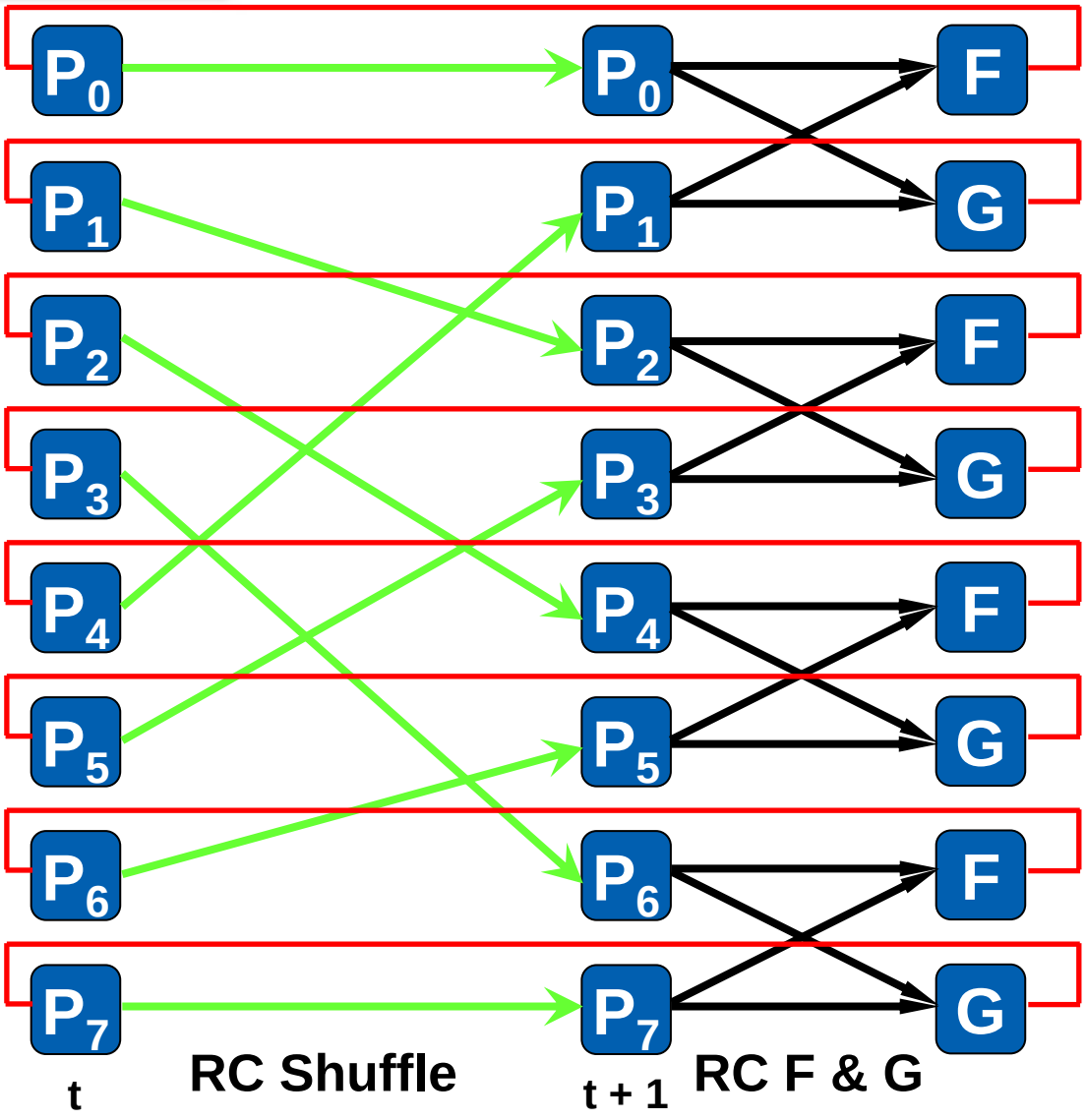


Shuffle – 8 Procesoare





Shuffle – 8 Procesoare



- Doua RC:
  - Shuffle
  - Adiacenta (pt F & G):
    - F e pe Y_i = pare
    - G e pe Y_{i+1} = impare
  - $\log_2 n$ iteratii

```
for i = 1 to log2n
   shuffle[y]
   Yi   = F[Yi,Yi+1]
   Yi+1 = G[Yi,Yi+1]
end i loop
```

- Utilizare: sortari, FFT, transpuneri, functii cu operatii recurente

Shuffle – Aplicatii

- Pentru sortari:
  - F calculeaza maximul celor doi operanzi
  - G calculeaza minimul celor doi operanzi
- Pentru FFT:
  - F produce suma ponderata
  - G produce diferenta ponderata
  - Ponderile sunt calculate dinamic la fiecare pas al iteratiei
- Pentru transpunerea matricelor:
  - Organizarea directa a matricei
  - $F = Y_i$
  - $G = Y_{i+1}$

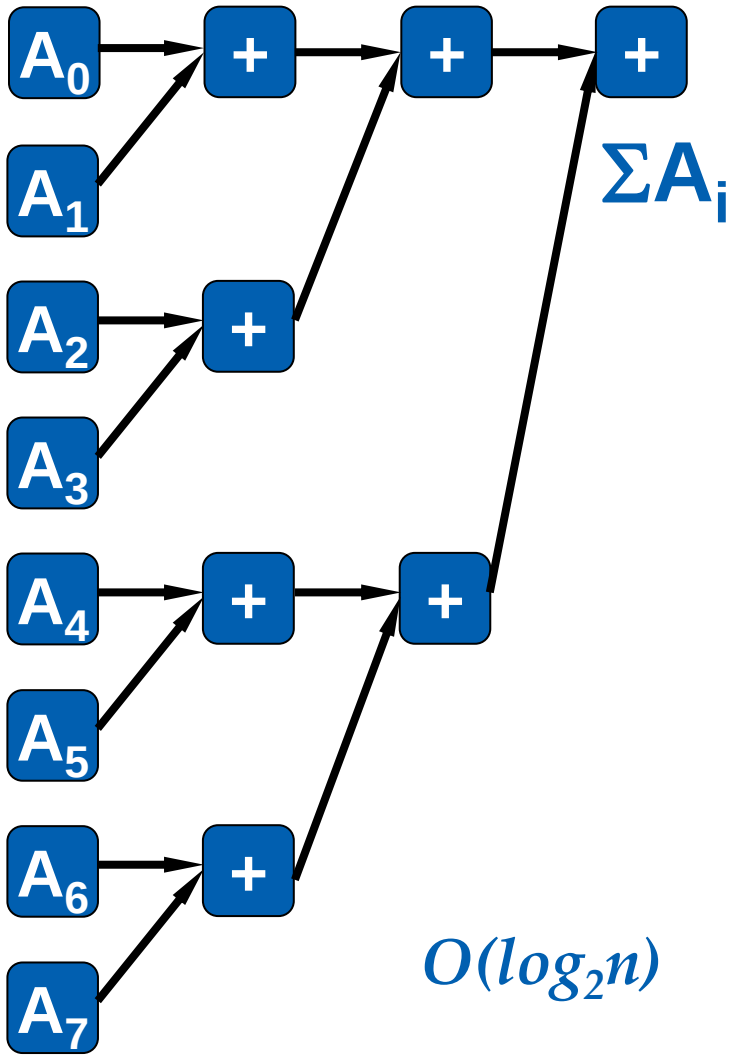
Shuffle – Aplicatii Iterative

- Suma elementelor unui vector
- Daca vectorul e intr-o singura memorie → secvential:

```
sum = 0
for i = 0 to n-1
   sum += a_i
end i loop
```

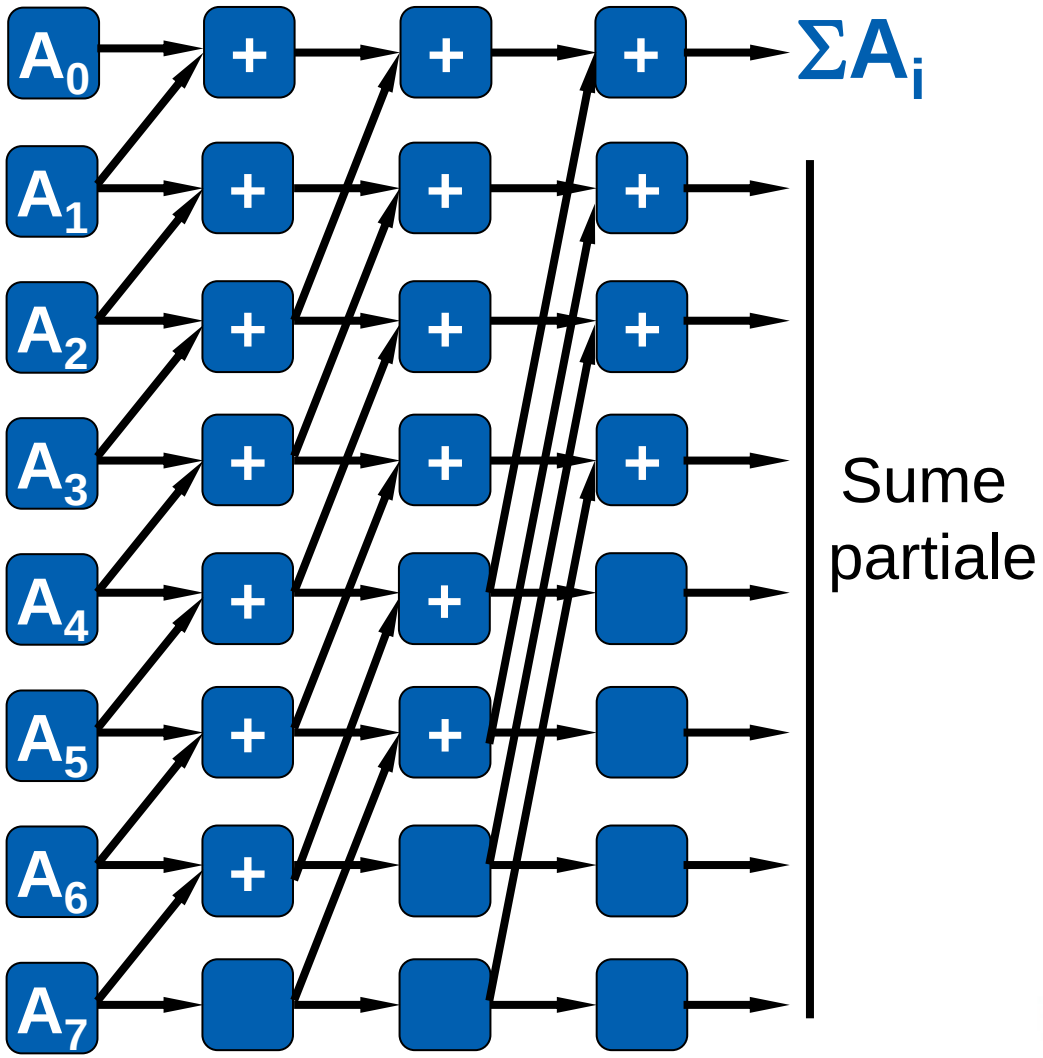
$O(n)$

- Se poate reduce $O(n)$ pe SIMD?
- Distribuim vectorul intre memorii
- Probleme:
  - Incarcarea neechilibrata a procs
  - La fiecare pas trebuie schimbata RC in mod dinamic



Shuffle – Aplicatii Iterative (cont)

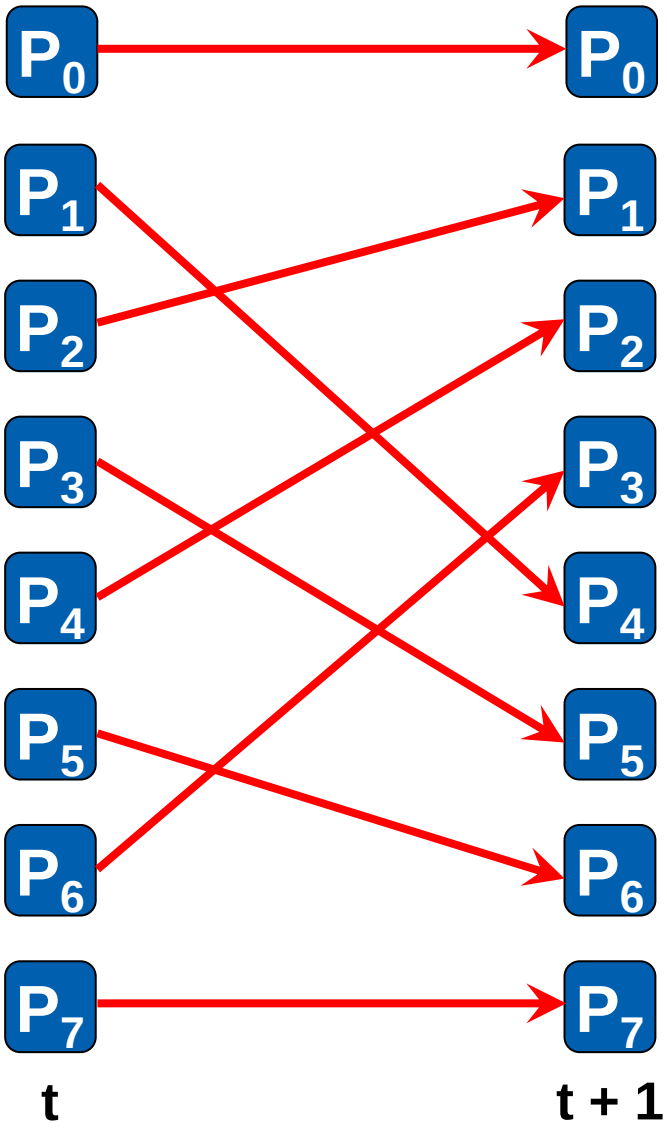
- Sunt necesare
  - Comunicare liniara
  - RC statica
- Reconfigurarea conexiunilor e mai simpla
- Dezavantaj: este in continuare necesara reconfigurarea la fiecare pas!
- Complexitate: $O(log_2 n)$



Cuprins

- Structura SIMD
- Probleme de Comunicatii intre Procesoarele unui Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

Inverse Shuffle

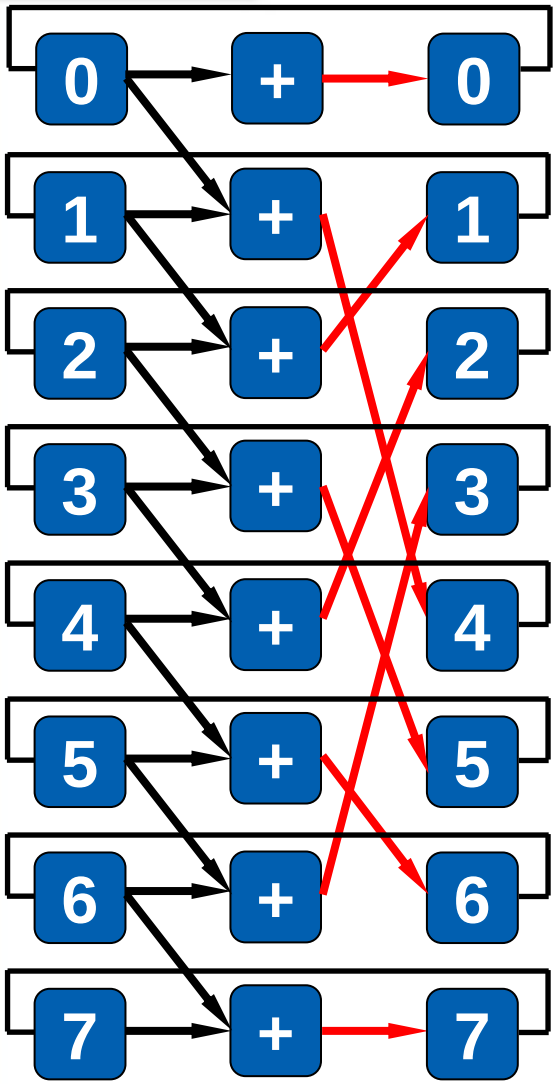


- Adecvat pentru operatii recurente, calculabile in $\log_2 n$ pasi
- Suma elementelor unui vector:

```
Y_i = A_i  (0 ≤ i ≤ n-1)
for j = 1 to log2n
   Y_j  = Y_j + Y_j-1
   invshuffle[Y], MASK
   compute MASK
end j loop
```

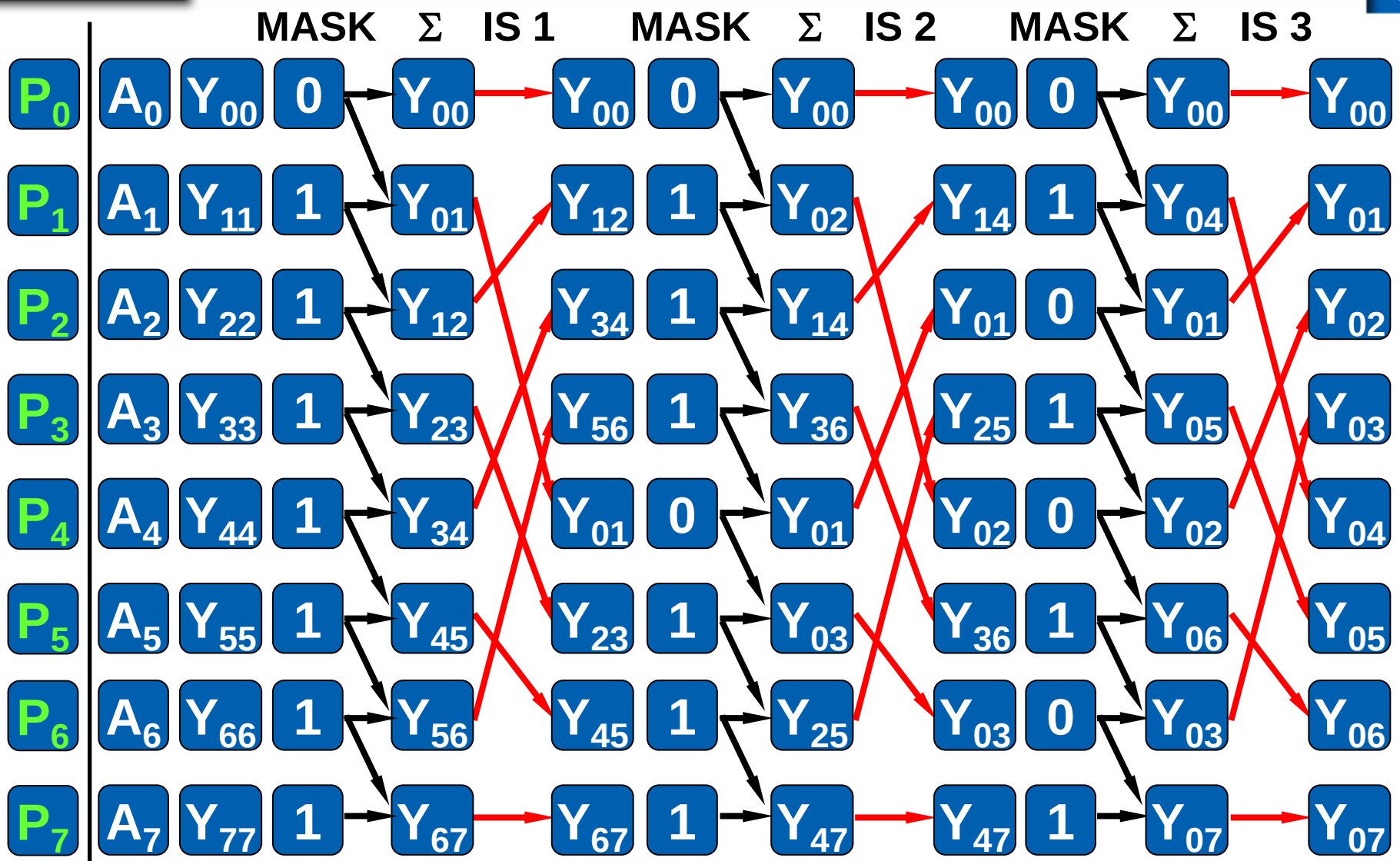
- Legatura identica pentru toate iteratiile:
  - O legatura directa
  - Un Inverse Shuffle

Inverse Shuffle – Aplicatii Iterative



- Legatura (RC) identica pentru toate iteratiile:
  - O legatura directa: $Y_j = Y_j + Y_{j-1}$
  - Un Inverse Shuffle
- Mastile se modifica si ele prin Inverse Shuffle la fiecare iteratie
  - MASK = 0 → procesor inactiv
  - MASK = 1 → procesor activ: (F & G = +)
- La anumite iteratii, unele procesoare pot fi inactivate de masti
- Notam $Y_{ij} = \sum_{k=i}^{j} A(k)$ $O(log_2 n)$

Inverse Shuffle – RC



Concluzii Structuri SIMD

- Probleme adecvate structurilor SIMD:
  - Calculele trebuiesc descrise de iteratii de tip vector a.i. majoritatea timpului sa fie ocupat cu operatii cu caracter identic
  - Operanzii (simultani) trebuiesc repartizati in module diferite de memorie
  - Trebuiesc identificate conexiunile intre operanzi a.i. ele sa fie cat mai aproape de conexiunile fizice intre procesoare
- Clasa problemelor SIMD nu este universala → doar probleme dedicate se preteaza!
- Algoritmii, sistemele de interconectare si structura datelor trebuiesc adaptate in consecinta

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 8
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

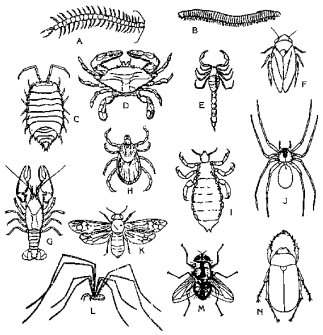
Cuprins

- Debugging and Performance Evaluation
- Performance Issues
- Profiling and Tracing
- Performance Counters

Debugging and Performance Evaluation

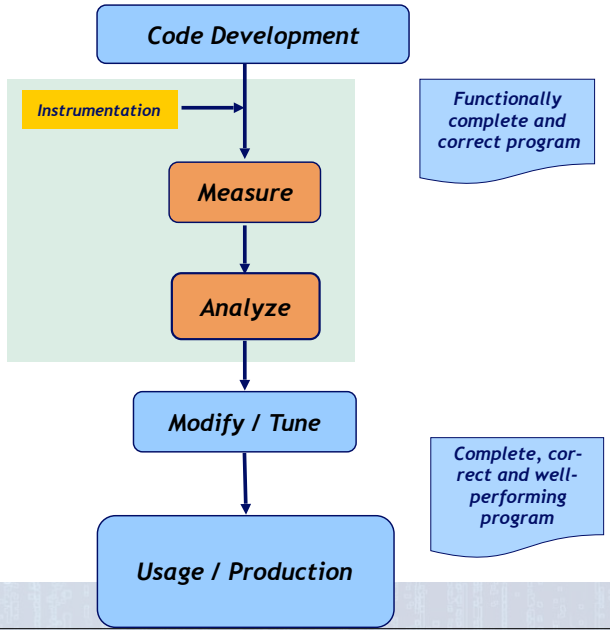
Debugging and Performance Evaluation

- Common errors in parallel programs
- Debugging tools
- Overview of benchmarking and performance measurements

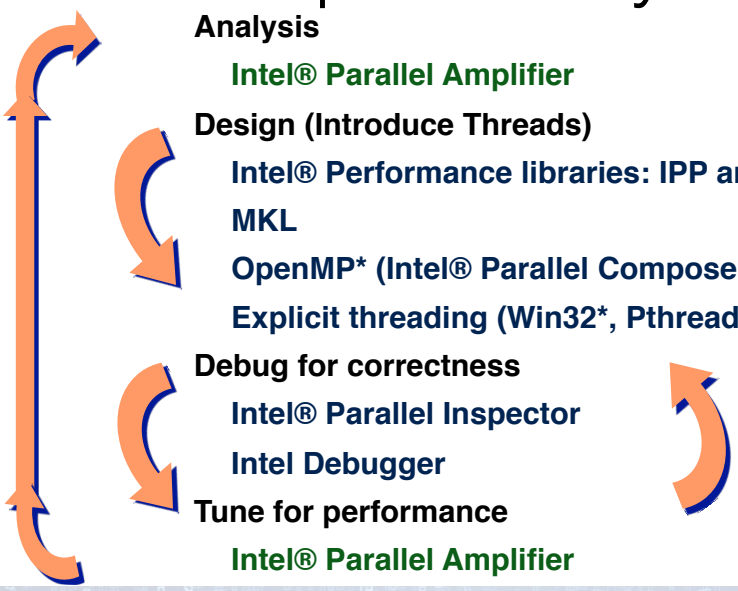


Concepts and Definitions

- The typical performance optimization cycle



Development Cycle

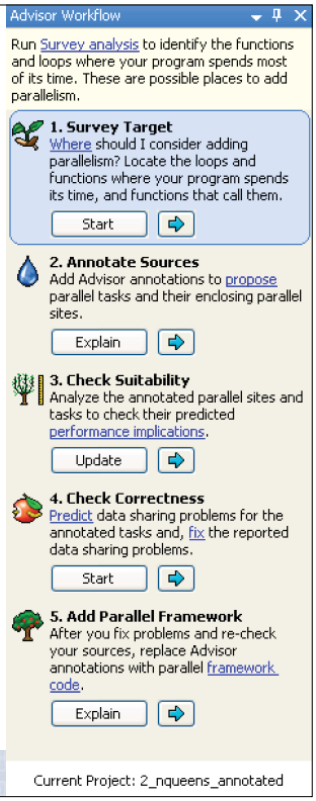


Intel® Parallel Studio

- Decide where to add the parallelism
  - Analyze the serial program
  - Prepare it for parallelism
- Add the parallelism
  - Threads, CUDA, OpenMP, Cilk, TBB, etc.
- Find logic problems
  - Only fails sometimes
  - Place of failure changes
- Find performance problems

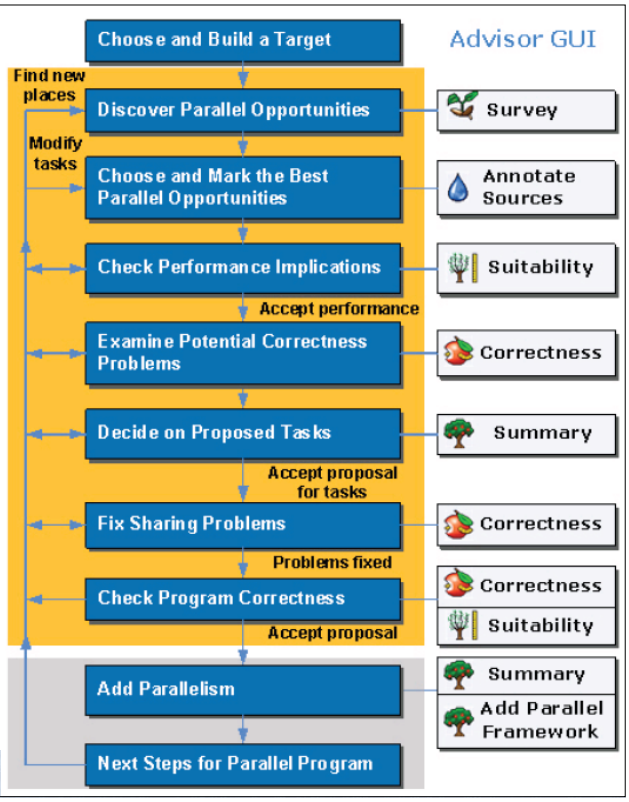
Workflow

- Transforming many serial algorithms into parallel form takes five easy high-level steps
- Often existing algorithms are over-constrained by serial language semantics, and the underlying mathematics has a natural parallel expression if you can just find it



Advisor Overview

- If you look at these steps in more detail, you find decisions you will need to make
- You do not have to choose the perfect answer the first time, so you can go back and modify your choices

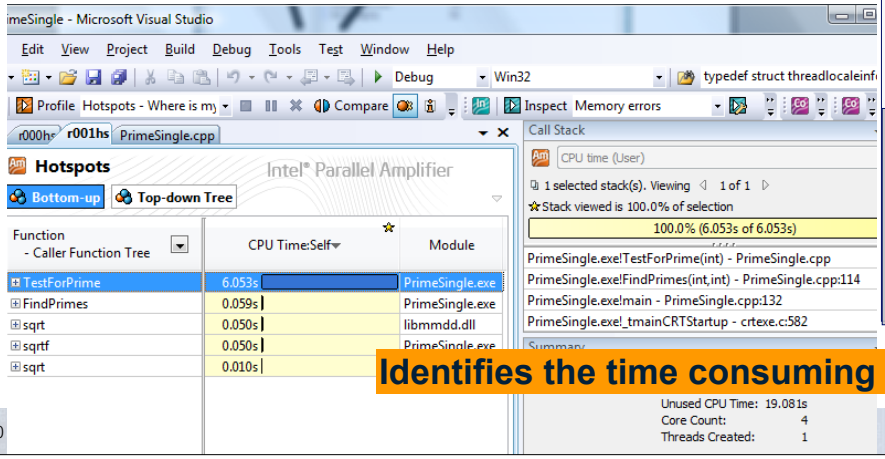


Hotspot Analysis

- Use Parallel Amplifier to find hotspots in application

```
bool TestForPrime(int val)
{   // let's start checking from 3
    int limit, factor = 3;
    limit = (long)(sqrtf((float)val)+0.5f);
    while( (factor <= limit) && (val % factor))
            factor ++;

    return (factor > limit);
}
```

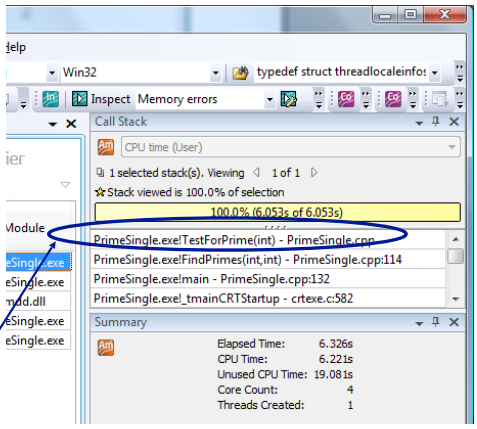
```
void FindPrimes(int start, int end)
{
    // start is always odd
    int range = end - start + 1;
    for( int i = start; i <= end; i+= 2 ){
        if( TestForPrime(i) )
            globalPrimes[gPrimesFound++] = i;
        ShowProgress(i, range);
    }
}
```



Identifies the time consuming regions

Analysis - Call Stack

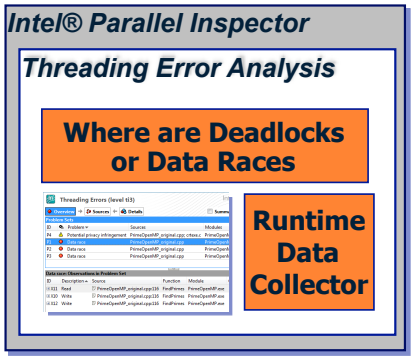
- Inspect the code for the leaf node
- Look for a loop to parallelize
  - If none are found, progress up the call stack until you find a suitable loop or function call to parallelize (FindPrimes)



Used to find proper level in the call-tree to thread

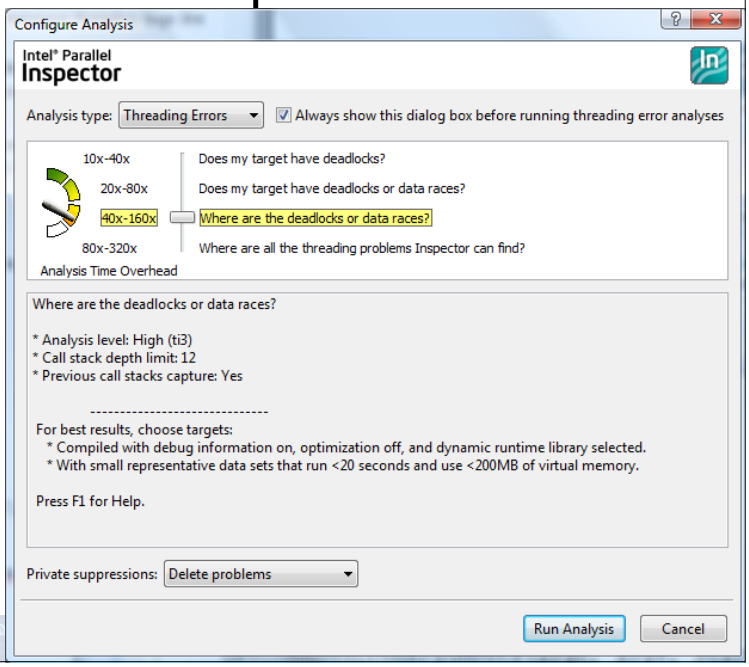
Debugging for Correctness

- Intel® Parallel Inspector pinpoints notorious threading bugs like data races and deadlocks



Intel® Parallel Inspector

- Select info regarding both data races & deadlocks
- View the Overview for Threading Errors
- Select a threading error and inspect the code



Motivation

- Developing threaded applications can be a complex task
- New class of problems are caused by the interaction between concurrent threads
  - Data races or storage conflicts
    - More than one thread accesses memory without synchronization
  - Deadlocks
    - Thread waits for an event that will never happen

Intel® Parallel Inspector

- Debugging tool for threaded software
  - Plug-in to Microsoft* Visual Studio*
- Finds threading bugs in OpenMP*, Intel® Threading Building Blocks, and Win32* threaded software
- Locates bugs quickly that can take days to find using traditional methods and tools
  - Isolates **problems**, not the **symptoms**
  - **Bug does not have to occur to find it!**

Parallel Inspector: Analysis

- Dynamic as software runs
  - Data (workload)-driven execution
- Includes monitoring of:
  - Thread and Sync APIs used
  - Thread execution order
    - Scheduler impacts results
  - Memory accesses between threads

Code path must be executed to be analyzed

Parallel Inspector: Before You Start

- Instrumentation: background
  - Adds calls to library to record information
    - Thread and Sync APIs
    - Memory accesses
  - Increases execution **time** and **size**
- Use **small** data sets (workloads)
  - Execution time and space is **expanded**
  - Multiple runs over different paths yield best results

Workload selection is important!

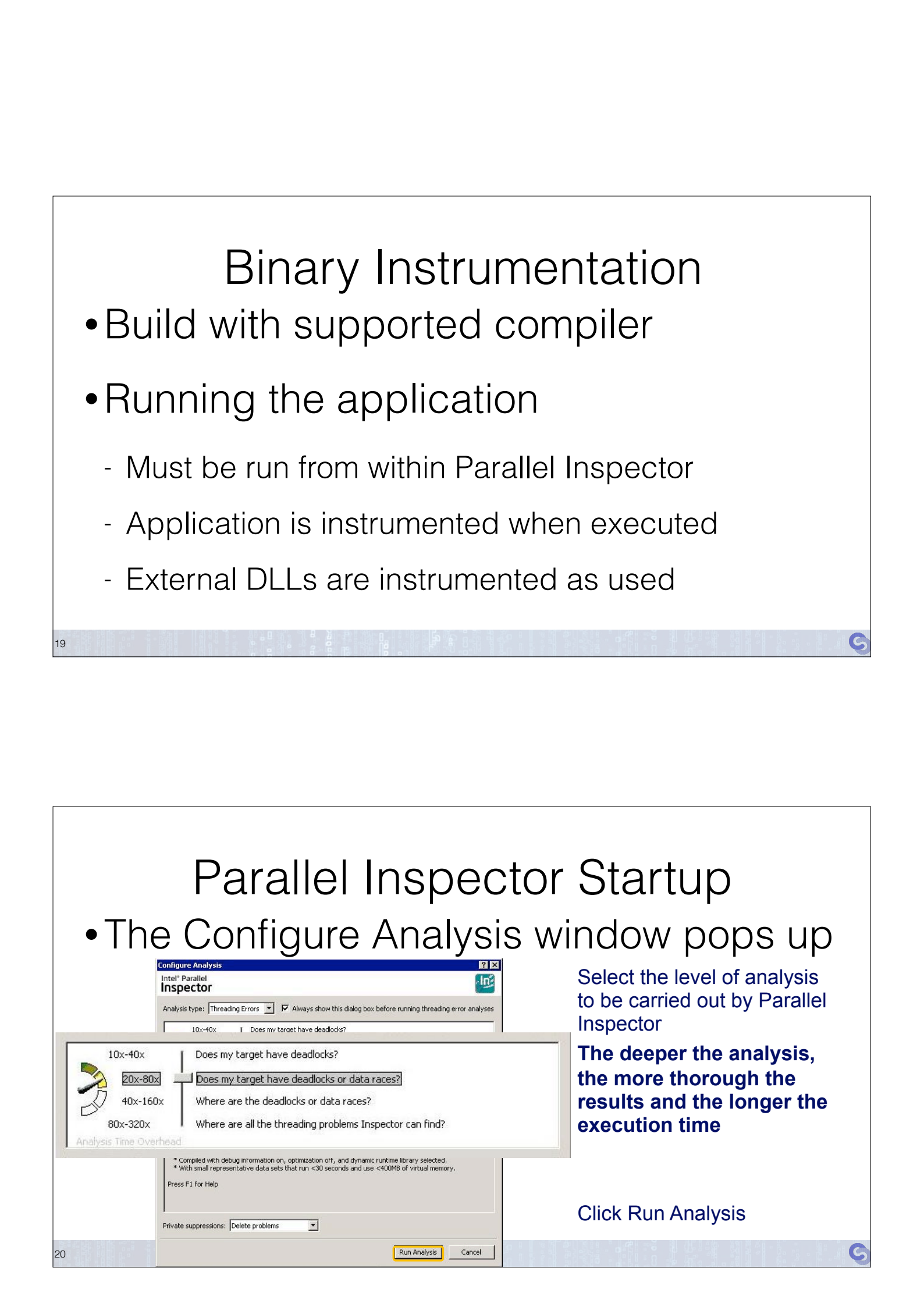
Workload Guidelines

- Execute problem code once per thread to be identified
- Use smallest possible working data set
  - Minimize data set size
    - Smaller image sizes
  - Minimize loop iterations or time steps
    - Simulate minutes rather than days
  - Minimize update rates
    - Lower frames per second

Finds threading errors faster!

Binary Instrumentation

- Build with supported compiler
- Running the application
  - Must be run from within Parallel Inspector
  - Application is instrumented when executed
  - External DLLs are instrumented as used



Parallel Inspector Startup

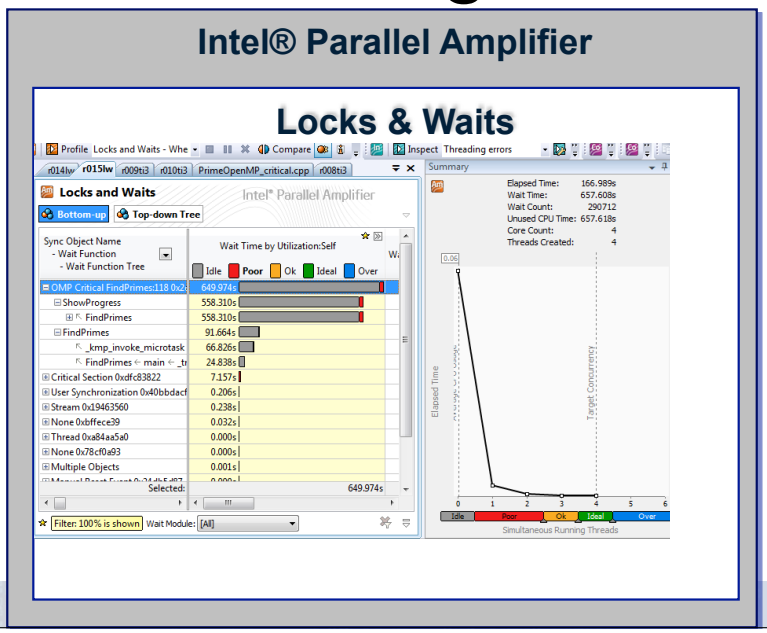
- The Configure Analysis window pops up

Select the level of analysis to be carried out by Parallel Inspector

The deeper the analysis, the more thorough the results and the longer the execution time

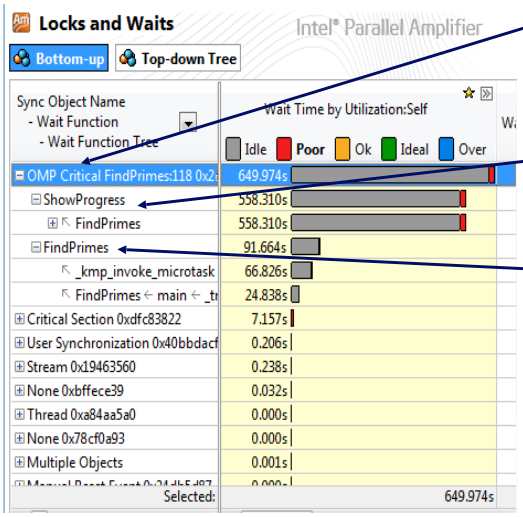
Click Run Analysis

Tuning for Performance



- Parallel Amplifier (Locks & Waits) pinpoints performance bottlenecks in threaded applications

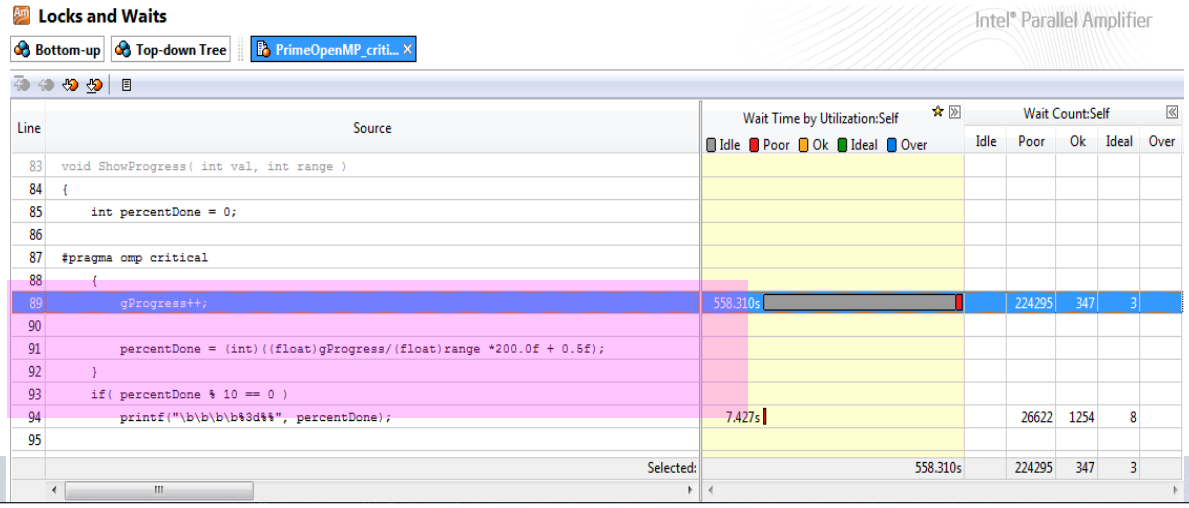
Parallel Amplifier - Locks & Waits



- Graph shows significant portion of time in idle condition as result of a critical section
- FindPrimes() & ShowProgress() are both excessively impacted by the idle time occurring in the critical section

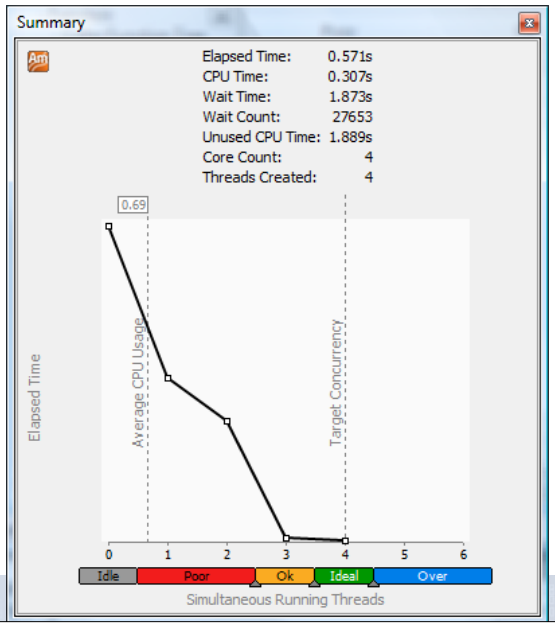
Parallel Amplifier - Locks & Waits

- ShowProgress() consumes 558/657 (85%) of the time idling in a critical section
- Double Click ShowProgress() in largest critical section to see the source code



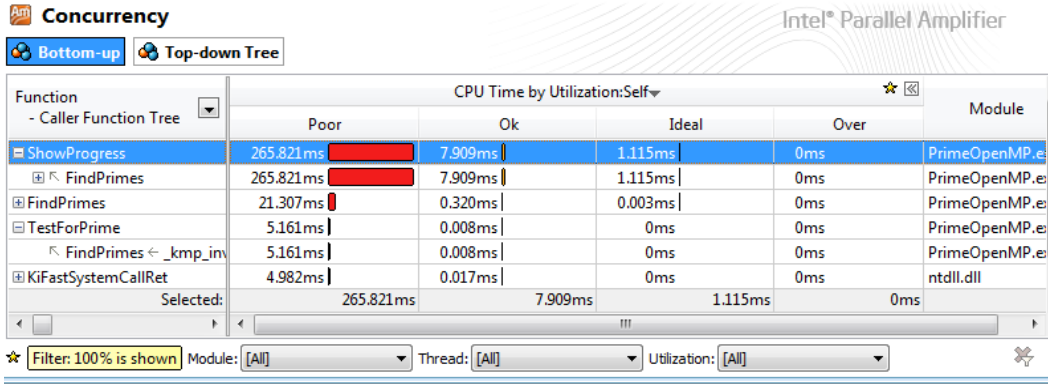
Parallel Amplifier Summary

- Elapsed Time shows 0.571 sec
- Wait Time/ Core Count:
  - 1.87/4 =0.47 sec
- Waiting 82% of elapsed time in critical section
- Most of the time 1 core and occasionally 2 are occupied



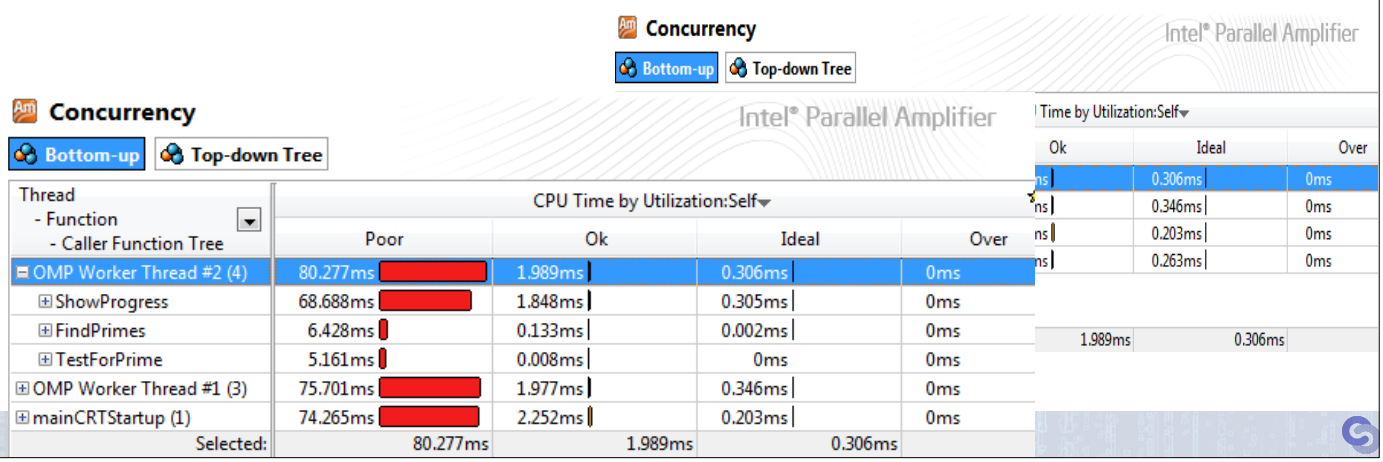
Parallel Amplifier - Concurrency

- Function - Caller Function Tree
- ShowProgress is called from FindPrimes and represent the biggest reason concurrency is poor



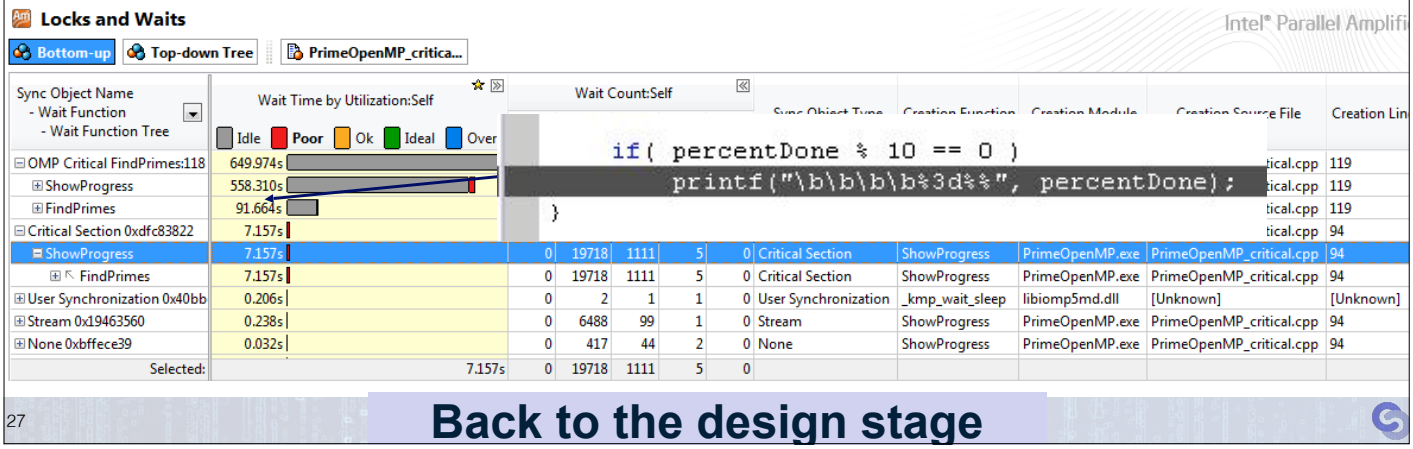
Parallel Amplifier - Concurrency

- Thread –Function –Caller Function Tree
  - This view shows how each thread contributes to the concurrency issue
- Expanding any thread will reveal the functions that contribute most



Performance

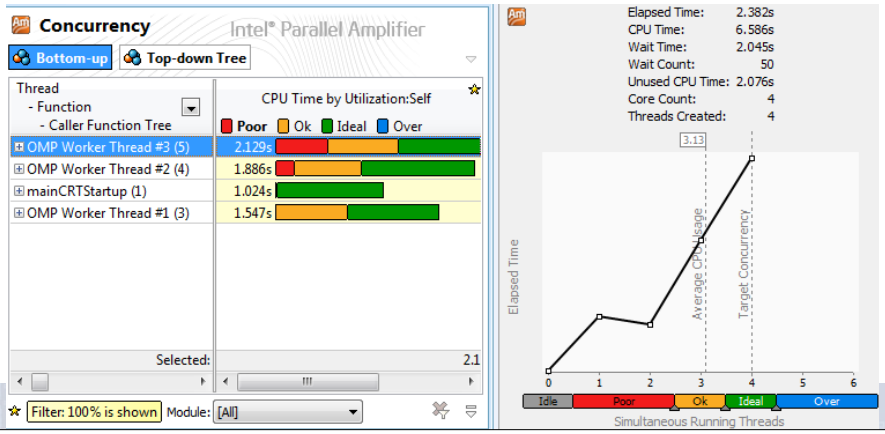
- Double Click ShowProgress in second largest critical section
- This implementation has implicit synchronization calls - printf
- This limits scaling performance due to the resulting context switches



Back to the design stage

Load Balance Analysis

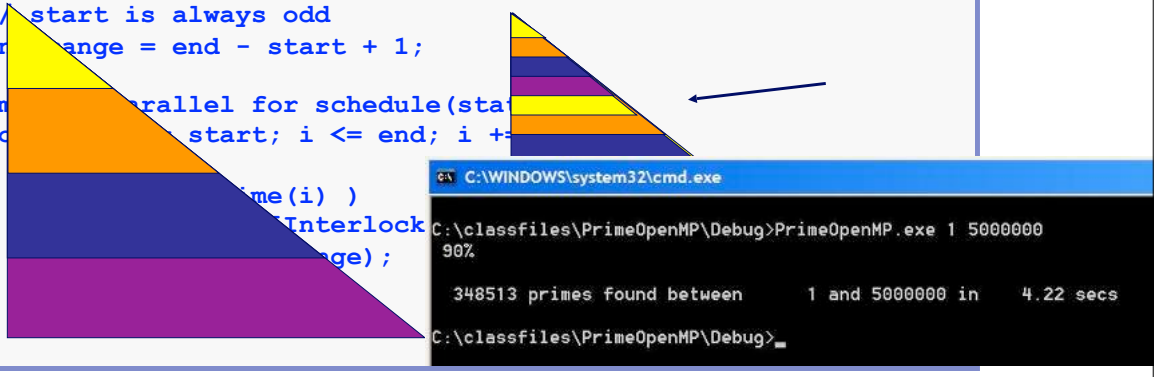
- Use Parallel Amplifier – Concurrency Analysis
- Select the “Thread – Function – Caller Function Tree”
- Observe that the 4 threads do unequal amounts of work



Fixing the Load Imbalance

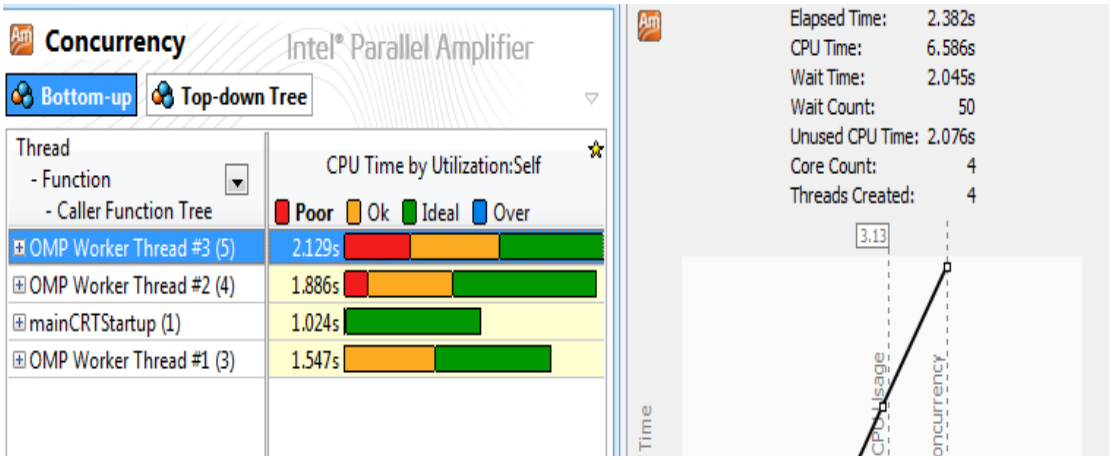
- Distribute the work more evenly

```
void FindPrimes(int start, int end)
{
    // start is always odd
    in  ange = end - start + 1;

#pragm  rallel for schedule(sta
    fo      start; i <= end; i +=
    {
                me(i) )
                Interlock
                ge);
    }
}
```



Speedup achieved is 1.68X

Comparative Analysis



Threading applications require multiple iterations of going through the software development cycle

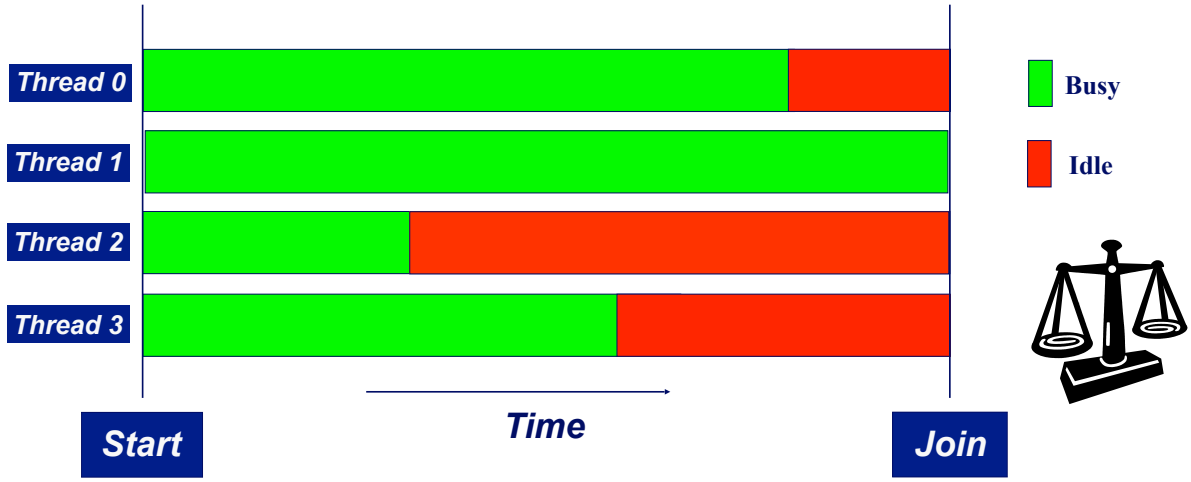
Performance Issues

Common Performance Issues

- **Load balance**
  - Improper distribution of parallel work
- **Synchronization**
  - Excessive use of global data / contention for same synchronization object
- Parallel Overhead
  - Due to thread creation, scheduling
- Granularity
  - Not sufficient parallel work

Load Imbalance

- Unequal work loads lead to idle threads and wasted time



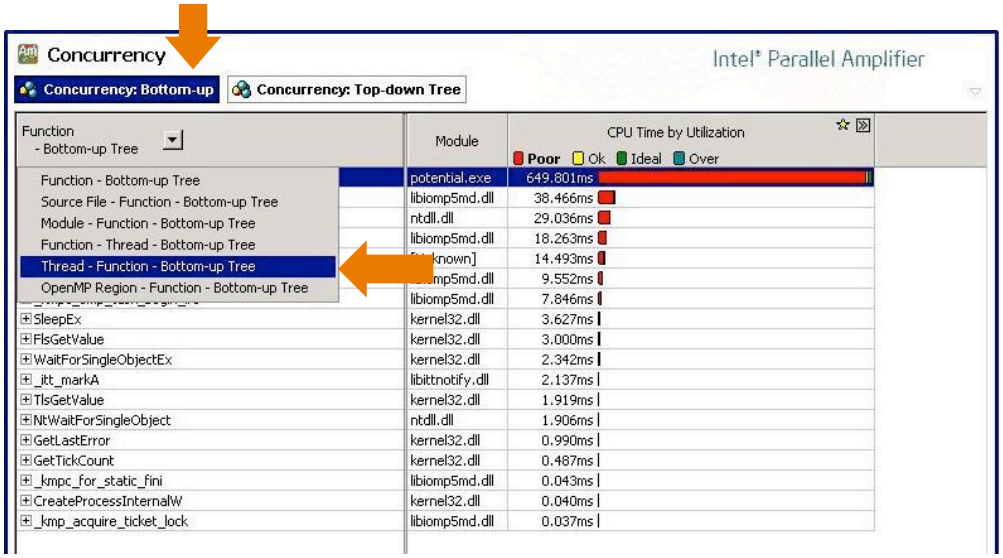
Redistribute Work to Threads

- Static assignment
  - Are the same number of tasks assigned to each thread?
  - Do tasks take different processing time?
    - Do tasks change in a predictable pattern?
    - Rearrange (static) order of assignment to threads
    - Use dynamic assignment of tasks

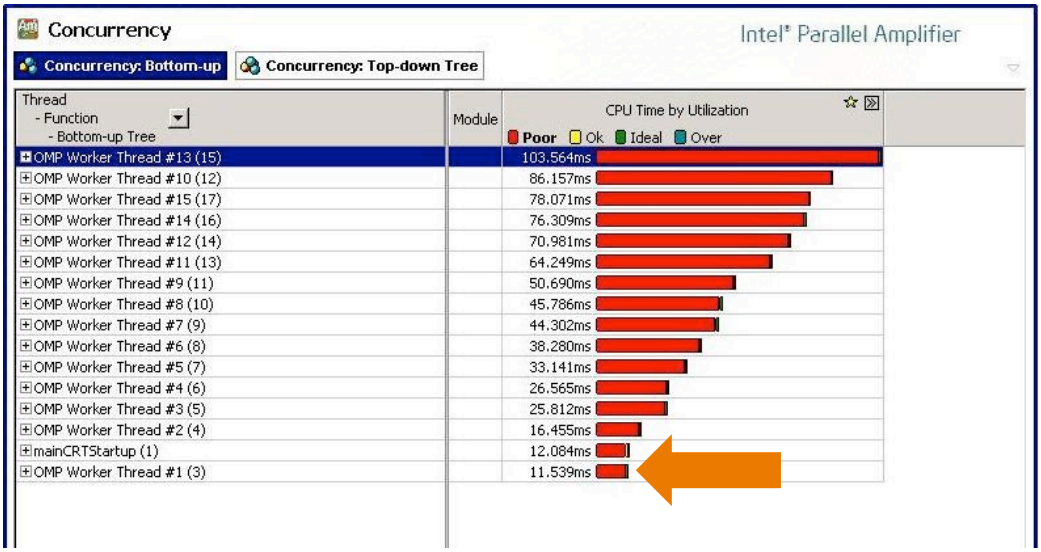
Redistribute Work to Threads

- Dynamic assignment
  - Is there one big task being assigned?
    - Break up large task to smaller parts
  - Are small computations agglomerated into larger task?
    - Adjust number of computations in a task?
    - More small computations into single task?
    - Fewer small computations into single task?

Unbalanced Workloads

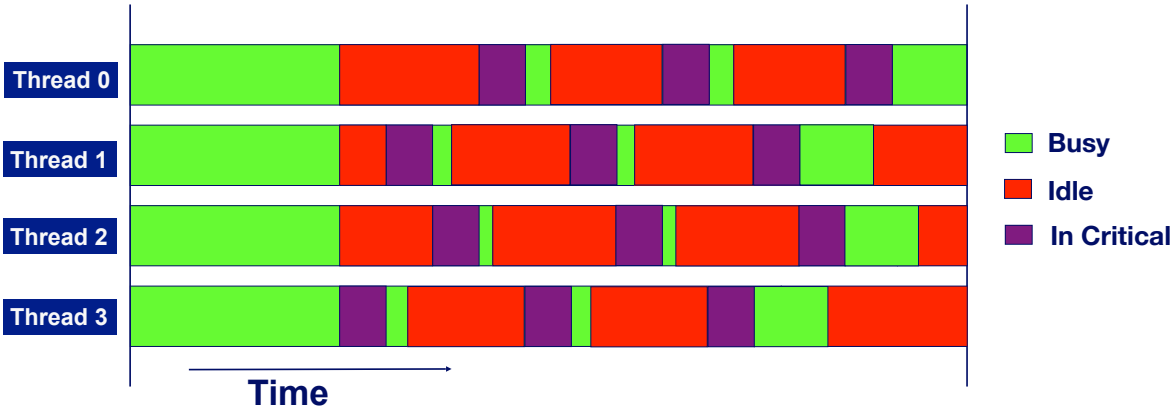


Unbalanced Workloads



Synchronization

- By definition, synchronization serializes execution
- Lock contention means more idle time for threads



Synchronization Fixes

- Eliminate synchronization
  - Expensive but necessary “evil”
  - Use storage local to threads
    - Use local variable for partial results, update global after local computations
    - Allocate space on thread stack (`alloca`)
    - Use thread-local storage API (`TlsAlloc`)
  - Use atomic updates whenever possible
    - Some global data updates can use atomic operations

General Optimizations

- Serial Optimizations
  - Serial optimizations along the critical path should affect execution time
- Parallel Optimizations
  - Reduce synchronization object contention
  - Balance workload
  - Functional parallelism
- Analyze benefit of increasing number of processors
- Analyze the effect of increasing the number of threads on scaling performance

Profiling and Tracing

Measurement: Profiling vs. Tracing

- Profiling
  - Summary statistics of performance metrics
    - Number of times a routine was invoked
    - Exclusive & inclusive time
    - Hardware performance counters
    - Number of child routines invoked, etc.
    - Structure of invocations (call-trees/call-graphs)
    - Memory, message communication sizes
- Tracing
  - When and where events took place along a global timeline
    - Time-stamped log of events
    - Message communication events (sends/receives) are tracked
    - Shows when and from/to where messages were sent
    - Large volume of performance data generated leads to more perturbation in the program

Measurement: Profiling

- Profiling
  - Helps to expose performance bottlenecks and hotspots
  - 80/20 – rule or Pareto principle: often 80% of the execution time in 20% of your application
  - Optimize what matters, don't waste time optimizing things that have negligible overall influence on performance
- Implementation
  - Sampling: periodic OS interrupts or hardware counter traps
    - Build a histogram of sampled program counter (PC) values
    - Hotspots will show up as regions with many hits
  - Measurement: direct insertion of measurement code
    - Measure at start and end of regions of interests, compute difference

Measurement: Tracing

- Tracing
  - Recording of information about significant points (events) during program execution
    - Entering/exiting code region (function, loop, block, …)
    - Thread/process interactions (e.g., send/receive message)
  - Save information in event record
    - Timestamp
    - CPU identifier, thread identifier
    - Event type and event-specific information
  - Event trace is a time-sequenced stream of event records
  - Can be used to reconstruct dynamic program behavior
  - Typically requires code instrumentation

Performance Counters

Hardware Performance Counters

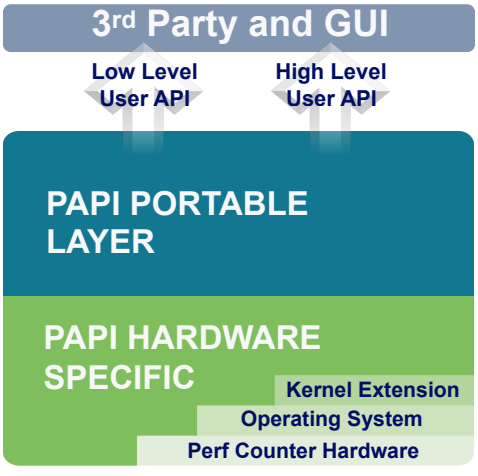
- Specialized hardware registers to measure the performance of various aspects of a microprocessor
- Originally used for hardware verification purposes
- Can provide insight into:
  - Cache behavior
  - Branching behavior
  - Memory and resource contention and access patterns
  - Pipeline stalls
  - Floating point efficiency
  - Instructions per cycle
- Counters vs. events
  - Usually a large number of countable events - hundreds
  - On a small number of counters (4-18)

What is PAPI

- Middleware that provides a consistent and efficient programming interface for the performance counter hardware found in most major microprocessors.
- Countable events are defined in two ways:
  - Platform-neutral Preset Events (e.g., PAPI_TOT_INS)
  - Platform-dependent Native Events (e.g., L3_CACHE_MISS)
- Preset Events can be derived from multiple Native Events
  - PAPI_L1_TCM (Total Cache Misses) **might** L1 Data Misses + L1 Instruction Misses
- Preset events are defined in a best-effort way
  - No guarantees of semantics portably
  - Figuring out what a counter actually counts and if it does so correctly can be hairy

PAPI Counter Interfaces

- PAPI provides 3 interfaces to the underlying counter hardware:
  - A low level API manages **hardware events** in user defined groups called EventSets.
  - A high level API provides the ability to **start**, **stop** and **read** the **counters** for a specified list of events.
  - Graphical and end-user tools provide facile data collection and visualization.



PAPI Example Low Level API Usage

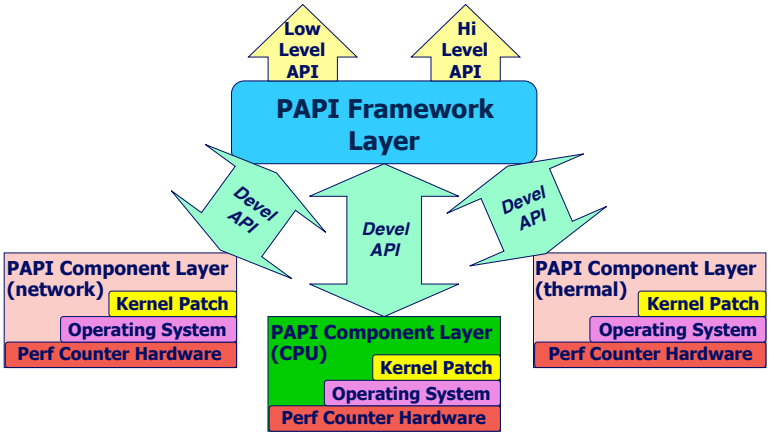
```
#include "papi.h"
#define NUM_EVENTS 2
int Events[NUM_EVENTS]={PAPI_FP_OPS,PAPI_TOT_CYC},
int EventSet;
long long values[NUM_EVENTS];
/* Initialize the Library */
retval = PAPI_library_init (PAPI_VER_CURRENT);
/* Allocate space for the new eventset and do setup */
retval = PAPI_create_eventset (&EventSet);
/* Add Flops and total cycles to the eventset */
retval = PAPI_add_events (&EventSet,Events,NUM_EVENTS);
/* Start the counters */
retval = PAPI_start (EventSet);

do_work();  /* What we want to monitor*/

/*Stop counters and store results in values */
retval = PAPI_stop (EventSet,values);
```

Using PAPI Through Tools

- You can use PAPI directly in your application, but most people use it through tools
- The tool might have a predefined set of counters or lets you select counters through a configuration file/environment variable, etc.
- Tools using PAPI
  - Scalasca, KOJAK (FZ Juelich, UTK)
  - TAU (UO)
  - PerfSuite (NCSA)
  - HPCToolkit (Rice)
  - Open|Speedshop (SGI)
  - ompP (UCB)
  - IPM (LBNL)

Component PAPI Design

Re-Implementation of PAPI w/ support for multiple monitoring domains (CPU, network, thermal)



Ce am invatat in aces curs?

- Debugging and Performance Evaluation
- Performance Issues
- Profiling and Tracing
- Performance Counters

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 9
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

In cursul anterior

- Debugging and Performance Evaluation
- Performance Issues
- Profiling and Tracing
- Performance Counters

Cuprins

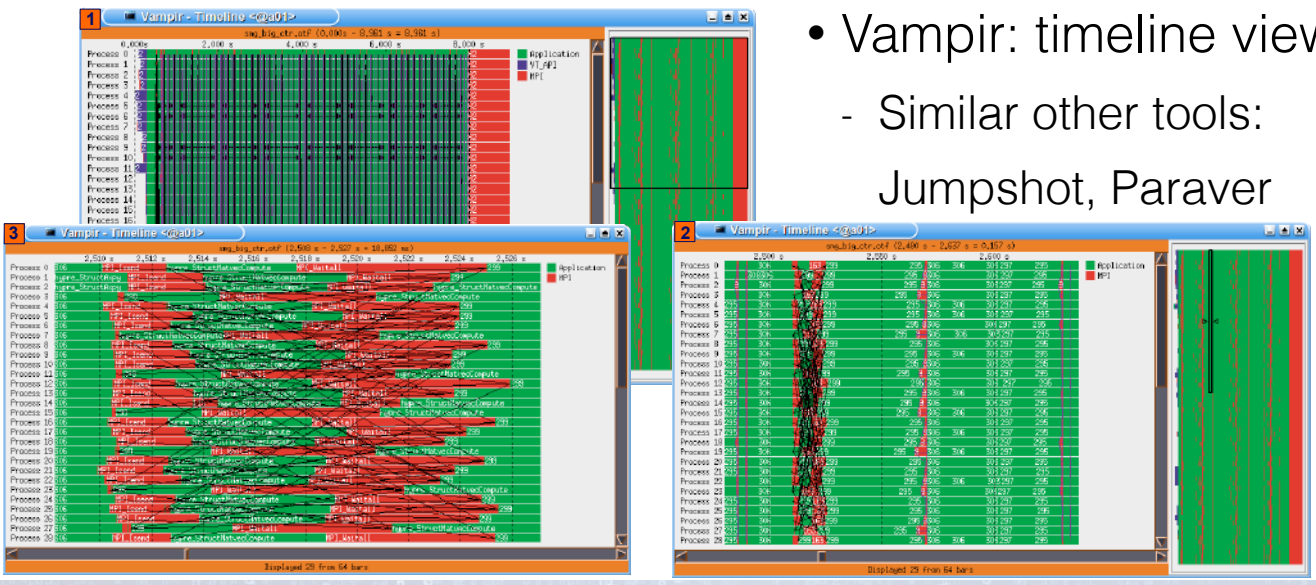
- Performance Analysis
  - Programming
  - Tools
  - Techniques
- Parallel Correctness Challenges
  - Debugging
  - Testing

Performance Analysis - Tools & Techniques

Performance Data Analysis

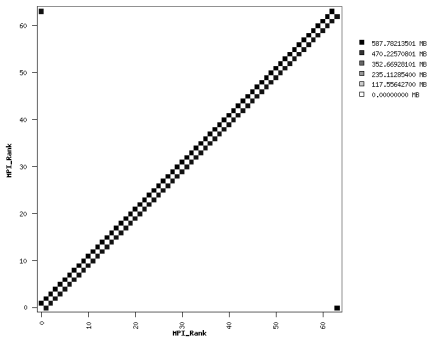
- Draw conclusions from measured performance data
- Manual analysis
  - Visualization
  - Interactive exploration
  - Statistical analysis
  - Modeling
- Automated analysis
  - Try to cope with huge amounts of performance by automation
  - Examples: Paradyn, KOJAK, Scalasca, Periscope

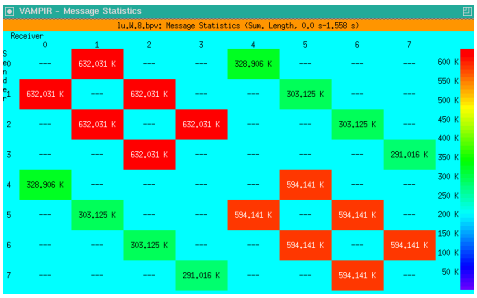
Trace File Visualization

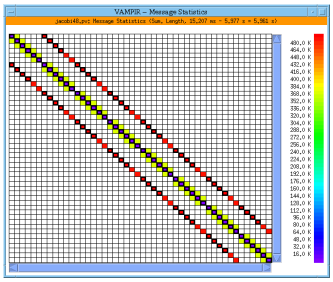


- Vampir: timeline view
  - Similar other tools: Jumpshot, Paraver

Trace File Visualization



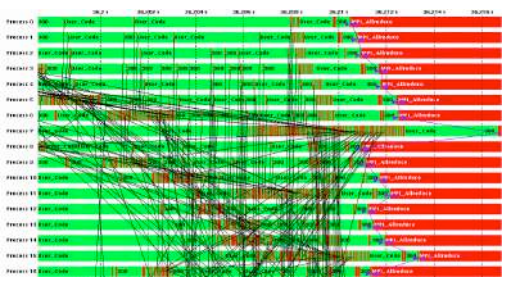




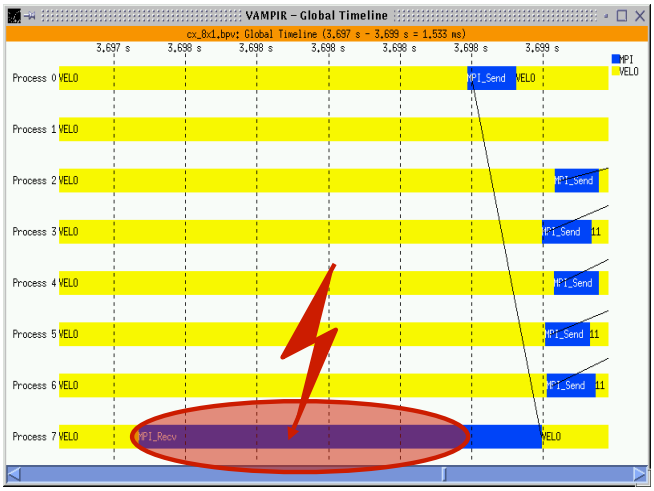
- Vampir/IPM: message communication statistics

Automated Performance Analysis

- Reason for Automation
  - Size of systems: several tens of thousand of processors
  - LLNL Sequoia: 1.6 million cores
  - Trend to multi-core
- Large amounts of performance data
  - Several gigabytes or even terabytes
- Not all programmers are performance experts
  - Scientists want to focus on their domain
  - Need to keep up with new machines
- Automation can solve some of these issues

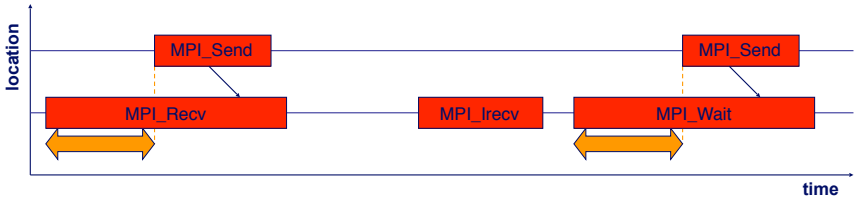


Automation Example

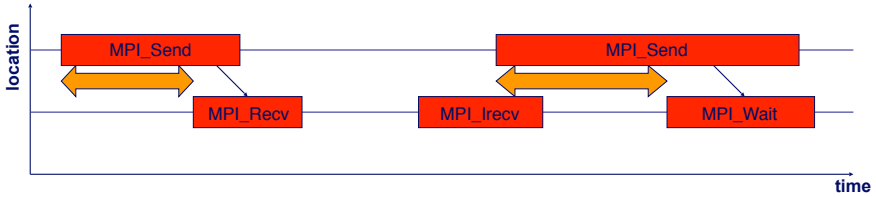


- „Late sender“ pattern
- This pattern can be detected automatically by analyzing the trace

MPI Pattern: Late Sender / Receiver

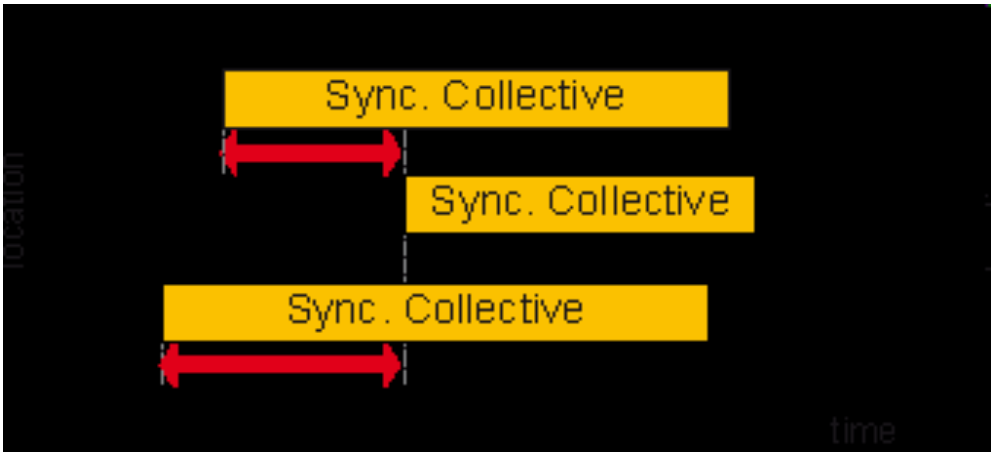


- Late Sender: Time lost waiting caused by a blocking receive operation posted earlier than the corresponding send operation



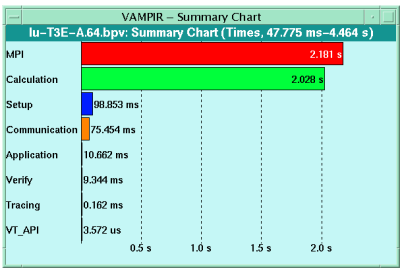
- Late Receiver: Time lost waiting in a blocking send operation until the corresponding receive operation is called

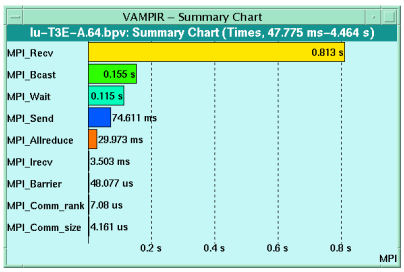
MPI Pattern: Wait at Barrier



- Time spent in front of MPI synchronizing operation such as barriers

Vampir Overview Statistics



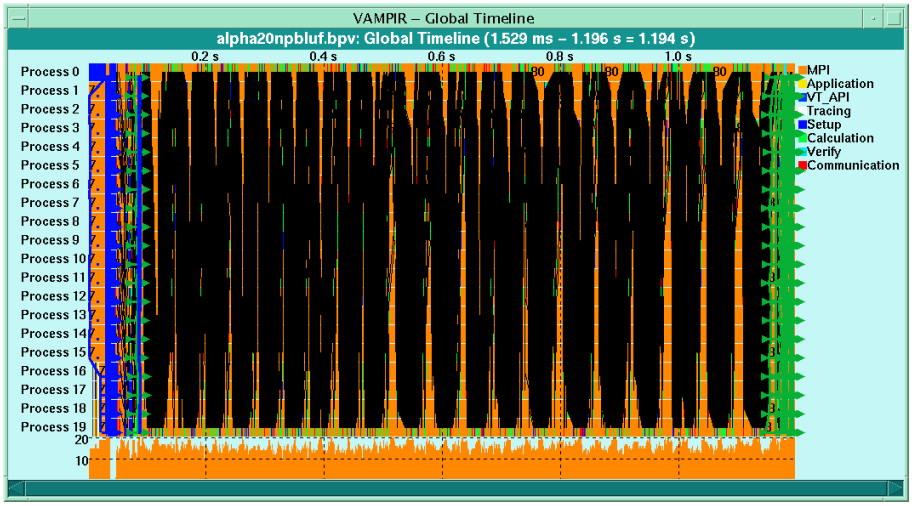


- Aggregated profiling information
  - Execution time
  - Number of calls
- This profiling information is computed from the trace
  - Change the selection in main timeline window
- Inclusive or exclusive of called routines

Timeline Display – Contents

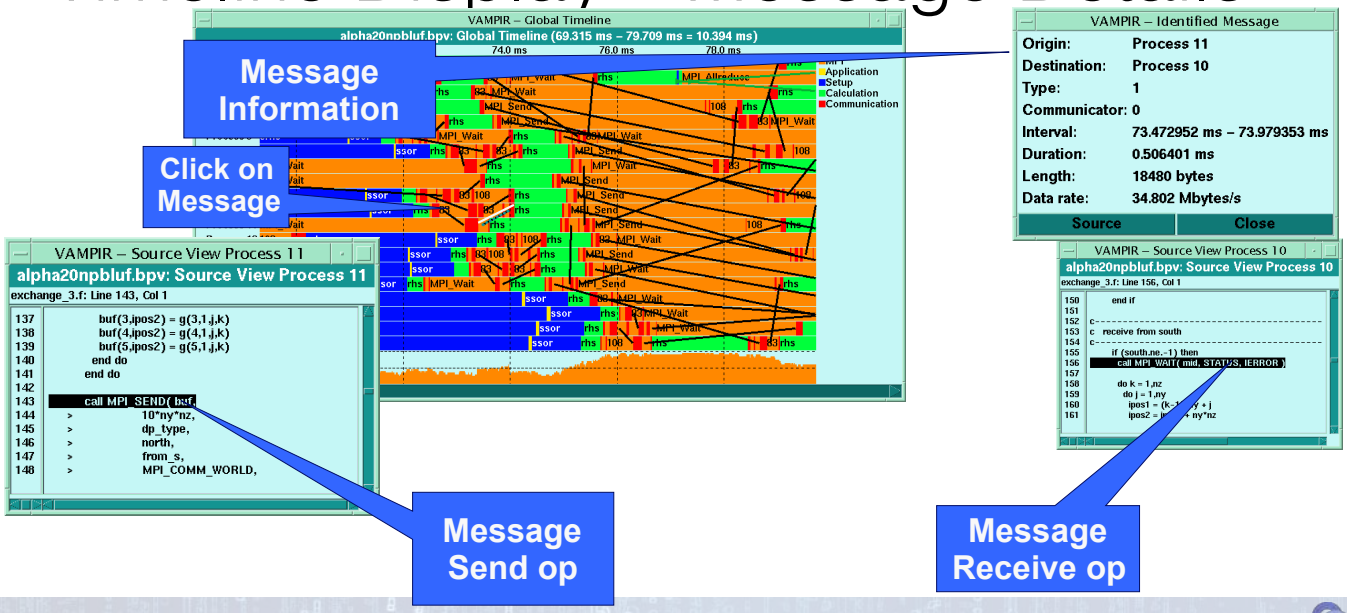
- Shows all selected processes
- Shows state changes (activity color)
- Shows messages, collective and MPI–IO operations
- Can show parallelism display at the bottom

Timeline Display

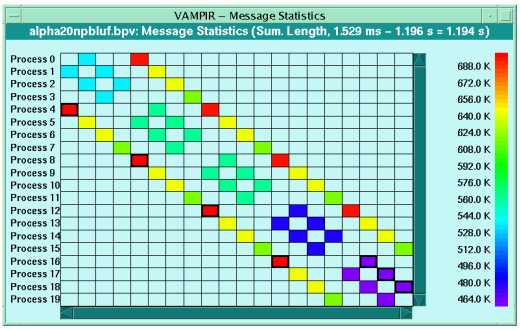


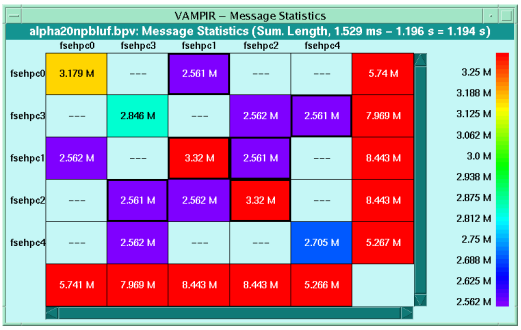
- To zoom, mark region with the mouse

Timeline Display – Message Details



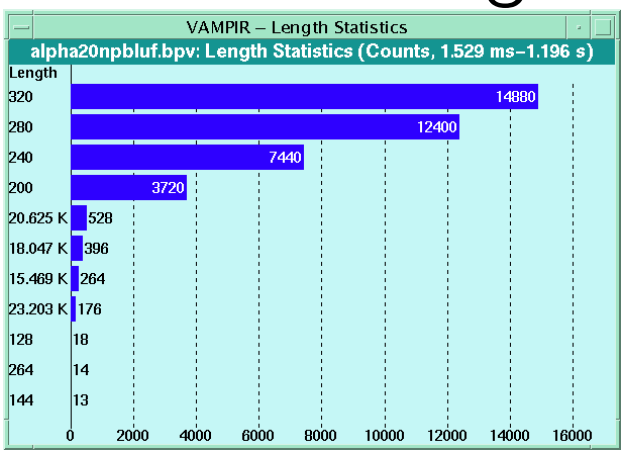
Communication Statistics

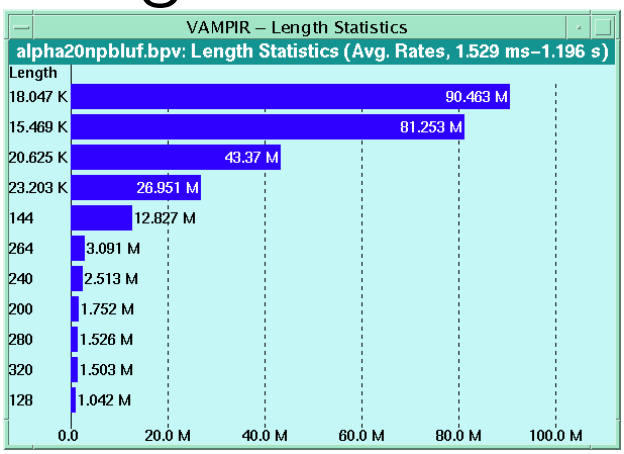




- Message statistics for each process/node pair:
  - Byte and message count
  - Min/max/avg message length, bandwidth

Message Histograms

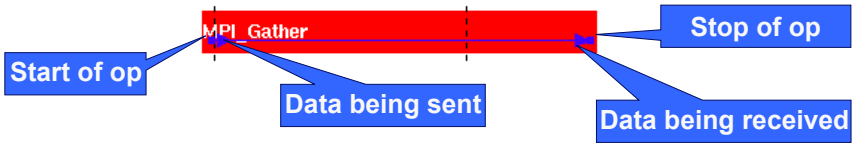


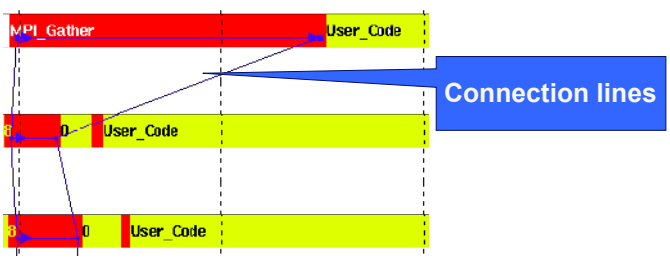


- Message statistics by length, tag or communicator
  - Byte and message count
  - Min/max/avg bandwidth

Collective Operations

- For each process: mark operation locally

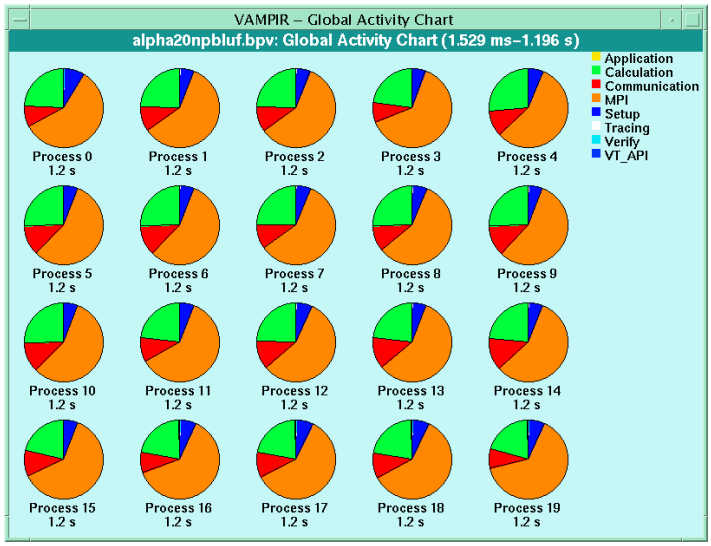




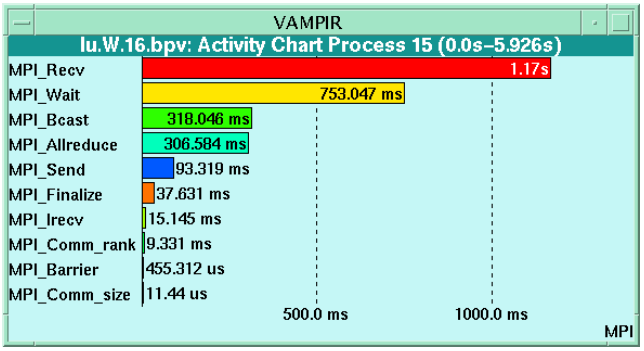
- Connect start/stop points by lines

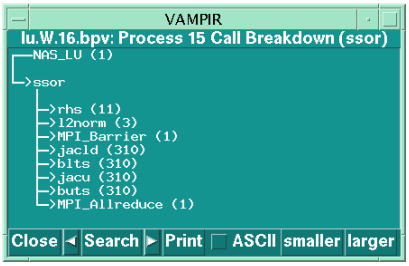
Activity Chart

- Profiling information for all processes

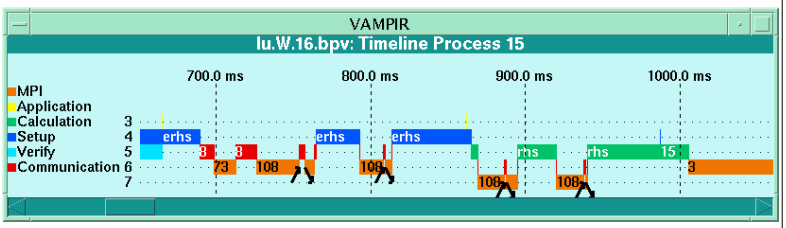


Process – Local Displays

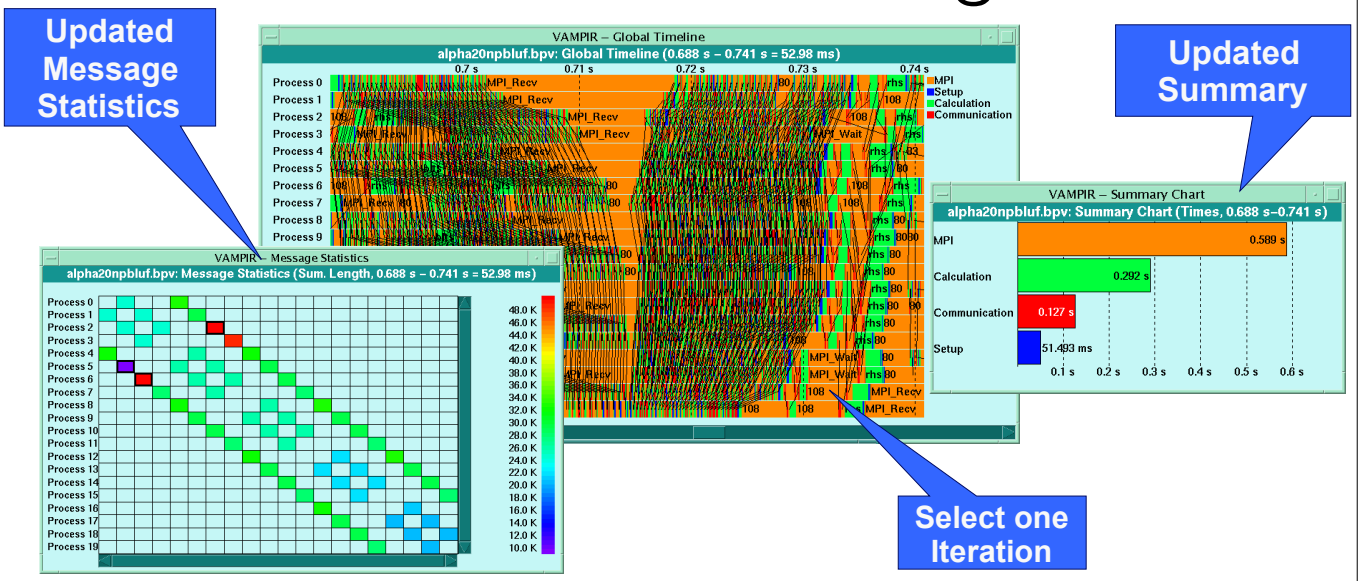




- Timeline: calling levels
- Activity chart
- Calling tree: number of calls

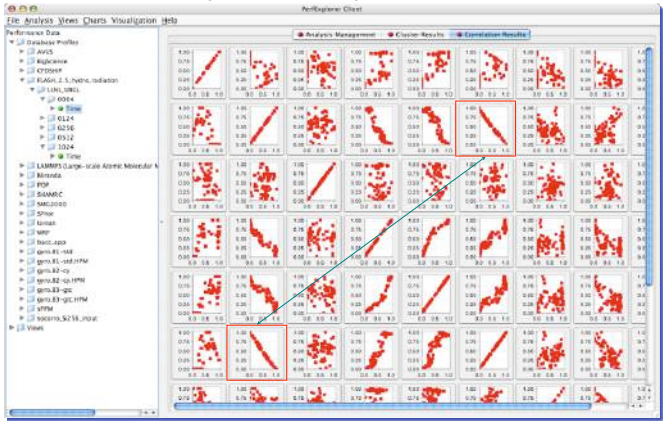


Effects of Zooming



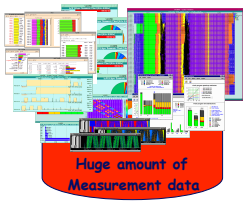
PerfExplorer – Correlation Analysis (Flash)

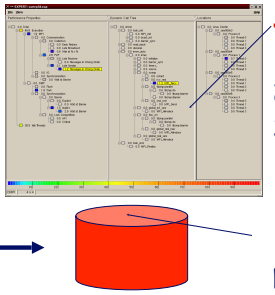
- Describes strength and direction of a linear relationship between two variables (events) in the data



Scalasca / KOJAK – Basic Idea

- "Traditional" Tool
- Automatic Tool





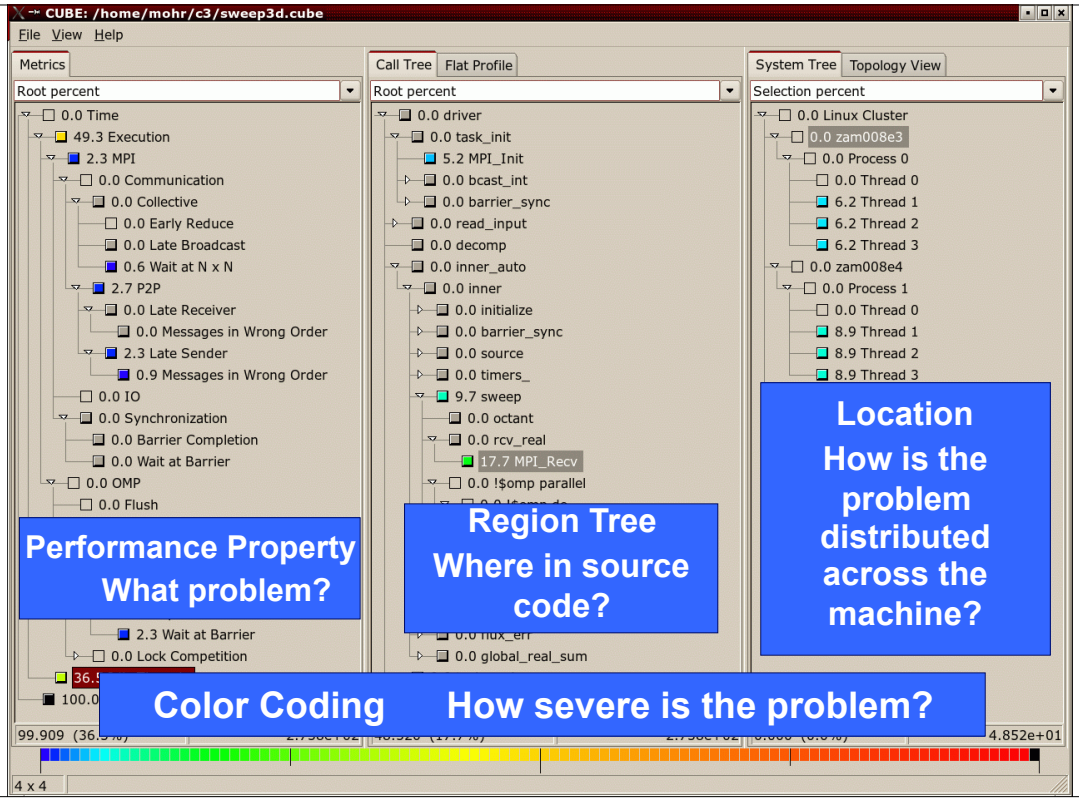
Simple:
1 screen +
2 commands +
3 panes

Relevant problems and data

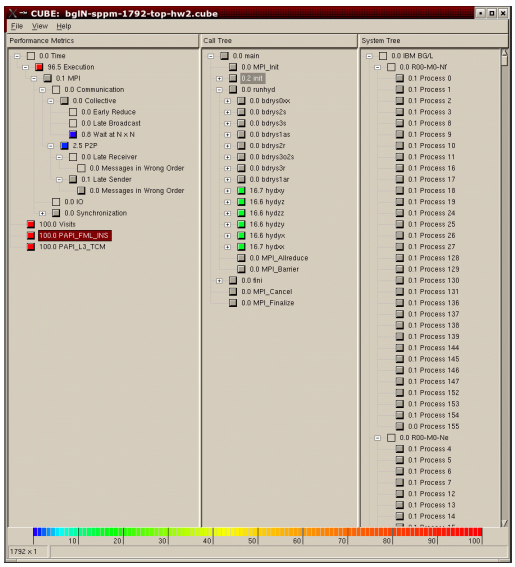
- For non-standard / tricky cases (10%)
- For expert users

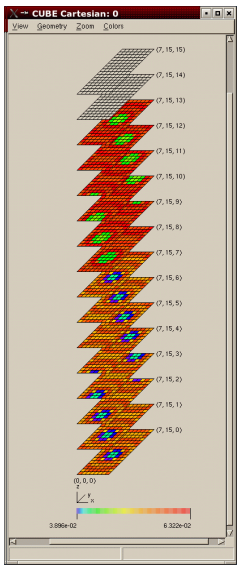
- For standard cases (90% ?!)
- For "normal" users
- Starting point for experts

⇨ **More productivity for performance analysis process!**



KOJAK: sPPM run on (8x16x14) 1792 PEs





- Topology display
- Shows distribution of pattern over HW topology
- Easily scales to even larger systems

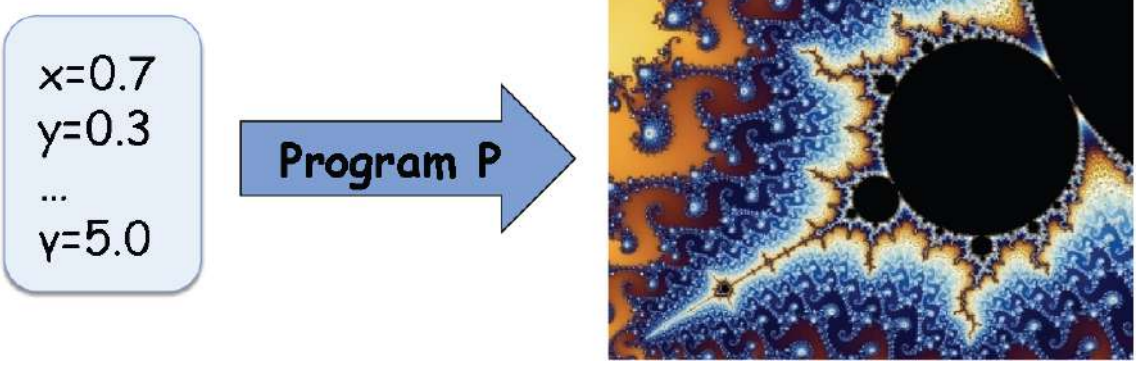
Parallel Correctness Challenges

Parallel Correctness Challenges

- Parallel programming presents a number of new challenges to writing correct software
  - New kinds of bugs: data races, deadlocks, etc.
  - More difficult to test programs and find bugs
  - More difficult to reproduce errors
- Key Difficulty: Potential non-determinism
  - Order in which threads execute can change from run to run
  - Some runs are correct while others hit bugs

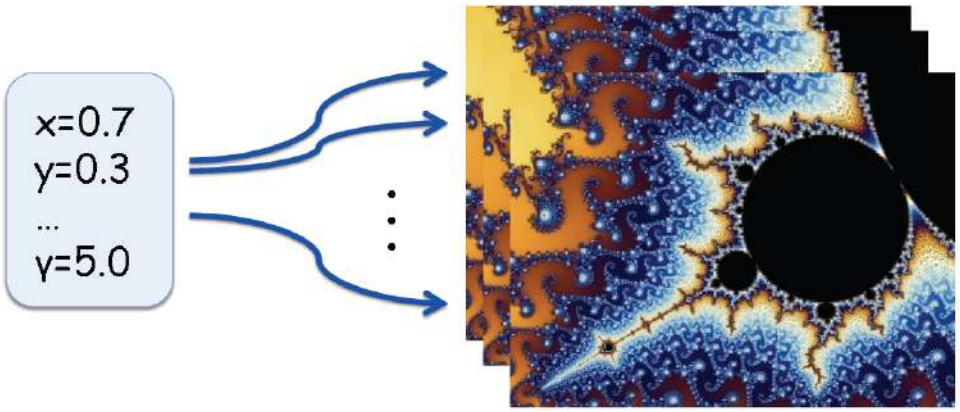
Parallel Correctness Challenges

- For sequential programs, we typically expect that same input → same output



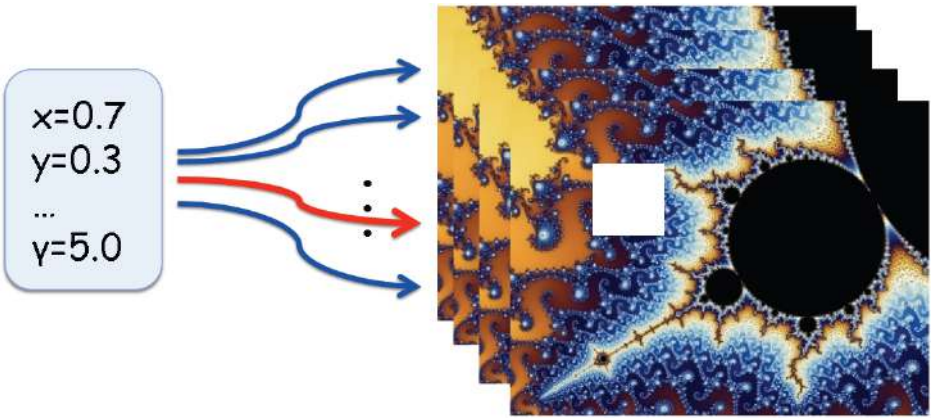
Parallel Correctness Challenges

- But for parallel programs, threads can be scheduled differently each run



Parallel Correctness Challenges

- But for parallel programs, threads can be scheduled differently each run



Parallel Correctness Challenges

- But for parallel programs, threads can be scheduled differently each run
- A bug may occur under only rare schedules
  - In 1 run in 1000 or 10,000 or …
- May occur only under some configurations:
  - Particular OS scheduler
  - When machine is under heavy load (> 80%)
  - Only when debugging/logging is turned off!

Testing Parallel Programs

- For **sequential** programs:
  - Create several test inputs with known answers.
  - Run the code on each test input.
  - If all tests give correct output, have some confidence in the program
  - Have intuition about which “edge cases” to test.
- But for **parallel** programs:
  - Each run tests only a single schedule.
  - How can we test many different schedules?
  - How confident can we be when our tests pass?

Testing Parallel Programs

- Possible Idea: Can we just run each test thousands of times?
- Problem: Often not much randomness in OS scheduling
  - May waste much effort, but test few different schedules
  - Recall: Some schedules tend to occur only under certain configurations – hardware, OS, etc
  - One easy parameter to change: load on machine

Stress Testing

- Idea: Test parallel program while oversubscribing the machine
  - On a 4-core system, run with 8 or 16 threads
  - Run several instances of the program at a time
  - Increase size to overflow cache/memory
  - Effect: Timing of threads will change, leading to
    - **Different thread schedules**
- Pro: Very simple idea, easy to implement
  - And often works!

Noise Making / Random Scheduling

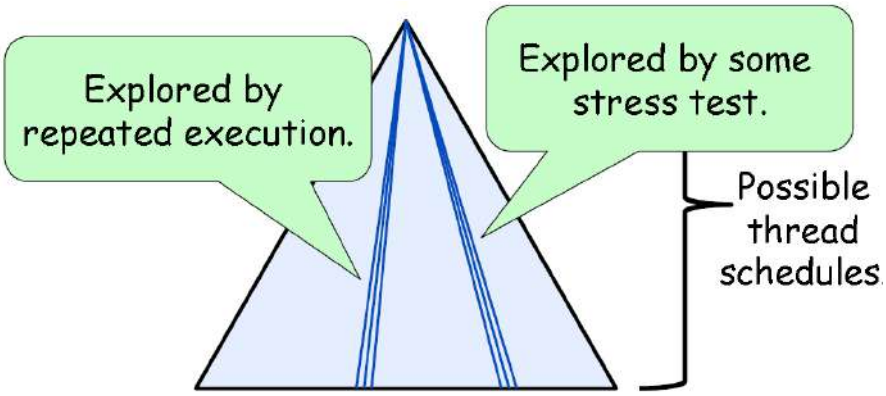
- Idea: Run with random thread schedules
  - E.g., insert code like:
    - if (rand() < 0.01) usleep(100)
    - if (rand() < 0.01) yield()
  - Can add to “suspicious” or “tricky” code
  - Or use tool to seize control of thread scheduling
- Pros: Still fairly simple and often effective
  - Explores different schedules than stress testing
  - Many tools can perform this automatically

Noise Making / Random Scheduling

- IBM’s ConTest: Noise-making - Java
- Berkeley’s Thrille - C + pThreads)
- CalFuzzer does simple random scheduling - Java
- Microsoft Research’s Cuzz - .NET
- Many of these tools provide replay
  - Same random number seed → same schedule

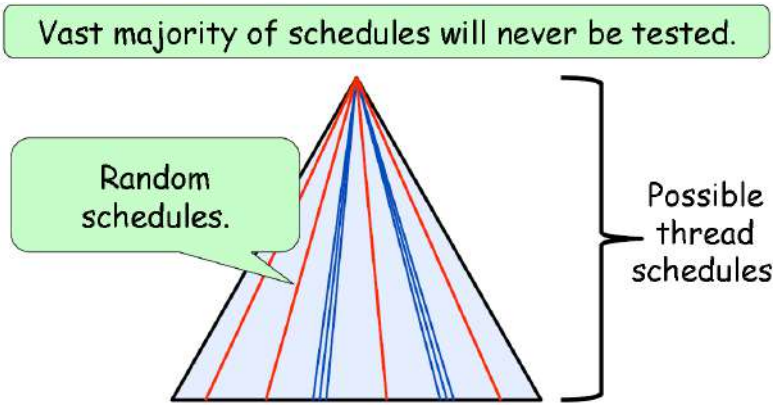
Limitations of Random Scheduling

- Parallel programs have huge number of schedules – exponential in length of a run



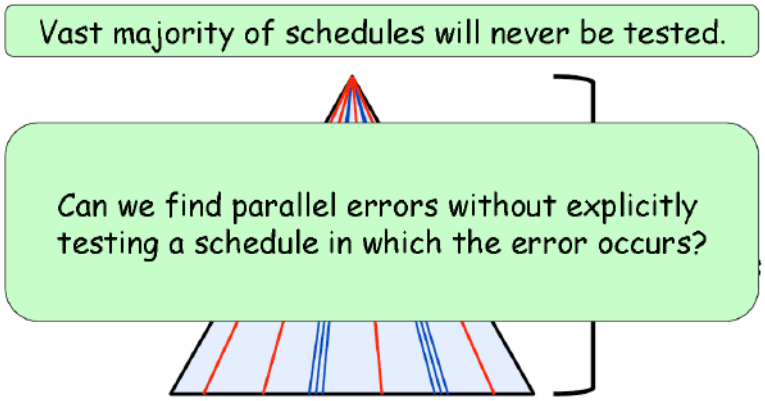
Limitations of Random Scheduling

- Parallel programs have huge number of schedules – exponential in length of a run



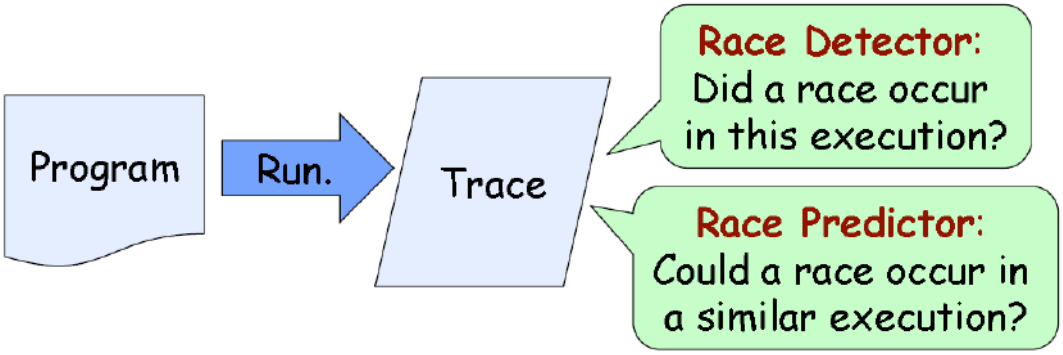
Limitations of Random Scheduling

- Parallel programs have huge number of schedules – exponential in length of a run



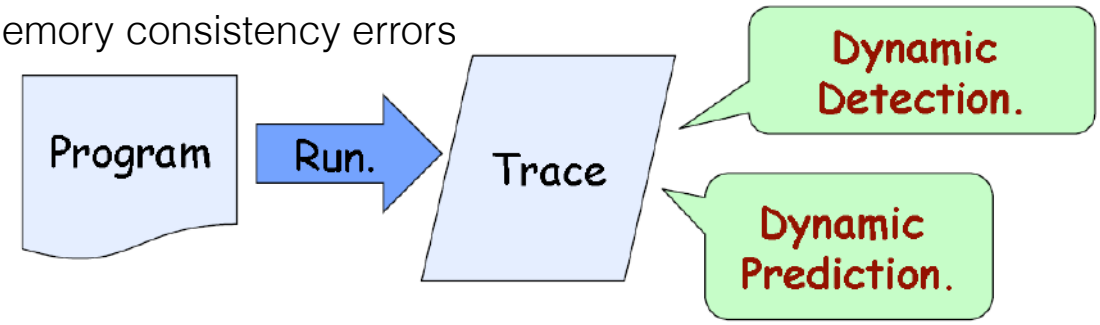
Detecting/Predicting Parallel Bugs

- Say we observe a test run of a parallel program that doesn't obviously fail
- Key Question: Can we find possible parallel bugs by examining the execution?



Detecting/Predicting Parallel Bugs

- Techniques/tools exist for:
  - Data races
  - Atomicity violations
  - Deadlocks
  - Memory consistency errors



Static Analysis

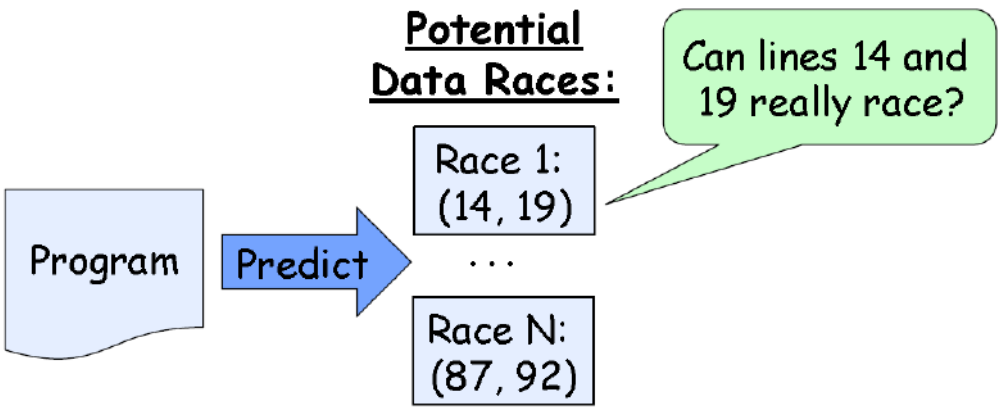
- So far we've only discussed dynamic analyses
  - Examine a real run/trace of a program
- Static analyses predict data races, deadlocks, etc., **without** running a program
  - Only examine the source code
  - Area of active research for ~20 years
  - Potentially much better coverage than dynamic analysis – examines all possible runs
  - But typically also more false warnings

Active Random Testing Overview

- **Problem**: Random testing can be very effective for parallel programs, but can miss many potential bugs
- **Problem**: Predictive analyses find many bugs, but can have false warnings
  - Time consuming and difficult to examine reported bugs and determine whether or not they are real
- **Key Idea**: Combine them – use predictive analysis to find potential bugs, then **biased** random testing to actually create each bug

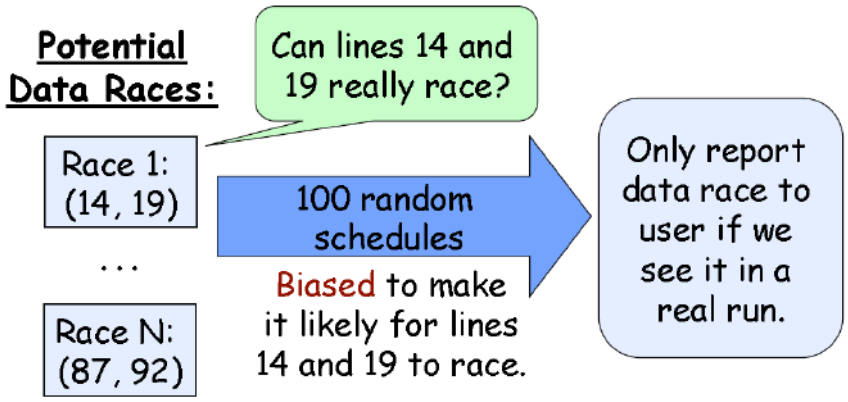
Active Random Testing

- **Key Idea**: Combine them – use predictive analysis to find potential bugs, then **biased** random testing to actually create each bug



Active Random Testing

- **Key Idea**: Combine them – use predictive analysis to find potential bugs, then **biased** random testing to actually create each bug



Active Random Testing Tools

- CalFuzzer - extensible, open-source tool for active testing of Java programs
  - Data races, atomicity bugs, and deadlocks
  - RaceFuzzer is the active testing algorithm for data races
- Thrille for C / pThreads & Data Races
- And UPC-Thrille for Unified Parallel C
  - Part of the Berkeley UPC system

Testing & Debugging Conclusions

- Many tools available right now to help find bugs in parallel software
  - Data races, atomicity violations, deadlocks
- **No silver bullet solution!**
  - Design how threads will coordinate and share/protect data
  - Tools help catching mistakes when design is not followed
  - Ad hoc parallelization likely to **never** be correct, even with help from tools

Tools References

- IBM's ConTest: Noise-making - Java
  - https://www.research.ibm.com/haifa/projects/verification/contest/index.html
- Cuzz: Random scheduling for C++/.NET
  - http://research.microsoft.com/en-us/projects/cuzz/
- Intel Thread Checker and Parallel Inspector C/C++
  - http://software.intel.com/en-us/articles/intel-parallel-studio-xe/
- Helgrind, DRD, ThreadSanitizer Dynamic Data Race Detection/Prediction for C/C++
  - http://valgrind.org/docs/manual/hg-manual.html
  - http://code.google.com/p/data-race-test/
- CHORD - Static Race/Deadlock Detection for Java
  - http://code.google.com/p/jchord/

Tools References

- CalFuzzer - Java
  - http://srl.cs.berkeley.edu/~ksen/calfuzzer/
- Thrille - C
  - http://github.com/nicholasjalbert/Thrille
- CHESS - C++/.NET Model Checking, Race Detection
  - http://research.microsoft.com/en-us/projects/chess/default.aspx
- Java Path Finder - Model Checking for Java
  - http://babelfish.arc.nasa.gov/trac/jpf
- Tau Performance System - Fortran, C, C++, Java, Python
  - http://www.cs.uoregon.edu/research/tau/home.php
- Vampir/GuideView - C/C++ and Fortran
  - https://computing.llnl.gov/code/vgv.html
- Performance Application Programming Interface - PAPI
  - http://icl.cs.utk.edu/papi/

Ce am invatat in aces curs?

- Performance Analysis
  - Programming
  - Tools
  - Techniques
- Parallel Correctness Challenges
  - Debugging
  - Testing

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 10
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

Cuprins

- Arhitecturi MIMD – Caracteristici
- Sisteme Multiprocesor / Multicalculator
- Sisteme cu Memorie Partajata
- Sisteme cu Memorie Distribuita
- Sisteme Strans / Slab Cuplate
- Accesul la Resursele Locale – Switch-ul S
- Structura si Functiile K_{MAP}
- Arhitectura Procesorului de Mapare P_{MAP}
- Comunicarea in Cadrul unui Cluster
- Comunicarea Intercluster

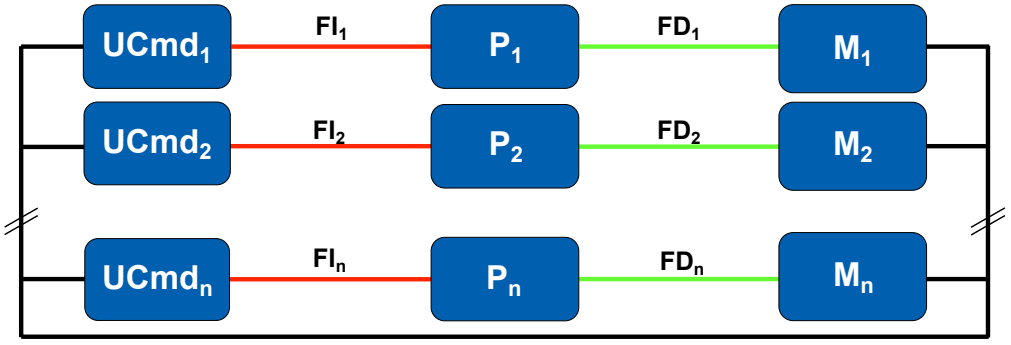
MiMD

Arhitecturi MIMD

MIMD

- MIMD pot comunica: (P-P sau P-M)
- Toate Procesoarele participa la acelasi program
- Mult mail des folosite ca SIMD
- Mod programare:
  - Shared memory (strans cuplate) – Memorie Partajata (e.g. OpenMP)
  - Distributed memory (slab cuplate) – Transfer de Mesaje (e.g. MPI)

MIMD = n FI & n FD



Arhitecturi MIMD – Caracteristici

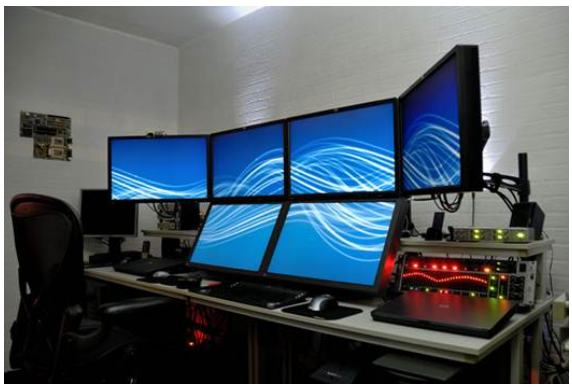
- Structura & arhitectura procesoarelor e eterogena
- Procesoarele pot functiona autonom ca un sistem universal
  - Pot prelua task-urile altor procesoare din sistem
  - Astfel se previne degradarea performantelor sistemului
- Organizarea memoriei
  - Fiecare procesor ar trebui sa aiba acces la toata memoria sistemului, dar…
    - Capacitatea de adresare a fiecarui procesor e mai mica ca dimensiunea memoriei → mecanisme speciale
  - Trebuie asigurata protectia memoriei pentru asigurarea integritatii datelor din sistem

Arhitecturi MIMD – Caracteristici

- Modularitatea presupune
  - Expandabilitate
    - Numarul de procesoare poate fi modificat fara a fi necesara reconfigurarea sistemului
  - Reconfigurabilitate
    - Structurala
      - Procesul de refacere a starii sistemului la aparitia unor defecte → se realizeaza prin distribuirea structurala a functiilor
      - Permite preluarea sarcinilor unui procesor de catre alte procesoare din sistem
    - Arhitecturala: schimbarea caracteristicilor de baza
      - Dimensiunea registrilor de memorie
      - Lungimea cuvantului
      - Structura UAL

Arhitecturi MIMD – Probleme

- Decompozitia algoritmilor
  - Manual – la conceptia algoritmilor
  - Automat – prin limbaje de programare concurenta
- Planificarea pentru executie a task-urilor si maparea pe structura de procesoare utilizata
- Sincronizarea proceselor/task-urilor astfel incat sa nu se ajunga la blocarea sistemului
- Siguranta in functionare e asigurata de
  - Disponibilitate: capacitatea sistemului de a raspunde cererilor utilizatorului
  - Integritate: capacitatea sistemului de a mentine consistenta starii in conditiile aparitiilor unor defecte



System Multiprocesor / Multicalculator

Clasificarea Structurilor MIMD

- Din punct de vedere arhitectural, structurile MIMD se pot imparti in doua mari categorii:
  - Sisteme multiprocesor:
    - Sistem de calcul unitar unde o colectie de procesoare autonome functioneaza concurent si coopereaza la realizarea unei functii logice si de calcul de sub controlul unui “executiv” unic
  - Sisteme multicalculator:
    - Mai multe calculatoare autonome, interconectate, ce colaboreaza la realizarea unor task-uri din cadrul unei lucrari complexe

Sisteme Multiprocesor

- Au un singur spatiu de adresare - comunicare prin shared memory
- Se impart in sisteme:
  - Cu memorie distribuita / scalabilitate:
    - Gestiune dinamica a accesului la memoria distribuita
    - Gestiune statica a accesului la memoria distribuita
  - Cu memorie centrala / nu exista scalabilitate:
    - Comunicarea se face prin retele de comutatie (e.g. crossbar switch)
    - Conexiuni de tip magistrala (bus) multiplu

Sisteme Multicalculator

- Au spatii multiple de adresare - comunicare prin transfer de mesaje
- Se impart in sisteme:
  - Multicalculator distribuite:
    - Mesh – plasa de interconectare P-M, NEC…
    - Conexiuni Butterfly (Perfect Shuffle) – CM5
    - Hypercube – la sistemele NCUBE, BlueGene…
    - Retele LAN rapide: Myrinet, ATM, GigaEthernet, InfiniBand, MPLS …
  - Multicalculator centralizate

Clasificarea Structurilor MIMD

- Din punct de vedere al accesului la memorie, structurile MIMD sunt:
  - Sisteme cu memorie partajata - Shared Memory

  - Sisteme cu memorie distribuita - Distributed Memory

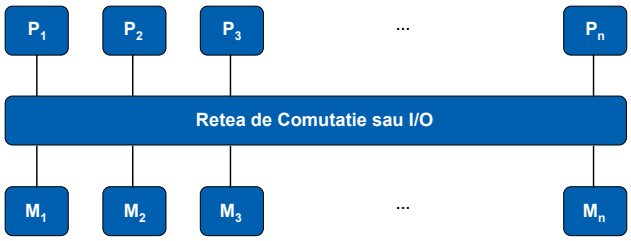
Sisteme cu Memorie Partajata

Sisteme cu Memorie Partajata

- Sistemele cu memorie partajata (Shared Memory) pot fi implementate in una din urmatoarele configuratii:
  - UMA = Uniform Memory Access
  - NUMA = Non-Uniform Memory Access
    - CC-NUMA = Cache-Coherent NUMA
  - COMA = Cache-Only Memory Architecture

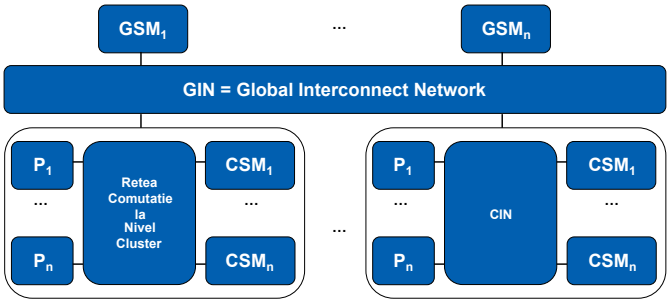
Structura UMA

- UMA = Uniform Memory Access
- Accesul este unic: orice procesor se poate conecta la un modul de memorie in mod uniform
- Reteaua de comunicatie (I/O) poate fi de orice tip



Structura NUMA

- CSM = Cluster Shared Memory
- CIN = Cluster Interconnect Network
- GSM = Global Shared Memory
- GIN = retea de comutare globala la care se conecteaza clustere formate din procesoare
- Accesul la memorii depinde de modulul unde trebuie efectuat accesul (CSM_i sau GSM_j)



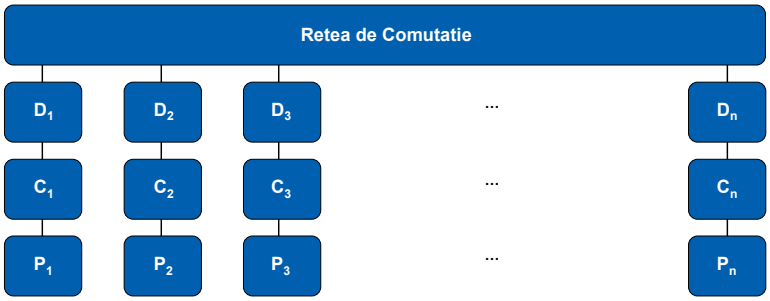
Structura CC-NUMA

- CC-NUMA = Cache Coherent NUMA
- Coerenta accesului la memorie e mai bine stabilita ca la NUMA:
  - Silicon Graphics - Origin 2000:
    - 2 procesoare pe nod
    - Acces divizat la componentele nodului printr-un Bus intern
    - Hub de conectare asigura accesul partajat la Cache-ul nodului → coerenta cache-ului

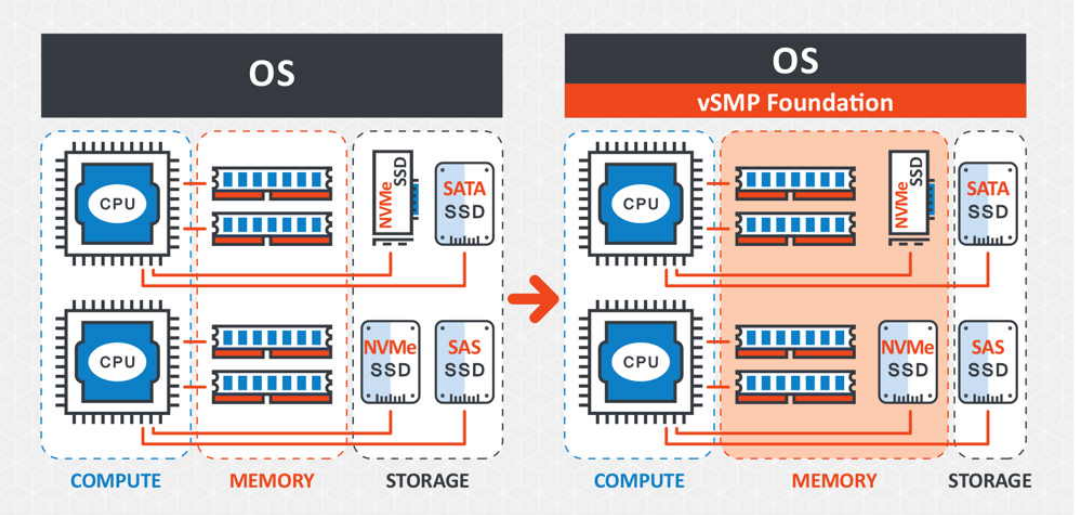
  - Sequent NUMA-Q - structuri de noduri interconectate cu structura inelara:
    - Standardul IEEE – SCI = Scalable Coherent Interface – structura de comunicare "rapida" (1GB/s in the 00's)
  - Standford DASH (Directory Architecture for SHared memory) – 4 MIPS R3000 Procs

Structura COMA

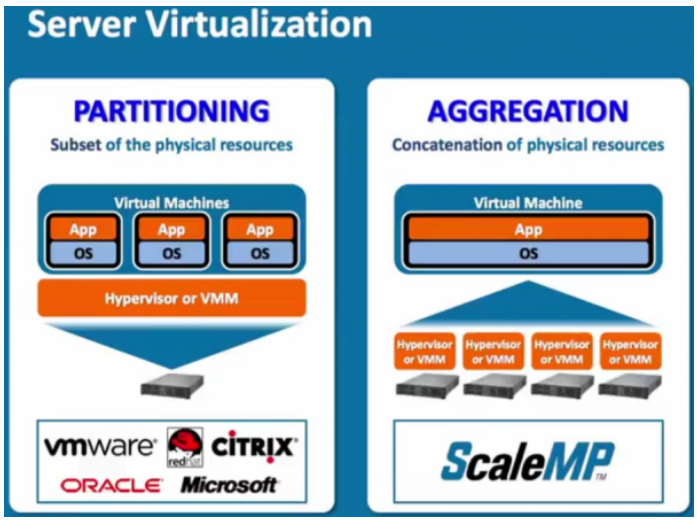
- Procesoarele nu au acces direct
  - La propria zona de date
  - La cea a celorlate procesoare
- Procesoarele au acces direct doar la Cache! → Cache Only
  - D_i = Memoria de Date
  - C_i = Memoria Cache
  - P_i = Procesoare

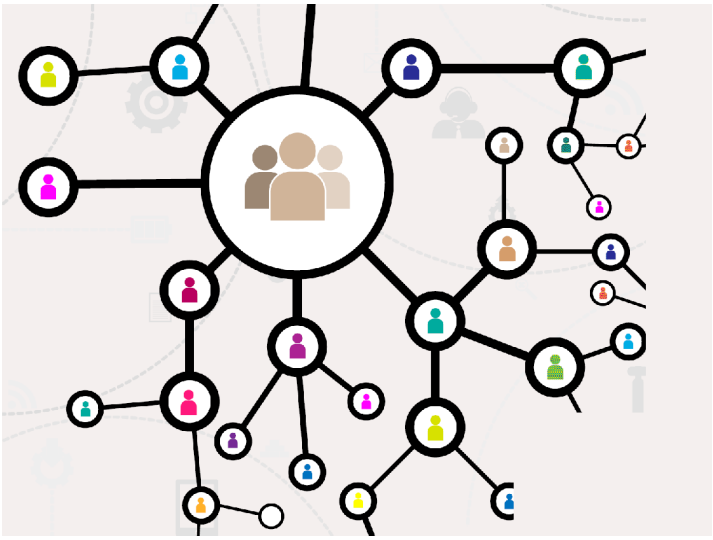


Exemplu de SW COMA - Scale MP



Scale MP Server Virtualization

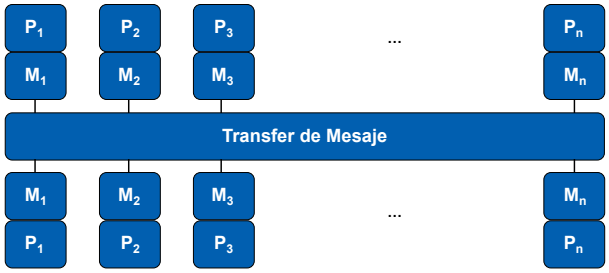




Sisteme cu Memorie Distribuita

Sisteme cu Memorie Distribuita

- Accesul se face prin transfer de mesaje
- Sistemele au la baza mai multe noduri legate printr-o retea de comutatie de mesaje (nu neaparat fizica)
- Conexiunea este statica, punct la punct intre noduri
- Memoria locala este adresabila direct doar de catre procesorul local



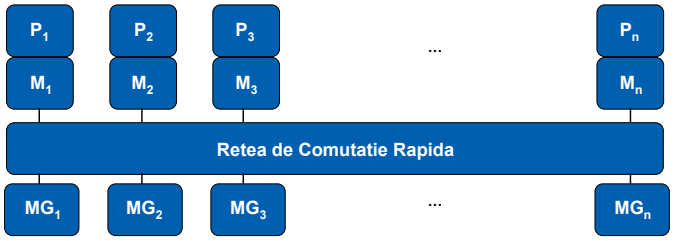
Exemplu @cs.pub.ro

- SUN Enterprise 10000 – Starfire (1996-2003)
  - Reteaua de comutatie este de tip Gigaplane XB (102.4GB/s; p2p: 1.6GB/s)
  - Comutare de pachete cu cai separate de adrese si date XB
  - Adresele distribuite printr-un router ce face broadcast
  - Acest fapt permite
    - Partitionarea dinamica a setului de procesoare si crearea unor partitii
    - Aceste partitii sunt complet izolate intre ele si pot fi utilizate ca atare pentru sarcini distincte
  - Suporta paralelism de tip shared-memory in Fortran & C/C++ OpenMP

Sisteme Strans / Slab Cuplate

Sisteme Strans / Puternic Cuplate

- Dispun de procesoare P_j cu memorie locala asociata M_i de tip multiport
- O retea de comutatie rapida RC (dar fixa) ce permite accesul uniform la memoria globala partajata MG_i
- Comunicarea si sincronizarea se face prin MG_i cu o rata de transfer similara cu a memoriei locale
- Asigura o scalabilitate redusa datorita faptului ca RC este fixa si de aceea au o structura dedicata
- Performantele sistemului sunt date de modul de acces la memoria partajata

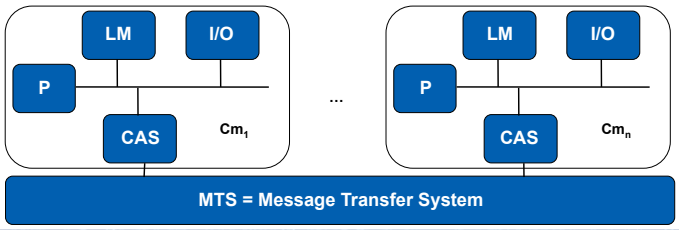


Sisteme Slab Cuplate

- De tip cluster → calculatoarele au resurse locale necesare functionarii independente
- Procesele se executa pe procesoare care dispun de toate resursele pentru implementarea task-ului respectiv
- Comunicarea intre procesoare se face prin
  - Transfer de mesaje
  - Comutare de circuite
- Sistemele slab cuplate (distribuite) sunt eficiente atunci cand interactiunea intre procesoare este minima (granularitatea problemei este mare)
  - Multicalculator centralizate

Sisteme Slab Cuplate

- Resurse locale ale CM_i = Computer Module:
  - LM = local memory (acces din interior/exterior)
  - I/O = magistrala de intrare/iesire (acces din interior/exterior)
  - MTS = Message Transfer System – poate fi implementata cu comutare de circuite sau transfer de mesaje
  - MTS dicteaza performantele prin lungimea mesajelor & viteza de lucru
  - CAS = Collision Avoidance System – legatura si accesul la MTS se fece prin excludere mutuala:
    - Un singur transfer odata daca MTS este magistrala
    - Mai multe transferuri simultane daca MTS este de tip retea

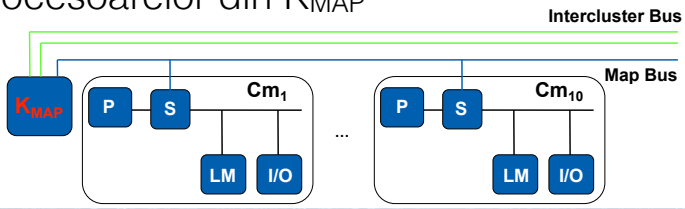


Exemplu – Cm*

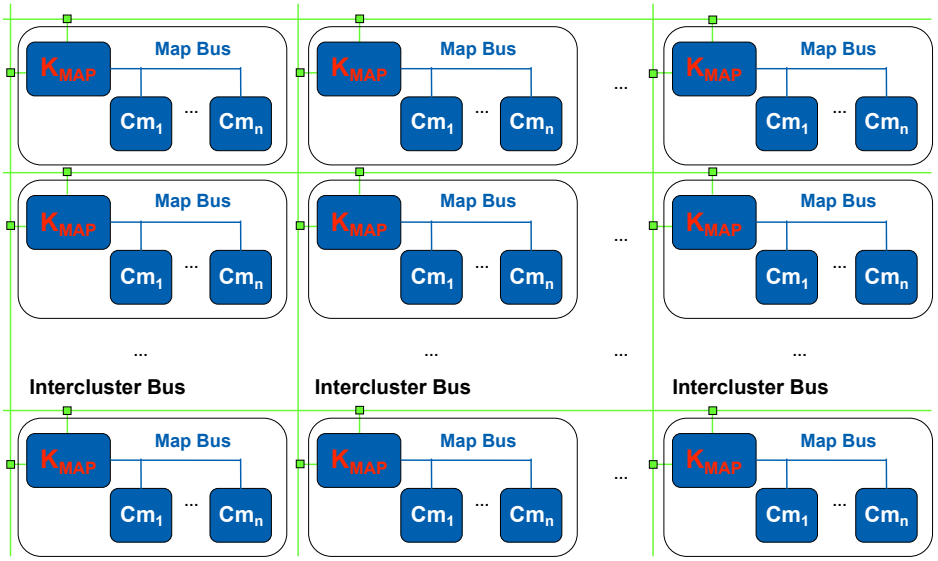
- Structura Cm* a fost dezvoltata arhitectural si implementata la Carnegie Mellon in anii ‘70
- Cm* e proiectat cu intentia de a obtine in acelasi timp:
  - Performante bune
  - Fiabilitate sporita
- In 1975 structura e compusa din 10 procesoare
- Ajunge in 1977 la 50 de procesoare
- Cm* beneficiaza de doua sisteme de operare distincte

Arhitectura Cm*

- Structura Cm* conecteaza mai multe module individuale → structura multipla de calculatoare == cluster
- K_{MAP} = gestioneaza magistrala Map Bus si cererile pentru Cm-uri
- Un cluster e format din mai multe Cm-uri, K_{MAP} & Map Bus
- Cm-urile sunt fiabile → asigura semiparalelism
- Performantele sunt determinate de interconectarea clusterelor & viteza de comunicatie a procesoarelor din K_{MAP}



Conectare Matriceala de Clustere



Caracteristicile Cluster-ului

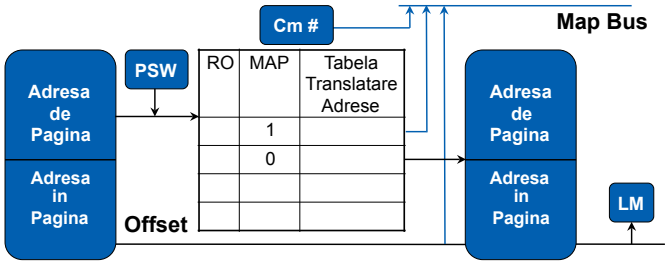
- Un Cluster faciliteaza cooperarea intre resursele sistemului cu un overhead minim
- Sursa de bottleneck intr-un cluster este Map Bus ↔ posibila doar o singura tranzactie la un moment dat → cea mai avantajoasa este comutarea de circuite
- Orice referire la memorie este tratata de K_{MAP} → rutare
- Clusterele comunica intre ele prin K_{MAP} conectate la Intercluster Bus
- Functii K_{MAP}:
  - Maparea adreselor
  - Gestionarea circuitelor
  - Transmiterea mesajelor pe Ethernet, TokenBus, TokenRing, etc.
  - Sincronizarea

Accesul la Resursele Locale – Switch-ul S

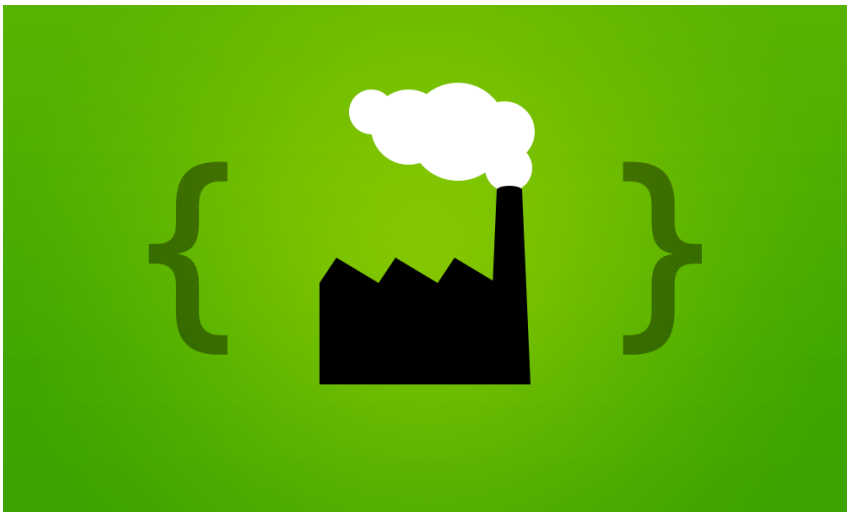
Accesul la Resursele Locale

- Functiile switch-ului S dintr-o structura Cm:
  - Primeste si interpreteaza cererile de acces de la P-uri locale si externe la memoria locala si la sistemul I/O
  - S permite unui P local sa acceseze resurse externe Cm
- Pentru a realiza interpretarea cererilor locale si externe S-ul asigura:
  - O translatie a adreselor la nivel local
  - Formarea adresei externe prin concatenarea adresei in cadrul paginii si numarul de identificare al procesorului
- Memoria este paginata cu dimensiunea paginii de 4K
  - 16 pagini in total → 64K
- Spatiul total de adresare este de 28 de biti - 256M

Mecanismul de Adresare pentru Referinte la Memoria Locala



- PSW = Processor Status Word – ofera adresa in tabela de translatare (adresa fizica/ virtuala)
- RO = Read Only bit (0 = RO; 1 = RW)
- MAP = Maparea adreselor (0 = adresa locala; 1 = adresa globala)
- Cm # = in ce Cm se afla adresa externa; K_{MAP} interpreteaza si decide daca adresa este in cluster-ul curent sau inafara lui



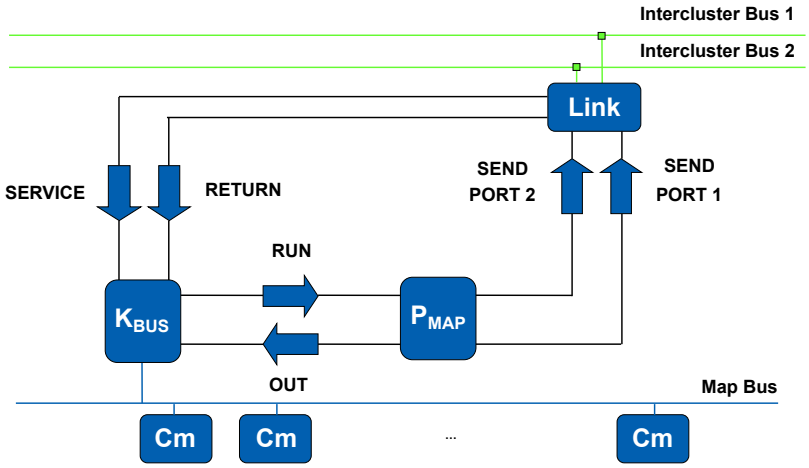
Structura si Functiile K_{MAP}

Functiile K_{MAP}

- K_{MAP} asigura:
  - Controlul in cadrul unui cluster – prin excluderea mutuala a accesului la MapBus si arbitrare intre Cm-urile din acelasi cluster
  - Comunicarea intre clustere prin Intercluster Bus
- Functiile de baza ale K_{MAP}:
  - Mapare de adrese
  - Comunicare
  - Sincronizare
- O serie din primitivele OS sunt implementate in K_{MAP}
- Nucleul OS din Cm se simplifica → transparenta

Structura K_{MAP}

- K_{MAP} este compus din 3 procesoare: P_{MAP}, Link, K_{BUS}:



Elementele K_{MAP}

- K_{BUS}: unitate de comanda a Map Bus si va controla toate tranzactiile de pe aceasta magistrala
- P_{MAP}: procesor de mapare care face toate translatarile de adresa
  - Maxim 8 procese simultan
- Link: gestioneaza si supravegheaza transmiterea mesajelor intercluster Send/Recv
- Exista doua tipuri de comutatie:
  - De circuite – in cadrul cluster-ului
  - De mesaje – intre clustere
- K_{MAP} prelucreaza tranzactii (contexte) concurente
- Fiecare context are asociate registre speciale si registre de legatura la subrutine pt translatarea adreselor

Comunicatia K_{BUS} – P_{MAP}

- K_{BUS} mentine starea a 8 contexte si le aloca cereri de serviciu
  - Fiecare va fi rezident in P_{MAP}
- Numarul contextului si alte informatii sunt plasate in coada RUN
- Din RUN sunt preluate de P_{MAP} care activeaza contextul si eventual translateaza adresa si o plaseaza in OUT
- In acest timp K_{BUS} poate citi alte cereri sau efectua alte functii cerute de P_{MAP}
- In P_{MAP} se executa o schimbare de context prin intermediul celor doua cozi: RUN/OUT

Comunicatia K_{BUS} – P_{MAP}

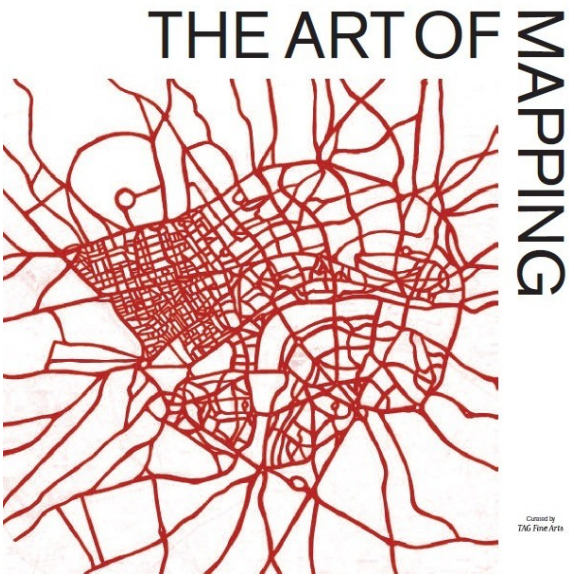
- K_{BUS} preia acum cererea din OUT si transfera adresa si datele la S-ul local al destinatiei; apoi se ocupa de alte cereri
- Cand accesul la memorie este terminat, K_{BUS} citeste confirmarea de terminare sau citeste datele si plaseaza iara contextul in RUN, reactivandu-l
- K_{BUS} trimite apoi confirmarea la procesul ce a initiat cererea de serviciu
- Contextul poate fi utilizat acum pentru o noua cerere de serviciu

Comunicatia Link – P_{MAP}

- P_{MAP} trimite in SEND o cerere de transmitere mesaj inter-cluster
- Se suspenda contextul initiator al mesajului intercluster & il trece intr-o coada de asteptare
- La primirea raspunsului pentru acest context se pune in RETURN
- Link reactiveaza contextul in RUN
- La primirea raspunsului inter-cluster in Link acesta il plaseaza in SERVICE
- Se emite spre K_{BUS} o cerere de alocare a unui nou context in RUN

Comunicatia K_{BUS} – Map Bus

- K_{BUS} trebuie sa execute o mare varietate de task-uri printre care si tratarea inteligenta a erorilor
- K_{BUS} este un procesor microprogramat cu o memorie de control ROM de 256 de cuvinte a cate 40 de biti fiecare
- Map Bus este o magistrala de comunicatii de 38 de biti dintre care 20 bidirectionali de adrese / date intre K_{BUS} si S-urile locale
- K_{BUS} controleaza toate accesele pe Map Bus precum si cozile si registrele pentru comunicarea cu P_{MAP}
- Maparea Map Bus este sincrona cu K_{BUS}

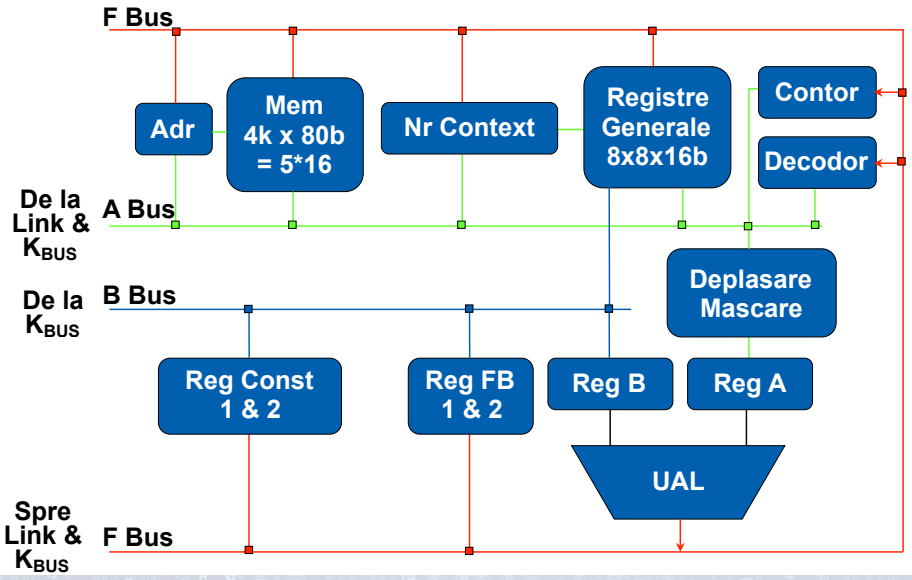


Arhitectura Procesorului de Mapare P_{MAP}

Complexitatea Arhitecturii P_{MAP}

- P_{MAP} este un procesor dedicat pe 16 biti, microprogramat
- Memoria de control este formata din 4k * 80 de biti
- Memoria de control RAM foarte rapida are cuvantul de 80 biti (5*16) – Writable Control Store
- Functiile fundamentale ale procesorului P_{MAP} sunt:
  - Translatarea adreselor nelocale din cluster
  - Implementarea primitivelor OS critice din punctul de vedere al timpului de executie
  - Culegerea informatiilor statistice pentru imbunatatirea solutiilor de proiectare

Arhitectura Procesorului P_{MAP}

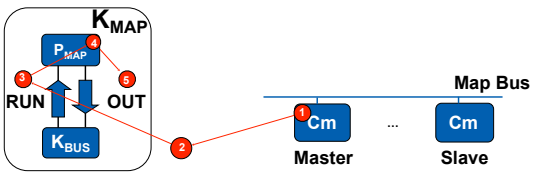


Componentele Procesorului P_{MAP}

- In registrele generale (8x128b sau 8x8x16b) este salvat contextul executiei in curs
- Este organizat in jurul a 3 magistrale:
  - A, B pentru operanzi
  - F pentru rezultate
- Pentru a putea mapa adresele PMAP trebuie sa aibe acces rapid la informatia pt translatarea adreselor virtuale in adrese fizice
- UAL are un operator la nivel de bit pentru Reg A ce asigura
  - Deplasarea
  - Rotirea
  - Mascarea (cu 30 de masti predefinite), permitand preluarea descriptorului de context
- Prelucrarea este in banda de asamblare – operatii UAL si operanzii ce vin simultan, separat, prin registrii tampon
- Accesul la o memorie in acelasi cluster este mult mai rapid decat in alt cluster

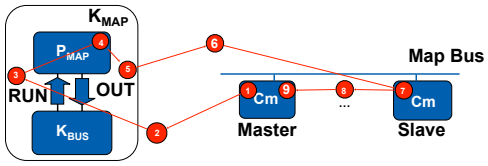
Comunicarea in Cadrul unui Cluster

Etapele de Acces in Cadrul unui Cluster



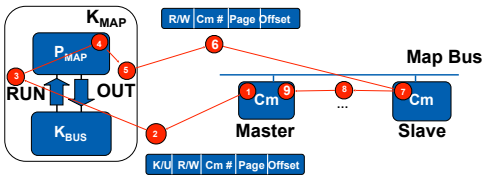
1. In Cm Master se initiaza acces la o memorie nelocala
2. Cm Master emite adresa virtuala catre K_{BUS}
3. K_{BUS} activeaza un context (crearea structurii de date specifica tranzitiei) ce se pune in coada RUN catre P_{MAP}
4. P_{MAP} trateaza contextul si face translatarea adresei
5. P_{MAP} pune in coada OUT o cerere pentru ciclu de memorie pentru Cm Slave din cluster-ul curent

Etapele de Acces in Cadrul unui Cluster

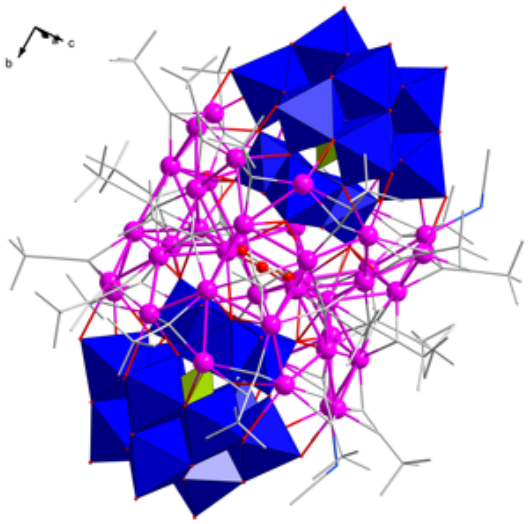


6. K_{BUS} trimite adresa fizica catre Cm Slave prin Map Bus
7. S-ul local din Cm Slave face acces la memoria locala prin furt de ciclu
8. K_{BUS} “permite” ca rezultatul operatiei de acces la memorie sa fie furnizat catre Cm Master
9. Cm Master preia datele, termina operatia in curs si continua executia

Formatul Pachetelor Transmise

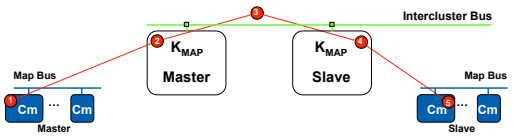


- Pachetele trimise au urmatorul format:
  - U/K – User/Kernel
  - R/W – Read/Write
  - Cm # – numarul Cm-ului ce efectueaza transferul
  - Page – utilizata pentru translatarea locala
  - Offset – In cadrul paginii, adresa relativa
- Memoriile Cm-urilor trebuie sa aiba posibilitatea de dublu acces



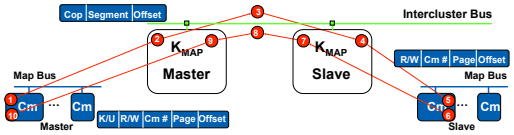
Comunicarea Intercluster

Comunicarea Intercluster



1.Cm Master trimite o cerere de transfer la K_{MAP} Master
2.K_{MAP} Master pregateste mesaj/pachet intercluster si codifica cererea
3.Mesajul intercluster e transmis pe Intercluster Bus pe baza algoritmilor de rutare
4.K_{MAP} Slave decodifica cererea primita si o trimite catre Clusterul sau local
5.Cererea de ciclu de memorie este trimisa la Cm Slave

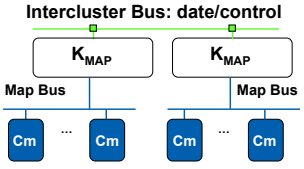
Comunicarea Intercluster



6. Cm Slave transmite rezultatul catre K_{MAP} Slave
7. K_{MAP} Slave pregateste mesajul intercluster - reactivarea contextului respectiv
8. K_{MAP} Slave transmite rezultatul catre K_{MAP} Master
9. K_{MAP} Master receptioneaza si interpreteaza mesajul primit
10. Rezultatul este trimis catre Cm Master

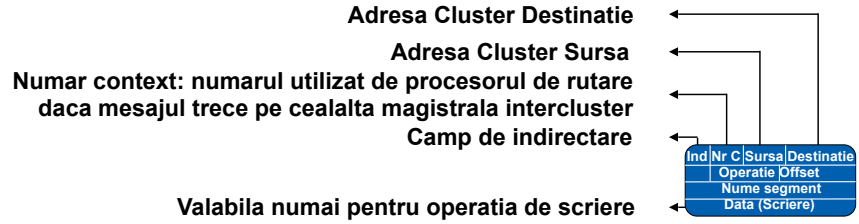
Protocolul Magistralei Intercluster

- Magistrala intercluster
  - Contine linii de date si control
  - Nu contine linii de adresa pt ca se realizeaza comutare de pachete si nu de circuite
- Comunicarea este:
  - Asicrona
  - De tip intrebare-raspuns (Q & A)
  - Cu interblocare completa – pentru fiecare actiune se primeste raspuns si abia apoi se anuleaza comanda
- Mesajele intercluster sunt formate din 1-8 cuvinte
- Cele mai utilizate: mesajele directe si de raspuns

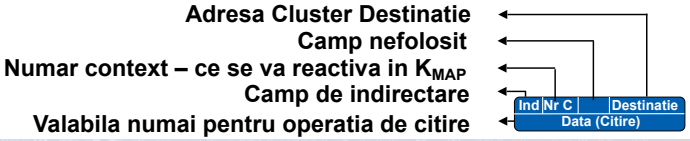


Formatul Mesajelor Intercluster

- Mesajul Direct – format din 4 cuvinte



- Mesajul de Raspuns – format din 2 cuvinte



Ce am invatat in aces curs?

- Caracteristicile arhitecturilor MIMD
- Sisteme Multiprocesor / Multicalculator
- Sisteme cu Memorie Partajata
- Sisteme cu Memorie Distribuita
- Sisteme Strans / Slab Cuplate
- Accesul la Resursele Locale – Switch-ul S
- Structura si Functiile K_{MAP}
- Arhitectura Procesorului de Mapare P_{MAP}
- Comunicarea in cadrul unui Cluster
- Comunicarea Intercluster

Arhitectura Sistemelor de Calcul – Curs 10

Universitatea Politehnica Bucuresti

Facultatea de Automatica si Calculatoare

cs.ncit.pub.ro

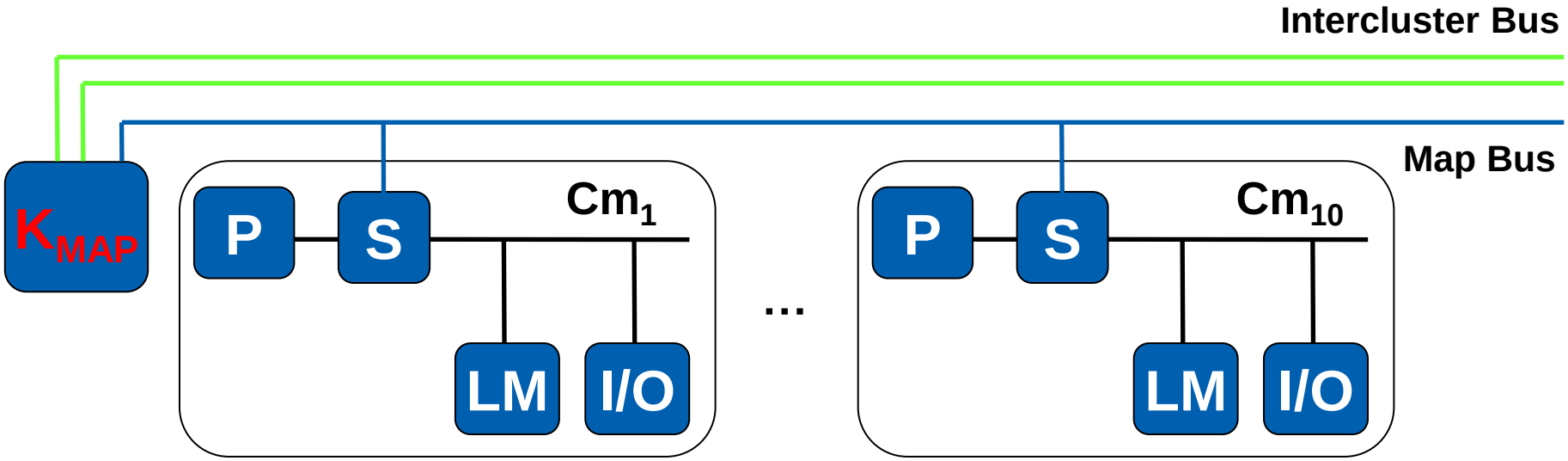
curs.cs.pub.ro

Cuprins

- Accesul la resursele locale – Switch-ul S
- Structura si functiile K_{MAP}
  - Comunicatia K_{BUS} – P_{MAP}
  - Comunicatia Link – P_{MAP}
  - Comunicatia K_{BUS} – Map Bus
- Arhitectura procesorului de mapare P_{MAP}
- Comunicarea in cadrul unui cluster
- Comunicarea intercluster

Arhitectura Cm*

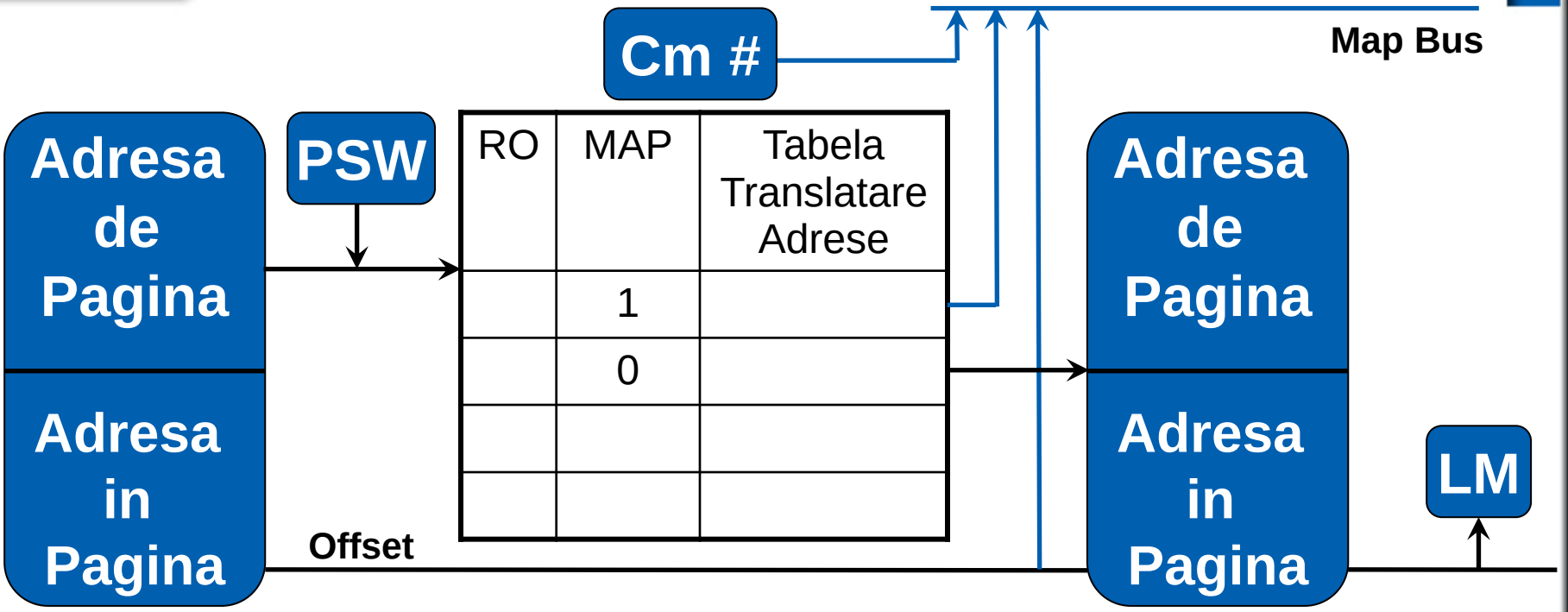
- Structura Cm* conecteaza mai multe module individuale → structura multipla de calculatoare == cluster
- K_{MAP} = gestioneaza magistrala Map Bus si cererile pentru Cm-uri
- Un cluster e format din mai multe Cm-uri, K_{MAP} & Map Bus
- Performantele sunt determinate de interconectarea clusterelor & viteza de comunicatie a procesoarelor din K_{MAP}



Accesul la Resursele Locale

- Functiile switch-ului S dintr-o structura Cm:
  - Primeste si interpreteaza cererile de acces de la P-uri locale si externe la memoria locala si la sistemul I/O
  - S permite unui P local sa acceseze resurse externe Cm
- Pentru a realiza interpretarea cererilor locale si externe S-ul asigura:
  - O translatie a adreselor la nivel local
  - Formarea adresei externe prin concatenarea adresei in cadrul paginii si numarul de identificare al procesorului
  - Memoria este paginata cu dimensiunea paginii de 4K
  - Spatiul total de adresare este de partajat de toate procesoarele

Mecanismul de Adresare pentru Referinte la Memoria Locala



- PSW = Processor Status Word – ofera adresa in tabela de translatare (adresa fizica/virtuala)
- RO = Read Only bit (0 = RO; 1 = RW)
- MAP = Maparea adreselor (0 = adresa locala; 1 = adresa globala)
- Cm # = in ce Cm se afla adresa externa; K_{MAP} interpreteaza si decide daca adresa este in cluster-ul curent sau inafara lui

Cuprins

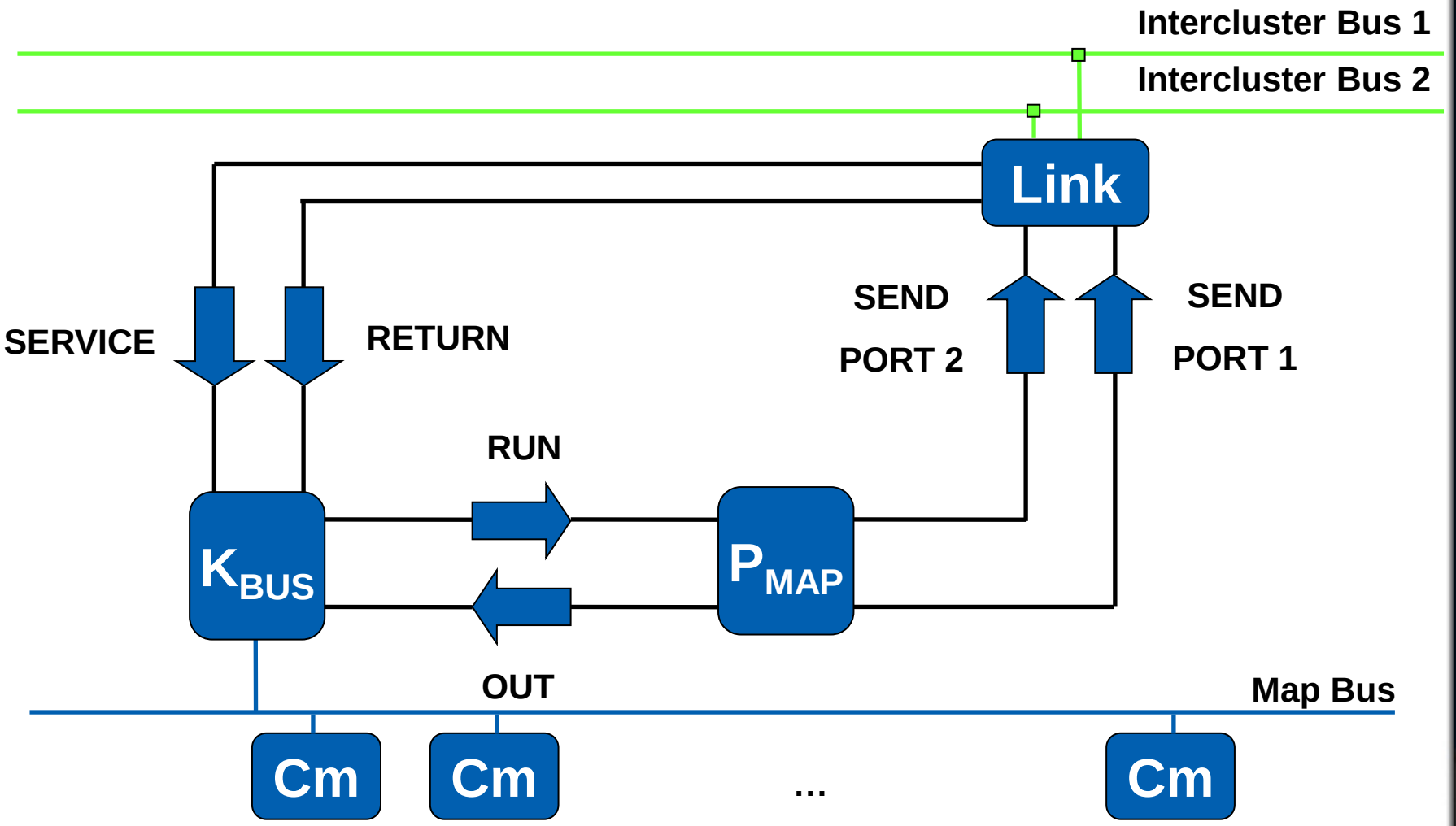
- Accesul la resursele locale – Switch-ul S
- Structura si functiile K_{MAP}
  - Comunicatia K_{BUS} – P_{MAP}
  - Comunicatia Link – P_{MAP}
  - Comunicatia K_{BUS} – Map Bus
- Arhitectura procesorului de mapare P_{MAP}
- Comunicarea in cadrul unui cluster
- Comunicarea intercluster

Functiile K_{MAP}

- K_{MAP} asigura:
  - Controlul in cadrul unui cluster – prin excluderea mutuala a accesului la MapBus (arbitrare intre Cm-urile din acelasi cluster)
  - Comunicarea intre clustere prin Intercluster Bus
- Functiile de baza ale K_{MAP}:
  - Mapare de adrese
  - Comunicare
  - Sincronizare
- O serie din primitivele OS sunt implementate in K_{MAP}
- Nucleul OS din Cm se simplifica → transparenta

Structura K_{MAP}

- K_{MAP} este compus din 3 procesoare: P_{MAP}, Link, K_{BUS}



Elementele K_{MAP}

- K_{BUS}: unitate de comanda a Map Bus si va controla toate tranzactiile de pe aceasta magistrala
- P_{MAP}: procesor de mapare care face toate translatarile de adresa (maxim 8 procese simultan)
- Link: este cel care gestioneaza si supravegheaza transmiterea mesajelor intercluster (send/recv)
- Exista doua tipuri de comutatie:
  - De circuite – in cadrul cluster-ului
  - De mesaje – intre clustere
- K_{MAP} prelucreaza tranzactii (contexte) concurente
- Fiecare context are asociate registre speciale si registre de legatura la subrutine pt translatarea adreselor

Cuprins

- Accesul la resursele locale – Switch-ul S
- Structura si functiile K_{MAP}
  - Comunicatia K_{BUS} – P_{MAP}
  - Comunicatia Link – P_{MAP}
  - Comunicatia K_{BUS} – Map Bus
- Arhitectura procesorului de mapare P_{MAP}
- Comunicarea in cadrul unui cluster
- Comunicarea intercluster

Comunicatia K_{BUS} – P_{MAP}

- K_{BUS} mentine starea a 8 contexte si le aloca cereri de serviciu (fiecare va fi rezident in P_{MAP})
- Numarul contextului si alte informatii sunt plasate in coada RUN
- Din RUN sunt preluate de P_{MAP} care activeaza contextul si eventual translateaza adresa si o plaseaza in OUT
- In acest timp K_{BUS} poate citi alte cereri sau efectua alte functii cerute de P_{MAP}
- In P_{MAP} se executa o schimbare de context prin intermediul celor doua cozi (RUN/OUT)

Comunicatia K_{BUS} – P_{MAP}

- K_{BUS} preia acum cererea din OUT si transfera adresa si datele la S-ul local al destinatiei; apoi se ocupa de alte cereri
- Cand accesul la memorie este terminat, K_{BUS} citeste confirmarea de terminare sau citeste datele si plaseaza iara contextul in RUN, reactivandu-l
- K_{BUS} trimite apoi confirmarea la procesul ce a initiat cererea de serviciu
- Contextul poate fi utilizat acum pentru o noua cerere de serviciu

Cuprins

- Accesul la resursele locale – Switch-ul S
- Structura si functiile K_{MAP}
  - Comunicatia K_{BUS} – P_{MAP}
  - Comunicatia Link – P_{MAP}
  - Comunicatia K_{BUS} – Map Bus
- Arhitectura procesorului de mapare P_{MAP}
- Comunicarea in cadrul unui cluster
- Comunicarea intercluster

Comunicatia Link – P_{MAP}

- P_{MAP} trimite in SEND o cerere de transmitere mesaj inter-cluster
- Se suspenda contextul initiator al mesajului intercluster & il trece intr-o coada de asteptare
- La primirea raspunsului pentru acest context se pune in RETURN
- Link reactiveaza contextul in RUN
- La primirea raspunsului inter-cluster in Link acesta il plaseaza in SERVICE
- Se emite spre K_{BUS} o cerere de alocare a unui nou context in RUN

Cuprins

- Accesul la resursele locale – Switch-ul S
- Structura si functiile K_{MAP}
  - Comunicatia K_{BUS} – P_{MAP}
  - Comunicatia Link – P_{MAP}
  - Comunicatia K_{BUS} – Map Bus
- Arhitectura procesorului de mapare P_{MAP}
- Comunicarea in cadrul unui cluster
- Comunicarea intercluster

Comunicatia K_{BUS} – Map Bus

- K_{BUS} trebuie sa execute o mare varietate de task-uri printre care si tratarea inteligenta a erorilor
- K_{BUS} este un procesor microprogramat cu o memorie de control ROM
- Map Bus este o magistrala de comunicatii adrese/date intre K_{BUS} si S-urile locale
- K_{BUS} controleaza toate accesele pe Map Bus precum si cozile si registrele pentru comunicarea cu P_{MAP}
- Maparea Map Bus este sincrona cu K_{BUS}

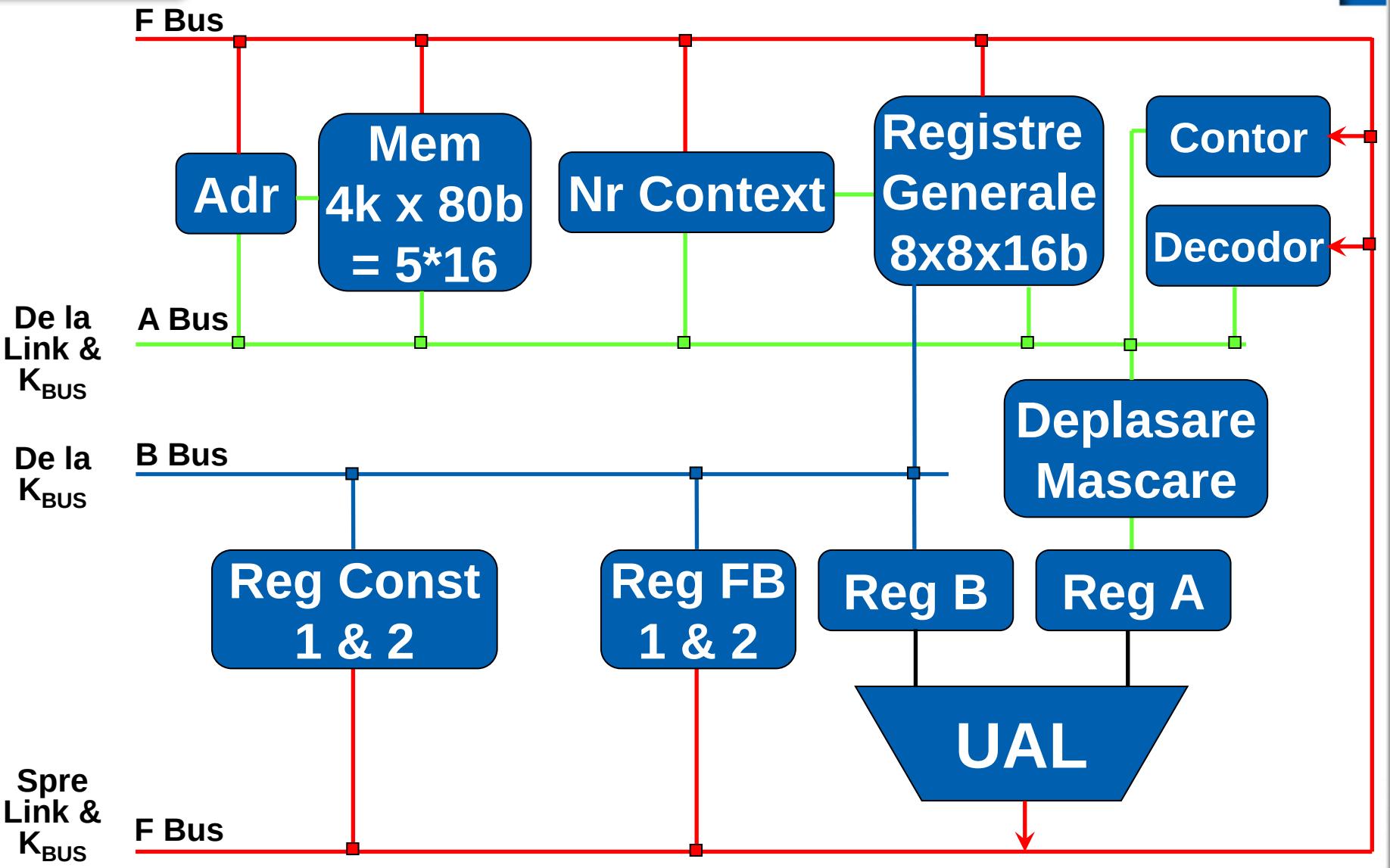
Cuprins

- Accesul la resursele locale – Switch-ul S
- Structura si functiile K_{MAP}
  - Comunicatia K_{BUS} – P_{MAP}
  - Comunicatia Link – P_{MAP}
  - Comunicatia K_{BUS} – Map Bus
- Arhitectura procesorului de mapare P_{MAP}
- Comunicarea in cadrul unui cluster
- Comunicarea intercluster

Complexitatea Arhitecturii P_{MAP}

- P_{MAP} este un procesor dedicat pe 16 biti, microprogramat, cu codificare orizontala
- Memoria de control este formata din 4k * 80 de biti
- Memoria de control RAM foarte rapida are cuvantul de 80 biti (5*16) – Writable Control Store
- Functiile fundamentale ale procesorului P_{MAP} sunt:
  - Translatarea adreselor nelocale din cluster
  - Implementarea primitivelor OS critice din punctul de vedere al timpului de executie
  - Culegerea informatiilor statistice pentru imbunatatirea solutiilor de proiectare

Arhitectura Procesorului P_{MAP}



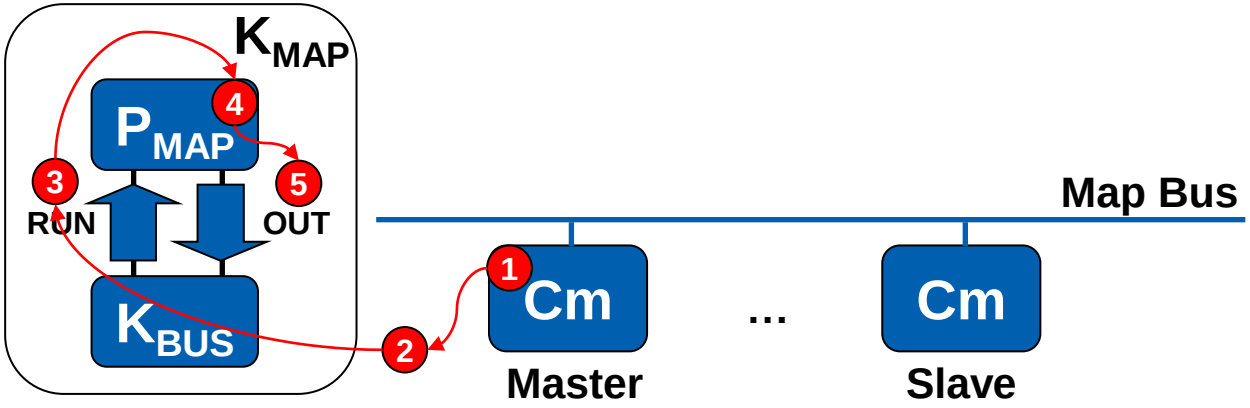
Componentele Procesorului P_{MAP}

- In registrele generale (8x128b sau 8x8x16b) este salvat contextul executiei in curs
- Este organizat in jurul a 3 magistrale:
  - A, B pentru operanzi
  - F pentru rezultate
- Pentru a putea mapa adresele P_{MAP} trebuie sa aibe acces rapid la informatia pt translatarea adr virtuale in adr fizice
- UAL are un operator la nivel de bit pentru Reg A ce asigura:
  - Deplasarea
  - Rotirea
  - Mascarea (cu 30 de masti predefinite), permitand preluarea descriptorului de context
- Prelucrarea este in banda de asamblare – operatii UAL si operanzii ce vin simultan, separat, prin registrii tampon
- Accesul la o memorie in acelasi cluster este mult mai rapid decat in alt cluster

Cuprins

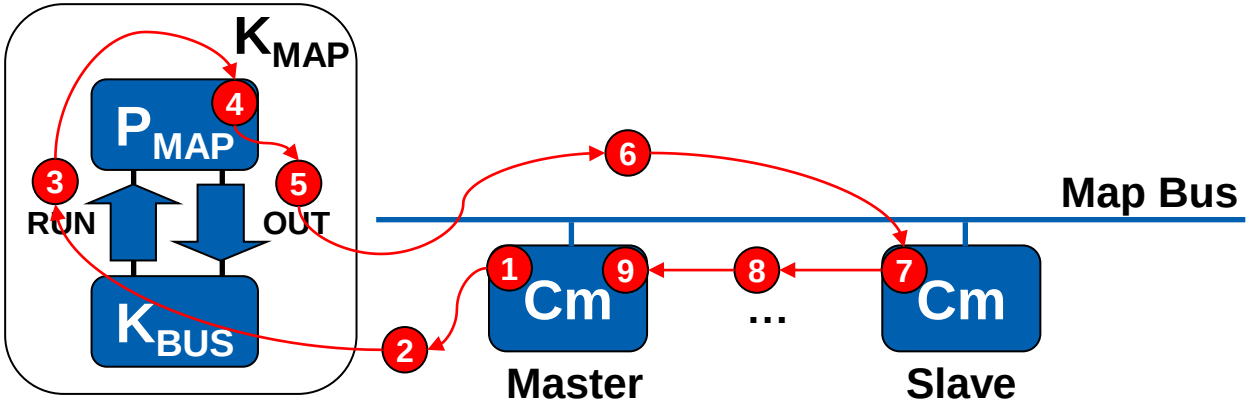
- Accesul la resursele locale – Switch-ul S
- Structura si functiile K_{MAP}
  - Comunicatia K_{BUS} – P_{MAP}
  - Comunicatia Link – P_{MAP}
  - Comunicatia K_{BUS} – Map Bus
- Arhitectura procesorului de mapare P_{MAP}
- Comunicarea in cadrul unui cluster
- Comunicarea intercluster

Etapele de Acces in Cadrul unui Cluster



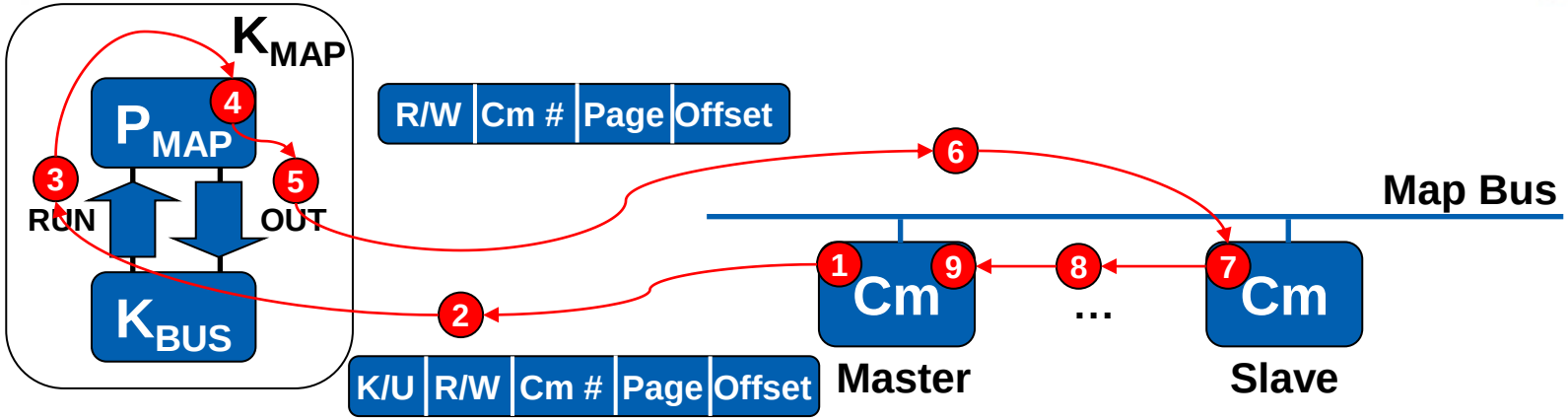
1. In Cm Master se initiaza acces la o memorie nelocala
2. Cm Master emite adresa virtuala catre K_{BUS}
3. K_{BUS} activeaza un context (crearea structurii de date specifica tranzitiei) ce se pune in coada RUN catre P_{MAP}
4. P_{MAP} trateaza contextul si face translatarea adresei
5. P_{MAP} pune in coada OUT o cerere pentru ciclu de memorie pentru Cm Slave din cluster-ul curent

Etapele de Acces in Cadrul unui Cluster



6. K_{BUS} trimite adresa fizica catre Cm Slave prin Map Bus
7. S-ul local din Cm Slave face acces la memoria locala prin furt de ciclu
8. K_{BUS} “permite” ca rezultatul operatiei de acces la memorie sa fie furnizat catre Cm Master
9. Cm Master preia datele, termina operatia in curs si continua executia

Formatul Pachetelor Transmise

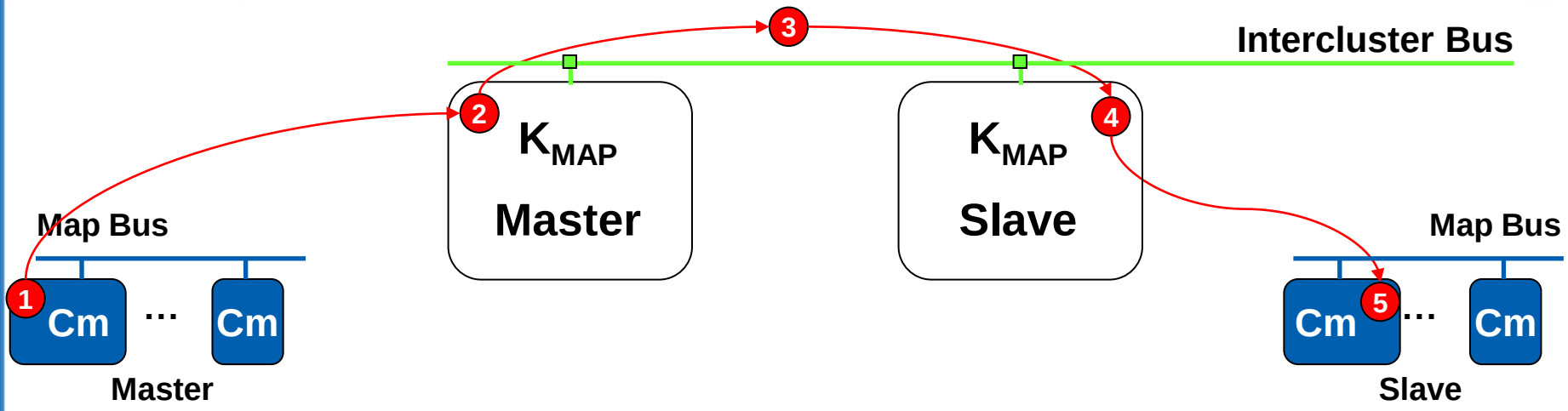


- Pachetele trimise au urmatorul format:
  - U/K – User/Kernel
  - R/W – Read/Write
  - Cm # – numarul Cm-ului ce efectueaza transferul
  - Page – utilizata pentru translatarea locala
  - Offset – In cadrul paginii, adresa relativa
- Memoriile Cm-urilor trebuie sa aiba posibilitatea de dublu acces

Cuprins

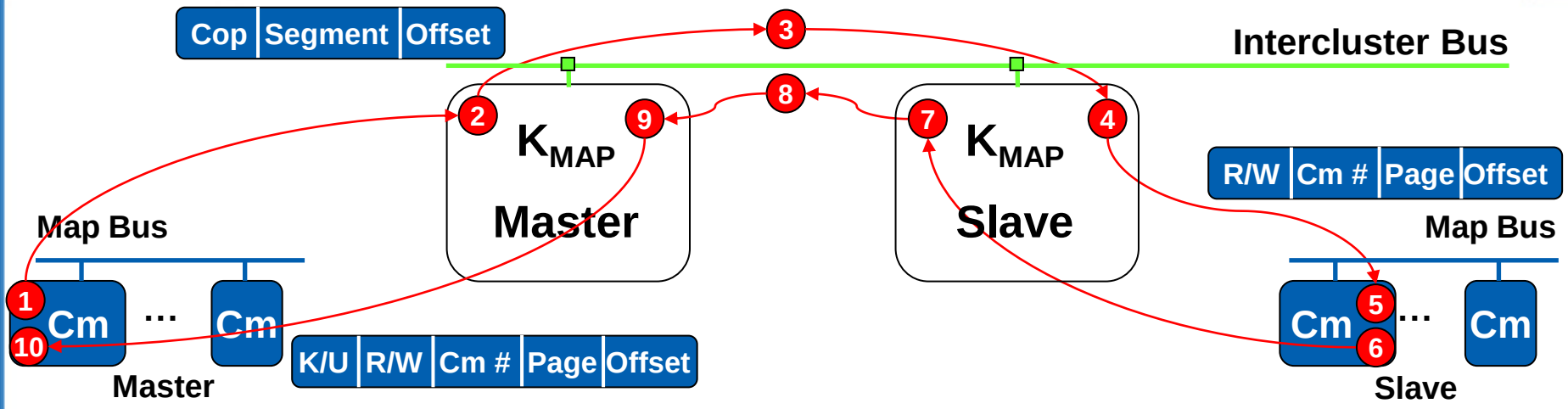
- Accesul la resursele locale – Switch-ul S
- Structura si functiile K_{MAP}
  - Comunicatia K_{BUS} – P_{MAP}
  - Comunicatia Link – P_{MAP}
  - Comunicatia K_{BUS} – Map Bus
- Arhitectura procesorului de mapare P_{MAP}
- Comunicarea in cadrul unui cluster
- Comunicarea intercluster

Comunicarea Intercluster



1. Cm Master trimite o cerere de transfer la K_{MAP} Master
2. K_{MAP} Master pregateste mesaj/pachet intercluster si codifica cererea
3. Mesajul intercluster e transmis pe Intercluster Bus pe baza algoritmilor de rutare
4. K_{MAP} Slave decodifica cererea primita si o trimite catre Clusterul sau local
5. Cererea de ciclu de memorie este trimisa la Cm Slave

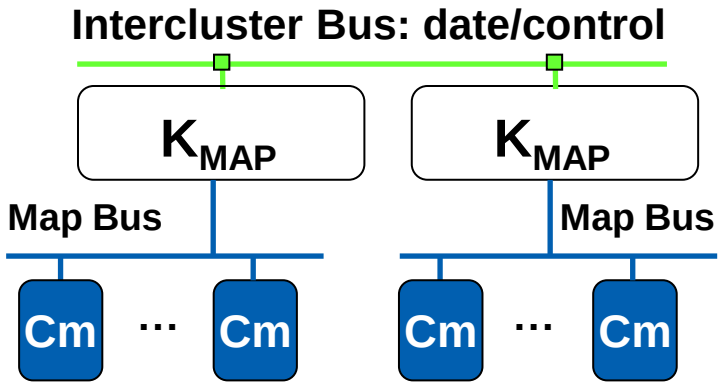
Comunicarea Intercluster



6. Cm Slave transmite rezultatul catre K_{MAP} Slave
7. K_{MAP} Slave pregateste mesajul intercluster (reactivarea contextului respectiv)
8. K_{MAP} Slave transmite rezultatul catre K_{MAP} Master
9. K_{MAP} Master receptioneaza si interpreteaza mesajul primit
10. Rezultatul este trimis catre Cm Master

Protocolul Magistralei Intercluster

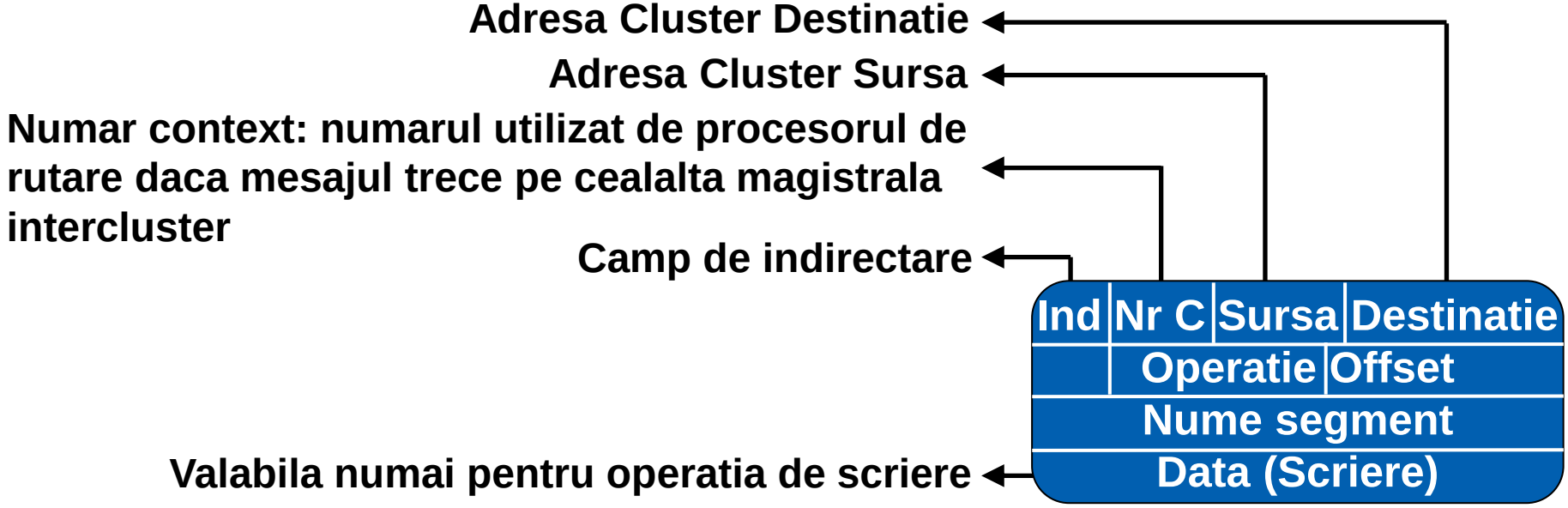
- Magistrala intercluster
  - Contine linii de date si control
  - **Nu** contine linii de adresa pt ca se realizeaza comutare de pachete si nu de circuite



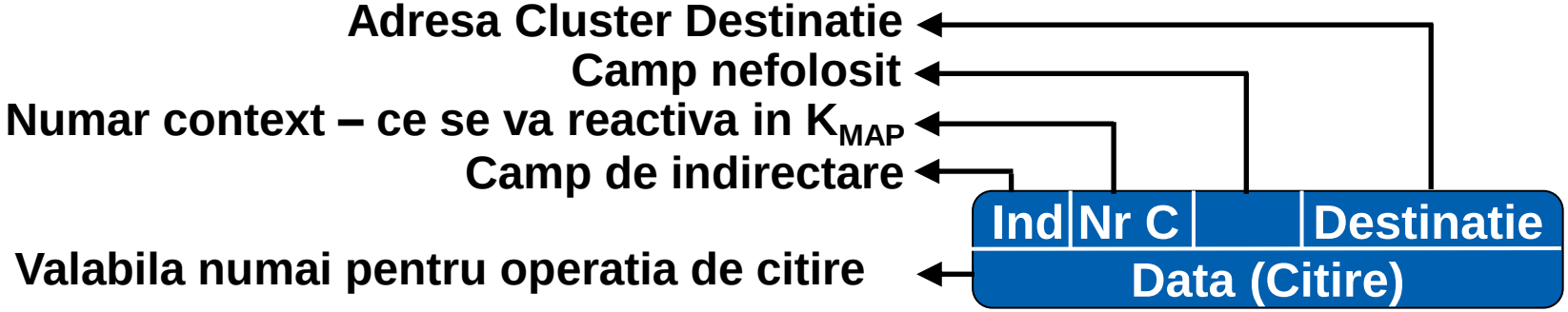
- Comunicarea este:
  - Asicrona
  - De tip intrebare-raspuns (Q & A)
  - Cu interblocare completa – pentru fiecare actiune se primeste raspuns si abia apoi se anuleaza comanda
- Mesajele intercluster sunt formate din 1-8 cuvinte
- Cele mai utilizate: mesajele directe si de raspuns

Formatul Mesajelor Intercluster

- Mesajul Direct – format din 4 cuvinte



- Mesajul de Raspuns – format din 2 cuvinte



What Next?

- Q & A?

- Next time:
  - Benchmarking – nevoia de a compara sisteme de calcul
  - Benchmark-uri pentru masini seriale
  - HPCC: HPC Challenge Benchmark
    - Motivatie
    - Prezentarea componentelor software
    - Grafice & Date

Arhitectura Sistemelor de Calcul

Universitatea Politehnica Bucuresti

Facultatea de Automatica si Calculatoare

cs.ncit.pub.ro

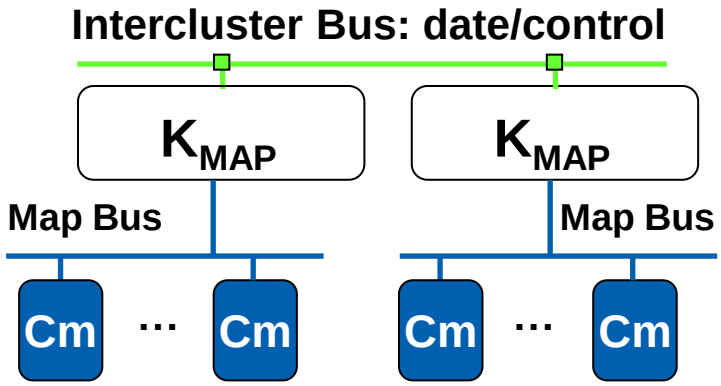
curs.cs.pub.ro

Cuprins

- Protocolul Magistralei Intercluster
- Modele de Acces – Capabilitati
- Sisteme cu Procesoare Omogene
  - Simetrice
  - Asimetrice
- Interconectarea in Sistemele cu Resurse Multiple

Protocolul Magistralei Intercluster

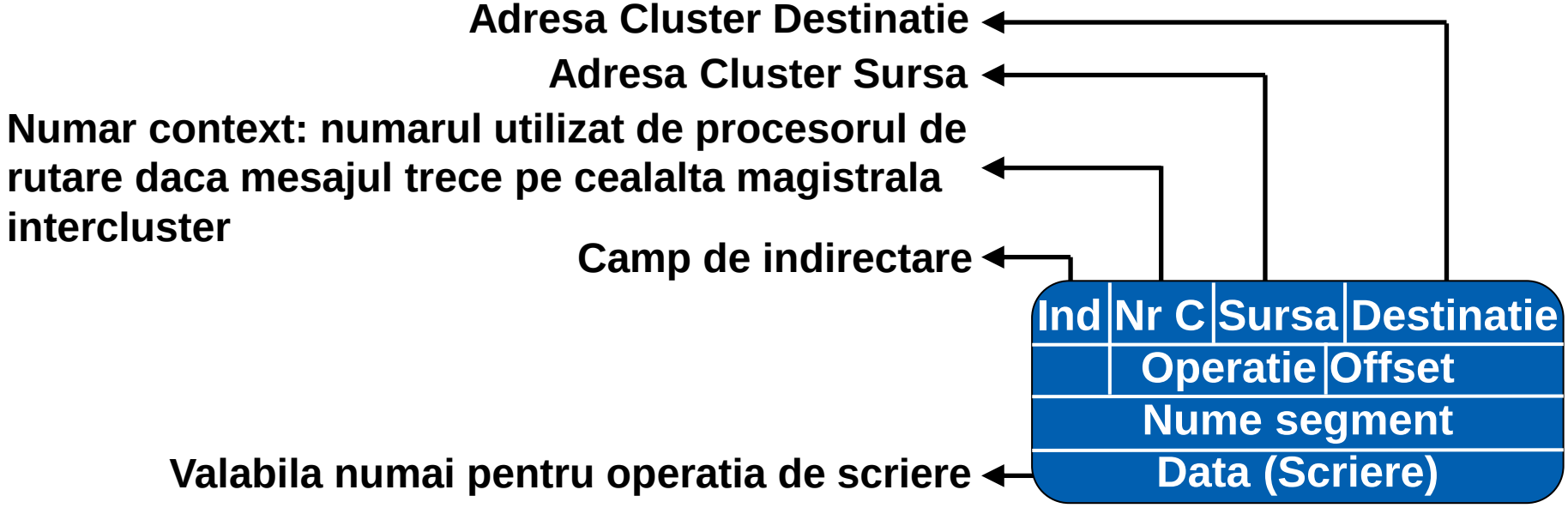
- Magistrala intercluster
  - Contine linii de date si control
  - **Nu** contine linii de adresa pt ca se realizeaza comutare de pachete si nu de circuite



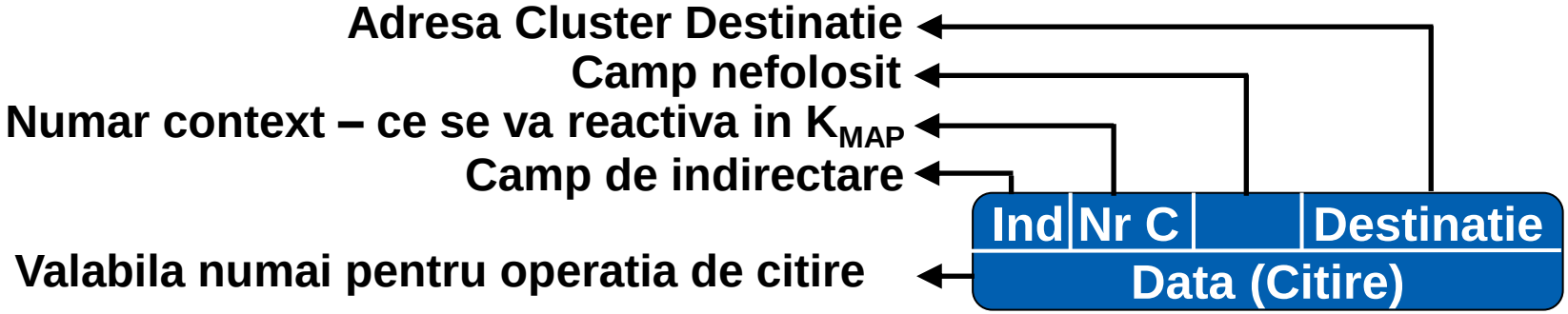
- Comunicarea este:
  - Asicrona
  - De tip intrebare-raspuns (Q & A)
  - Cu interblocare completa – pentru fiecare actiune se primeste raspuns si abia apoi se anuleaza comanda
- Mesajele intercluster sunt formate din 1-8 cuvinte
- Cele mai utilizate: mesajele directe si de raspuns

Formatul Mesajelor Intercluster

- Mesajul Direct – format din 4 cuvinte



- Mesajul de Raspuns – format din 2 cuvinte



Cuprins

- Protocolul Magistralei Intercluster
- Modele de Acces – Capabilitati
- Sisteme cu Procesoare Omogene
  - Simetrice
  - Asimetrice
- Interconectarea in Sistemele cu Resurse Multiple

Modele de Acces la Memorie

- Cu capabilitati:
  - Cu mascare de biti
  - Cu liste – Partitioning/C-lists
  - Cu criptare locala – Sparse Encrypted Capabilities
- Access Control Lists:
  - Pentru fiecare obiect retin o lista de subiecti si drepturi de acces
- Encrypted Methods Model:
  - Pentru fiecare cerere de acces la un anumit obiect, atasez si o parola de acces
  - Exista o lista globala cu reuniunea drepturilor & cheilor pentru fiecare obiect → compararea cheile

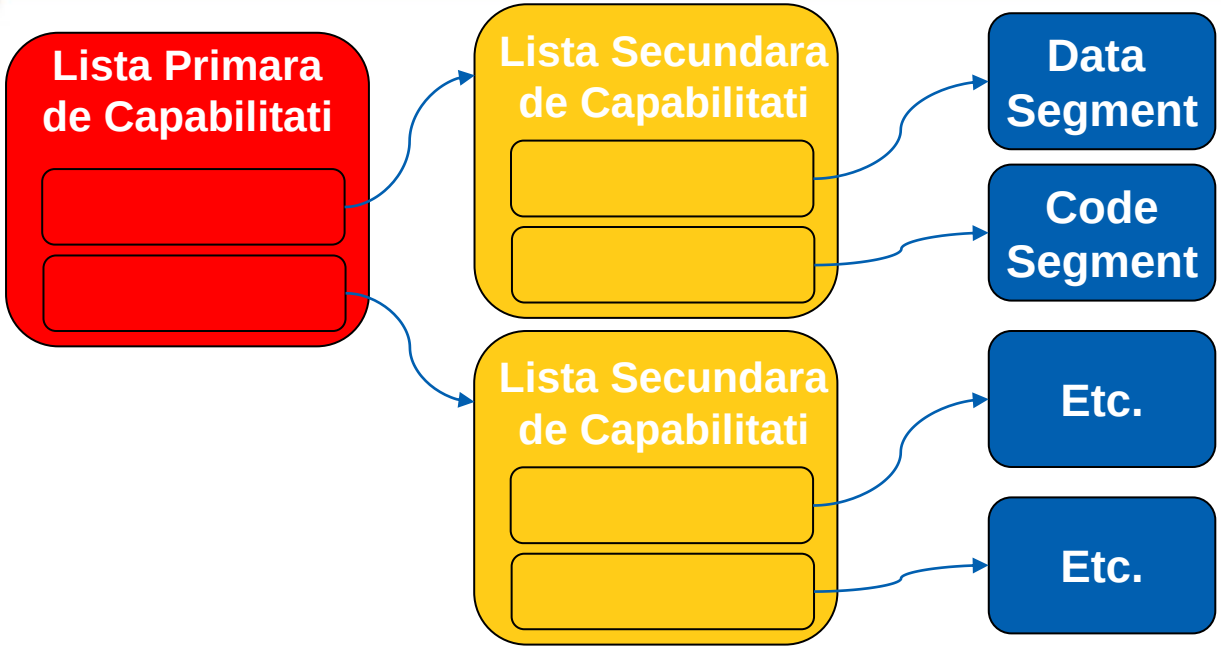
Capabilitati

- Un segment de memorie e descris de un descriptor de segment – informatie asupra adresei fizice de baza a segmentului & lungimea segmentului
- Segmentele memoriei nu sunt niciodata adresabile direct, ci doar prin capabilitati – din 2 cuvinte:
  - Numele segmentului
  - Drepturile de acces ale operatiei la segment
- Capabilitatile sunt utilizate ca suport pentru
  - Schimbarea eficienta a contextului
  - Transmiterea de mesaje
- Capabilitatile trebuie sa contina informatii privind structura unui proces definit la nivelul SO

Capabilitati (2)

- Fiecare proces e reprezentat printr-un mediu (environment) == structuri de date organizate pe 3 nivele/tipuri
  - Nivelul 1 contine lista primara de capabilitati (8):
    - Intrarea 0 o reprezinta segmentul sau vectorul de stare al procesului ce asteapta – bun pt IDLE
    - Restul (1-7) sunt legaturi la capabilitati secundare ce reprezinta segmentele la care are acces procesul in rulare
  - Nivelul 2 contine lista secundara de capabilitati
  - Nivelul 3 contine segmente de date, cod, s.a.m.d.
- Un proces poate
  - Accesa doar segmente pentru care exista capabilitati
  - Efectua numai acele operatii permise de aceste capabilitati
- Listele de capabilitati ofera posibilitatea de utilizare in comun de segmente sau seturi de segmente de catre procesoare care coopereaza

Capabilitati (3)



- In memorie, o pagina e rezervata pt interactiunea directa a programului din memoria locala cu K_{MAP}
- Este necesar un index la lista de capabilitati
  - Selectarea de subliste de 8 capabilitati
  - Offset-ul in cadrul liste de capabilitati

Capabilitati – Implementari

1. Cu mascare de biti
   - Prin adaugarea la fiecare cuvant din memorie a unor informatii – drepturi de acces codificate → consum mare de memorie
   - Se face AND intre drepturile de acces ale utilizatorului si drepturile cuvantului de date din memorie → usor de implementat HW
2. Cu liste – Partitioning/C-Lists
   - SO-ul retine liste de drepturi de acces pentru diverse obiecte (C-Lists)
   - SO-ul valideaza dreptul de acces al utilizatorului
   - SO-ul e strans legat aici de HW = extensie a acestuia
3. Cu criptare locala – Sparse Encrypted Capabilities
   - SO-ul are liste de obiecte cu cererea/cheia de acces/drepturile de acces → face verificarea cheii

Cuprins

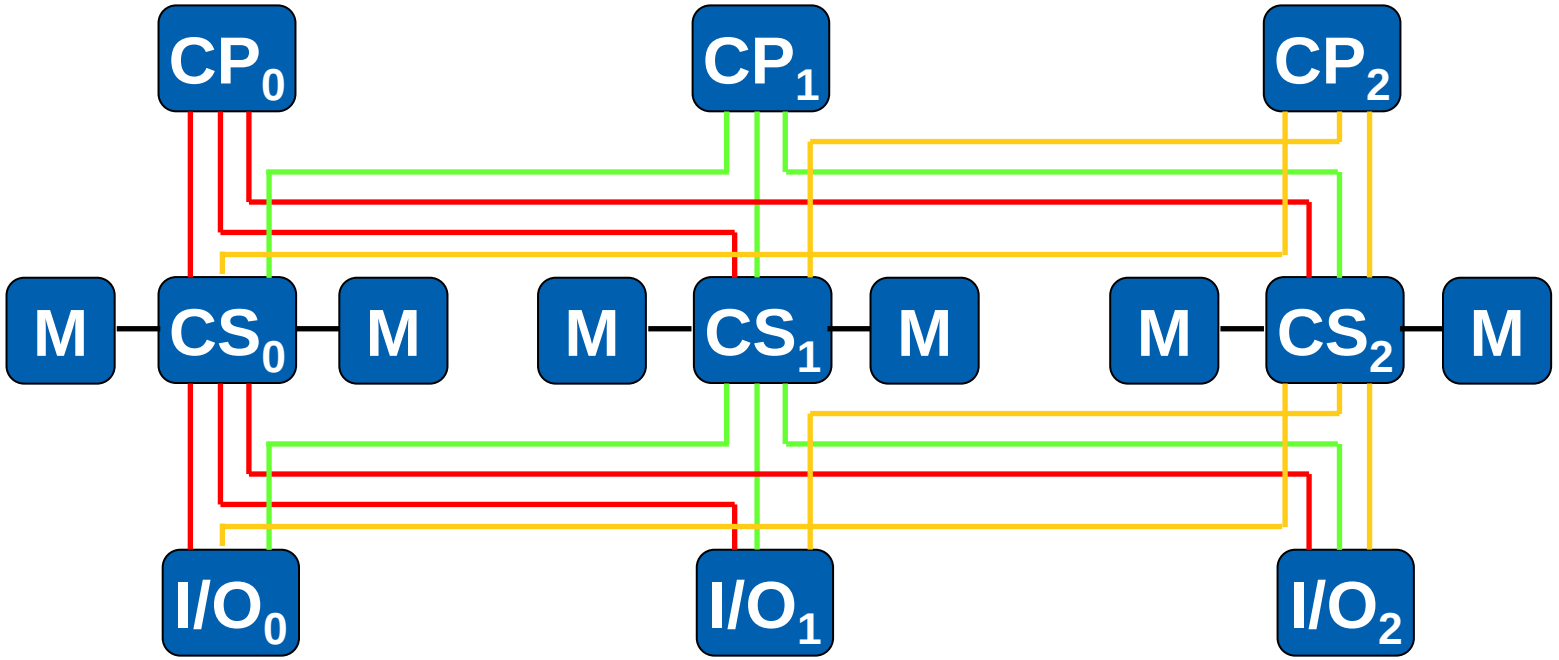
- Protocolul Magistralei Intercluster
- Modele de Acces – Capabilitati
- Sisteme cu Procesoare Omogene
  - Simetrice
  - Asimetrice
- Interconectarea in Sistemele cu Resurse Multiple

Sisteme MIMD

- In sistemele MIMD procesoarele pot fi
  - Omogene
  - Eterogene
- Un sistem este omogen daca
  - Procesoarele sunt identice
  - Elementele I/O sunt identice
- Sistemele cu procesoare omogene pot fi
  - Cu procesoare simetrice – identice functional si I/O
    - Mai usor de programat – programarea resurselor e similara pt toate procesoarele
    - Ofera facilitati de prelucrare al proceselor in caz de defectiune al unor resurse (degradare lenta a performatelor)
    - In general include si elemente de redundanta
  - Cu procesoare asimetrice – identice dpdv functional dar diferite ca subsisteme I/O

Sisteme Omogene Simetrice

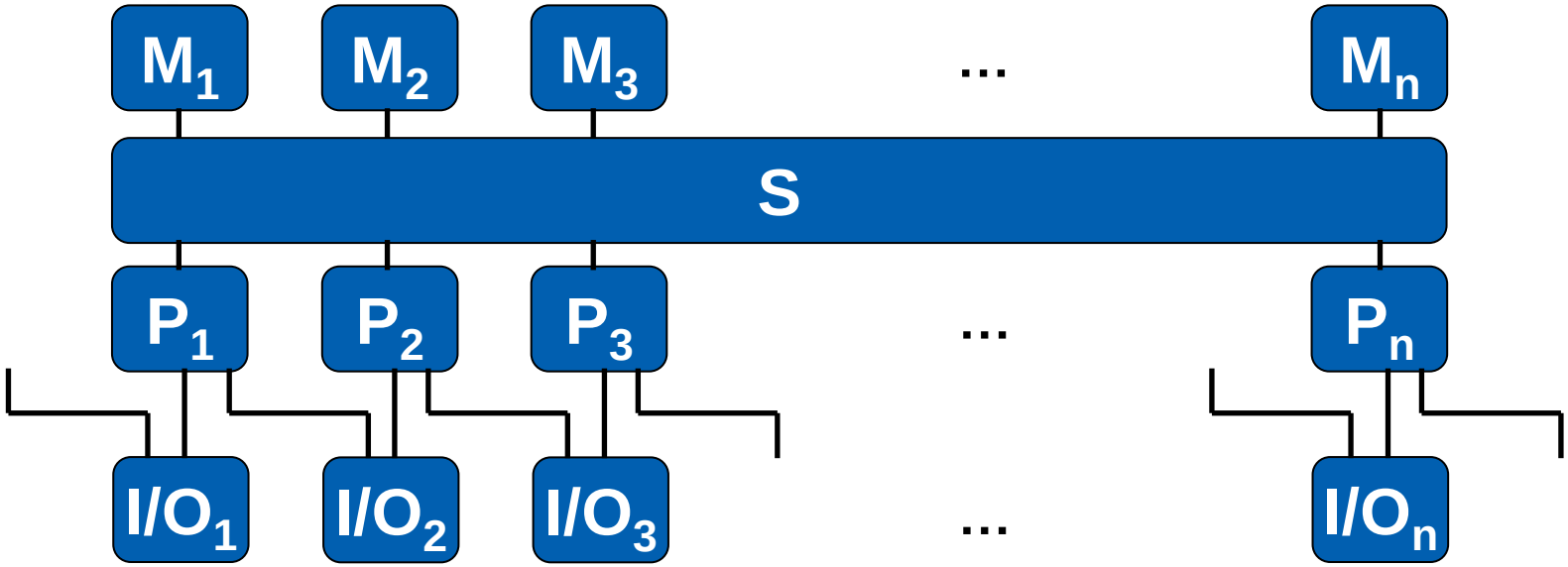
- Arhitectura Honeywell contine:
  - CP – Central Processor = procesoare centrale
  - CS – Command Switch = switch-uri cu functii de comutare si control
  - M – Memory = doua module/proc, accesabile de orice procesor
  - I/O – I/O System = subsistem de intrare/iesire partajat
- Conexiunile directe ofera redundanta cailor de comunicare pentru o disponibilitate maxima a sistemului



Sisteme Omogene Simetrice (2)

- CP-urile trebuie sa acceseze 3 componente:
  - Subsistemul I/O
  - Doua module de memorie
- Pentru ca CP-urile sa poata comunica intre ele, se conecteaza CS-urile intre ele
- Daca se defecteaza vreun controler de sistem, orice subsistem I/O e accesibil de la orice procesor
- CS-urile asigura si transmiterea intreruperilor intre CP-uri si subsistemele I/O
- Aceste structuri se incadreaza in cadrul sistemelor multiprocesor “strans cuplate”

Alt Sistem Strans Cuplat



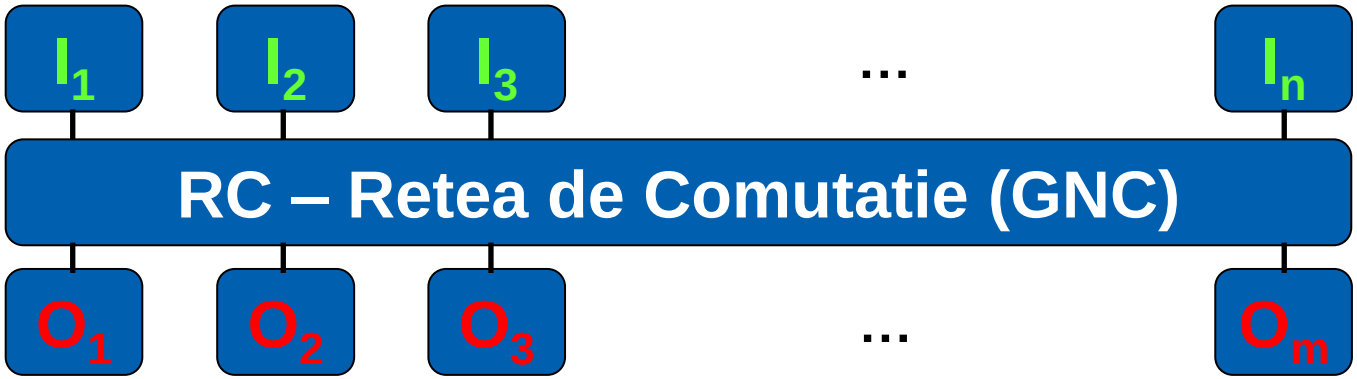
- Procesoarele P_i si Memoriile M_j au asociate niste subsisteme de I/O_k
- Un procesor are acces la cel putin doua subsisteme I/O succesive
- In cazul unui eventual defect al unui procesor, nu este afectat nici accesul la I/O respectiv si nici preluarea sarcinilor acelui procesor

Cuprins

- Protocolul Magistralei Intercluster
- Modele de Acces – Capabilitati
- Sisteme cu Procesoare Omogene
  - Simetrice
  - Asimetrice
- Interconectarea in Sistemele cu Resurse Multiple

Interconectarea in Sistemele cu Resurse Multiple

- In structurile SIMD si MIMD, retelele de comutare (de interconectare a resurselor), au rolul cel mai important in asigurarea performantelor sistemului!
- Realizarea interconectarii resurselor (P, M & I/O) se face printr-o retea de comutatie (RC)



- I_i = Intrari & O_j = Iesiri; RC conecteaza n·m resurse
- Net (n, m) = Generalised Connection Network – graf in care arcele conecteaza perechi de resurse

Retele de Comutatie

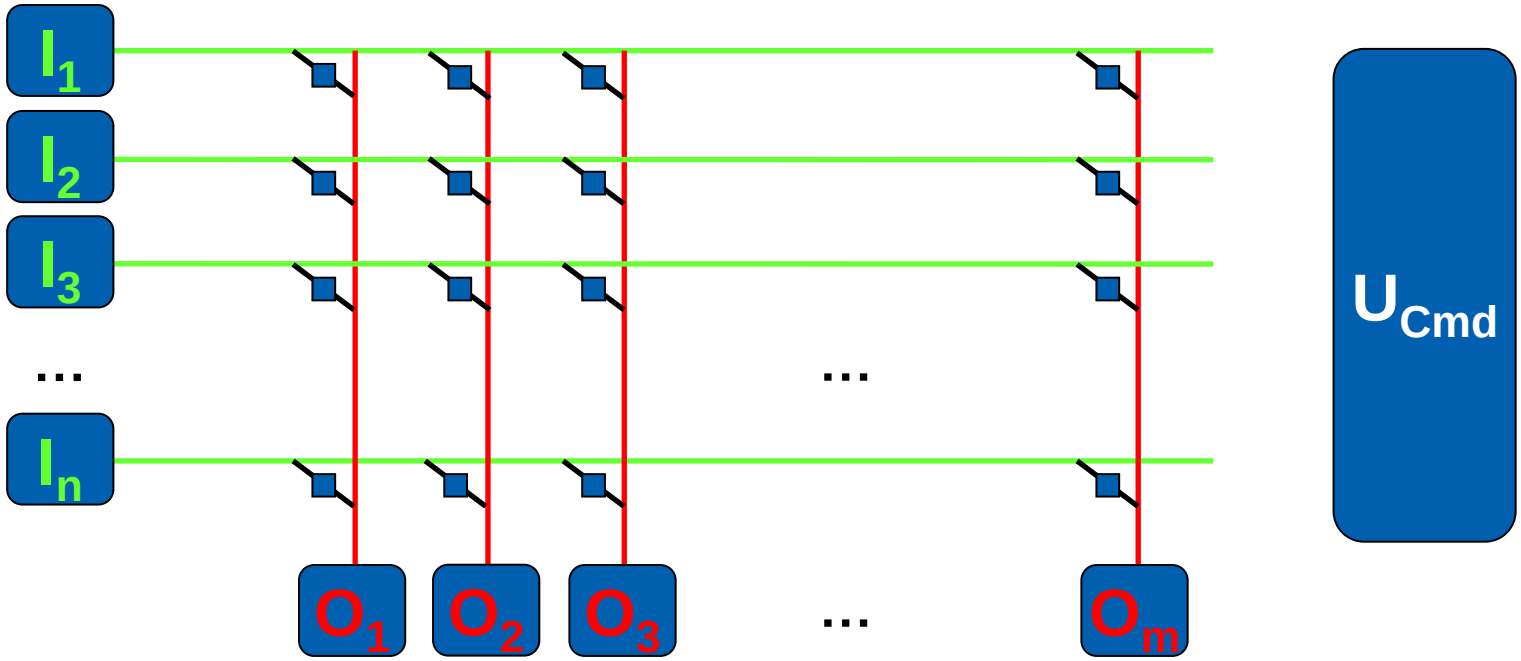
- O astfel de RC trebuie sa respecte niste reguli:
  - O intrare poate fi conectata la orice iesire
  - O intrare poate fi conectata simultan la mai multe iesiri
    - Astfel vom avea Broadcast partial sau total
  - O iesire poate fi conectata numai la o intrare (la un anumit moment dat – de exemplu 2 P pot accesa aceeasi M in acelasi timp)
  - Concurenta maxima e data de min (m, n)
- Din aceste reguli rezulta ca RC ar trebui sa realizeze n^m corespondente == GCN!

GCN – Generalised Connection Network

- Un GCN poate fi implementat
  - Printr-o structura originizata pe un singur nivel in care se stabilesc legaturi directe intre intrari si iesiri (crossbar)
  - Prin utilizarea unor subretele de dimensiuni reduse conectate ierarhic pe mai multe nivele
- Caracteristici pentru GCN sunt doi parametrii
  - Intarzierea de propagare (Δt) – timpul de raspuns ce depinde de numarul maxim de comutatoare elementare (de tip poarta) ce asigura legatura intre intrare si iesire
  - Complexitatea (nC) – numarul total de comutatoare elementare utilizate in implementarea GCN-ului

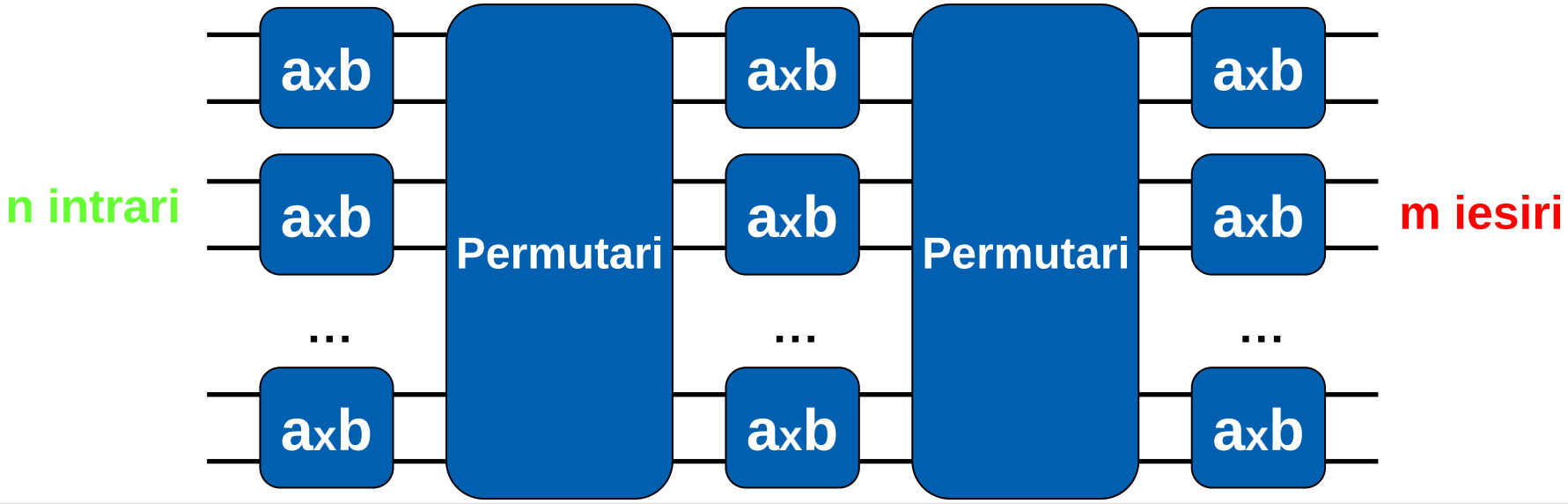
Implementari ale RC (GCN)

- Comutare directa – Cross Bar cu n·m comutatoare ($\Delta t = 1$)
- Unitatea de comanda trebuie sa rejecteze 2 I / 1 O
- O implementare cu comutare directa
  - Este greu de proiectat fizic (nC = n·m)
  - Nu se preteaza la VLSI & este inflexibila la dezvoltare ulterioara
- U_{Cmd} controleaza toate switch-urile elementare



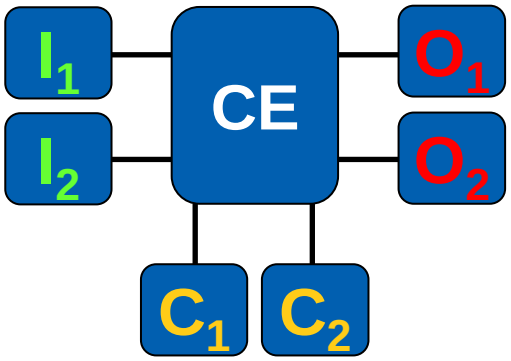
Implementari ale RC (GCN)

- Prin realizarea unor subretele de comutare organizate ierarhic, pe mai multe niveluri
- Aceasta implementare necesita $k \approx \log_2 n$ comutatoare elementare → reduce complexitatea (nC) & creste timpul de comutare (Δt)
- Comutatoarele elementare sunt de tip Cross Bar (axb), unde a « n si b « m
- Intre nivele se aseaza permutarile de interconectare



Comutatoare Elementare

- Un comutator elementar are 2 intrari, 2 iesiri si 2 biti de comanda:



C_1	C_2	Conexiuni	
0	0	$O_1 = I_1; O_2 = I_2$	Direct
0	1	$O_1 = I_2; O_2 = I_1$	Inversat
1	0	$O_1 = I_1; O_2 = I_1$	Broadcast Superior
1	1	$O_1 = I_2; O_2 = I_2$	Broadcast Inferior

- Aceste module se pot integra pe baza permutarilor elementare
- In acelasi mod se pot construi si comutatoare elementare 4x4, 8x8, etc.

What Next?

- Q & A?
- Next time:
  - Retele de Comutare Ierarhice
  - Retele de Comutare de tip Delta
  - Retele Bazate pe Rutare
  - Performantele Retelelor de Comutare
    - Analiza Retelei de Tip CrossBar
    - Analiza Retelei de Tip Delta

Computer Science & Engineering Department

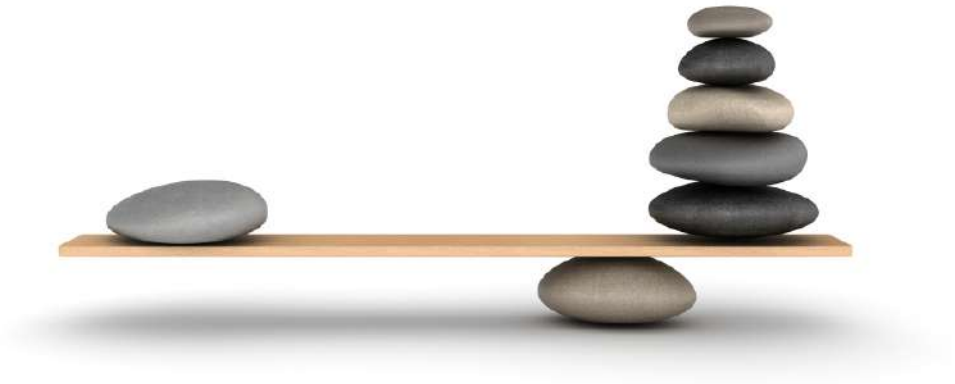
Arhitectura Sistemelor de Calcul

Curs 11
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

Cuprins

- Nevoia de a compara sisteme de calcul
- Cum masuram Performanta?
- Benchmark-uri pentru masini “seriale”
- Benchmark-uri pentru masini “paralele”



Nevoia de a Compara Sisteme de Calcul

Motivatie

- Lectura specificatiilor unor sisteme de calcul nu este suficienta pentru a ne oferi o imagine completa asupra performantelor acestora
- Asadar, a fost necesara crearea unor teste (benchmark-uri) care sa permita compararea performantelor intre sisteme/arhitecturi diverse
- Testele constau din seturi de programe care sunt menite sa scoata in evidenta performantele diverselor componente ale unui SC:
  - Floating Point CPU Performance
  - Caracteristici de interconectare (Bandwidth/Latenta)
  - Memorii externe (Hard-Drives)

Caracteristici & Probleme

- Sunt create pentru a “simula” comportamentul aplicatiilor reale (sintetice)
- Cele mai bune benchmark-uri sunt aplicatiile utilizator in sine…
- Trebuie sa fim intotdeauna circumspecti cand utilizam benchmark-uri!
- Cateva probleme:
  - Producatorii HW fac “tuning”
  - Performantele masinilor se degradeaza cand load-ul depaseste o valoare de prag
  - Securitatea, disponibilitatea, fiabilitatea, serviceabilitatea sau scalabilitatea NU sunt obiectul unui benchmark

Tipuri de Benchmark-uri

- Aplicatii reale: procesoare de text, aplicatii utilizator
- Kernel: contine doar parti esentiale dintr-o varietate de programe (i.e. Linpack), cu rezultate in MFlops
- Micro-benchmark-uri: dezvoltate de utilizatori pentru a testa componente ale sistemului
- Sintetice: se strang date statistice asupra operatiilor efectuate de aplicatii si se construiesc programe care sa respecte proportia aflata astfel (Whetstone/ Dhrystone)
- Pentru sisteme I/O
- Pentru sisteme paralele

Cum Masuram Performanta?

Latency vs. Throughput

- **Latency** (timpul de executie): timpul necesar terminarii unui task anume
- **Throughput** / bandwidth (latimea de banda): numarul de taskuri/transferuri intr-un timp fix
  - Exploateaza paralelismul pentru throughput si nu pentru latenta
  - Deseori latenta si throughputul pun presiuni contradictorii pe sisteme
  - Definitii diferite in functie de cod
    - Simulare Stiintifica - **Latency**
    - Web-Server - **Throughput**

Exemplu: Latency vs. Throughput

- Cum mutam oameni 10 km?
  - Masina: capacitate = 5; viteza = 60 km/h
  - Autobuz: capacitate = 60; viteza = 20 km/h
- Latency: **Masina = 10 min**; Autobuz = 30 min
- Throughput: Masina = 15 ppl/h; **Autobuz = 60 ppl/h**

Comparand Performanta - Speedup

- A este de X ori mai rapid ca B daca
  - X = Latency(B)/Latency(A)
    - Imparte la cel mai rapid
  - X = Throughput(A)/Throughput(B)
    - Imparte la cel mai incet
- A este cu X% mai rapid ca B daca
  - Latency(A) = Latency(B) / (1 + X/100)
  - Throughput(A) = Throughput(B) * (1 + X/100)

Exemplu: Latency vs. Throughput

- Latency
  - Masina este de **3** ori (si **200%**) mai "rapida" ca Autobuzul

- Throughput
  - Autobuzul este de **4** ori (si **300%**) mai "rapid" ca Masina

Speedup si % Crestere/Descrestere

- Programul A ruleaza 200 cicli - facand un task X
- Programul B ruleaza 350 cicli - facand acelasi task X
- % crestere si descrestere **nu sunt la fel**:
  - % crestere: ((350-200)/200)*100 = **75%**
  - % descrestere: ((350-200)/350)*100 = **42,3%**
- Speedup:
  - 350/200 = 1.75: A este de 1.75 ori mai rapid ca B
  - % crestere: (1.75 - 1)*100 = 75%
- Daca C e 1.5x mai rapid ca A, cati cicli ruleaza C?
  - 133 cicli, adica 50% mai rapid ca A

Calculul Performantei - Medii

- Media Aritmetica: (1/N) * $\sum_{P=1..N}$ Latenta(P)
  - Pentru marimi ce sunt proportionale cu timpul
- Media Armonica: N / $\sum_{P=1..N}$ 1/Throughput(P)
  - Pentru marimi ce sunt invers proportionale cu timpul
- Media Geometrica: $\sqrt[N]{\prod_{P=1..N}}$ Speedup(P)
  - Pentru marimi adimensionale

Exemplu de Medie Armonica

- Conduceti 2 km
  - 30 km/h pentru primul km
  - 90 km/h pentru al doilea km
- Care este viteza medie?
  - Atentie: raspunsul **nu este 60 km/h**
  - 0.033 h / km pentru km unu
  - 0.011 h / km pentru km doi
  - 0.022 h / km in medie = **45 km/h**

Performanta Procesoarelor vs. Workloads

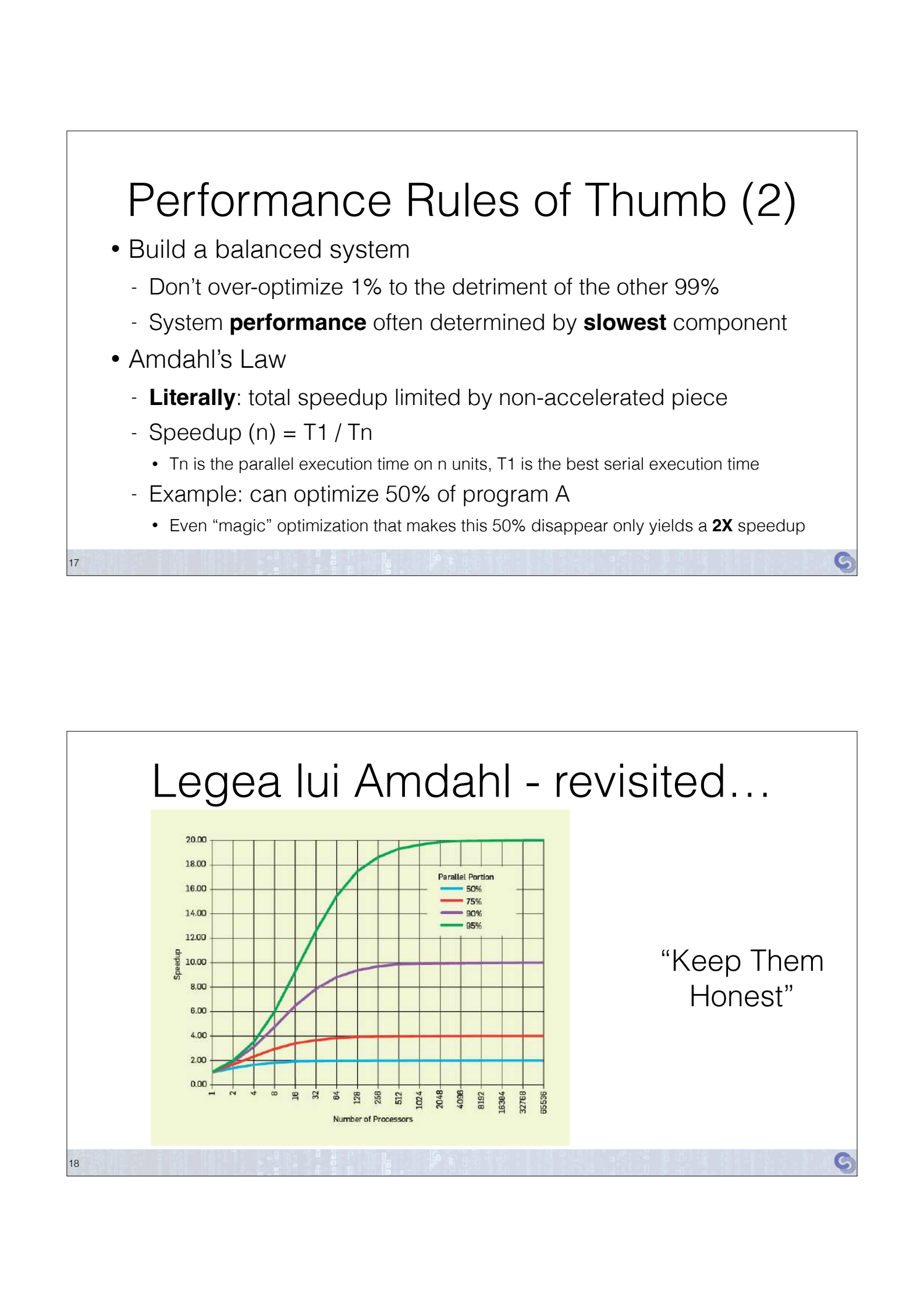
- Q: Ce inseamna performanta unui procesor?
- A: Nimic fara un workload asociat.
- Benchmarks
  - Reprezinta un set standard de workload-uri
  - Utilizate pentru a compara performanta mai multor sisteme
  - Reprezentative programelor reale sau chiar programe reale
- Micro-Benchmarks
  - Set non-standard de non-workload-uri
  - Programe mici utilizate pentru izolarea unor chestiuni de performanta
  - Nereprezentative pentru manifestari complexe: turnuri Hanoi, 8-queens, etc.

Performance Rules of Thumb

- Design for actual performance, not peak performance
  - Peak performance: “Performance you are guaranteed not to exceed”
  - Greater than “actual” or “average” or “sustained” performance
  - For actual performance X, machine capability must be > X
- Easier to “buy” bandwidth than latency
  - Which approach is easier, to transport more cargo via train?
    - Build another track?
    - Make a train that goes twice as fast?
  - Use bandwidth to reduce latency

Performance Rules of Thumb (2)

- Build a balanced system
  - Don't over-optimize 1% to the detriment of the other 99%
  - System **performance** often determined by **slowest** component
- Amdahl's Law
  - **Literally**: total speedup limited by non-accelerated piece
  - Speedup (n) = T1 / Tn
    - Tn is the parallel execution time on n units, T1 is the best serial execution time
  - Example: can optimize 50% of program A
    - Even "magic" optimization that makes this 50% disappear only yields a **2X** speedup



Legea lui Amdahl - revisited…

"Keep Them Honest"

Benchmark-uri pentru Masini “Seriale”

SPEC

- SPEC (Standard Performance Evaluation Corporation) realizeaza benchmark-uri pentru sisteme de calcul si le publica pe web
- Incearca sa simuleze situatii reale:
  - SPECweb testeaza performantele unui server de web pentru cereri HTTP paralele
  - SPEC CPU testeaza CPU-ul ruland 29 de aplicații INT si FP
- SPEC sunt scrise in C sau Fortran
  - Compilatoarele pot avea optimizari pentru acest tip de aplicatie
- Se distribuie cu o licenta
- Exista un conflict cu GPL (foloseste cod GPL)… http://www.spec.org/

Performanta SPEC

- Un sistem cu SPEC 150 este de 1.5 ori mai rapid decat un sistem cu SPEC 100
- Numarul SPEC generic se calculeaza din performanta celor 29 de aplicatii:
  - Medie geometrica
  - Medie aritmetica
  - Medie aritmetica ponderata in functie de prioritatile utilizatorului

GeekBench

- Benchmark-uri multicore multi-platforma
  - Laptop, workstation, iPhone, Android, etc.
- Performanta: Int, FP, Memorie, Bandwidth, etc.
- Salvarea rezultatelor online
  - Usor de comparat cu alte sisteme/procesoare
- Problema: workload-uri simple ce nu “streseaza” complet sistemele

Exemple GeekBench

- Laptop:
  - Intel “Ivy bridge” at 3.4 GHz (4 cores) 11,456
- Workstation
  - Intel Core i7-3520M at 2.9 GHz (2 cores) 7,807
- Platforme mobile:
  - iPhone 4S - Apple A5 at 0.8 GHz (2 cores) 642
  - iPhone 5 - Apple A6 at 1 GHz (2 cores) 1,589
  - Samsung G S3 - Snapdragon S3 – 1.5 GHz (2 cores) 1,429

TPC & BAPCo & Sandra

- TPC (Transaction Processing Performance Council) a definit benchmark-uri pentru procesarea tranzactiilor si a bazelor de date:
  - http://www.tpc.org/
- Consortiul BAPCo (Business Application Performance Corporation) este format din Intel, AMD, Dell, HP, NVidia, Apple & Microsoft, etc:
  - WEBmark, SYSmark, si MobileMark
  - http://www.bapco.com/
- SiSoftware Sandra (System ANalyser, Diagnostic and Reporting Assistant) este un program de testare si diagnoza:
  - http://www.sisoftware.net/

Benchmark-uri pentru Masini “Paralele”

The LINPACK Benchmark

- Linpack folosit pentru sisteme paralele pentru ca
  - Este extrem de raspandit si utilizat
  - Date si teste pentru acest program sunt disponibile pentru **toate** sitemele relevante din lume
- Benchmark-ul masoara cat de repede rezolva computer-ul un sistem dens de ecuatii liniare $A \cdot x = b$ de dimensiune $n \cdot n$
- Solutia se bazeaza pe metoda eliminarii Gaussiene, utilizand pivotarea partiala
- Conduce la $2/3 \cdot n^3 + n^2$ operatii in virgula mobila $O(n^3)$
- Se exclude utilizarea metodelor optimizate de inmultire a matricelor de genul metodei lui Strassen $O(n^{2.807})$ sau Coppersmith–Winograd $O(n^{2.376})$
- In acest mod se vor genera milioane de operatii in virgula mobila pe secunda (Mflop/s)
- Rezolvarea sistemelor de ecuatii liniare este o problema intalnita **frecvent** in aproape orice aplicatie stiintifica sau inginereasca

The LINPACK Benchmark (+)

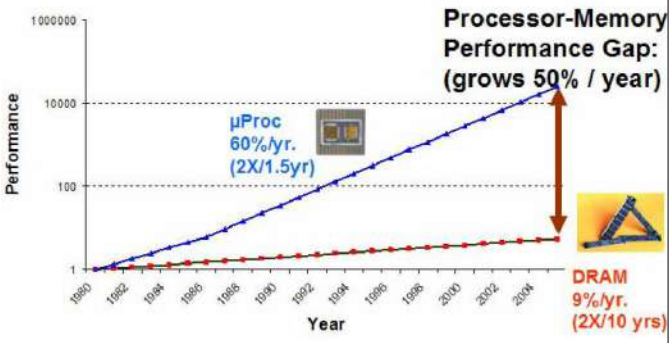
- Avantaje:
  - Ofera ca rezultat un **singur** numar
    - Rmax = performanta maxima (**obtinuta**)
    - Rpeak = performanta maxima teoretica de varf (**estimata**)
  - Usor de definit si usor de clasificat pe baza lui
  - Permite modificarea dimensiunii problemei (n x n)
  - Ocupa sistemul testat cu un job de lunga durata
  - Disponibil la: http://www.netlib.org/benchmark/hpl/

The LINPACK Benchmark (-)

- Dezavantaje
  - Scoate in evidenta **doar** viteza “peak” si numarul de procesoare
  - **Nu** evidentiaza (explicit) rata de transfer locala (bandwidth)
  - **Nu** evidentiaza (explicit) reteaua de comutare
  - **Nu** testeaza (explicit) mecanisme de tipul gather/scatter
  - Ignora legea lui Amdahl – permite doar scalare “slaba”
- Un singur numar nu poate caracteriza performanta totala a unui sistem de calcul!

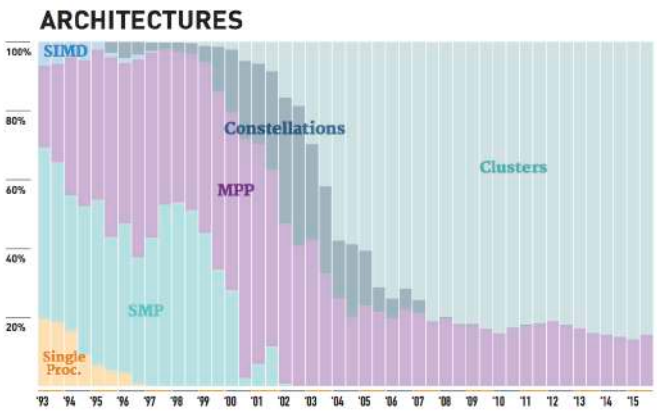
Relevanta LINPACK

- Daca luam in considerare anii 80'-90', poate LINPACK era cel mai potrivit benchmark
- Zidul memoriei era mai mult o treapta
- Memoria era mai "apropiata" de CPU:
  - n ciclii / executie instructiune
  - n ciclii / aducerea unui cuvant din memorie
- LINPACK arata de asemenea gradul de optimizare al compilatoarelor



Schimbari ale Sistemelor de Calcul

- In ultimii 40 de ani s-au schimbat foarte multe in hardware-ul sistemelor de calcul:
  - Arhitecturi vectoriale, superscalare, distribuite, cu memorie partajata, sisteme multicore, etc
- Cu toate ca LINPACK a fost modificat, nu toate modificarile reflecta progresele hardware
- Ierarhia memoriilor este mult mai complexa acum



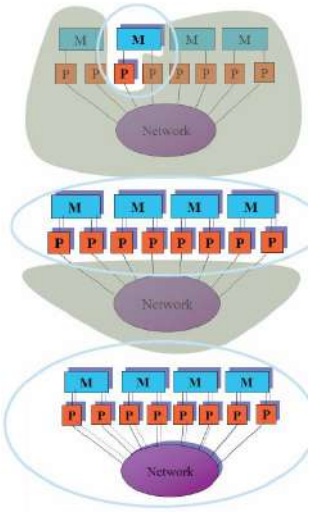
HPC Challenge Benchmark

- Un Benchmark ce evidentiaza
  - Procesoarele
  - Memoria
  - Reteaua de interconectare
- Avantajul major al HPCC
  - Arhitecturile pot fi descrise cu mult mai multe metrici decat FLOPS-urile LINPACK
- Scopuri ale HPCC
  - Complementar cu TOP 500
  - Exprimarea performatei aplicatiilor ca functie a caracteristilor accesului la memorie (localitate spatiala / temporala)
  - Permite optimizari
  - Inregistreaza efortul necesar pentru tuning-ul aplicatiilor
  - Rezultate arhivate si verificabile

HPCS = High Productivity Computing Systems
DARPA = Defense Advanced Research Projects Agency

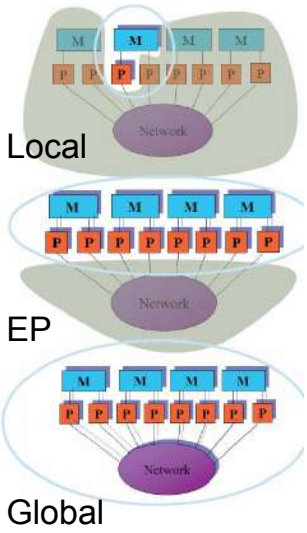
Testarea unui singur Procesor si a intregului Sistem de Calcul

- **Local**: un singur procesor participa la calculele dorite
- **Embarrasingly Parallel**: fiecare procesor din sistem participa la calcule, insa ele nu comunica explicit intre ele
- **Global**: toate procesoarele din sistem participa la calcule si comunica explicit intre ele

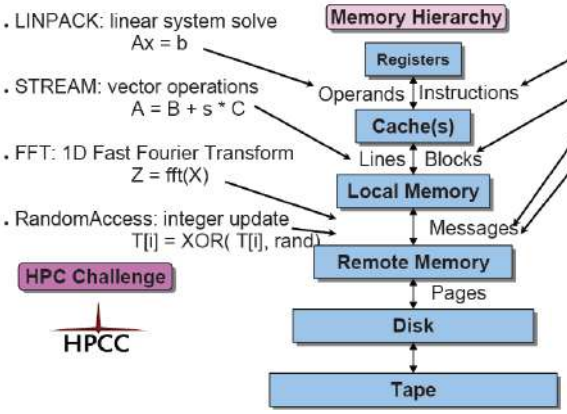


Componente HPCC

- HPCC e format din 7 benchmark-uri relevante:
  1. **HPL** (LINPACK) – MPI Global (Ax=b)
  2. **STREAM** – Local / ***STREAM** – EP (A ← B + s*C)
  3. **PTRANS** – MPI Global (A ← A + B^T)
  4. **RandomAccess** – Local, MPI Global / ***RandomAccess** – EP (T[i] ← XOR(T[i], rand))
  5. **Bandwidth** and **Latency** – MPI
  6. **FFT** – Local, Global & EP (Z ← fft(X))
  7. **Matrix Multiply** – Local & EP
- HPCC == framework pentru adaugarea de benchmark-uri



Scopul HPCC



Max	Relative
2 Pflop/s	8x
6.5 Pbyte/s	40x
0.5 Pflop/s	200x
64000 GUPS	2000x

DARPA HPCS

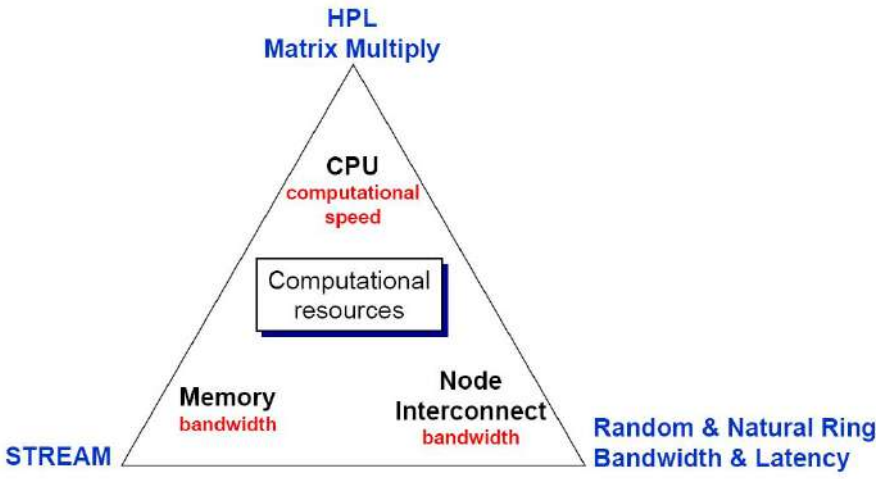
Performance Targets

- HPCS incearca prin HPCC sa:
  - Uniformizeze diferentele existente in ierarhia de memorie
  - Imbunatateasca performantele aplicatiilor reale
  - Usureze programarea sistemelor (paralele) de calcul

Functionarea HPCC

- Benchmark-ul functioneaza ca un program de sine statator
- Similar ca fisier de intrare/specificare cu HPL
- HPCC este format dintr-un “Base Run”, urmat de eventuale “Optimization Runs”:
  - Base Run este rularea Benchmark-ului “out of the box”
  - Utilizatorul trebuie sa instaleze pe masinile de test MPI si BLAS
  - In Optimized Runs se pot inlocui anumite subrutine, pentru a creste performantele unei anume componente
- Rezultatele se arhiveaza la: http://icl.cs.utk.edu/hpcc/
- Se pot genera apoi tabele html sau Excel cu rezultatele obtinute:
  - In mod intentionat nu se va oferi un singur criteriu de evaluare
  - Astfel NU se vor obtine clasificari generale!
- Scopul declarat al HPCC: sa NU necesite mai mult de dublul unei rulari HPL

Resurse vs HPCC



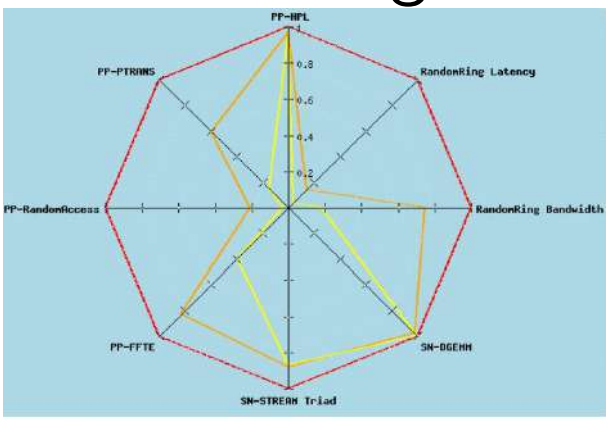
- Din ce sunt formate componentele HPCC?

Condensed Results - Base Runs Only - 114 Systems - Generated on Tue Nov 14 17:13:55 2006

System Information System - Processor - Speed - Count - Threads - Processes MA/PT/PS/PC/TH/PR/CM/CS/IC/LA/SD				G-HPL TFlop/s	G-PTRANS GB/s	G-Random Access Gup/s	G-FFTE GFlop/s	EP-STREAM Sys GB/s	EP-STREAM Triad GB/s	EP-DGEMM GFlop/s	RandomRing Bandwidth GB/s	RandomRing Latency usec
Cray Inc. Red Storm/XT3 AMD Opteron	2.4GHz	12960	125920	91.0250000	2355.9700	1.7401300	1354.0700	34840.499	2.1138	4.39989	0.03911	16.29
IBM Blue Gene/L PowerPC 440	0.7GHz	65536	165536	80.6830000	339.2840	0.0657312	2178.1100	53553.868	0.8172	1.80619	0.01084	8.84
IBM p5-575 Power5	1.9GHz	10240	110240	57.8670000	553.0090	0.1693440	842.5000	55184.179	5.3891	7.08552	0.11015	110.59
IBM p5-575 Power5	1.9GHz	8192	16192	45.7019000	2626.1700	0.3239760	908.6920	44455.936	5.4268	7.06423	0.08871	11.05
Cray Inc. XT3 Dual-Core AMD Opteron	2.6GHz	10404	110404	43.4033000	778.3650	0.8235630	1107.2100	25774.557	2.4774	4.78935	0.06537	14.32
IBM Blue Gene/L PowerPC 440	0.7GHz	65536	165536	37.0540000	4665.9100	0.1648000	1762.8200	62899.707	0.9596	2.47017	0.01039	9.63
IBM p5-575 Power5	1.9GHz	8192	26192	33.3173000	575.8230	0.2066390	965.6680	43802.460	5.3470	6.08616	0.07698	51.99
Cray Inc. XT3 AMD Opteron	2GHz	10330	110330	32.9869000	1013.0600	1.0176560	1119.2900	43581.780	4.2109	3.66719	0.16189	10.32
IBM Blue Gene/L PowerPC 440	0.7GHz	32768	132768	31.2381000	87.7818	0.2786090	1112.8100	29913.678	0.9129	2.17447	0.01197	9.91
Cray Inc. XT3 AMD Opteron	2.4GHz	5200	15200	20.5270000	874.8990	0.2653830	644.7300	26020.800	5.0040	4.39535	0.14682	25.80
Cray Inc. XT3 AMD Opteron	2.4GHz	5208	15208	20.4086000	944.2270	0.8724120	761.7290	24268.442	4.6598	4.41173	0.20636	9.20
Cray Inc. XT3 AMD Opteron	2.6GHz	4096	14096	16.9752000	302.9790	0.3330720	905.5690	20656.456	5.0431	4.75166	0.16886	9.44
Cray Inc. XT3 AMD Opteron	2.6GHz	4128	14128	16.6421000	674.7660	0.6767580	821.6770	19295.676	4.6743	4.75946	0.22245	8.22
Cray Inc. XT3 AMD Opteron	2.4GHz	3744	12744	14.3040000	608.5660	0.2202980	417.1720	18146.382	4.8468	4.41330	0.16164	25.32
Cray Inc. X1 Cray E	1.13GHz	1008	11008	12.0263000	168.0190	0.0861199	82.3684	15522.691	15.3983	14.50000	0.13667	16.30
HP XC Intel Itanium 2	1.6GHz	2048	12048	10.5616000	306.4190	0.2633320	339.8360	4606.031	1.9591	6.26484	0.15920	6.04

System Information System - Processor - Speed - Count - Threads - Processes MA/PT/PS/PC/TH/PR/CM/CS/IC/LA/SD				G-HPL TFlop/s	G-PTRANS GB/s	G-Random Access Gup/s	G-FFTE GFlop/s	EP-STREAM Sys GB/s	EP-STREAM Triad GB/s	EP-DGEMM GFlop/s	RandomRing Bandwidth GB/s	RandomRing Latency usec
SGI Columbia 2048 Intel Itanium 2	1.6GHz	2024	12024	9.0196500	18.1901	0.0481621	45.7760	3996.493	1.9758	6.23637	0.12271	6.98
NEC SX-8	2GHz	576	1 576	9.0085900	312.7670	0.0192617	160.9490	22555.750	40.9954	15.22220	0.92024	22.27
IBM p655 Power4+	1.5GHz	2048	12048	6.2054900	103.0640	0.1417230	106.3440	3651.636	1.7703	5.35073	0.08560	15.36
SGI Altix 3700 Bx2 Intel Itanium 2	1.6GHz	1008	11008	5.1283200	105.6660	0.0325982	15.6619	1907.509	1.8924	5.88404	0.20288	6.82
HP XC Intel Itanium 2	1.6GHz	1024	11024	4.8532700	154.1190	0.1615660	184.1270	2602.423	1.9555	6.24250	0.13878	5.94
Cray Inc. XT3 AMD Opteron	2.6GHz	1100	11100	4.7823400	217.9230	0.1370020	266.6600	5274.696	4.7952	4.81050	0.23638	25.94
Cray Inc. XT3 AMD Opteron	2.6GHz	1100	11100	4.7276600	253.3460	0.3035680	328.2860	5161.124	4.6919	4.77440	0.39964	7.29
IBM PS P575+ Power5+	1.9GHz	512	1 512	3.4100800	219.5560	0.0848539	52.2538	3997.745	5.8550	7.62342	0.05243	36.14
Cray Inc. X1E Cray X1 MSP	1.13GHz	252	1 252	3.1940900	65.2640	0.0140684	15.5052	2438.908	9.6823	14.10470	0.36024	14.90
IBM p655 Power4+	1.5GHz	1024	11024	3.1102600	51.4248	0.0983533	82.1055	1733.284	1.6927	3.72701	0.09024	10.13

Diagrame Kiviat



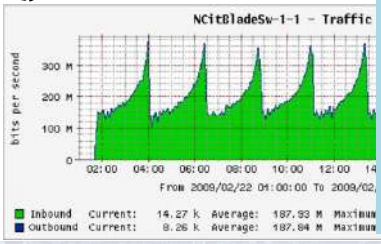
Cray XD1 AMD Opteron - 64 procs - 2.2 GHz
1 thread/MPI process (64) - RapidArray Interconnect System - 11-22-2004
Dalco Opteron/GbNet Linux Cluster AMD Opteron - 64 procs - 2.2 GHz
1 thread/MPI process (64) - GbNet11 - 11-04-2004
Sun Fire V20z Cluster AMD Opteron - 64 procs - 2.2 GHz
1 thread/MPI process (64) - Gigabit Ethernet, Cisco 6509 switch - 03-06-2005

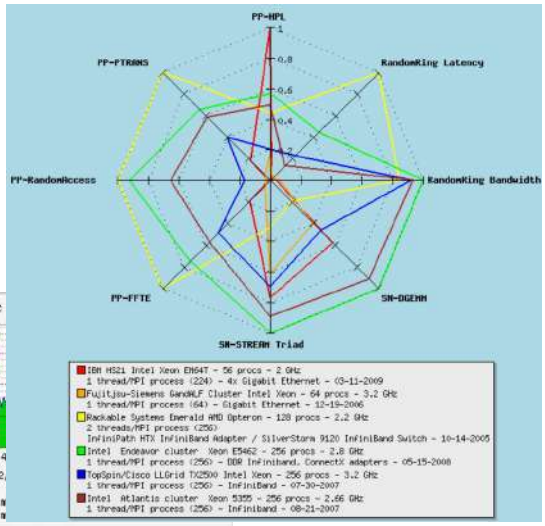
- HPL, STREAM, DGEMM: tehnologia de interconectarea nu conteaza; HPL scaleaza bine iar STREAM si DGEMM nu comunica (sunt locale)
- RandomRing Bandwidth: Cray-ul are cel mai mare bandwidth
- RandomRing Latency: Cray-ul are si cea mai mica latenta
- Random Access: Tehnologia de interconectare conteaza mult – senzitivitate la latenta
- PTRANS & FFTE: Tehnologia de interconectare conteaza mult – senzitivitate la bandwidth

HPL & HPCC @cs.pub.ro

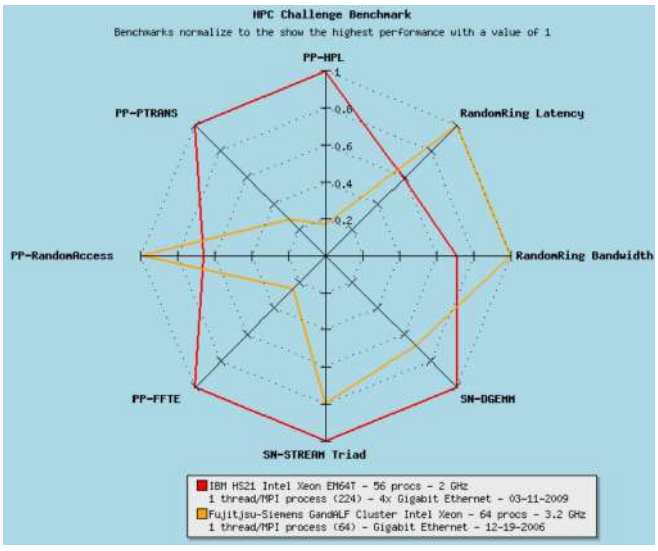
- In 2009 we broke the 1TFlops barrier with 28 of our 32 IBM HS21 blades (2xQuad-Core)
- HPL performance:
  - **1003 GFlops Max**
  - 1792 GFlops Peak
- System dimension:
  - 230.000 (Ax=b)

55.88%





Interconectare in Clustere cs.pub.ro



Dezvoltarea Viitoare a HPCC

- Proiectul doreste crearea unui cadru pentru dezvoltearea de benchmark-uri
- Se doreste dezvoltarea unor “semnaturi” ale fiecarei masini
- In viitor, colectia de benchmark-uri trebuie marita
  - De exemplu pentru operatii cu matrici rare
- Portarea HPCC pe cat mai multe sisteme
- Implementarea diversa a HPCC
  - In noi limbaje: UPC, Co-Array, Chapel, Charm++,…
  - In noi medii si paradigme: Spark, 0MQ, Flink,…

Alte Benchmark-uri Paralele

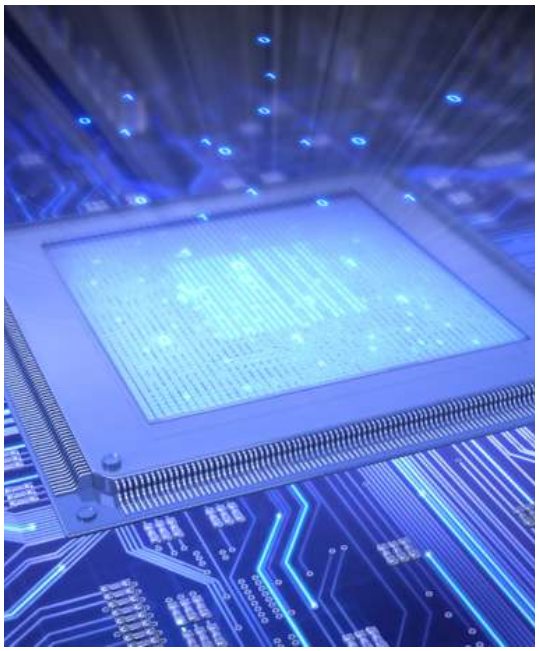
- SPLASH2 - Stanford Parallel Applications for Shared Memory
  - http://www.capsl.udel.edu/splash/
- NAS Parallel Benchmark
  - https://www.nas.nasa.gov/publications/npb.html
- SPECopenMP
  - Parallelized (OpenMP) versions of SPECfp 2000
- SPECjbb
  - Java multithreaded database-like workload

Ce am invatat in aces curs?

- Nevoia de a compara sisteme de calcul
- Cum masuram Performanta?
- Benchmark-uri pentru masini “seriale”
- Benchmark-uri pentru masini “paralele”

Slides & Support Material Acknowledgements

CIS 371 - University of Pennsylvania 2013
CS/ECE 3810 - University of Colorado 2017
University of Tennessee 2017
http://www.netlib.org 2017



Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 13
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

In cursul anterior

- Top 500 List
- Arhitecturi si Aplicatii Relevante

Cuprins

- Top 5
- Concluzii Top 500

Top 5

The HPCG Benchmark

- The High Performance Conjugate Gradients (HPCG) Benchmark
  - New metric for ranking HPC systems
  - **Complementary** to the High Performance LINPACK (HPL) benchmark
  - Designed to exercise computational and data access patterns that more closely match a different and broad set of important applications
  - Implementation written in C++ with MPI and OpenMP support
  - Offers extensive GPU support

The HPCG Benchmark

- HPCG is a stand-alone code measuring performance of basic operations:
  - Sparse matrix-vector multiplication
  - Vector updates
  - Global dot products
  - Local symmetric Gauss-Seidel smoother
  - Sparse triangular solve
  - Driven by multi-grid preconditioned conjugate gradient algorithm exercising key kernels on a nested set of coarse grids

Top 5 @HPCG

Rank	TOP500 Rank	System	Cores	Rmax (PFlop/s)	HPCG (TFlop/s)
1	4	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	16004.50
2	1	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,206.00	14054.00
3	2	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	5612.60
4	5	**LUMI** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,752,704	379.70	4586.95
5	6	**Alps** - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Swiss National Supercomputing Centre (CSCS) Switzerland	1,305,600	270.00	3671.32

MAY 2024

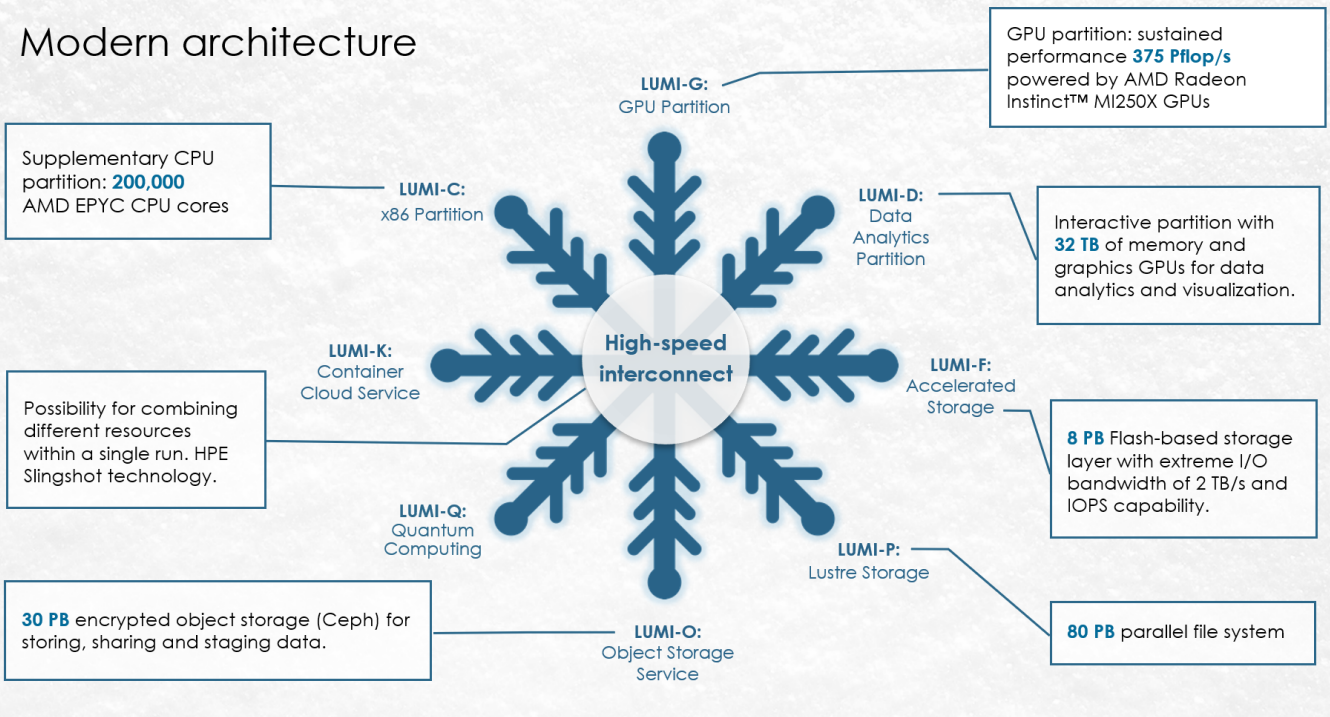
			SITE	COUNTRY	CORES	RMAX PFLOP/S	POWER MW
1	**Frontier**	HPE Cray EX235a, AMD Opt 3rd Gen EPYC (64C 2GHz), AMD Instinct MI250X, Slingshot-11	DOE/SC/ORNL	USA	8,699,904	1,206.0	22.7
2	**Aurora**	HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 (52C 2.4GHz), Intel Data Center GPU Max, Slingshot-11	DOE/SC/ANL	USA	9,264,128	1,012.0	38.7
3	**Eagle**	Microsoft NDv5, Xeon Platinum 8480C (48C 2GHz), NVIDIA H100, NVIDIA Infiniband NDR	Microsoft Azure	USA	1,123,200	561.2	
4	**Fugaku**	Fujitsu A64FX (48C, 2.2GHz), Tofu Interconnect D	RIKEN R-CCS	Japan	7,630,848	442.0	29.9
5	**LUMI**	HPE Cray EX235a, AMD Opt 3rd Gen EPYC (64C 2GHz), AMD Instinct MI250X, Slingshot-11	EuroHPC/CSC	Finland	2,220,288	379.7	6.01

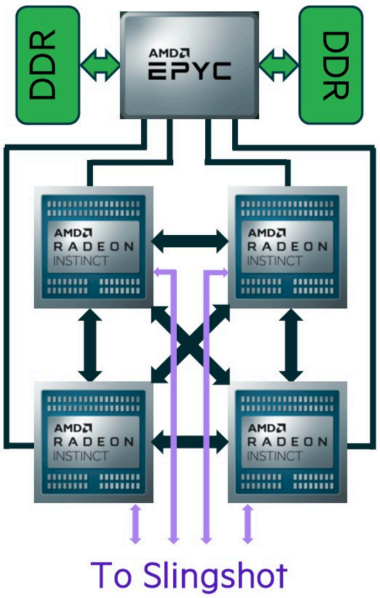
Top 5 @HPL

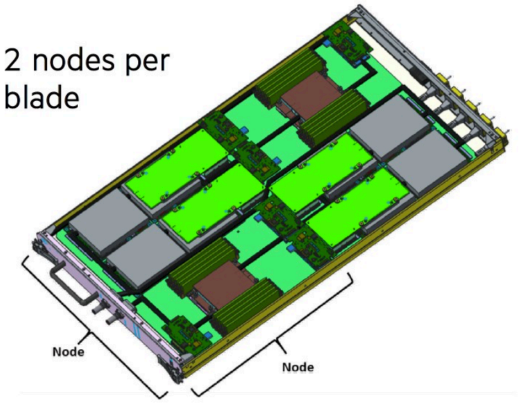
5 - Lumi - HPE Cray EX235a

- Site: EuroHPC/CSC
- Familia de sisteme: HPE Cray EX235a
- Procesor: AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X
- OS: HPE Cray OS
- Arhitectura: MPP
- Aplicatii: Cercetare
- Anul instalarii: 2023
- Numar de core-uri: 2,220,288
- Rmax(PFlops): **309**,1
- Rpeak(PFlops): **428**,7
- HPCG(PFlops): **3,**4
- Consum: **6.015** kW
- Interconectare: Slingshot-11

Modern architecture



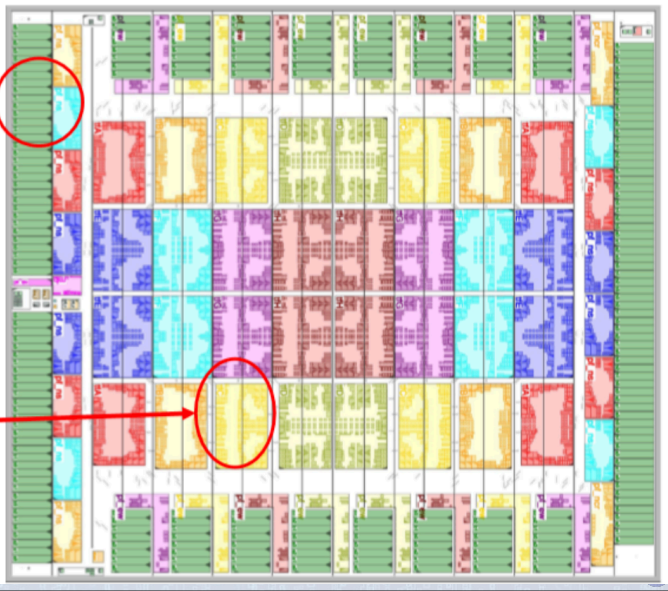




Node Architecture

Slingshot 11

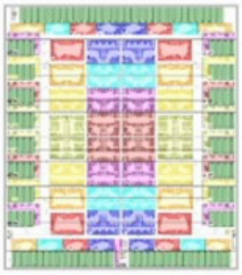
- TSMC 16nm FF
- 64 ports x 200 Gbps/dir
  - PAM4 56G
- ~250W
- Tiled architecture:
  - 32 peripheral function blocks
    - Network SerDes, MAC/PCS/LLR
    - Ethernet Lookup functions
  - 32 tile blocks
    - All other port functionality
  - Management Block (MB)
- First silicon Sept '18, Production Q4'19



Slingshot Interconnect

Rosetta Switch

- Multiple QoS levels
- Aggressive adaptive routing
- Advanced congestion control
- Very low average and tail latency
- High performance multicast and reduction



64 ports x 200 Gbps

SS-10 (100Gb)
Injection: ~14 TB/s
Bisection: ~24 TB/s

SS-11 (200Gb)
Injection: ~28 TB/s
Bisection: ~24 TB/s

Mellanox ConnectX NIC

Slingshot 10

- HPE Cray MPI stack
- Ethernet functionality
- RDMA offload

Cassini NIC

Slingshot 11

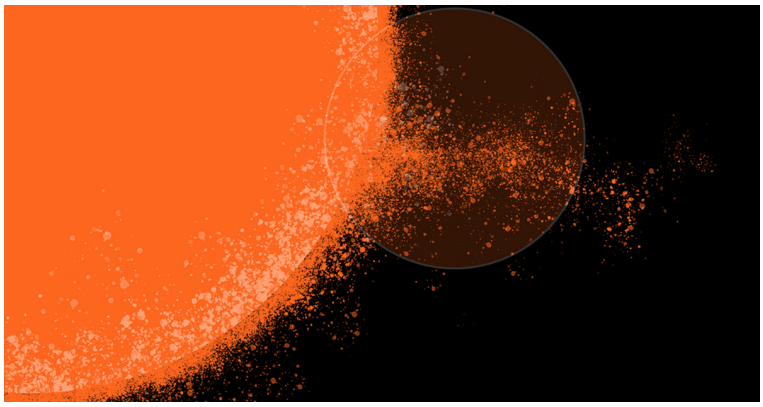
- MPI hardware tag matching
- MPI progress engine
- One-sided operations
- Collectives
- 2X injection bandwidth

Slingshot 11

- Liquid or air cooled 1U form-factor
- Up to 8 blades / chassis & up to 64 blades / cabinet
- 100-200 Gb/s in a TOR pattern
- 1.2 B packets / s (600M / direction)

Lumi R&D Highlight

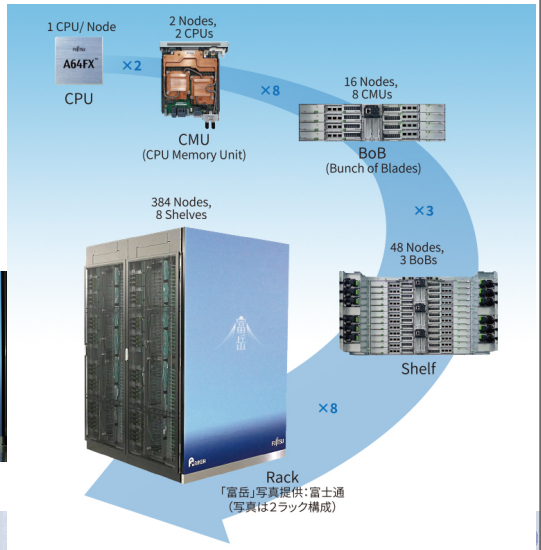
- Information from under the Sun's surface about the emergence of solar storms
  - Predict solar storms



https://www.lumi-supercomputer.eu/researchers-discovered-information-from-under-the-suns-surface-about-the-emergence-of-solar-storms-using-the-lumi-supercomputer/

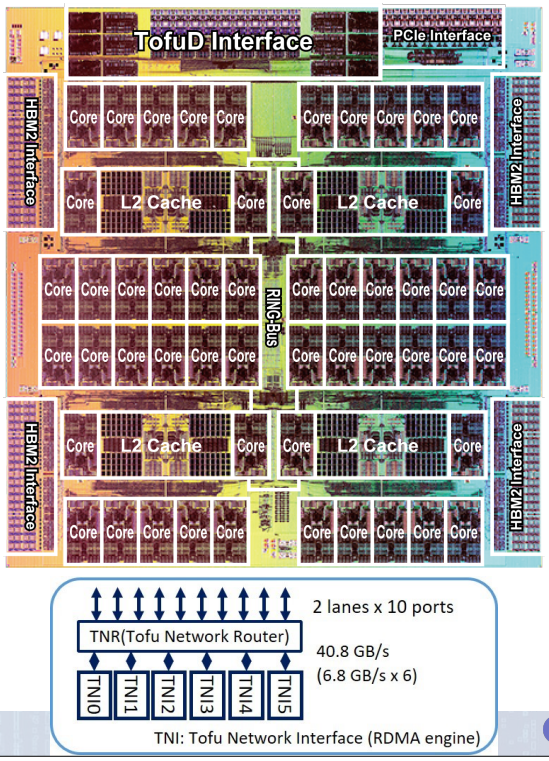
4 Fugaku K Computer Update

- Site: RIKEN Advanced Institute for Computational Science
- Familia de sisteme: Fujistu – SPARC64
- Procesor: A64FX 48C 2.2GHz
- OS: Linux
- Arhitectura: MPP
- Aplicatii: Cercetare
- Anul instalarii: 2020
- Numar de core-uri: 7.630.848
- Rmax(PFlops): **442**
- Rpeak(PFlops): **537**
- HPCG(PFlops): **16**
- Memorie: **5,087** TB
- Consum: **29.899** kW
- Interconectare: Custom – Tofu

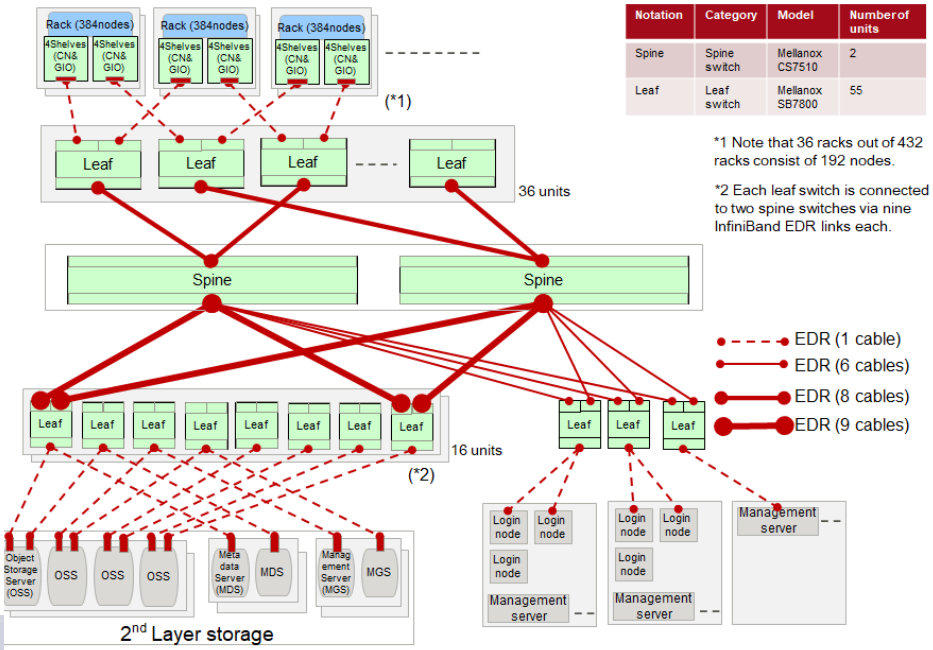


Node & Interconnect Stats

- 48+2 cores @2-2.2GHz 7nm FinFET
- Cache:
  - L1D/core: 64 KiB, 4way, 256 GB/s (load), 128 GB/s (store)
  - L2/CMG: 8 MiB, 16way
  - L2/node: 4 TB/s (load), 2 TB/s (store)
  - L2/core: 128 GB/s (load), 64 GB/s (store)
- Memory 1024 GB/s



I/O Networking



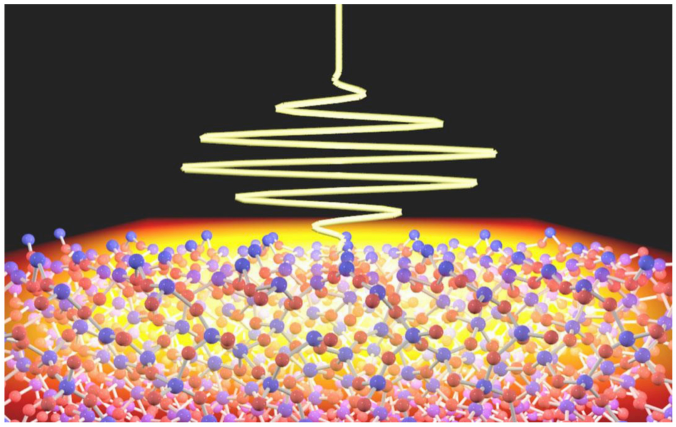
Notation	Category	Model	Number of units
Spine	Spine switch	Mellanox CS7510	2
Leaf	Leaf switch	Mellanox SB7800	55

*1 Note that 36 racks out of 432 racks consist of 192 nodes.

*2 Each leaf switch is connected to two spine switches via nine InfiniBand EDR links each.

Fugaku R&D Highlights

- COVID-19 Projects
  - Drug candidates
  - Pandemic Phenomena
- Model interactions between matter and light at the atomic scale



https://www.r-ccs.riken.jp/en/fugaku/research/

3 - Eagle -Microsoft NDv5

- Site: Microsoft Azure
- Familia de sisteme: IBM Power Systems
- Procesor: Intel Xeon Platinum 8480C 48C 2GHz, NVidia H100
- OS: Ubuntu 22
- Arhitectura: Cluster
- Aplicatii: Industrie
- Anul instalarii: 2018
- Numar de core-uri: 2,073,600
- Rmax(PFlops): **561**,2
- Rpeak(PFlops): **846**,2
- Memorie: **N/A** TB
- Consum: **N/A** kW
- Interconectare: Infiniband NDR

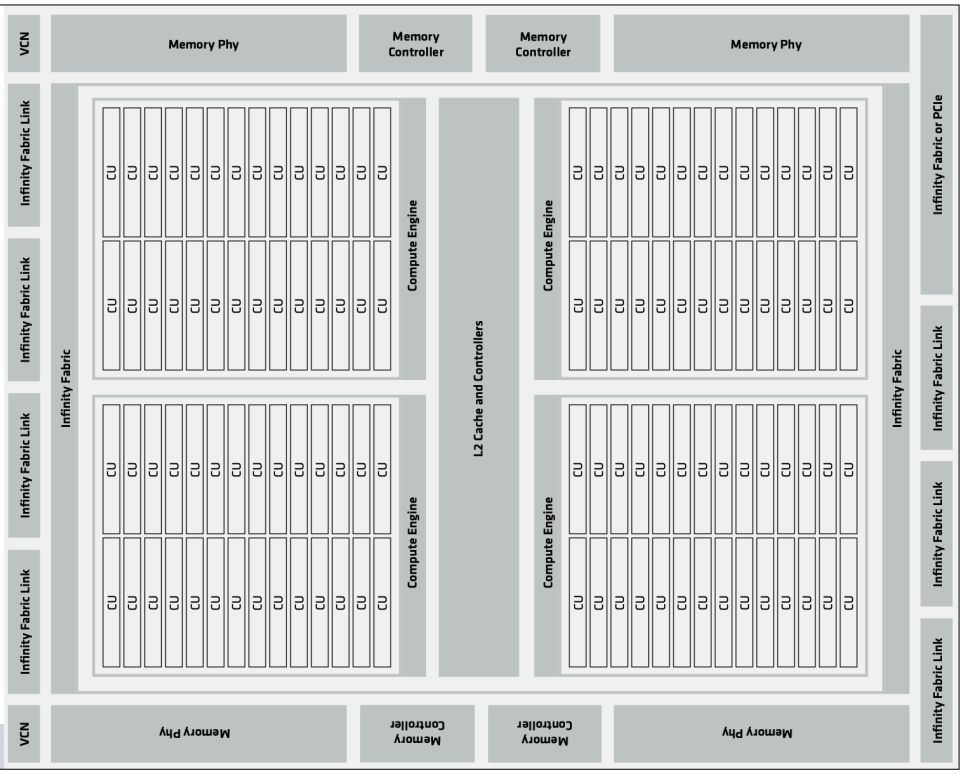
2 - Aurora - HPE Cray EX

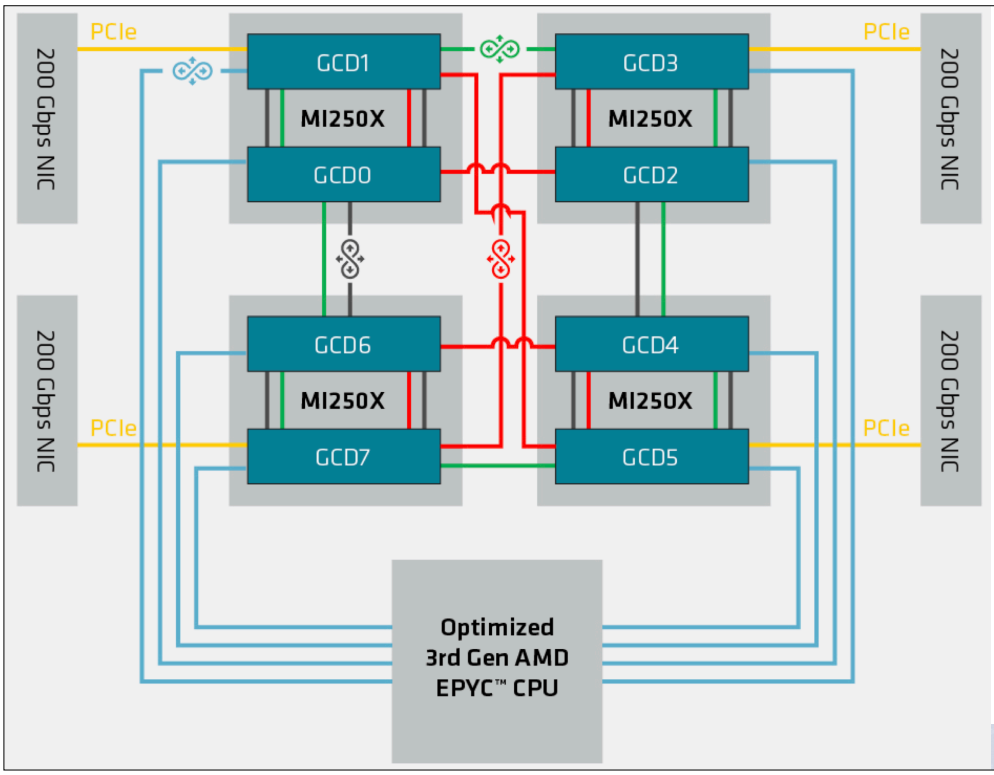
- Site:DOE/SC/Argonne National Laboratory
- Familia de sisteme: HPE Cray EX
- Procesor: Xeon CPU Max 9470 52C 2.4GHz, NVIDIA A100 SXM4 64GB
- OS: Suse Linux
- Arhitectura: MPP
- Aplicatii: Cercetare
- Anul instalarii: 2023/2024
- Numar de core-uri: 9,264,128
- Rmax(**EFlops**): **1**,02
- Rpeak(**EFlops**): **1**,98
- HPCG(**PFlops**): **5,**612
- Consum: **38.7 MW**
- Interconectare: Slingshot-11

1 - Frontier - HPE Cray EX235a

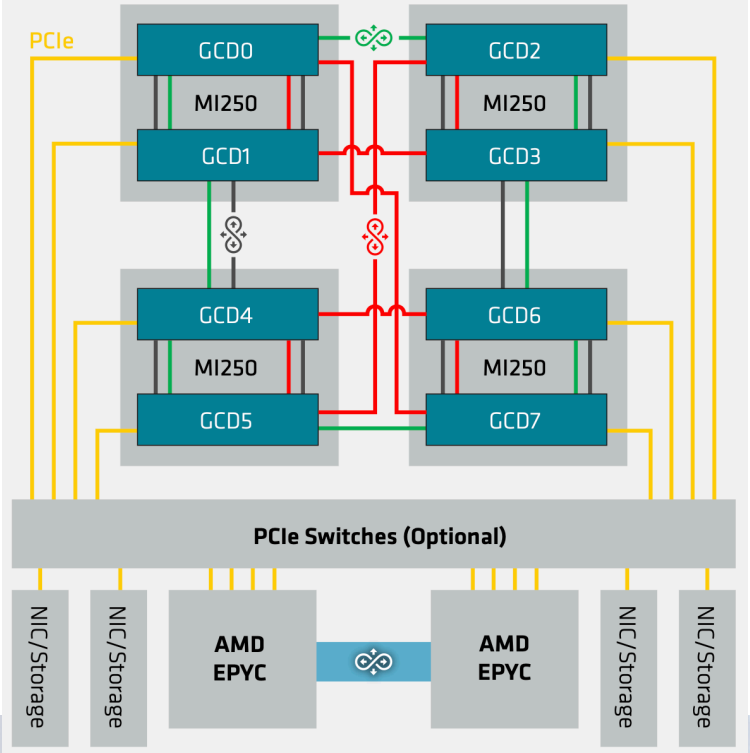
- Site: DOE/SC/Oak Ridge National Laboratory
- Familia de sisteme: HPE Cray EX235a
- Procesor: AMD Optimized 3rd Gen EPYC 64C 2GHz, AMD Instinct MI250X
- OS: HPE Cray OS
- Arhitectura: MPP
- Aplicatii: Cercetare
- Anul instalarii: 2021
- Numar de core-uri: 8,699,904
- Rmax(**EFlops**): **1**,194
- Rpeak(**EFlops**): **1**,679
- HPCG(PFlops): **14,**05
- Memorie: **3,1** PB
- Consum: **22.703** kW
- Interconectare: Slingshot-11

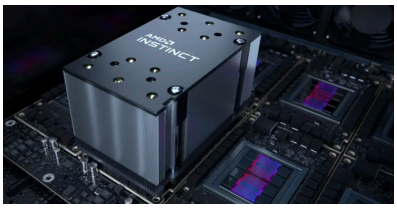
AMD Instinct MI250X





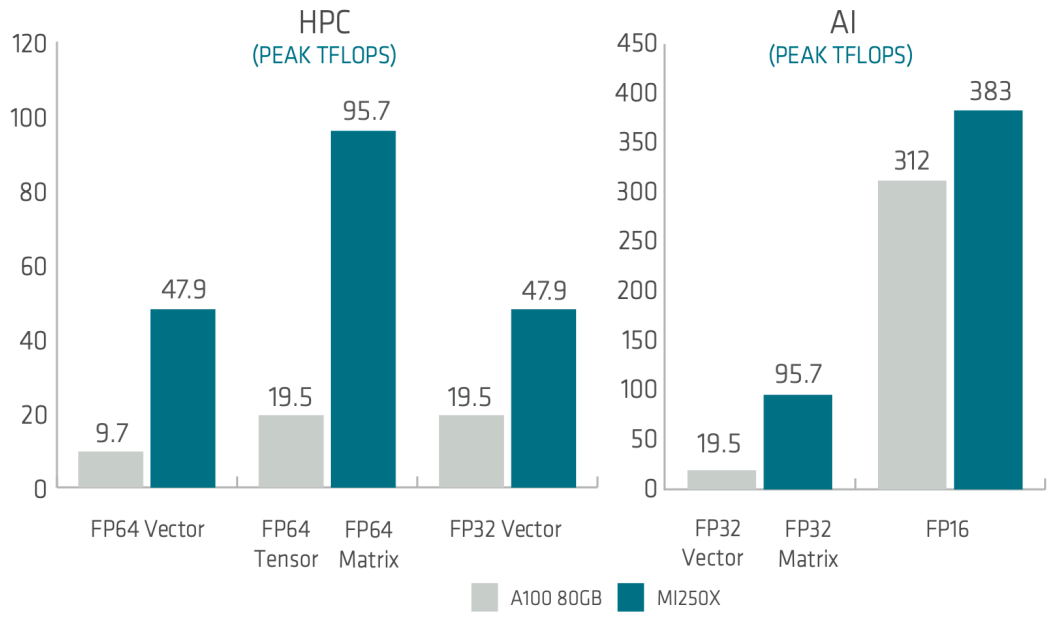
AMD Instinct MI250X HPC Node





AMD Instinct MI250X HPC/ML Node

Performance	MI210	MI250	MI250X
CDNA2 Graphics Compute Die (GCD)	1	2	2
Compute Units	104 CU	208CU	220CU
Stream processors	6,656	13,312	14,080
Matrix Cores	416	832	880
Peak FP64/FP32 Vector	22.6 TF	45.3 TF	47.9 TF
Peak FP64/FP32 Matrix	45.3 TF	90.5 TF	95.7 TF
Peak FP16/BF16	181.0 TF	362.1 TF	383.0 TF
Peak INT4/INT8	181.0 TOPS	362.1 TOPS	383.0 TOPS
Memory			
Memory Size	64GB HBM2e	128GB HBM2e	128GB HBM2e
Memory Interface	4,096 bits	8,192 bits	8,192 bits
Memory Clock	1.6GHz	1.6GHz	1.6GHz
Memory Bandwidth	up to 1.6 TB/sec	up to 3.2TB/sec2	up to 3.2TB/sec2
Scalability			
Infinity Fabric Links	up to 3	up to 6	up to 8
xGMI Bridge Card Configuration	Yes (Dual | Quad Hives)	NA	NA
Coherency Enabled	No	No	Yes



Graph 1: Peak TFLOPS across range of mixed-precision Compute[1]

Original

```
float vxi = 0.0f, vyi = 0.0f, vzi = 0.0f;
for (int j = threadIdx.x; j < count1; j += blockDim.x) {
  float dx = xx1[j] - xxi;
  float dy = yy1[j] - yyi;
  float dz = zz1[j] - zzi;
  float dist2 = dx*dx + dy*dy + dz*dz;
  if (dist2 < fsrrmax2) {
    float rtemp = (dist2 + rsm2)*(dist2 + rsm2)*(dist2 + rsm2);
    float f_over_r = massi*mass1[j]*(1.0f/sqrt(rtemp) -
        (ma0 + dist2*(ma1 + dist2*(ma2 + dist2*(ma3 +
        dist2*(ma4 + dist2*ma5))))));
    vxi += fcoeff*f_over_r*dx;
    vyi += fcoeff*f_over_r*dy;
    vzi += fcoeff*f_over_r*dz;
  }
}
```

Modified to use Packed FMA32

```
float vxi = 0.0f, vyi = 0.0f, vzi = 0.0f;
  for (int j = threadIdx.x; j < count1; j += 2*blockDim.x) {
    float2 dx = {xx1[j] - xxi, xx1[j + blockDim.x] - xxi};
    float2 dy = {yy1[j] - yyi, yy1[j + blockDim.x] - yyi};
    float2 dz = {zz1[j] - zzi, zz1[j + blockDim.x] - zzi};
    float2 dist2 = dx*dx + dy*dy + dz*dz;
    bool check[2] = {dist2.x < fsrrmax2, dist2.y < fsrrmax2};
    if (check[0] || check[1]) {
      float2 rtemp = (dist2 + rsm2)*(dist2 + rsm2)*(dist2 + rsm2);
      float2 mass1_2 = {mass1[j],mass1[j + blockDim.x]};
      float2 sqrt_rtemp = {sqrtf(rtemp.x),sqrtf(rtemp.y)};
      float2 f_over_r = massi*mass1_2*(1.0f/sqrt_rtemp -
          (ma0 + dist2*(ma1 + dist2*(ma2 + dist2*(ma3 +
          dist2*(ma4 + dist2*ma5))))));
      float2 vxi_tmp = fcoeff*f_over_r*dx;
      float2 vyi_tmp = fcoeff*f_over_r*dy;
      float2 vzi_tmp = fcoeff*f_over_r*dz;
      vxi += check[0] ? vxi_tmp.x : 0.0f;
      vxi += check[1] ? vxi_tmp.y : 0.0f;
      vyi += check[0] ? vyi_tmp.x : 0.0f;
      vyi += check[1] ? vyi_tmp.y : 0.0f;
      vzi += check[0] ? vzi_tmp.x : 0.0f;
      vzi += check[1] ? vzi_tmp.y : 0.0f;
    }
  }
```

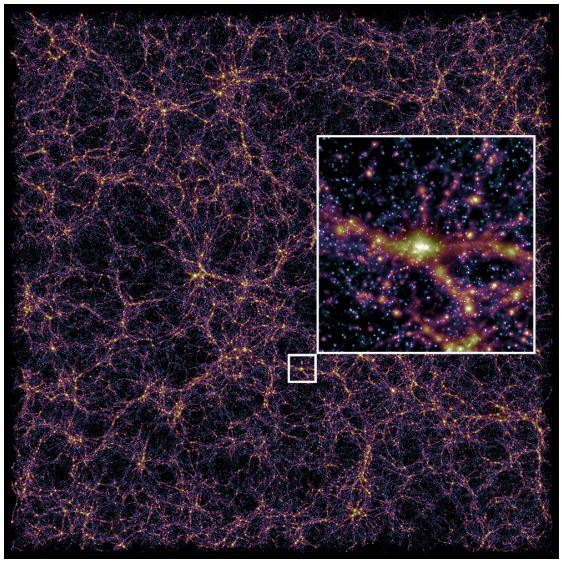
AMD ROCm™ Platform

Deployment Tools	ROCm Validation Suite	ROCm Data Center Tool	ROCm SMI			
Libraries	rocBLAS	rocRAND	rocFFT	MIGraphX	MIVisionX	rocPRIM
	rocSOLVER	rocALUTION	rocSPARSE	rocTHRUST	MIOpen	RCCL
Compilers & Tools	Compiler	Profiler	Debugger	hipify	TENSILE	
Programming Models	OpenMP® API	HIP API	OpenCL™			
Drivers/Runtimes	RedHat®, SLES & Ubuntu® Device Drivers and Run-Time					

Frontier R&D Highlight

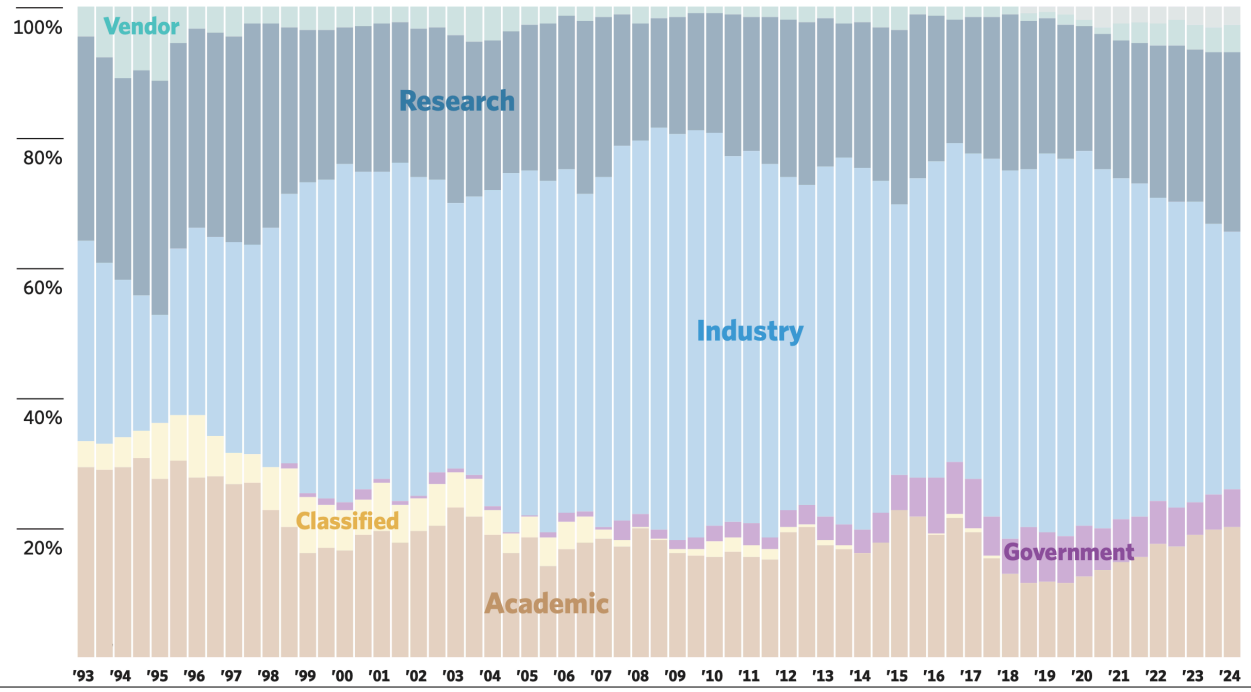
- More detailed Universe
- HACC (Hardware/Hybrid Accelerated Cosmology Code)
- Port from NVidia GP-GPUs to AMD Instinct

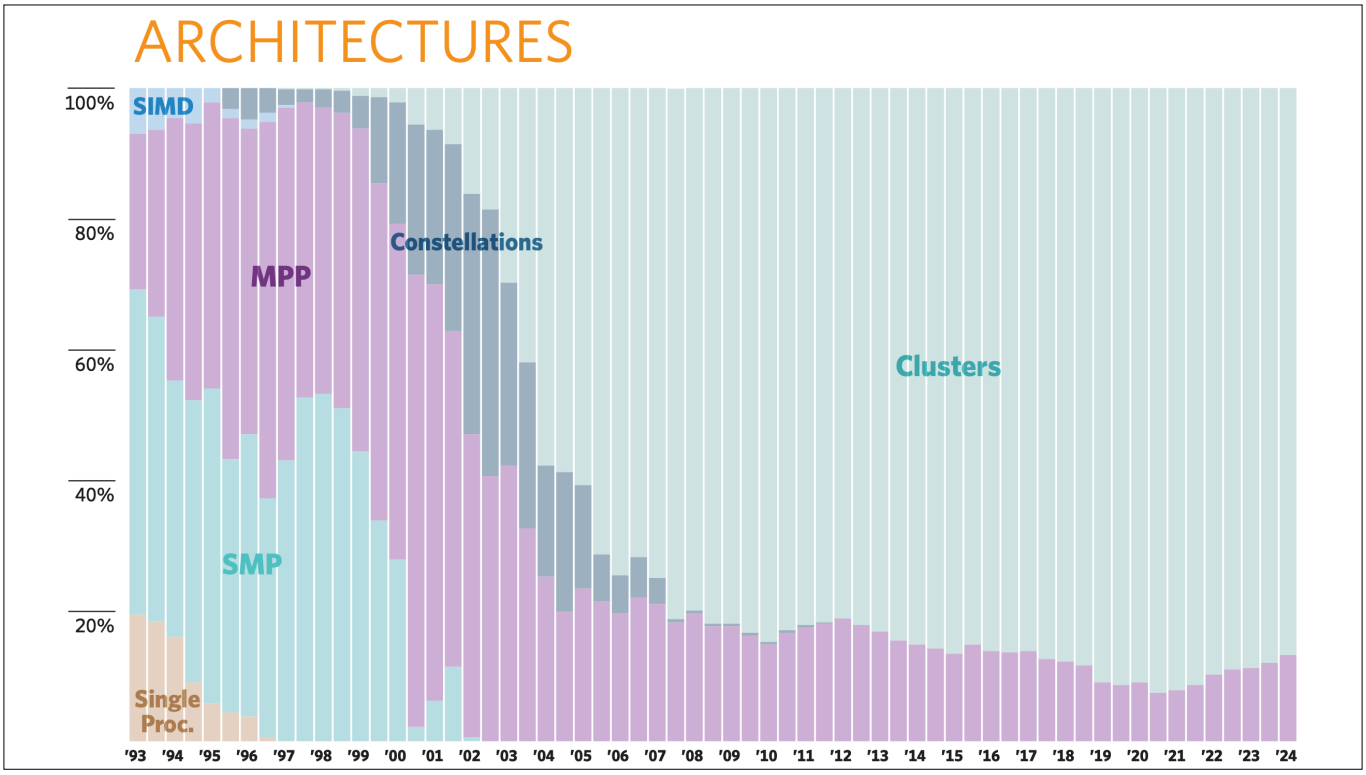
https://www.olcf.ornl.gov/2023/04/26/simulating-a-more-detailed-universe-with-frontier/

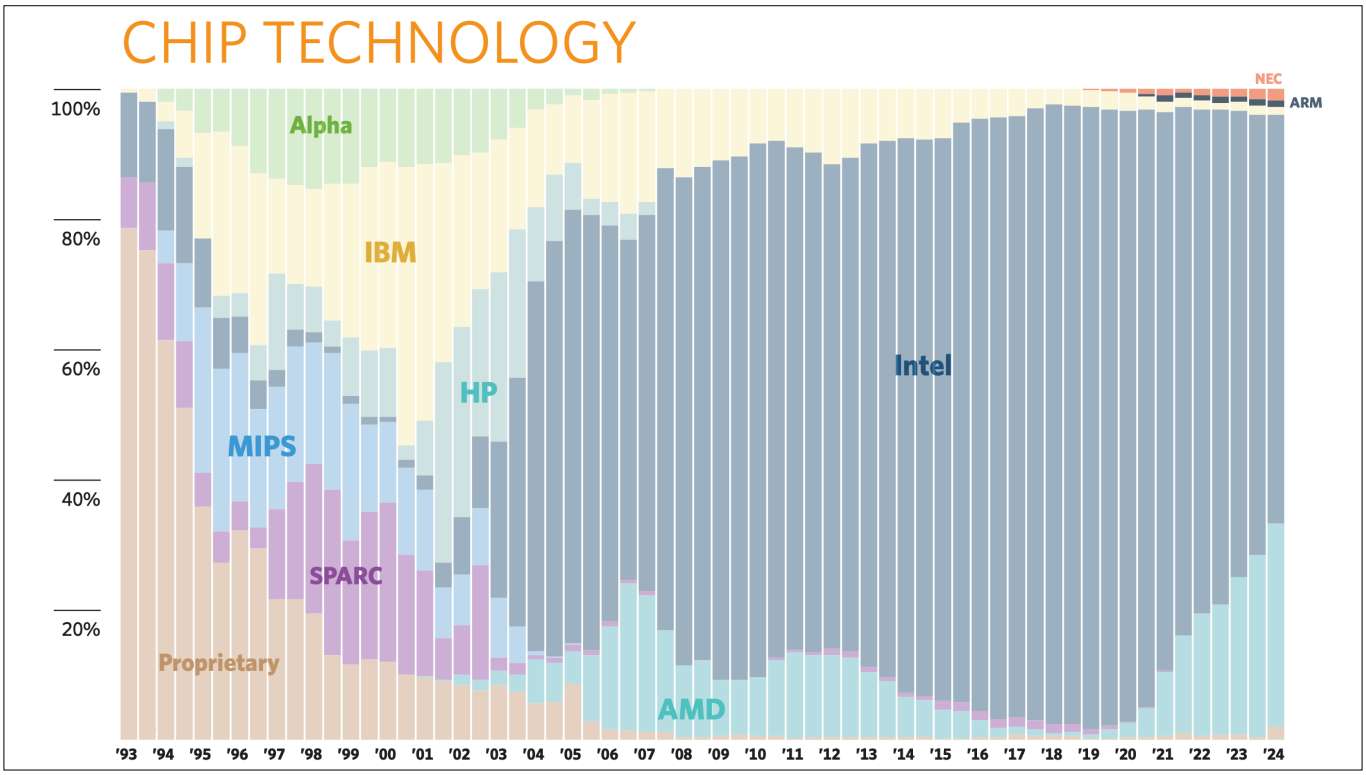


Concluzii Top 500

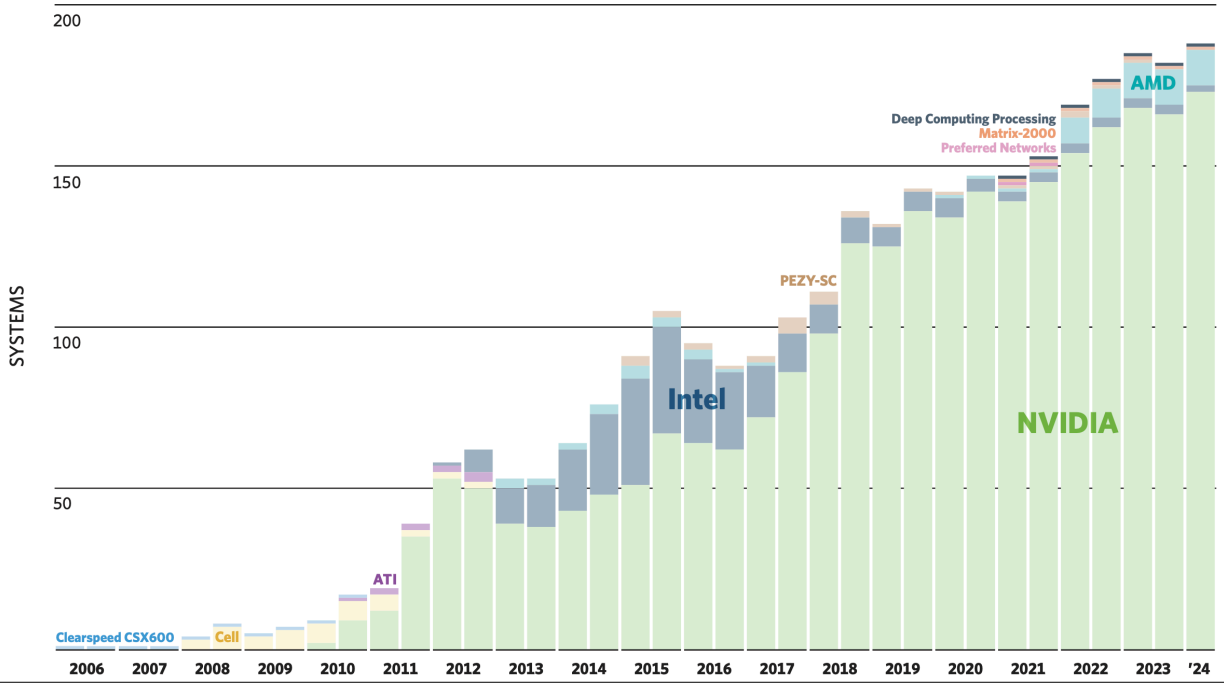
INSTALLATION TYPE




ARCHITECTURES
100%
80%
60%
40%
20%
SIMD
MPP
Constellations
Clusters
SMP
Single Proc.
'93 '94 '95 '96 '97 '98 '99 '00 '01 '02 '03 '04 '05 '06 '07 '08 '09 '10 '11 '12 '13 '14 '15 '16 '17 '18 '19 '20 '21 '22 '23 '24


CHIP TECHNOLOGY
100%
80%
60%
40%
20%
NEC
ARM
Alpha
IBM
HP
Intel
MIPS
SPARC
Proprietary
AMD
'93 '94 '95 '96 '97 '98 '99 '00 '01 '02 '03 '04 '05 '06 '07 '08 '09 '10 '11 '12 '13 '14 '15 '16 '17 '18 '19 '20 '21 '22 '23 '24

ACCELERATORS/CO-PROCESSORS



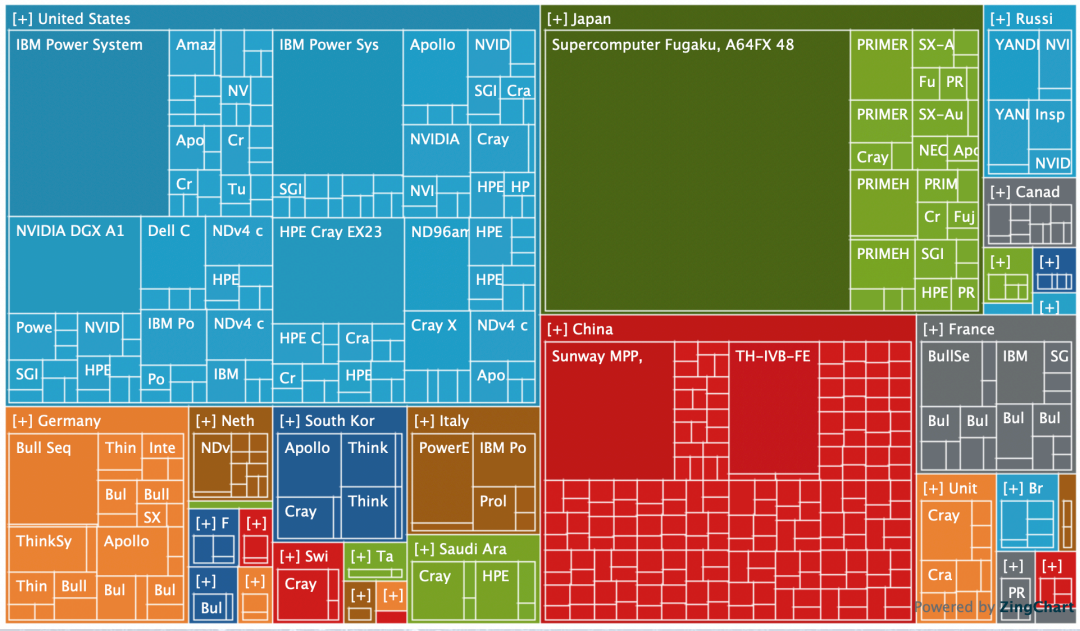
Vendors

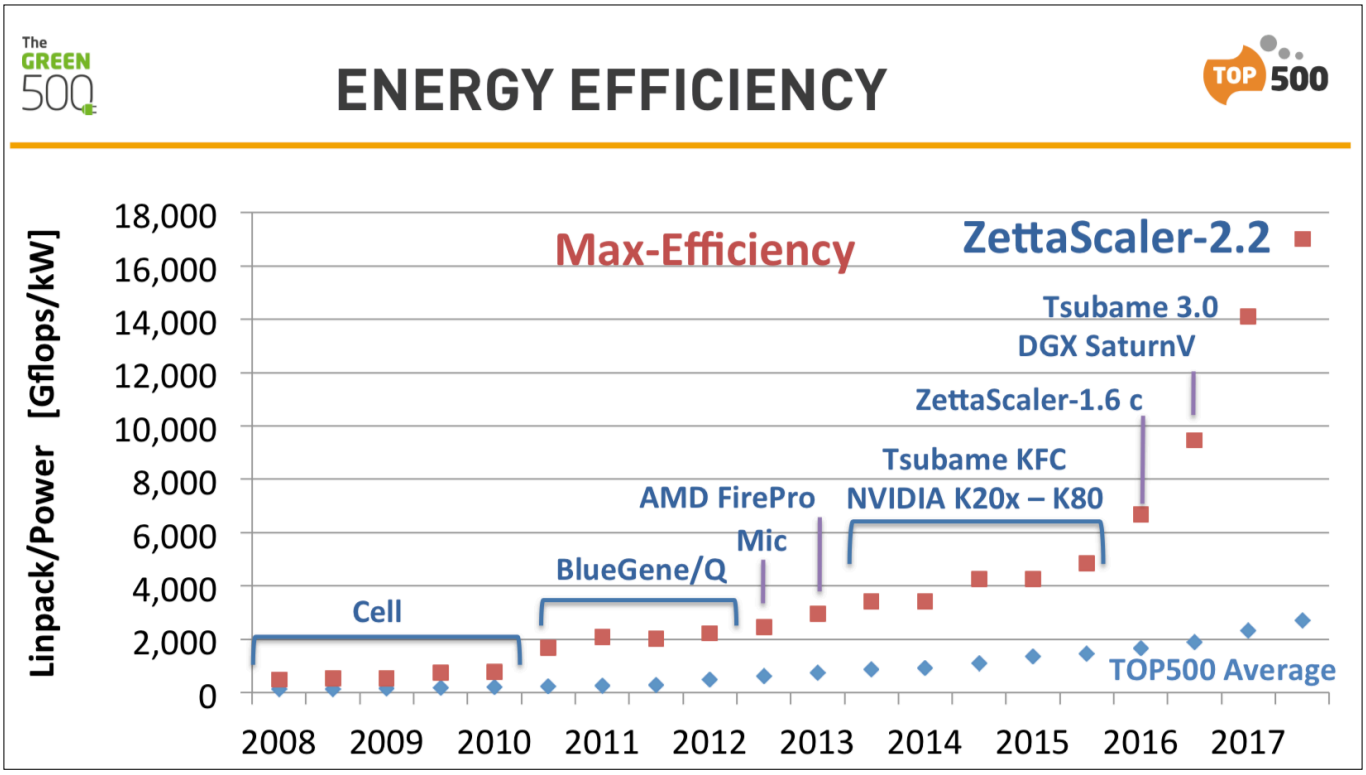
		Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
1	Lenovo	173	34.6	330,547	610,259	12,339,312
2	Sugon	71	14.2	119,286	301,610	5,182,664
3	Inspur	66	13.2	102,121	219,012	2,666,744
4	HPE	35	7	122,353	179,809	3,174,784
5	Cray/HPE	35	7	162,476	249,470	6,029,800
6	Atos	23	4.6	64,828	103,337	2,117,592
7	Fujitsu	13	2.6	60,538	96,398	1,589,640
8	IBM	13	2.6	216,919	287,902	6,005,272
9	Penguin Computing	11	2.2	20,034	25,248	614,136
10	Dell EMC	11	2.2	52,309	89,404	1,276,988

Interconnect

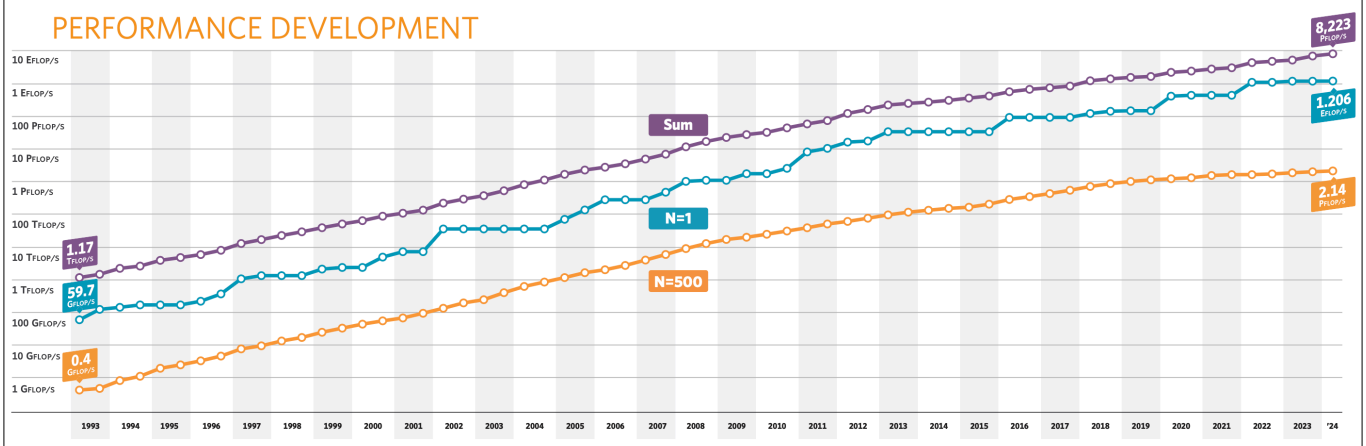
		Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
1	Gigabit Ethernet	259	51.8	430,575	897,359	16,913,772
2	Infiniband	140	28	662,896	1,013,372	15,946,634
3	Omnipath	50	10	181,500	285,493	4,704,264
4	Custom Interconnect	45	9	356,933	524,811	24,991,220
5	Proprietary Network	5	1	13,009	16,406	508,496
6	Myrinet	1	0.2	1,975	6,595	89,600

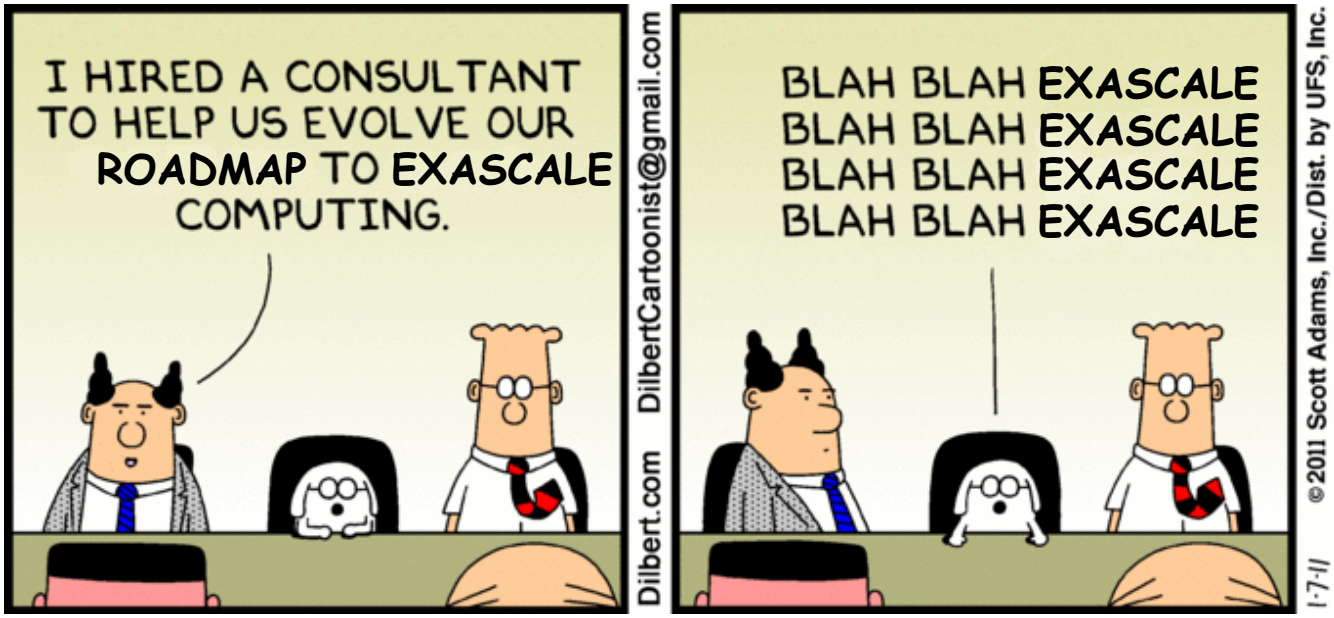
Instalari @Tari





Performanta in timp…



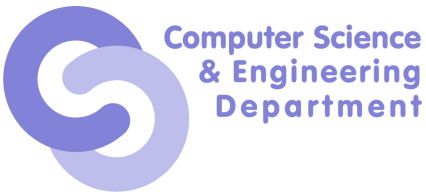


Feedback @ASC

Ce am invatat in aces curs?

- Top 5
- Concluzii Top 500

Thank you for your attention.



emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 14
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

Cuprins

- Scurta recapitulare a cursului
- Examen si Feedback

Scurta recapitulare a cursului

Un Scurt Sumar al Cursului

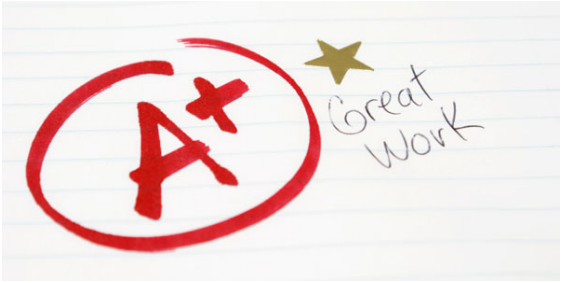
- Intro & Recapitulare
- Nivelul structural de descriere a sitemelor numerice - PMS
- Ierarhia de memorii, localitatea datelor, cache
- Comutatoare ierarhice & neierarhice
- Structuri de calcul cu prelucrare paralela – taxonomii
- Structuri SIMD
- Intercalarea perfecta si conectarea inversa
- Arhitecturi omogene/eterogene: CPU / GP-GPU

Un Scurt Sumar al Cursului (2)

- Arhitecturi MIMD – Caracteristici
- Arhitectura Cm* - comunicare intra- si inter-cluster
- Interconectarea in sisteme cu resurse multiple
- Retele de comutare (Ierarhice & Delta)
- Debugging, profiling, analiza de performanta
- Analiza de corectitudine a codurilor paralele
- Top 500, Benchmarking & the current Top 5

Notarea la ASC

- Laborator: 5/5.5
  - Trei teme: 4.5:
    - Python: 1.5
    - GP-GPU: 1.5
    - Optimizare C: 1.5
  - Activitate la laborator: 1
  - Bonusuri:
    - Max 20% @teme
    - Participări concursuri
- Examen: 5 (teorie)
  - Min > 2.5 lab
  - Oral || Scris



Examen la ASC

- Examenul consta din doua parti distincte:
  - Teorie – 50min:
    - Subiecte din temele prezentate la curs / Poate fi sustinut oral
- Nota finala == Teorie*0.50 + Laborator*0.50
  - Punctajul fiecarei parti este in intervalul 0..10
- Intrarea in examen si promovarea nu poate avea loc decat daca nota finala de la laborator este strict mai mare ca 2.5
- Examen:
  - CA/CC – 11.06 / CB-CD 10.06 / CA/CC oral – 12.06

Bibliografie

- Structura si Arhitectura Sistemelor Numerice; T. Moisa, N. Tapus 1999
- Introduction to Parallel Computing: Design & Analysis of Algorithms; V. Kumar, A. Grama, A. Gupta, G. Karypis; Addison Wesley; 2nd Edition 2003
- The Sourcebook of Parallel Computing; J. Dongarra, I. Foster, W. Grapp, K. Kennedy; Morgan Kaufmann 2002
- Computer Organization & Design – The Hardware Software Interface; D.A. Patterson, J. Hennesy; Morgan Kaufmann; 5th Edition 2014
- Computer Architecture: A Quantitative Approach; J. Hennesy, D.A. Patterson; Morgan Kaufmann; 4th Edition 2007
- Introduction to Parallel Processing; Algorithms and Architectures; Behrooz Parhami; Kluwer 2002
- Techniques for Optimizing Applications: High Performance Computing; Rajat P. Garg, Ilya Sharapov; Sun 2001.
- Practical Computing on the Cell Broadband Engine; Sandeep Koranne; Springer 2009.
- Inside the Machine; Jon Stokes; No Starch Press 2006.
- www.top500.org

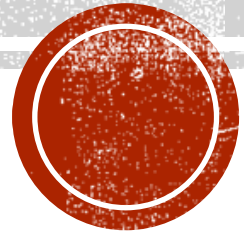
What next?

- Q&A?
- Next time:
  - Examen
  - Last but not least: Feedback

THE END

EXAMEN ASC

Întrebări & răspunsuri



Descrieți folosind primitivele PMS componenta Unitatea de Comandă cu minim 5 primitive.

Unitatea de Comanda – K

- Rolul K:
  - Componenta care exprima controlul in diverse subansamble ale sistemului
  - Are o functie de comanda a resurselor asociate unitatilor functionale
    - Nu citeste & interpreteaza instructiuni aritmetice logice, ci doar **comanda** & **control**
- Atribute:
  - Functia: comanda resursa I
  - Implementare: conventionala | μprogramata
  - Numar de stari: n
    - Daca are 2 stari este un bistabil
    - Daca are 64 de stari este un automat complex

15

Unitatea de Comanda – K

- Citireste, interpreteaza si executa instructiuni masina de **comanda si control**
- Legata de:
  - Setul de instructiuni
    - Intructiuni Universale; Ciclu variabil
    - RISC cu ciclu fix
    - Complex CISC – emularea unei masini intermediare
- Codificarea instructiunilor
  - 0 adrese

  - 1 adresa

  - 2 adrese

  - 3 adrese

- Moduri de adresare multiple la ora actuala

16

Descrieți comutatorul ierarhic de tip trunchi K.

Comutatoare Ierarhice Trunchi K

- Imbina conectivitatea oferita de S legatura multipla cu costul redus al celorlalte tipuri de comutatoare
  - S {TK, ma, nb, c = k, k*(m+n)Sp}
- Orice a poate comunica cu orice b
- Nu totdeauna vei avea min(m, n) comunicatii simultane
  - Asa a aparut idea de a folosi k linii pentru comunicatie
- In functie de m si n si de trafic, se va determina un k optim
  - 8 procs si 4 memorii:
    - Pot fi maxim 4 accese la memorii → k=4
    - Nu are sens mai mult → k < n si k < m

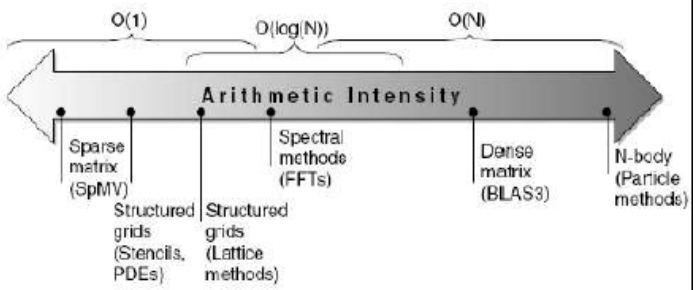
Ce reprezintă intensitatea computațională/aritmetică și cum se leagă ea de modelul Roofline?

<u>GPT4o POV:</u>

Intensitatea computațională (sau aritmetică) reprezintă raportul dintre numărul de operații în virgulă mobilă (floating-point operations) și numărul de bytes citiți din memorie. Mai exact, intensitatea aritmetică măsoară câte operații de calcul se fac pentru fiecare byte de date transferat din memorie.

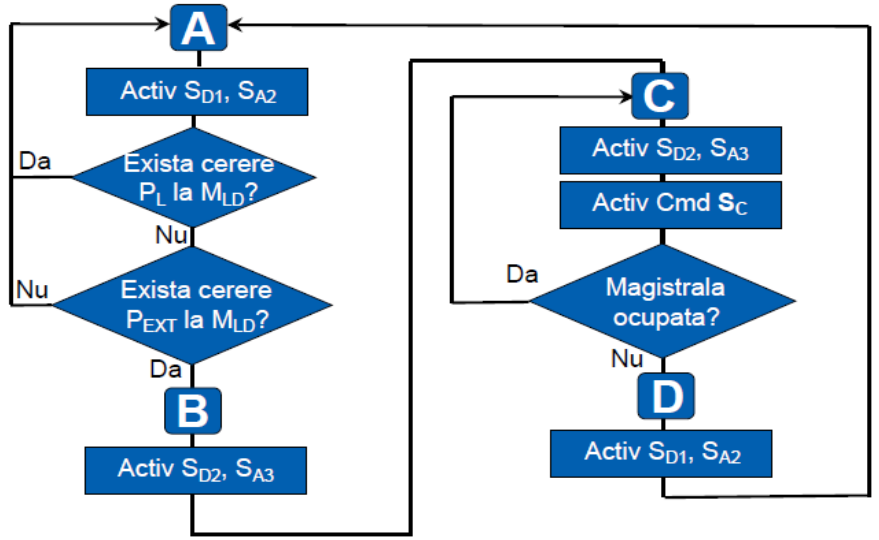
The Roofline Performance Model

- Basic Idea
  - Plot peak floating-point throughput as a function of arithmetic intensity
  - Ties together floating-point performance and memory performance for a target machine
- Arithmetic Intensity
  - Floating-point operations per byte read



Realizați schema logică și descrieți funcționarea unității K_{ML} din dispozitivele SBC.

K_{ML}



Descriere K_{ML}

- K_{ML} – asigura accesul la Mem locala a PL prin intermediul S_{D1} si S_{A2} & a proc externe prin S_{D2} si S_{A3}
- PL are prioritate fata de proc externe (P_{EXT}) == daca vrea P_L la M_{LD} NU ma intereseaza ce vor celelalte P_{EXT}
- Starea A: P_L poate avea comenzi locale
- Starea B: activez accesul ext la M_{LD} – semnalele asociate starii
- Obs: Mem locala e vazuta de fiecare procesor acelasi spatiu de memorie, iar de procesoarele ext in spatiul complementar → trebuie sa existe o unitate de translatare a adreselor
- Mag ocupata: nu stii ciclii masina ai unui proc extern. Numai cand magistrala e libera un P_{EXT} a terminat ciclul masina curent → abordare top-down

Descrieți o rețea de comutare de tip Delta cu 25 de intrări și 16 ieșiri Și explicați modul de rutare corespunzător ieșirii 9.

Calculul nivelelor

Pentru o rețea Delta cu 25 de intrări și 16 ieșiri, putem folosi comutatoare 5×4, deoarece:

- $25 = 5^2$
- $16 = 4^2$

Acest lucru înseamnă că vom avea 2 nivele de comutatoare 5×4.

Structura rețelei Delta

- **Nivelul 0:** conține 5×4 comutatoare pentru a conecta 25 de intrări la 20 de ieșiri intermediare.
- **Nivelul 1:** conține 5×4 comutatoare pentru a conecta 20 de ieșiri intermediare la 16 ieșiri finale.

Mod de rutare pentru ieșirea 9

Pentru a explica modul de rutare corespunzător ieșirii 9, trebuie să urmărim pașii prin rețea. Adresa ieșirii 9 în baza 4 este 21_4.

1. **Nivelul 0:** primul digit 2 indică faptul că pachetul va trece prin al treilea comutator 5×4 de la nivelul 0.
2. **Nivelul 1:** al doilea digit 1 indică faptul că pachetul va trece prin al doilea comutator 5×4 de la nivelul 1.

Sisteme cu memorie partajată:
CC-NUMA și COMA.
Care sunt asemănările și care sunt deosebirile între aceste tipuri de sisteme. Oferiți exemple de astfel de sisteme.

Asemănări între CC-NUMA și COMA

1. **Memorie Partajată:** Ambele sisteme sunt tipuri de arhitecturi cu memorie partajată, unde multiple procesoare au acces la o memorie comună.
2. **Cache Coherency:** Ambele sisteme se bazează pe mecanisme de coerență a cache-ului pentru a asigura că toate procesoarele au o vedere consistentă a memoriei partajate.
3. **Scalabilitate:** Atât CC-NUMA, cât și COMA sunt proiectate pentru a fi scalabile, permițând adăugarea de mai multe procesoare și noduri pentru a crește performanța.

Deosebiri între CC-NUMA și COMA

1. **Accesul la Memorie:**
   - **CC-NUMA (Cache Coherent Non-Uniform Memory Access):**
     - Fiecare procesor are acces direct la propria sa memorie locală și acces non-uniform la memoria celorlalte procesoare. Accesul la memoria locală este mai rapid decât accesul la memoria la distanță.
     - Exemple: Silicon Graphics Origin 2000, Sequent NUMA-Q.
   - **COMA (Cache Only Memory Architecture):**
     - Procesoarele nu au acces direct la propria zonă de date sau la datele altor procesoare. Accesul la date se face exclusiv prin intermediul cache-ului.
     - Exemple: Nu sunt specificate în detaliu în imagine, dar se poate presupune că sunt implementate în sisteme experimentale sau specifice.
2. **Arhitectura și Structura:**
   - **CC-NUMA:**
     - Structura nodurilor include procesoare, memorie locală și interconectări pentru accesul la memoriile altor noduri. Nodurile sunt conectate printr-o structură inelară sau bus intern, iar hub-urile de conectare asigură coerența cache-ului.
   - **COMA:**
     - Memoria este organizată doar în cache-uri, iar datele sunt migrate către cache-ul procesorului care are nevoie de ele. Nu există memorie principală accesibilă direct de procesoare, doar cache-uri.
     - Structura tipică implică o rețea de comutație care conectează cache-urile și procesoarele.
3. **Exemple de Implementări:**
   - **CC-NUMA:**
     - Silicon Graphics Origin 2000: 2 procesoare pe nod, acces divizat la componentele nodului printr-un bus intern.
     - Sequent NUMA-Q: Noduri interconectate cu structura inelară, utilizând standardul IEEE SCI (Scalable Coherent Interface).
     - Stanford DASH: Directory Architecture for Shared memory, cu 4 MIPS R3000 Procs.
   - **COMA:**
     - Specificitățile legate de sistemele COMA nu sunt detaliate, dar sunt cunoscute din literatura de specialitate și implementări teoretice.

Ce speedup maxim poate fi atins, cu un număr infinit de unități de procesare, dacă 70% din cod este paralelizabil?

Legea lui Amdahl

Legea lui Amdahl stipulează că potențialul maxim de accelerare al unui program folosind mai multe procesoare este limitat de partea secvențială a programului. Formula de bază a legii este:

$$\text{Speedup} = \frac{1}{(1-P)+\frac{P}{N}}$$

unde:

- P este fracțiunea de cod paralelizabilă.
- N este numărul de procesoare.

În cazul nostru:

- $P = 0.7$ (70% din cod este paralelizabil).
- $N = \infty$ (număr infinit de procesoare).

Calculul Speedup-ului Maxim

Cu un număr infinit de procesoare, fracțiunea de timp necesară pentru partea paralelizabilă a codului tinde spre zero. Formula devine:

$$\text{Speedup}_{\infty} = \frac{1}{(1-P)}$$

Substituind valorile noastre:

$$\text{Speedup}_{\infty} = \frac{1}{(1-0.7)}$$
$$\text{Speedup}_{\infty} = \frac{1}{0.3}$$
$$\text{Speedup}_{\infty} = 3.33$$

Concluzie

Speedup-ul maxim care poate fi atins cu un număr infinit de unități de procesare, dacă 70% din cod este paralelizabil, este de **3.33**. Aceasta înseamnă că, indiferent de câte unități de procesare sunt adăugate, programul nu va rula mai rapid decât de 3.33 ori față de execuția sa secvențială.

Prezentați pe scurt benchmarkul LINPACK. Prezentați avantajele și dezavantajele acestuia.

The LINPACK Benchmark

- Linpack folosit pentru sisteme paralele pentru ca
  - Este extrem de raspandit si utilizat
  - Date si teste pentru acest program sunt disponibile pentru **toate** sitemele relevante din lume
- Benchmark-ul masoara cat de repede rezolva computer-ul un sistem dens de ecuatii liniare $A \cdot x = b$ de dimensiune $n \cdot n$
- Solutia se bazeaza pe metoda eliminarii Gaussiene, utilizand pivotarea partiala

Avantaje	Dezavantaje
Oferă ca rezultat un singur număr (Rmax = performanța maximă obținută, Rpeak = performanța maximă teoretică de vârf estimată)	Scoate în evidență doar viteza 'peak' și numărul de procesoare
Ușor de definit și ușor de clasificat pe baza lui	Nu evidențiază (explicit) rata de transfer locală (bandwidth)
Permite modificarea dimensiunii problemei (n x n)	Nu evidențiază (explicit) rețeaua de comutare
Ocupă sistemul testat cu un job de lungă durată	Nu testează (explicit) mecanisme de tipul gather/scatter
	Ignoră legea lui Amdahl – permite doar scalare 'slabă'
	Un singur număr nu poate caracteriza performanța totală a unui sistem de calcul

Ce înseamnă random scheduling și ce limitări are această tehnică?

Random scheduling (programarea aleatorie) este o tehnică utilizată în testarea programelor paralele pentru a introduce variații aleatoare în ordinea în care sunt programate și executate firele (threads). Scopul acestei tehnici este de a explora diferite programe de execuție pentru a identifica potențiale erori care apar doar în anumite secvențe de execuție.

Cum funcționează:

- Introducerea de întârzieri aleatoare sau yield-uri (cedări voluntare ale controlului) în execuția firelor.

- Utilizarea unor unelte care controlează programarea firelor, modificând ordinea lor de execuție în mod aleatoriu.

Limitări ale Random Scheduling

1. **Număr Exponențial de Scheduluri:**
   - Programele paralele au un număr foarte mare de posibile programe de execuție, care cresc exponențial cu lungimea unui run.
   - Chiar și cu programarea aleatorie, este dificil de acoperit toate programele posibile de execuție, lăsând posibilitatea ca unele erori să nu fie detectate.
2. **Potențial Limitat de Detecție:**
   - Random scheduling poate să nu fie suficient pentru a detecta toate erorile, deoarece unele erori pot apărea doar în anumite condiții sau configurații foarte specifice, care pot fi rare.
3. **Incapacitatea de a Garanța Completitudinea Testării:**
   - Deși poate detecta multe erori, nu poate garanta că toate programele posibile de execuție au fost testate și că programul este complet lipsit de erori.
4. **Complexitate în Reproducere:**
   - Programele de execuție aleatorii pot fi greu de reprodus, ceea ce face dificilă depanarea erorilor detectate. Totuși, unele unelte oferă capacitatea de a salva și reda programele de execuție pentru a ajuta la reproducerea erorilor.

Care este diferența dintre memoria partajată (shared) și cea globală în sistemele de calcul de tip accelerator matematic dezvoltate de Nvidia, ATI/AMD sau Intel?

Diferența principală dintre memoria partajată și cea globală în sistemele de calcul de tip accelerator matematic constă în **viteză** și **accesibilitate**. Memoria partajată este rapidă și locală, oferind acces concurent firelor de execuție dintr-un SM, în timp ce memoria globală este mai lentă, dar accesibilă tuturor firelor de execuție din întregul accelerator.

Caracteristici ale Memoriei Partajate:

1. **Viteză Mare:** Memoria partajată are o latență foarte scăzută comparativ cu alte tipuri de memorie, făcând-o ideală pentru operații care necesită acces rapid și frecvent la date.
2. **Acces Concurent:** Permite accesul concurent al multiplelor fire de execuție, facilitând sincronizarea și partajarea rapidă a datelor.
3. **Localitate:** Este locală în cadrul SM-urilor (Streaming Multiprocessors) și este distribuită între acestea.
4. **Control al Utilizatorului:** Programatorul poate controla explicit utilizarea memoriei partajate, alocând și gestionând această memorie în codul sursă.

Caracteristici ale Memoriei Globale:

1. **Acces Universal:** Este accesibilă de către toate firele de execuție, indiferent de SM-ul pe care rulează.
2. **Capacitate Mare:** Memoria globală oferă o capacitate de stocare mult mai mare decât memoria partajată.
3. **Latență Mare:** Accesul la memoria globală are o latență ridicată, ceea ce poate încetini performanța dacă nu este gestionat eficient.
4. **Acces Asincron:** Firele de execuție pot citi și scrie în memoria globală independent de alte fire, ceea ce poate duce la probleme de coerență a datelor dacă nu sunt utilizate mecanisme de sincronizare adecvate.

Ce este specific core-urilor de GPU fata de cele de CPU?

1. **Paralelism și Performanță:**
   - **GPU:**
     - **Paralelism Masiv:** Pot procesa mii de fire de execuție simultan (SIMD/SIMT).
     - **Throughput Mare:** Ideal pentru grafica 3D, simulări fizice și machine learning.
2. **Arhitectură și Design:**
   - **GPU:**
     - **Core-uri Simple și Ușoare:** Multe core-uri de procesare (Streaming Processors - SP) în fiecare multiprocesor (SM).
     - **Unități Specializate:** Tensor Cores pentru AI și SFU pentru calcule complexe.
3. **Memorie și Latență:**
   - **GPU:**
     - **Memorie Globală și Partajată:** Optimizată pentru acces rapid și larg la date.
     - **Lățime de Bandă Mare:** Concentrează pe transferuri de date rapide.

Concluzie:

Core-urile GPU sunt proiectate pentru paralelism masiv și throughput mare, optimizate pentru aplicații cu cerințe intense de procesare paralelă. În contrast, core-urile CPU sunt optimizate pentru performanță serială și execuție generală, având arhitecturi complexe pentru a gestiona o varietate largă de instrucțiuni și pentru a minimiza latența accesului la memorie.

- **GPU:** Core-uri simple, paralelism masiv, throughput mare.

Cand si pentru ce avem nevoie sa facem benchmarking ?

Definiție:

Benchmarking-ul este procesul de evaluare și comparare a performanțelor unui sistem de calcul sau a componentelor sale prin rularea unui set de teste standardizate, cunoscute sub numele de benchmark-uri.

Nevoia de Benchmarking:

- **Compararea Sistemelor de Calcul:** Specificațiile tehnice ale sistemelor de calcul nu oferă o imagine completă asupra performanței lor reale. Benchmark-urile permit compararea obiectivă a performanțelor între diferite sisteme și arhitecturi.
- **Evaluarea Performanței:** Benchmark-urile ajută la măsurarea performanței diverselor componente ale unui sistem de calcul, cum ar fi CPU, memoria, și subsistemele I/O.
- **Identificarea Bottleneck-urilor:** Ajută la identificarea componentelor sistemului care limitează performanța generală și la evaluarea impactului posibilelor îmbunătățiri hardware sau software.

Care este rolul si utilitatea arbitrului de magistrala intr-un system de calcul? Dati exemplu si descrieti un loc unde apare in cursul de ASC.

Definiție și Rol:

Arbitrajul de magistrală este mecanismul care gestionează accesul la magistrală într-un sistem de calcul, asigurând că doar un dispozitiv la un moment dat poate transmite date pe magistrală. Acesta previne conflictele de acces și garantează utilizarea eficientă a resurselor comune.

Utilitatea Arbitrajului de Magistrală:

- **Previne Conflictele de Acces:** Asigură că doar un singur dispozitiv poate utiliza magistrala la un moment dat, evitând coliziuni și coruperea datelor.
- **Prioritizează Accesul:** Permite stabilirea unei ordini de prioritate între dispozitivele care solicită acces la magistrală, asigurând că dispozitivele mai importante sau cu prioritate mai mare obțin accesul mai rapid.
- **Optimizează Performanța:** Ajută la utilizarea eficientă a magistralei, reducând timpii de așteptare și maximizând throughput-ul sistemului.

Exemplu de Utilizare:

În cursul de Arhitectura Sistemelor de Calcul, arbitrajul de magistrală este descris în contextul **Single Board Computers (SBC)**. Fiecare SBC are mecanisme pentru gestionarea accesului la magistrală, inclusiv unități de comandă specifice pentru accesul la magistrală, cum ar fi KBUS, KML, și altele.

Care este rolul si utilitatea arbitrului de magistrala intr-un system de calcul? Dati exemplu si descrieti un loc unde apare in cursul de ASC.

1. Disponibilitate (High Availability):

- **Definiție:** Disponibilitatea unui sistem de calcul înseamnă capacitatea acestuia de a fi operațional și accesibil pentru utilizatori în cea mai mare parte a timpului. Într-un context MIMD (Multiple Instruction, Multiple Data), aceasta implică faptul că sistemul poate răspunde cererilor de procesare și poate menține funcționalitatea chiar și în condiții de sarcină mare sau în cazul unor defecțiuni ale componentelor.
- **Caracteristici cheie:**
  - **Redundanță:** Implementarea componentelor redundante (ex. procesoare, module de memorie, surse de alimentare) pentru a asigura că, dacă o componentă eșuează, alta poate prelua sarcinile fără întrerupere.
  - **Failover automat:** Capabilitatea de a comuta automat la un sistem de rezervă sau la o componentă redundantă în cazul unei defecțiuni.
  - **Monitorizare și mentenanță:** Monitorizarea constantă a stării sistemului și realizarea mentenanței preventive pentru a evita timpii de nefuncționare neplanificați.

2. Toleranță la Defecte (Fault Tolerance):

- **Definiție:** Toleranța la defecte este capacitatea unui sistem de a continua să funcționeze corect chiar și în cazul apariției unor defecte hardware sau software. În arhitecturile MIMD, acest lucru înseamnă că sistemul poate detecta și izola defectele pentru a preveni afectarea întregului sistem.
- **Caracteristici cheie:**
  - **Detectarea și izolare defectelor:** Sistemele MIMD trebuie să fie capabile să detecteze rapid defectele și să izoleze componentele defecte pentru a preveni propagarea erorilor.
  - **Recuperare automată:** Capacitatea de a reconfigura sistemul astfel încât să ocolească componentele defecte și să redistribuie sarcinile de lucru către componentele funcționale.
  - **Reconfigurabilitate:** Procesoarele și resursele de memorie din sistemul MIMD pot prelua sarcinile altor procesoare în caz de defecțiune, prevenind astfel degradarea performanței.

Ce inseamna ca un system MIMD trebuie sa fie disponibil? Dar tolerant la defecte?

1. Disponibilitate (High Availability):

- **Definiție:** Disponibilitatea unui sistem de calcul înseamnă capacitatea acestuia de a fi operațional și accesibil pentru utilizatori în cea mai mare parte a timpului. Într-un context MIMD (Multiple Instruction, Multiple Data), aceasta implică faptul că sistemul poate răspunde cererilor de procesare și poate menține funcționalitatea chiar și în condiții de sarcină mare sau în cazul unor defecțiuni ale componentelor.
- **Caracteristici cheie:**
  - **Redundanță:** Implementarea componentelor redundante (ex. procesoare, module de memorie, surse de alimentare) pentru a asigura că, dacă o componentă eșuează, alta poate prelua sarcinile fără întrerupere.
  - **Failover automat:** Capabilitatea de a comuta automat la un sistem de rezervă sau la o componentă redundantă în cazul unei defecțiuni.
  - **Monitorizare și mentenanță:** Monitorizarea constantă a stării sistemului și realizarea mentenanței preventive pentru a evita timpii de nefuncționare neplanificați.

2. Toleranță la Defecte (Fault Tolerance):

- **Definiție:** Toleranța la defecte este capacitatea unui sistem de a continua să funcționeze corect chiar și în cazul apariției unor defecte hardware sau software. În arhitecturile MIMD, acest lucru înseamnă că sistemul poate detecta și izola defectele pentru a preveni afectarea întregului sistem.
- **Caracteristici cheie:**
  - **Detectarea și izolare defectelor:** Sistemele MIMD trebuie să fie capabile să detecteze rapid defectele și să izoleze componentele defecte pentru a preveni propagarea erorilor.
  - **Recuperare automată:** Capacitatea de a reconfigura sistemul astfel încât să ocolească componentele defecte și să redistribuie sarcinile de lucru către componentele funcționale.
  - **Reconfigurabilitate:** Procesoarele și resursele de memorie din sistemul MIMD pot prelua sarcinile altor procesoare în caz de defecțiune, prevenind astfel degradarea performanței.

Primitivele PMS: minim 3 perechi atribut+valoare pentru a descrie procesorul.

Procesorul Central – P_C

- Rolul PC:
  - Citeste, interpreteaza si executa instructiuni masina
  - In acest proces are loc generarea comenzilor spre toate resursele sistemului + citirea starilor acestora
- Atribute:
  - Functie: universal | specializat (de semnale, etc)
  - Implementarea: conventionala | μprogramata | μprogramata specializata
  - Formatul instructiunilor: fix | variabil
  - Lungimea instructiunilor: 8 | 16 | 32 | 64 | 128 biti
  - Ciclu instructiune: ciclu fix | ciclu varibil (S cicli masina)
  - Tehnologia: LSI | VLSI | …

Ex:

Atribut	Valoare
Functie	universal
Tip implementare	μprogramata
Formatul instr	8

Care sunt pasii de comunicare intra-cluster in Cm* ?
Evidentiati componentele KMAP implicate. Ce rol are procesorul Link?

---

1. **Inițierea accesului:**
   - În Cm Master se inițiază o cerere de acces la o memorie nelocală.
   - Cm Master emite adresa virtuală către KBUS.
2. **Activarea contextului:**
   - KBUS activează un context, adică creează structura de date specifică tranzacției și o plasează în coada RUN către PMAP.
3. **Translatarea adresei:**
   - PMAP tratează contextul și face translatarea adresei virtuale în adresă fizică.
   - PMAP pune în coada OUT o cerere pentru ciclul de memorie destinat Cm Slave din clusterul curent.
4. **Transmiterea adresei fizice:**
   - KBUS trimite adresa fizică către Cm Slave prin Map Bus.
   - S-ul local din Cm Slave face acces la memoria locală prin furt de ciclu.
5. **Furnizarea rezultatului:**
   - KBUS permite ca rezultatul operației de acces la memorie să fie furnizat către Cm Master.
   - Cm Master preia datele, termină operația în curs și continuă execuția .

Rolul procesorului Link:

Procesorul Link în arhitectura KMAP are rolul crucial de a gestiona comunicarea inter-cluster. Acesta supraveghează transmiterea mesajelor între clustere și se ocupă de următoarele sarcini:

- **Transmiterea mesajelor:** Link trimite cererile de transmitere mesaje inter-cluster și primește răspunsurile aferente.
- **Gestionarea contextelor:** Suspendă contextul inițiator al mesajului inter-cluster și îl trece într-o coadă de așteptare până la primirea răspunsului, moment în care reintroduce contextul în coada RUN pentru a continua procesarea.
- **Supravegherea comunicării:** Asigură că mesajele inter-cluster sunt corect gestionate și că răspunsurile sunt primite și tratate adecvat, menținând astfel integritatea și eficiența comunicării între clustere .

Ce trebuie să facem ca să putem citi în paralel într-un sistem SIMD atât o linie întreagă cât și o coloană întreagă a unei matrice?

Pentru a citi în paralel atât o linie întreagă, cât și o coloană întreagă a unei matrice într-un sistem SIMD, trebuie să organizăm datele în memorie astfel încât să facilităm accesul simultan la ambele structuri.

Organizarea Datelor

Există mai multe metode de organizare a datelor pentru a permite acest tip de acces:

1. **Reprezentarea Directă pe Linii:**
   - Datele sunt organizate astfel încât fiecare linie a matricei să fie stocată în module de memorie separate, permițând accesul paralel la întreaga linie.
   - **Dezavantaj:** Accesul la coloane va fi secvențial, deoarece elementele unei coloane sunt distribuite pe mai multe module de memorie.
2. **Reprezentarea Directă pe Coloane:**
   - Datele sunt organizate astfel încât fiecare coloană a matricei să fie stocată în module de memorie separate, permițând accesul paralel la întreaga coloană.
   - **Dezavantaj:** Accesul la linii va fi secvențial.
3. **Reprezentarea Deplasată Ciclic:**
   - Aceasta este metoda cea mai eficientă pentru a permite accesul paralel atât la linii, cât și la coloane.
   - Elemente ale unei linii sau coloane sunt distribuite ciclic în modulele de memorie, permițând accesul paralel la ambele structuri.

Necesitatea rețelei de comutare

Pentru a facilita accesul paralel la date, este esențială utilizarea unei rețele de comutare eficiente:

- **Retea de comutare P-P (Procesor-Procesor):** Permite comunicarea între procesoare pentru a partaja datele necesare.
- **Retea de comutare P-M (Procesor-Memorie):** Asigură accesul eficient al procesoarelor la modulele de memorie, permitând citirea paralelă a datelor distribuite ciclic.

Concluzie

Pentru a citi în paralel atât o linie întreagă, cât și o coloană întreagă a unei matrice într-un sistem SIMD, trebuie să organizăm datele folosind reprezentarea deplasată ciclic și să utilizăm o rețea de comutare eficientă pentru accesul rapid la date. Acest aranjament permite accesul paralel și eficient la structurile de date necesare, maximizând performanța sistemului SIMD.

Comparati cache-ul total asociativ cu cel set asociativ.

Caracteristică	Cache Total Asociativ	Cache Set Asociativ
Definiție	O linie de memorie poate fi plasată oriunde în cache	O linie de memorie poate fi plasată într-un subset specific de poziții din cache
Avantaje	- Reducerea thrashing-ului	- Complexitate hardware mai mică decât cache-ul total asociativ
Dezavantaje	- Complexitate crescută a unității de comandă	- Poate fi vulnerabil la thrashing
Performanță	- Oferă performanțe mai bune în reducerea conflictelor de plasare a datelor	- Performanță intermediară între cache-ul mapat direct și cel total asociativ
Complexitate hardware	- Crescută, necesită hardware suplimentar pentru gestionarea flexibilității	- Mai mică decât cache-ul total asociativ, dar mai mare decât cache-ul mapat direct

Cum ne asiguram ca un program paralel este fara bug-uri ?

1. **Identificarea Erorilor Comune**:
   - **Data races**: Accesul simultan la aceeași locație de memorie de către mai multe thread-uri, unde cel puțin unul dintre accesuri este o scriere.
   - **Deadlocks**: Situații în care thread-urile așteaptă resurse deținute de altele, blocându-se reciproc.
2. **Utilizarea Instrumentelor de Depanare**:
   - **Intel® Parallel Inspector**: Detectează data races și deadlocks prin analiză dinamică în timpul execuției.
   - **CalFuzzer, Thrille, ConTest**: Instrumente pentru testarea activă și detectarea erorilor de date și sincronizare.
3. **Testarea Paralelă**:
   - **Testing multi-schedule**: Rularea programului de mai multe ori sub diferite configurații și sarcini de lucru pentru a explora diverse programe de execuție și a identifica bug-uri.
4. **Tehnici de Testare și Metode**:
   - **Stress testing**: Rularea programului în condiții de supraîncărcare pentru a schimba sincronizarea thread-urilor și a explora diverse programe de execuție.
   - **Random scheduling**: Introducerea de pauze aleatorii în cod pentru a detecta programele de execuție care duc la bug-uri.
   - **Static analysis**: Analiza codului sursă pentru a prezice erori fără a rula programul, oferind o acoperire mai mare decât analiza dinamică.
5. **Analiza și Modificarea Programului**:
   - **Hotspot analysis**: Identificarea regiunilor din cod care consumă cel mai mult timp și paralelizarea buclelor.
   - **Load balancing**: Distribuirea echitabilă a sarcinilor între thread-uri pentru a evita dezechilibrul de încărcare și timpul irosit de thread-urile inactive.

Ce rol are unitatea de comanda a circuitului crossbar pentru ansamblul Procesoare+Memorii dintr-un system SIMD?

Unitatea de Comandă a Circuitului Crossbar

Unitatea de comandă a circuitului crossbar are un rol esențial în asigurarea comunicării eficiente și rapide între procesoare și modulele de memorie într-un sistem SIMD (Single Instruction, Multiple Data). Aceasta gestionează conexiunile dinamice dintre procesoare și memoriile lor asociate, facilitând astfel accesul paralel la date și optimizând performanța generală a sistemului.

Funcționalitățile Unitații de Comandă:

1. **Citirea și Interpretarea Instrucțiunilor de Control:**
   - Unitatea de comandă (UCmd) citește și interpretează instrucțiunile de control, asigurând coordonarea tuturor operațiunilor de procesare și transfer de date.
2. **Gestionarea Conexiunilor din Crossbar:**
   - Unitatea de comandă configurează conexiunile în circuitul crossbar pentru a asigura transferurile corecte de date între procesoare și modulele de memorie.
   - Aceasta permite maparea dinamică a conexiunilor, asigurând că fiecare procesor poate accesa datele din orice modul de memorie fără conflicte.
3. **Sincronizarea Procesoarelor:**
   - Într-un sistem SIMD, toate procesoarele execută aceeași instrucțiune în același timp. Unitatea de comandă asigură sincronizarea acestor operațiuni, garantând că fiecare procesor accesează corect memoria în conformitate cu instrucțiunile primite.
4. **Rezolvarea Conflictului de Acces:**
   - Unitatea de comandă utilizează diverse tehnici pentru a rezolva conflictele de acces la memorie. Acestea pot include mecanisme de prioritizare și arbitraj pentru a decide ordinea în care procesoarele accesează memoria.

Care sunt pattern-urile de acces la date din programe pe care se bazeaza cache-urile actuale?

Moduri de Accesare a Datelor

1. **Acces Secvențial:**
   - Datele sunt accesate într-o ordine liniară. Aceasta este foarte comună în buclele care parcurg vectori sau matrici.
   - Cache-urile exploatează acest tipar prin preîncărcarea liniilor de memorie adiacente celor curente, minimizând astfel cache miss-urile.
2. **Acces Stride:**
   - Datele sunt accesate la intervale regulate (de exemplu, fiecare a N-a locație de memorie). Aceasta apare frecvent în procesarea matricilor și a structurilor de date multidimensionale.
   - Cache-urile pot fi optimizate pentru a detecta și a se adapta la accesul stride prin ajustarea strategiilor de preîncărcare.
3. **Acces Aleator (Random Access):**
   - Datele sunt accesate în ordine aleatorie, fără un tipar previzibil. Acesta este cel mai dificil de gestionat pentru cache-uri.
   - Performanța cache-ului poate fi îmbunătățită prin tehnici de asociativitate completă și politici avansate de înlocuire a blocurilor de cache.

Ce inseamna profiling si care ar fi pasii prin care facem profiling pe o aplicatie?

Profiling este procesul de măsurare a diferitelor aspecte legate de performanța unui program pentru a identifica zonele care necesită optimizare. Profiling-ul poate ajuta la detectarea problemelor de performanță, cum ar fi secțiunile de cod care consumă prea mult timp sau resurse.

Pașii prin care facem profiling pe o aplicație:

1. **Selectarea instrumentului de profiling:** Alegerea unui tool de profiling adecvat, cum ar fi gprof, Valgrind, perf, sau instrumente specifice de profiling pentru limbajele de programare utilizate (de exemplu, Visual Studio Profiler pentru C#, Py-Spy pentru Python).

2. **Configurarea mediului de execuție:** Asigurarea că aplicația este rulată într-un mediu de test care imită cât mai bine mediul de producție pentru a obține date relevante.

3. **Rularea aplicației cu profiler:** Execuția aplicației cu profilerul activat pentru a colecta date de performanță. Acest lucru implică, de obicei, rularea aplicației cu comanda profilerului sau prin atașarea profilerului la un proces existent.

4. **Analiza datelor colectate:** Interpretarea rapoartelor generate de profiler. Aceste rapoarte vor indica cât timp este petrecut în fiecare funcție, numărul de apeluri la fiecare funcție și alte metrici relevante.

5. **Identificarea bottleneck-urilor:** Localizarea secțiunilor de cod care consumă cele mai multe resurse sau timp de execuție. Acestea sunt punctele unde optimizarea poate avea cel mai mare impact.

6. **Optimizarea codului:** Modificarea codului pentru a îmbunătăți performanța în zonele identificate. Aceste modificări pot include algoritmi mai eficienți, optimizări la nivel de compilator, reducerea accesărilor la memorie, etc.

7. **Reprofiling:** Rularea profilerului din nou pentru a verifica dacă optimizările au avut efectul dorit și pentru a identifica noi oportunități de îmbunătățire.

De ce nu sunt suficiente primitivele PMS pentru a putea trage concluzii vizavi de o ierarhizare a arhitecturii? (din punct de vedere al rulării unei aplicații date) În cazul în care cunoașteți, menționați o soluție mai bună.

Primitivele PMS (Processor Memory Switches) descriu componentele fundamentale ale unui sistem de calcul, cum ar fi procesorul central (PC), memoria (M), și comutatorul (S). Aceste primitive sunt utile pentru a înțelege structura de bază a sistemului, dar nu sunt suficiente pentru a trage concluzii detaliate despre performanța și ierarhizarea arhitecturii din câteva motive:

1. **Lipsa detaliilor despre comportamentul dinamic:** Primitivele PMS oferă o descriere statică a sistemului. Ele nu capturează comportamentul dinamic al componentelor în timpul execuției unui program, cum ar fi latentele de acces la memorie, timpul de comutare al procesorului, sau variațiile de performanță cauzate de cache-uri.

2. **Interacțiunea complexă între componente:** Performanța unui sistem de calcul depinde foarte mult de interacțiunea dintre diferitele componente, cum ar fi interconectările dintre procesor și memorie, bus-ul de date, și canalele de intrare/ieșire. PMS nu descrie aceste interacțiuni în detaliu.

3. **Lipsa detaliilor despre arhitectura cache-ului:** Cache-ul are un impact semnificativ asupra performanței unui sistem de calcul, dar PMS nu include detalii despre nivelurile de cache, dimensiunile acestora, sau algoritmii de gestionare a cache-ului.

4. **Nu se iau în considerare optimizările software:** Performanța unui sistem de calcul depinde nu doar de hardware, ci și de optimizările software, cum ar fi tehnicile de paralelizare, algoritmii de gestionare a memoriei și politicile de scheduling.

De ce nu sunt suficiente primitivele PMS pentru a putea trage concluzii vizavi de o ierarhizare a arhitecturii? (din punct de vedere al rulării unei aplicații date) În cazul în care cunoașteți, menționați o soluție mai bună.

Soluție mai bună:

Pentru a trage concluzii mai precise despre performanța și ierarhizarea arhitecturii, se poate folosi **profiling-ul sistemului**. Profiling-ul este o tehnică care măsoară performanța reală a unui program în timpul execuției sale, oferind informații detaliate despre utilizarea resurselor, latentele de acces, și punctele de blocaj. Acesta poate include:

- **Profiler de performanță:** Instrumente precum gprof, Valgrind, sau perf pot măsura timpul de execuție, frecvența apelurilor de funcție și utilizarea memoriei.
- **Simulatoare de arhitectură:** Instrumente precum Gem5 sau Simics pot simula comportamentul arhitectural detaliat, inclusiv gestionarea cache-urilor și interconectarea componentelor.
- **Benchmarks specifice:** Utilizarea seturilor de benchmark-uri standardizate (SPEC, LINPACK) pentru a evalua performanța și a compara diferite arhitecturi.

Prin utilizarea acestor tehnici, se poate obține o înțelegere detaliată și dinamică a performanței arhitecturii, permițând o ierarhizare și optimizare mai precise.

Să se scrie codul pentru un procesor SIMD cu n elemente de procesare pentru o transformare a tuturor elementelor unui vector A cu m >> n elemente (m multiplu de n) conform relației: A[i] = sqrt(A[i]), știind că A[i] sunt numere pozitive.

Pseudocod pentru SIMD

1. Încărcarea datelor în registrul acumulativ (ACC).
2. Compararea elementelor pentru a verifica dacă sunt pozitive.
3. Încărcarea măștii pentru a activa procesoarele doar pentru numerele pozitive.
4. Aplicarea funcției $\sqrt{}$ asupra elementelor pozitive.
5. Stocarea rezultatelor înapoi în vector.

Aici retinem cat putem, eventual nimic ca-mi bagpl

```c
void transform_array(float* A, int m) {
    // Pasul 1: Încărcarea datelor în registrul acumulativ
    float ACC[N];
    int MASK[N];
    int i, j;

    for (i = 0; i < m; i += N) {
        // Procesăm câte N elemente simultan
        for (j = 0; j < N && (i + j) < m; j++) {
            ACC[j] = A[i + j];
        }

        // Pasul 2: Compararea elementelor pentru a verifica dacă sunt pozitive
        for (j = 0; j < N && (i + j) < m; j++) {
            if (ACC[j] >= 0) {
                MASK[j] = 1;  // Activare procesor
            } else {
                MASK[j] = 0;  // Dezactivare procesor
            }
        }

        // Pasul 3: Aplicarea funcției sqrt asupra elementelor pozitive
        for (j = 0; j < N && (i + j) < m; j++) {
            if (MASK[j]) {
                ACC[j] = sqrt(ACC[j]);
            }
        }

        // Pasul 4: Stocarea rezultatelor înapoi în vector
        for (j = 0; j < N && (i + j) < m; j++) {
            if (MASK[j]) {
                A[i + j] = ACC[j];
            }
        }
    }
}
```

Cunoscand functiile principale ale KMAP in Cm*, sa se enumere si explice pe scurt functionalitatile de baza care trebuie sa existe intr-un cluster cu memorie partajata , pe modelul Cm*

Context și Importanță

KMAP (Key Management and Arbitration Processor) în arhitectura Cm* este esențial pentru gestionarea magistralei de comunicare între modulele computerului (Cm-uri) într-un cluster cu memorie partajată. Într-un sistem multiprocesor cu memorie partajată, KMAP joacă un rol crucial în asigurarea accesului coordonat și eficient la resursele comune.

Funcționalitățile de Bază care Trebuie să Existe într-un Cluster cu Memorie Partajată pe Modelul Cm*

1. **Translatarea Adreselor:**
   - Este necesară o unitate dedicată (PMAP) pentru translatarea rapidă și eficientă a adreselor virtuale în adrese fizice.
   - Aceasta funcționalitate asigură accesul transparent și eficient la memoria partajată.
2. **Arbitrarea Magistralei:**
   - Un arbitru de magistrală (KMAP) trebuie să gestioneze toate cererile de acces la magistrala comună.
   - Asigură prevenirea conflictelor și accesul ordonat la resursele comune.
3. **Sincronizarea Operațiilor:**
   - Sincronizarea accesului la memorie și a operațiilor de calcul între diferitele Cm-uri este crucială pentru coerența datelor și performanța sistemului.
   - Implementarea mecanismelor de sincronizare previne condițiile de cursă și blocarea sistemului.
4. **Comutarea Circuitelor:**
   - Gestionarea eficientă a comutării circuitelor pentru transferul de date rapid între Cm-uri și clustere.
   - Comutarea eficientă asigură utilizarea optimă a lățimii de bandă disponibile.

Cunoscand functiile principale ale KMAP in Cm*, sa se enumere si explice pe scurt functionalitatile de baza care trebuie sa existe intr-un cluster cu memorie partajata , pe modelul Cm*

5. **Transmiterea Mesajelor:**
   - Un mecanism de transmitere a mesajelor între clustere este necesar pentru coordonarea operațiilor distribuite.
   - Asigură comunicarea eficientă și sincronizată între diferitele componente ale sistemului.
6. **Gestionarea Contextului:**
   - Menținerea și gestionarea stării mai multor contexte pentru a manipula eficient cererile simultane de serviciu.
   - Permite procesarea paralelă și continuă a operațiilor fără întreruperi.

Cele de baza pe care trebuie sa le tinem minte ar fi :
- Maparea adreselor
- Sincronizare
- Comunicare

Rolul si functionalitatea unui arbitru de magistral

Rolul Arbitru de Magistrală

Arbitrul de magistrală este o componentă critică într-un sistem de calcul care utilizează o magistrală comună pentru comunicarea între diverse componente (procesor, memorie, dispozitive de intrare/ieșire). Rolul principal al arbitrului de magistrală este de a gestiona accesul la magistrală pentru a preveni conflictele și pentru a asigura o utilizare eficientă și echitabilă a resurselor.

Funcționalitatea Arbitru de Magistrală

1. **Gestionarea Cererilor de Acces:**
   - Arbitrul monitorizează cererile de acces la magistrală din partea tuturor dispozitivelor conectate.
   - Fiecare dispozitiv care dorește să acceseze magistrala trebuie să trimită o cerere de acces către arbitrul de magistrală.
2. **Arbitrarea și Prioritizarea:**
   - Arbitrul decide care cerere va fi servită pe baza unor reguli de arbitraj predefinite.
   - Prioritizarea poate fi bazată pe algoritmi de arbitraj cum ar fi Round Robin, arbitraj bazat pe prioritate sau alte scheme personalizate.
   - Algoritmul de arbitraj asigură că accesul la magistrală este echitabil și eficient, prevenind blocarea unor dispozitive.
3. **Acordarea Dreptului de Acces:**
   - Odată ce o cerere a fost selectată, arbitrul acordă dreptul de acces la magistrală dispozitivului solicitat.
   - Arbitrul generează semnale de control pentru a permite dispozitivului selectat să utilizeze magistrala.

Rolul si functionalitatea unui arbitru de magistral

4. **Monitorizarea și Sincronizarea:**
   - Arbitrul monitorizează durata de utilizare a magistralei pentru fiecare dispozitiv.
   - Sincronizează accesul la magistrală pentru a asigura că nu există conflicte și că datele sunt transferate corect și eficient.
   - În caz de conflict, arbitrul intervine pentru a rezolva problema și a reasigna drepturile de acces.
5. **Gestionarea Termenelor Limită:**
   - În sisteme în timp real, arbitrul poate gestiona termenele limită pentru anumite operații critice.
   - Asigură că cererile cu termene limită stringente sunt procesate în timp util.

Rolul si functionalitatea unui arbitru de magistral

Optimizarea **loop unrolling** (desfășurarea buclei) este o tehnică folosită pentru a îmbunătăți performanța unui program prin reducerea numărului de iterații ale buclei și, implicit, a numărului de salturi și teste condiționate.
Aceasta presupune replicarea corpului buclei de mai multe ori și ajustarea corespunzătoare a condițiilor de terminare.

Motivul pentru Degradarea Performanței

Optimizarea `loop unrolling` nu furnizează performanță decât până la un anumit factor de unrolling din cauza mai multor factori, printre care:

1. **Localitatea Datelor și Cache-ul:**
   - **Localitatea Spațială:** `Loop unrolling` poate îmbunătăți localitatea spațială a datelor, dar numai până la un anumit punct. După un anumit nivel de unrolling, blocurile de date accesate pot depăși dimensiunea cache-ului, ceea ce duce la creșterea ratei de cache miss.
   - **Cache Miss Penalty:** Cu cât mai multe date sunt accesate, cu atât mai mare este probabilitatea ca acestea să nu fie găsite în cache, ceea ce duce la penalități de timp mari datorită necesității de a încărca datele din memoria principală.
2. **Creșterea Dimensiunii Codului:**
   - **Ineficiență în Instrucțiuni:** `Loop unrolling` mărește dimensiunea codului generat. Un cod mai mare poate depăși capacitatea cache-ului de instrucțiuni, ceea ce duce la cache misses pentru instrucțiuni și, astfel, la o performanță scăzută.
   - **Prezența Blocurilor de Cod:** Blocurile de cod replicate pot introduce overhead-uri suplimentare în gestionarea instrucțiunilor, afectând predicția salturilor și execuția paralelă a instrucțiunilor.
3. **Alinierea Datelor:**
   - **Acces Ineficient la Memorie:** Factorii mari de unrolling pot duce la alinierea necorespunzătoare a datelor în memorie, ceea ce poate afecta negativ performanța accesului la date.
4. **Constrângeri Hardware:**
   - **Registre Limitate:** Factorii mari de unrolling necesită mai multe registre pentru a stoca variabilele temporare. Dacă numărul de registre disponibile este depășit, se va recurge la salvarea temporară a datelor în memorie, ceea ce încetinește execuția.
   - **Unități Funcționale:** Un număr mare de instrucțiuni paralele poate depăși capacitatea unităților funcționale ale procesorului, conducând la conflicte și penalități de execuție.

Prezentati permutarea shuffle si dati exemplu de o situtatie in care este utilizata

Permutarea Shuffle este o tehnică de intercalare a datelor în cadrul unei arhitecturi SIMD (Single Instruction, Multiple Data). Permutarea shuffle este utilizată pentru a reorganiza datele astfel încât să fie adecvate pentru anumite operații vectoriale sau matriciale.

Utilizări ale Permutării Shuffle:

1. Transformata Fourier Rapidă (FFT - Fast Fourier Transform)

1. Permutarea shuffle este utilizată pentru a rearanja elementele astfel încât să fie adecvate pentru aplicații de tip FFT, care necesită o anumită ordine a datelor pentru a optimiza calculele.

2. Algoritmi de Sortare

1. Shuffle poate fi folosit în algoritmii de sortare pentru a distribui datele în mod uniform și a facilita comparările și schimburile necesare în procesul de sortare.

3. Transpuneri de Matrice

1. În operațiile de transpunere a matricelor, shuffle ajută la reorganizarea elementelor pentru a fi procesate eficient de către procesoarele SIMD.

Prezentati permutarea shuffle si dati exemplu de o situtatie in care este utilizata

Permutarea Shuffle este o tehnică de intercalare a datelor în cadrul unei arhitecturi SIMD (Single Instruction, Multiple Data). Permutarea shuffle este utilizată pentru a reorganiza datele astfel încât să fie adecvate pentru anumite operații vectoriale sau matriciale.

Exemplu de Situație când Permutarea Shuffle este Utilizată:

Exemplu: Suma Elementelor unui Vector

În cazul în care dorim să calculăm suma elementelor unui vector utilizând o arhitectură SIMD, permutarea shuffle poate fi folosită pentru a organiza datele în mod eficient. Să considerăm un vector distribuit între mai multe procesoare:

1. **Inițializarea Vectorului**
   - Fiecare procesor primește o parte din vector și inițializează suma locală.
   - De exemplu, pentru 8 procesoare (P0, P1, ..., P7), fiecare procesor primește o parte a vectorului.
2. **Aplicarea Permutării Shuffle**
   - Permutarea shuffle este aplicată pentru a rearanja datele astfel încât fiecare procesor să poată accesa elementele necesare pentru suma parțială.
   - Fiecare procesor calculează suma elementelor sale și trimite rezultatul către următorul procesor conform permutării shuffle.
3. **Calculul Sumei Finale**
   - După log2(n) iterații (unde n este numărul de procesoare), fiecare procesor va avea acces la toate elementele vectorului.
   - Suma totală este calculată prin adăugarea sumelor parțiale rezultate din fiecare procesor.

Prezentati arhitectura Cell , facand o paralela cu GPGPU , studiat la curs si la laborator

Efectiv cea mai buna comparatie

Compararea Arhitecturii IBM Cell cu GPGPU

Paralelism:

- **IBM Cell:** Utilizează un set de 8 SPE-uri pentru a oferi paralelism. Fiecare SPE poate executa operații SIMD.
- **GPGPU:** Utilizează sute sau mii de nuclee CUDA pentru a oferi un paralelism masiv, fiecare SM conținând multiple SP-uri care execută instrucțiuni SIMD.

Memorie:

- **IBM Cell:** Fiecare SPE are un Local Store de 256 KB pentru acces rapid la date. Transferurile de date între memoria principală și Local Store sunt gestionate de MFC.
- **GPGPU:** Utilizează memorie globală partajată între toate SM-urile, dar are și memorie partajată locală pe fiecare SM pentru acces rapid la date comune.

Flexibilitate:

- **IBM Cell:** Mai specializat pentru anumite tipuri de aplicații datorită designului SPE-urilor.
- **GPGPU:** Mai flexibil și capabil să execute o gamă largă de aplicații datorită arhitecturii general-purpose.

Performanță:

- **IBM Cell:** Excelent pentru aplicații specifice care pot beneficia de designul heterogen al SPE-urilor.
- **GPGPU:** Excelent pentru aplicații care pot exploata paralelismul masiv și SIMD.

Comparati din punct de Vedere structural core-urile de CPU cu cele de GPU. Subliniati diferentele.

Diferențele Principale

1. **Complexitate și Flexibilitate:**
   - **CPU:** Mai complex, capabil să execute un set larg de instrucțiuni și să gestioneze operații complexe și secvențiale.
   - **GPU:** Mai simplu, specializat pentru operații masive în paralel și execuția unui set restrâns de instrucțiuni.
2. **Optimizare pentru Tipul de Sarcină:**
   - **CPU:** Optimizat pentru sarcini secvențiale, unde latența și performanța per fir de execuție sunt critice.
   - **GPU:** Optimizat pentru sarcini paralele, unde throughput-ul și numărul total de operații pe secundă sunt critice.
3. **Structura Memoriei Cache:**
   - **CPU:** Cache ierarhic extins pentru a reduce latența memoriei.
   - **GPU:** Cache și memorie partajată optimizate pentru acces paralel.
4. **Paralelism:**
   - **CPU:** Paralelism la nivel de instrucțiune (ILP) și un număr mic de nuclee puternice.
   - **GPU:** Paralelism masiv la nivel de date și fir de execuție, cu un număr foarte mare de nuclee simple.

Prezentati comutatorul trunchi K si explicati durata de propagare a semnalului prin acesta, in comparative cu comutatorul crossbar

Descrierea Comutatorului Trunchi K:

Comutatorul trunchi K este un tip de comutator utilizat pentru a conecta multiple componente de tip a (de exemplu, procesoare) cu multiple componente de tip b (de exemplu, memorii). Acesta combină avantajele conectivității oferite de comutatoarele crossbar cu costul redus al altor tipuri de comutatoare.

Structura Comutatorului Trunchi K:

- Este descris ca: S {TK, ma, nb, c = k, k*(m+n)Sp}.
- Orice componentă de tip a poate comunica cu orice componentă de tip b.
- Nu se permite întotdeauna comunicarea simultană maximă (min(m, n)), ci se folosește un parametru k optim, care reprezintă numărul de linii utilizate pentru comunicare.

Funcționarea Comutatorului Trunchi K:

- Dacă avem, de exemplu, 8 procesoare și 4 memorii, putem avea maxim 4 accese la memorii în același timp (k=4).
- Dacă numărul de componente a și b este mare, k se ajustează în funcție de necesitățile traficului, pentru a asigura eficiența și a minimiza conflictele.

Compararea cu Comutatorul Crossbar:

- Comutatorul crossbar oferă o conexiune directă între fiecare pereche de componente a și b, asigurând cea mai scurtă durată de propagare posibilă (Δt = 1), deoarece fiecare cerere de acces la memorie este servită direct printr-un singur comutator.
- Complexitatea unui crossbar crește semnificativ odată cu mărimea rețelei, făcându-l dificil de implementat în sisteme mari din cauza numărului mare de comutatoare elementare necesare (nC = n*m).
- Spre deosebire de trunchi K, un crossbar oferă performanțe superioare în termeni de latență, dar la un cost mult mai mare în ceea ce privește hardware-ul și scalabilitatea.

Comparati cele 3 tipuri de cache : total ascociativ, set asociativ si cu mapare directa . Ce reprezinta fenomenul de trashing ?

Cache cu Mapare Directă (Direct Mapped Cache):

•**Funcționare**: Fiecare bloc de memorie poate fi mapat doar într-o singură locație din cache. Adresa de memorie este împărțită în trei părți: tag, index și offset.

•**Avantaje**:

- Design simplu și rapid.
- Necesită puțin hardware pentru implementare.

•**Dezavantaje**:

- Vulnerabil la fenomenul de thrashing (murdarire), deoarece două adrese diferite de memorie care sunt mapate pe aceeași locație din cache vor cauza un număr mare de cache misses.

2. **Cache Set Asociativ (Set Associative Cache):**

- **Funcționare:** Fiecare bloc de memorie poate fi mapat într-un subset de locații din cache. Cache-ul este împărțit în seturi, fiecare set conținând mai multe linii de cache (blocuri).
- **Avantaje:**
  - Reduce probabilitatea de thrashing comparativ cu cache-ul cu mapare directă.
  - Echilibrează complexitatea și performanța.
- **Dezavantaje:**
  - Mai complicat și costisitor de implementat comparativ cu cache-ul cu mapare directă.
  - Performanța poate fi mai mică decât a unui cache total asociativ, dar mai bună decât a unui cache cu mapare directă.

Comparati cele 3 tipuri de cache : total ascociativ, set asociativ si cu mapare directa . Ce reprezinta fenomenul de trashing ?

3. **Cache Total Asociativ (Fully Associative Cache):**
    - **Funcționare:** Orice bloc de memorie poate fi mapat în orice locație din cache. Adresa de memorie este împărțită doar în două părți: tag și offset.
    - **Avantaje:**
        - Cel mai mic risc de thrashing, deoarece blocurile pot fi plasate oriunde în cache.
    - **Dezavantaje:**
        - Foarte complex și costisitor de implementat.
        - Necesită mai mult hardware pentru gestionarea tag-urilor și determinarea locului unde să fie plasate blocurile de memorie.

Fenomenul de Thrashing

Thrashing:

- **Definiție:** Thrashing-ul apare atunci când două sau mai multe regiuni de memorie sunt accesate repetat și nu pot coexista în același timp în cache. Acest lucru duce la un număr mare de cache misses și la performanțe scăzute.
- **Exemplu:** Într-un cache cu mapare directă, dacă două blocuri de memorie diferite se mapează pe aceeași linie de cache, fiecare acces la aceste blocuri va cauza un cache miss și va înlocui blocul precedent, ceea ce duce la thrashing.

Care sunt pasii rularii unei aplicatii pe GPU, pe modelul GPGPU? De ce thread-urile de pe Gpu sunt organizate in blocuri si blocurile intr-un grid ?

Pașii Rulării unei Aplicații pe GPU

1. **Inițializarea Mediului GPU:**
   - Alocarea memoriei pe dispozitivul GPU pentru datele care vor fi procesate.
   - Transferul datelor de pe CPU (host) pe GPU (device).
2. **Definirea și Lansarea Kernel-ului:**
   - Kernel-ul este funcția care va fi rulată pe GPU. Este definită folosind un limbaj specific (e.g., CUDA pentru NVIDIA, OpenCL pentru multiple platforme).
   - Lansarea kernel-ului implică specificarea numărului de blocuri și a numărului de thread-uri pe bloc.
3. **Execuția Kernel-ului:**
   - Fiecare thread execută instanța kernel-ului în paralel.
   - Blocurile de thread-uri sunt distribuite pe diferitele unități de calcul (multiprocesoare) ale GPU-ului.
4. **Sincronizarea și Finalizarea:**
   - Așteptarea ca toate thread-urile să termine execuția.
   - Transferul rezultatelor din memoria GPU în memoria CPU dacă este necesar.
5. **Dezalocarea Resurselor:**
   - Eliberarea memoriei alocate pe GPU.

Care sunt pasii rularii unei aplicatii pe GPU, pe modelul GPGPU? De ce thread-urile de pe Gpu sunt organizate in blocuri si blocurile intr-un grid ?

Organizarea Thread-urilor în Blocuri și Grid

1. **Scalabilitate și Flexibilitate:**
   - Blocurile și grid-urile permit scalarea aplicațiilor pentru a folosi eficient toate unitățile de calcul disponibile pe GPU.
   - Programatorii pot lansa un număr mare de thread-uri care se vor împărți automat pe resursele disponibile.
2. **Gestionarea Resurselor:**
   - Blocurile de thread-uri permit gestionarea eficientă a resurselor precum registrele și memoria partajată.
   - Fiecare bloc are acces la o memorie partajată rapidă, care poate fi utilizată pentru a reduce accesările costisitoare la memoria globală.
3. **Sincronizarea:**
   - Thread-urile din același bloc pot fi sincronizate ușor folosind instrucțiuni de sincronizare, asigurând astfel corectitudinea execuției.
   - Sincronizarea între blocuri este mai dificilă și în general necesită transferuri explicite de date între GPU și CPU.
4. **Eficiența Memoriei:**
   - Organizarea în blocuri permite utilizarea optimă a cache-urilor și memoriei partajate, reducând latențele de memorie și îmbunătățind performanța.
   - Blocurile și grid-urile ajută la maximizarea utilizării memoriei de tip cache prin accesări locale și contigue.

Descrieți folosind primitivele PMS componenta Memorie cu minim 5 primitive.

Memoria – M

- Rolul memoriei:
  - Pastreaza informatia
  - Actualizeaza informatia prin operatii de citire/scriere
  - Necesita un sistem de adresare cu o corespondenta liniara intre adresa si continut
- Atribute:
  - Functie: primara | auxiliara
  - Tehnologie: bipolara | MOS | statica | dinamica
  - Operatii: citire | citire/scriere
  - Mod acces: aleator | secvential | FIFO | LIFO | asociativ
  - Lungime cuvant: 8+1 | 16+2 | 32+4 | 64+8
  - Capacitatea: 4Mb | 8Mb | 16Mb | …| 1Gb | 2Gb | 4Gb | 8Gb
  - Ciclu de lucru: 2 | 4ms | … | 400ns | 200ns | 100ns

Descrieți comutatorul ierarhic de tip legatura multipla(CB).

Comutatoare Ierarhice Legatura Multipla

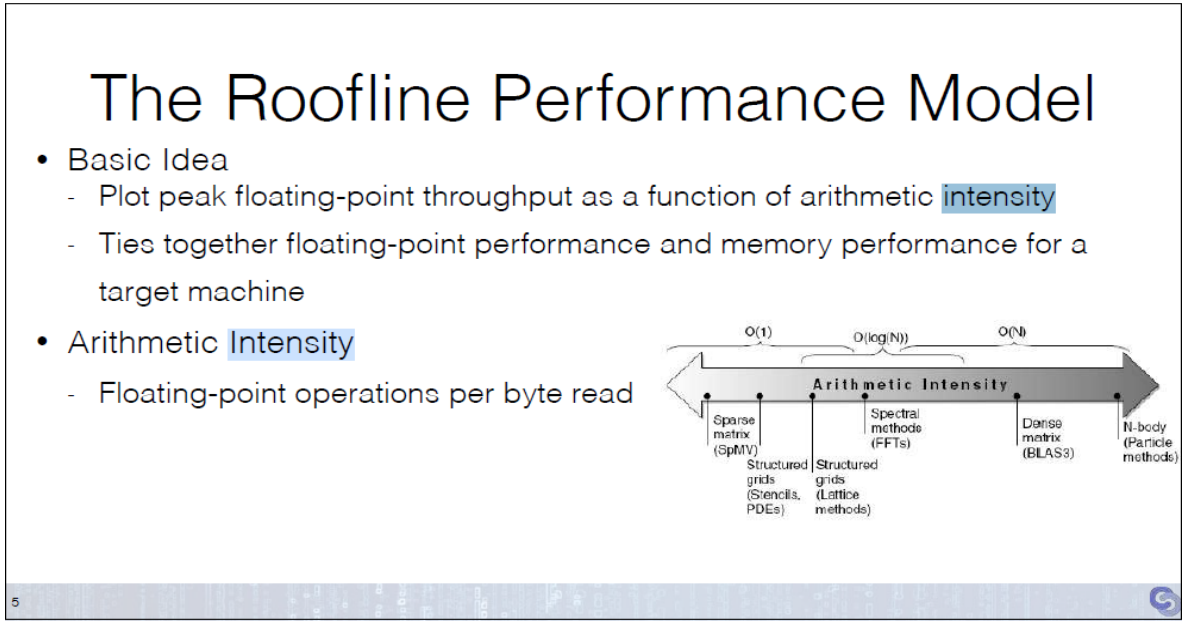
- Asigura comunicatia intre m componente de tip a si n componente de tip b
- Este cunoscut si ca Cross-Bar-Switch
  - S {CB, ma, nb, c = min(m, n), m*nSp}

- Orice a poate comunica cu orice b
- Pot avea loc transferuri multiple si simultane!
- Cel mai dificil este de implementat U_{Cmd} pentru S_{CB}
  - Trebuie sa rezolve toate posibilele conflicte

Explicati modelul Roofline si modul cum poate fi utilizat

GPT4o POV:

Modelul Roofline este un instrument grafic folosit pentru a evalua și optimiza performanța aplicațiilor pe arhitecturi de procesare, precum CPU și GPU. Acest model ajută la vizualizarea limitărilor performanței și la identificarea punctelor în care aplicaţiile pot fi îmbunătăţite.



Exemple de Utilizare

1. Optimizarea Aplicațiilor de Simulare

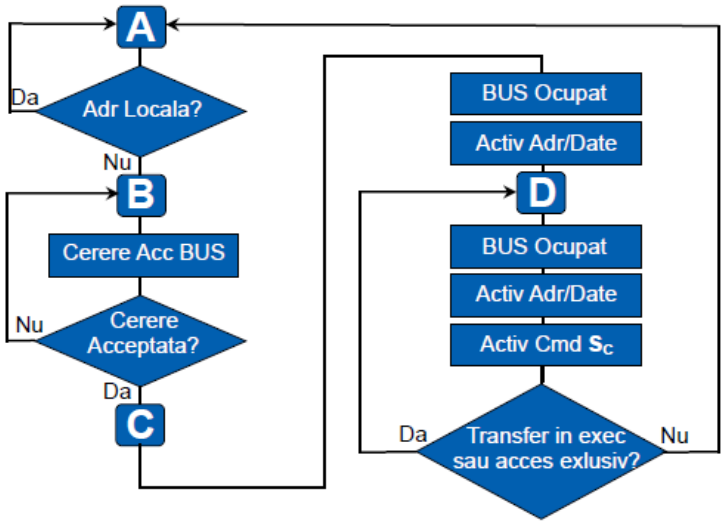
- Aplicațiile de simulare științifică, care implică multe calcule și accesări de date, pot fi optimizate prin identificarea dacă sunt limitate de memorie sau de calcul și aplicarea tehnicilor corespunzătoare de optimizare.

2. Analiza Performanței GPU-urilor

- Pentru aplicațiile care rulează pe GPU-uri, modelul Roofline poate ajuta la determinarea dacă intensitatea aritmetică a unui kernel este suficient de mare pentru a maximiza utilizarea capacității de calcul a GPU-ului.

Realizați schema logică și descrieți funcționarea unității KBUS din dispozitivele SBC.

KBUS



Descriere KBUS

- KBUS – reprezinta procesorul local in relatie cu magistrala externa
- Starea A: adresa e locala?
  - Da = se adreseaza resurse locale, nu trebuie iesit pe magistrala
  - Nu = adr nu e locala → se trece in starea B
- Starea B: cerere de acces la magistrala; daca cererea e acceptata se trece in starea C
- Starea C: se emite Bus Busy (mag ocupata) – doar in C & D; accesul pe BUS e exclusiv & cu rafala (overwrite) raman in D pt un nou ciclu daca mai e cazul; automatul preia functia procesorului; Dupa preluarea comenzii, procesorul face functia de secventiere a Adr, Date, Cmd
- Intrebare: de ce si starea C si starea D? Din cauza ciclului instructiune curent. Trebuie sincronizate comenzile cu decodificarea adreselor & stabilirea datelor pe magistrala (e un Δ_t acolo = T_{wait} datorat interpretarii adreselor) → se introduce starea D pentru a compensa acest timp de latenta: starea de overwrite
- Obs: KBUS trebuie sa dispuna de unitati logice combinationale ce identifica:
  - Daca adresa mem/port I/O implicate in transfer sunt locale
  - Semnalul de cerere acceptata e combinat din mai multe conditii: mag libera & modul SBC activ

Descrieți o rețea de comutare de tip Delta cu 16 intrări și 25 de ieșiri Și explicați modul de rutare corespunzător ieșirii 6.

Nu cred ca asa se face

Calculul nivelelor

Pentru o rețea Delta cu 16 intrări și 25 ieșiri, putem folosi comutatoare 5×5 și 4×5, deoarece:

- $25 = 5^2$
- $16 = 4^2$

Acest lucru înseamnă că vom avea 2 nivele de comutatoare.

Structura rețelei Delta

- **Nivelul 0:** conține 4 × 5 comutatoare pentru a conecta 16 de intrări la 20 de ieșiri intermediare.
- **Nivelul 1:** conține 5 × 5 comutatoare pentru a conecta 20 de ieșiri intermediare la 25 de ieșiri finale.

Mod de rutare pentru ieșirea 6

Pentru a explica modul de rutare corespunzător ieșirii 6, trebuie să urmărim pașii prin rețea. Adresa ieșirii 6 în baza 5 este 11.

1. **Nivelul 0:** primul digit 1 indică faptul că pachetul va trece prin al doilea comutator 4×5 de la nivelul 0.
2. **Nivelul 1:** al doilea digit 1 indică faptul că pachetul va trece prin al doilea comutator 5×5 de la nivelul 1.

Detalii Rutare

Nivelul 0:

- Intrările sunt grupate în 4 comutatoare 4x5.
- Intrarea care urmează să ajungă la ieșirea 6 trece prin al doilea comutator 4x5.

Nivelul 1:

- Ieșirile intermediare de la nivelul 0 devin intrările pentru nivelul 1.
- Pachetul trece prin al doilea comutator 5x↓in nivelul 1 pentru a ajunge la ieșirea finală 6.

Sisteme cu memorie partajată:
UMA și NUMA.
Care sunt asemănările și care sunt deosebirile între aceste tipuri de sisteme. Oferiți exemple de astfel de sisteme.

Asemănări

1. **Memorie Partajată:**
   - Ambele arhitecturi, UMA (Uniform Memory Access) și NUMA (Non-Uniform Memory Access), utilizează un model de memorie partajată. Procesoarele pot accesa o zonă comună de memorie.
   - Sunt utilizate în arhitecturi multiprocesor, unde mai multe procesoare colaborează pentru a executa taskuri comune.
2. **Acces la Memorie:**
   - În ambele sisteme, orice procesor poate accesa oricare locație de memorie.
   - Utilizează tehnici de coerență a cache-ului pentru a menține consistența datelor în memoriile cache locale ale procesoarelor.

Deosebiri între CC-NUMA și COMA

1. **Uniformitatea Accesului la Memorie:**
   - **UMA (Uniform Memory Access):**
     - Timpul de acces la orice locație de memorie este uniform și constant, indiferent de procesorul care face accesul.
     - Exemple: Sisteme SMP (Symmetric Multiprocessing).
   - **NUMA (Non-Uniform Memory Access):**
     - Timpul de acces la memorie depinde de locația memoriei față de procesorul care face accesul. Memoria locală a unui procesor poate fi accesată mai rapid decât memoria aflată la distanță, în alte noduri.
     - Exemple: Silicon Graphics Origin 2000, Sequent NUMA-Q, Stanford DASH.
2. **Structura de Interconectare:**
   - **UMA:**
     - Utilizează o rețea de interconectare simplă și poate utiliza o magistrală comună pentru toate procesoarele.
     - Scalabilitatea este limitată deoarece o magistrală comună devine un factor limitator pe măsură ce numărul de procesoare crește.
   - **NUMA:**
     - Utilizează o rețea de interconectare complexă, adesea sub forma unei rețele inelare, mesh sau alte topologii complexe.
     - Scalabilitatea este mai bună decât în UMA deoarece fiecare nod poate avea propria memorie locală și interconectări eficiente între noduri.
3. **Coerența Memoriei:**
   - **UMA:**
     - Gestionarea coerenței memoriei este simplificată deoarece toate accesele la memorie au aceleași caracteristici de timp.
   - **NUMA:**
     - Necesită mecanisme avansate de gestionare a coerenței cache-ului pentru a asigura consistența datelor între memoriile locale ale diferitelor noduri.

Ce speedup maxim poate fi atins, cu un număr infinit de unități de procesare, dacă 85% din cod este paralelizabil?

Legea lui Amdahl

Legea lui Amdahl stipulează că potențialul maxim de accelerare al unui program folosind mai multe procesoare este limitat de partea secvențială a programului. Formula de bază a legii este:

$$\text{Speedup} = \frac{1}{(1-P)+\frac{P}{N}}$$

unde:

- P este fracțiunea de cod paralelizabilă.
- N este numărul de procesoare.

În cazul nostru:

- $P = 0.85$ (85% din cod este paralelizabil).
- $N = \infty$ (număr infinit de procesoare).

Calculul Speedup-ului Maxim

Cu un număr infinit de procesoare, fracțiunea de timp necesară pentru partea paralelizabilă a codului tinde spre zero. Formula devine:

$$\text{Speedup}_{\infty} = \frac{1}{(1-P)}$$

Substituind valorile noastre:

$$\text{Speedup}_{\infty} = \frac{1}{(1-0.85)}$$
$$\text{Speedup}_{\infty} = \frac{1}{0.15}$$
$$\text{Speedup}_{\infty} \approx 6.67$$

Ce este un Benchmark? Oferiti o taxonomie a benchmarkurilor. Oferiti detalii pentru cel putin doua benchmarkuri pentru sisteme de calcul cu un singur CPU.

Ce este un Benchmark?

Un benchmark este un test sau un set de teste folosite pentru a evalua performanța unui sistem de calcul, a unei componente hardware sau a unui software. Benchmark-urile sunt folosite pentru a compara diferite sisteme sau configurații, pentru a determina cât de bine funcționează în diferite scenarii și pentru a identifica potențialele îmbunătățiri.

Taxonomia Benchmark-urilor

Benchmark-urile pot fi clasificate în funcție de diferite criterii. Iată o taxonomie a benchmark-urilor, conform cursului de benchmarking:

1. **Aplicații Reale:** Procesoare de text, aplicații utilizator
2. **Kernel:** Conțin doar părți esențiale dintr-o varietate de programe (ex. Linpack)
3. **Micro-benchmark-uri:** Dezvoltate de utilizatori pentru a testa componente ale sistemului
4. **Sintetice:** Construite pe baza datelor statistice asupra operațiilor efectuate de aplicații reale (ex. Whetstone, Dhrystone)
5. **Pentru sisteme I/O:** Evaluarea performanțelor subsistemelor de intrare/ieșire
6. **Pentru sisteme paralele:** Evaluarea performanțelor în mediul de calcul paralel

Ce este un Benchmark? Oferiti o taxonomie a benchmarkurilor. Oferiti detalii pentru cel putin doua benchmarkuri pentru sisteme de calcul cu un singur CPU.

1. **SPEC CPU**

Descriere:

- SPEC CPU (Standard Performance Evaluation Corporation) este un set de benchmark-uri folosit pentru a evalua performanța procesoarelor pe sarcini de calcul intensive.
- Testele includ 29 de aplicații de tip integer și floating-point (FP), scrise în C și Fortran.

Scop:

- Evaluarea performanței CPU în condiții de sarcini de calcul variate și realiste.

Metode de măsurare:

- Se folosesc două tipuri de scoruri: SPECint pentru sarcini integer și SPECfp pentru sarcini floating-point.
- Performanța este măsurată în raport cu un sistem de referință și rezultatele sunt prezentate ca un scor SPEC.

Exemplu de utilizare:

- Compararea performanțelor între diferite modele de CPU sau diferite arhitecturi de sisteme de calcul.

2. **GeekBench**

Descriere:

- GeekBench este un benchmark multi-platformă care evaluează performanța sistemelor de calcul, inclusiv laptopuri, stații de lucru, telefoane mobile, etc.
- Măsoară performanța procesorului, a memoriei și a lățimii de bandă.

Scop:

- Oferirea unui set de rezultate care să fie ușor comparabile între diferite dispozitive și platforme.

Metode de măsurare:

- GeekBench rulează un set de teste standardizate și generează un scor bazat pe performanța obținută.
- Rezultatele pot fi salvate online, permițând comparații facile între diferite dispozitive.

Exemplu de utilizare:

- Compararea performanței între un laptop și un telefon mobil sau între diferite generații de procesoare.

Ce este stress testing ? La ce e folosit ? Un exemplu

Stress testing (testarea la stres) este o metodă de testare utilizată pentru a evalua stabilitatea şi fiabilitatea unui sistem de calcul sub condiţii de încărcare extremă.

Scopul este de a determina limitele unui sistem şi de a observa comportamentul acestuia când este supus unei sarcini de lucru intense şi prelungite.

La ce este folosit Stress Testing?

Stress testing este folosit pentru:

1. **Evaluarea Stabilității:** Pentru a verifica dacă un sistem de calcul poate rămâne stabil și operațional sub sarcini de lucru mari și prelungite.
2. **Identificarea Punctelor Slabe:** Pentru a descoperi potențiale probleme legate de hardware sau software care pot apărea doar sub condiții de încărcare extremă.
3. **Validarea Performanței:** Pentru a confirma că performanța sistemului este la nivelul așteptat chiar și în condiții de utilizare intensă.

Exemplu de Stress Testing

Un exemplu clasic de stress testing este rularea unui program care utilizează toate nucleele procesorului la capacitate maximă pentru o perioadă prelungită de timp. Un astfel de test poate implica:

- Rularea de instanțe multiple ale unui program paralel pentru a satura toate resursele disponibile.
- Utilizarea unui instrument de testare, cum ar fi Prime95, care efectuează calcule intensive pentru a stresa CPU-ul și memoria.

Prime95 este un exemplu de tool folosit pentru stress testing:

- **Scop:** Testarea stabilității și a performanței procesorului și a memoriei.
- **Metodologie:** Prime95 rulează teste de calcul intensiv, cum ar fi testele Mersenne prime, pentru a stresa procesorul la maximum. Dacă CPU-ul sau sistemul de memorie au probleme, testul va detecta erori sau va determina ca sistemul să se blocheze sau să se repornească.
- **Utilizare:** Prime95 este adesea folosit de entuziaștii de overclocking pentru a valida stabilitatea unui sistem după ce au mărit frecvența de funcționare a procesorului.

Numiti cel putin 3 caracteristici distinctive ale ultimilor arhitecturi de procesoare Nvidia/AMD/Intel(2020+) pe care le-ati folosit.

. Intel: Arhitectura Tiger Lake

•Nuclee Willow Cove:

- Arhitectura Tiger Lake utilizează nuclee Willow Cove, care oferă îmbunătăţiri semnificative în performanţă şi eficienţă energetică faţă de generaţiile anterioare. Acestea includ optimizări în pipeline-ul de execuţie şi suport pentru instrucţiuni noi.

•Grafică integrată Intel Iris Xe:

- Tiger Lake introduce grafica integrată Intel Iris Xe, care oferă performanţe grafice mult îmbunătăţite, include suport pentru DirectX 12, Vulkan şi alte API-uri grafice moderne, fiind capabilă să gestioneze jocuri şi aplicaţii grafice complexe.

•Tehnologie SuperFin:

- Intel a implementat tehnologia de fabricare SuperFin în arhitectura Tiger Lake, care oferă un echilibru mai bun între performanţă şi eficienţă energetică. Aceasta permite procesoarelor să funcţioneze la frecvenţe mai mari cu un consum redus de energie.